

# **Understanding Health Insurance Cost Predictions Using Deep Neural Networks, Monte Carlo Simulation and Shapley Values**

by

Thabang Bongani Junior Baloyi

Dissertation submitted in partial fulfilment of the requirements in respect  
of the degree

**Master's Degree**

**(Mathematical Statistics & Actuarial Sciences)**

in the Department of Mathematical Statistics & Actuarial Science  
in the Faculty of Natural and Agricultural Sciences  
at the University of the Free State



UNIVERSITY OF THE FREE STATE  
UNIVERSITEIT VAN DIE VRYSTAAT  
YUNIVESITHI YA FREISTATA

Bloemfontein Campus, South Africa  
May 2026

Supervisor: Dr JM Blomerus

## Originality Declaration

I hereby declare that this work, submitted for the degree M.Sc. (Mathematical Statistics), at the University of the Free State, is my own original work and has not previously been submitted, for degree purposes or otherwise, to any other institution of higher learning. I further declare that all sources cited or quoted are indicated and acknowledged by means of a comprehensive list of references. All uses of AI tools are fully disclosed in relevant sections. Copyright hereby cedes to and is invested in the University of the Free State. I declare that all royalties as regards to intellectual property that was developed during the course of and/or in connection with the study at the University of the Free State, will accrue to the University.

---

Thabang Bongani Junior Baloyi

---

Date

## Declaration on the Use of Generative AI

In accordance with the University of the Free State's policies on academic integrity and the responsible use of artificial intelligence in research, the following declaration is made regarding the use of generative AI tools during the preparation of this dissertation.

Generative AI tools, including large language models, were used during certain stages of the research process. Their use was limited to the following purposes:

- Assisting with language editing, grammar correction and stylistic refinement of draft text written by the author.

All content generated or assisted by AI tools was critically reviewed, verified for accuracy and rewritten in the author's own words where necessary. The research design, statistical analysis, code implementation, interpretation of results and all scientific conclusions presented in this dissertation are the independent work of the author under the guidance of the supervisor, Dr J. M. Blomerus. No AI-generated output was accepted without verification against the data, code and relevant literature.

The author takes full responsibility for the accuracy and integrity of all material presented herein.

## Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr JM Blomerus, for his guidance, patience and constructive feedback throughout the development of this dissertation. His support helped me refine the direction of the research and strengthened the statistical and methodological quality of this work.

I would like to thank my family for their constant encouragement, sacrifices and belief in me throughout this journey. Their support gave me the strength to continue during demanding periods of the research process. I am equally thankful to my friends, who pushed and motivated me in ways they may not even have realised. Their presence, encouragement and belief in my ability helped me remain focused when the process became challenging.

I would also like to thank my partner for being understanding and patient whenever I had to prioritise my work. Her support, especially during the long hours and demanding stages of this dissertation, meant a great deal to me.

Finally, I acknowledge the personal discipline, resilience and perseverance required to complete this dissertation. This work represents not only an academic contribution, but also an important milestone in my growth as a researcher in mathematical statistics and actuarial science.

Xitsonga xiri, xihlovo xa le mahlweni imadyisa mbitsi.

## Abstract

This dissertation develops a three-stage statistical framework for health insurance cost prediction using a publicly available benchmark dataset of one million policyholder records. In the first stage, a deep neural network is trained to predict insurance charges, with a five-phase hyperparameter comparison selecting the final architecture and benchmarks provided by a Gamma generalised linear model, a random forest and an XGBoost ensemble. In the second stage, parametric Monte Carlo simulation generates synthetic policyholder profiles from fitted marginal distributions; the trained network is evaluated at each profile, producing a model-implied predicted-cost distribution validated through joint-distribution diagnostics and residual-augmented sensitivity analysis. In the third stage, Shapley values decompose each predicted cost into additive feature contributions at both the global level and conditionally within high-cost regions defined by upper-tail thresholds.

The deep neural network achieves test-set  $R^2 = 0.9957$ , outperforming the generalised linear model ( $R^2 = 0.9571$ ), random forest ( $R^2 = 0.9899$ ) and XGBoost ( $R^2 = 0.9955$ ) benchmarks. Shapley attribution identifies smoking status, coverage level and medical history as the dominant cost drivers globally. In the upper tail, the importance ranking shifts: coverage level, medical history and family medical history overtake smoking status, illustrating that the factors driving extreme predicted costs differ from those most important on average. All conclusions are internal to the fitted model and benchmark data and are not presented as causal claims.

**Keywords:** deep neural network, health insurance cost prediction, Monte Carlo simulation, Shapley values, explainable machine learning, generalised linear model, random forest, XGBoost, statistical learning.

# List of Figures

|    |                                                                                                                                 |     |
|----|---------------------------------------------------------------------------------------------------------------------------------|-----|
| 1  | Prediction, Simulation and Explanation framework. . . . .                                                                       | 41  |
| 2  | Position of the health-insurance cost prediction task within a standard machine-learning taxonomy. . . . .                      | 50  |
| 3  | Candidate feed-forward neural network architecture for health insurance cost prediction. . . . .                                | 56  |
| 4  | Illustration of the bias-variance trade-off. Adapted from (James et al., 2021). . . . .                                         | 59  |
| 5  | Construction of the input distribution $\hat{F}_X$ (equation (2.2.5)). . . . .                                                  | 68  |
| 6  | Empirical distributions of the numerical variables. . . . .                                                                     | 82  |
| 7  | Pearson correlation heatmap for the numerical variables. . . . .                                                                | 83  |
| 8  | Categorical predictor diagnostics: frequency distributions and conditional boxplots. . . . .                                    | 84  |
| 9  | Two-way interaction effects on <b>charges</b> . . . . .                                                                         | 86  |
| 10 | Encoded predictor correlation analysis on the training partition. . . . .                                                       | 87  |
| 11 | Funnel architecture of the deep neural network. . . . .                                                                         | 89  |
| 12 | Learning curves for the fitted DNN. . . . .                                                                                     | 102 |
| 13 | Sorted actual versus predicted insurance charges on the training and test sets. . . . .                                         | 103 |
| 14 | Violin plots comparing the distribution of actual and predicted charges on the training and test sets. . . . .                  | 104 |
| 15 | Residual diagnostics for the fitted DNN. . . . .                                                                                | 105 |
| 16 | PCA projections coloured by actual and predicted charges on training and test sets. . . . .                                     | 107 |
| 17 | PCA loading vectors for all 22 encoded predictors. . . . .                                                                      | 108 |
| 18 | Input-target correlation heatmaps for actual and predicted charges. . . . .                                                     | 110 |
| 19 | Correlation structure comparison: training data versus parametric MC sample. . . . .                                            | 115 |
| 20 | Cramér’s $V$ comparison: training versus parametric MC sample. . . . .                                                          | 116 |
| 21 | PCA projection of the joint input structure: (a) training data, (b) parametric MC sample, and (c) overlay of both sets. . . . . | 117 |
| 22 | Shapley attribution invariance: parametric MC versus bootstrap MC input regimes. . . . .                                        | 118 |
| 23 | Phase 1: validation MSE loss (left) and validation $R^2$ (right) by activation function over training epochs. . . . .           | 127 |
| 24 | Phase 2: validation MSE loss (left) and validation $R^2$ (right) by learning rate. . . . .                                      | 128 |
| 25 | Phase 3: validation MSE loss (left) and validation $R^2$ (right) by batch size. . . . .                                         | 129 |
| 26 | Phase 4: validation MSE loss (left) and validation $R^2$ (right) by optimiser. . . . .                                          | 129 |

|    |                                                                                                                                                                                                                                                               |     |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 27 | Phase 5: validation MSE loss (left) and validation $R^2$ (right) by first momentum coefficient $\beta_1$ . . . . .                                                                                                                                            | 130 |
| 28 | Best validation MSE loss across all hyperparameter configurations evaluated in Phases 1-5. . . . .                                                                                                                                                            | 130 |
| 29 | Test-set performance comparison across all four model classes: Generalised Linear Model (GLM) (Gamma, log-link), Random Forest (RF), eXtreme Gradient Boosting (XGBoost) and Deep Neural Network (DNN). (a) Test $R^2$ ; (b) test RMSE; (c) test MAE. . . . . | 133 |
| 30 | Overfitting diagnostic: training versus test $R^2$ for all four model classes. . .                                                                                                                                                                            | 133 |
| 31 | Continuous variable pairs: training (top row, blue) versus Monte Carlo (bottom row, red). . . . .                                                                                                                                                             | 135 |
| 32 | Conditional distributions of selected categorical variable pairs: training (solid bars) versus Monte Carlo (hatched bars). . . . .                                                                                                                            | 136 |
| 33 | Three-way cross-tabulations of the four key categorical variables: training (blue) versus Monte Carlo (red). . . . .                                                                                                                                          | 137 |
| 34 | Energy distance permutation test: null distribution and observed statistic. . . .                                                                                                                                                                             | 138 |
| 35 | t-SNE projection of training versus Monte Carlo joint distributions. . . . .                                                                                                                                                                                  | 138 |
| 36 | Marginal distributions of all 11 raw predictor variables: training versus Monte Carlo. . . . .                                                                                                                                                                | 139 |
| 37 | Monte Carlo predicted-cost distribution ( $B = 100\,000$ ): histogram with kernel density overlay (left) and box plot (right). . . . .                                                                                                                        | 140 |
| 38 | Monte Carlo convergence diagnostics by simulation size $B$ . . . . .                                                                                                                                                                                          | 141 |
| 39 | Overlay of the Monte Carlo predicted-cost distribution (red) and the observed training-set charge distribution (blue). . . . .                                                                                                                                | 143 |
| 40 | Centred training residual diagnostics for the augmented simulation. . . . .                                                                                                                                                                                   | 144 |
| 41 | Residual-augmented Monte Carlo sensitivity analysis. . . . .                                                                                                                                                                                                  | 144 |
| 42 | Global Shapley feature importance for all 11 raw variables. . . . .                                                                                                                                                                                           | 145 |
| 43 | SHAP beeswarm plot for all 22 encoded features. . . . .                                                                                                                                                                                                       | 147 |
| 44 | Shapley attribution invariance: training data versus Monte Carlo profiles. .                                                                                                                                                                                  | 148 |
| 45 | Conditional versus global mean  SHAP  importance at three upper-tail thresholds. . . . .                                                                                                                                                                      | 149 |
| 46 | Feature importance across thresholds: mean  SHAP  values at the global, $\tau_{90}$ , $\tau_{95}$ and $\tau_{99}$ levels for all 11 raw variables. . . . .                                                                                                    | 151 |
| 47 | Distributional profiles of the high-cost sets compared with the full Monte Carlo sample. . . . .                                                                                                                                                              | 152 |
| 48 | SHAP waterfall plot for a representative high-cost profile at the $\tau_{95}$ threshold (predicted cost R24 249). . . . .                                                                                                                                     | 153 |
| 49 | Test-set performance comparison across all four model classes: GLM (Gamma, log-link), Random Forest, XGBoost and DNN. . . . .                                                                                                                                 | 190 |

---

|    |                                                       |     |
|----|-------------------------------------------------------|-----|
| 50 | Charges KDE across partitions. . . . .                | 193 |
| 51 | Numeric feature KDE across partitions. . . . .        | 194 |
| 52 | Dummy variable proportions across partitions. . . . . | 194 |

# List of Tables

|    |                                                                                                                                                                    |     |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 1  | Variable dictionary and model-ready representation. . . . .                                                                                                        | 77  |
| 2  | Variable encoding conventions and reference categories. . . . .                                                                                                    | 79  |
| 3  | Descriptive statistics and distributional shape for numerical variables. . . .                                                                                     | 81  |
| 4  | Selected percentiles of the response variable <b>charges</b> . . . . .                                                                                             | 82  |
| 5  | Pearson correlation matrix for the numerical variables. . . . .                                                                                                    | 83  |
| 6  | Feed-forward DNN architecture and training configuration. . . . .                                                                                                  | 92  |
| 7  | DNN training algorithm. . . . .                                                                                                                                    | 94  |
| 8  | GLM fitting algorithm. . . . .                                                                                                                                     | 95  |
| 9  | Random Forest training algorithm. . . . .                                                                                                                          | 97  |
| 10 | Random Forest configuration. . . . .                                                                                                                               | 98  |
| 11 | XGBoost training algorithm. . . . .                                                                                                                                | 100 |
| 12 | XGBoost configuration. . . . .                                                                                                                                     | 101 |
| 13 | Monte Carlo input distributions for raw policyholder variables. . . . .                                                                                            | 112 |
| 14 | Monte Carlo convergence quantities recorded during the pilot run. . . . .                                                                                          | 113 |
| 15 | Base Monte Carlo simulation algorithm. . . . .                                                                                                                     | 114 |
| 16 | Comparison of the two Monte Carlo input regimes. . . . .                                                                                                           | 118 |
| 17 | Residual-augmented Monte Carlo sensitivity design. . . . .                                                                                                         | 121 |
| 18 | Residual-augmented Monte Carlo algorithm. . . . .                                                                                                                  | 121 |
| 19 | SHAP implementation settings. . . . .                                                                                                                              | 122 |
| 20 | Shapley summaries calculated for simulated profiles. . . . .                                                                                                       | 125 |
| 21 | Summary of modelling assumptions and interpretive boundaries. . . . .                                                                                              | 126 |
| 22 | Selected hyperparameters from the sequential comparison. . . . .                                                                                                   | 131 |
| 23 | Predictive performance of the final DNN on the original monetary scale. . .                                                                                        | 131 |
| 24 | Predictive performance comparison across four model classes on the original<br>monetary scale. . . . .                                                             | 132 |
| 25 | Distributional summary of the Monte Carlo predicted costs ( $B = 100\,000$ ). .                                                                                    | 140 |
| 26 | Monte Carlo convergence: running summaries and MCSE by simulation size. .                                                                                          | 142 |
| 27 | Global Shapley feature importance: mean signed and mean absolute values<br>for all 11 raw variables on the normalised scale. . . . .                               | 146 |
| 28 | High-cost thresholds from the Monte Carlo predicted-cost distribution. . .                                                                                         | 149 |
| 29 | Mean absolute Shapley values: global and conditional at each high-cost<br>threshold (normalised scale), with 95% bootstrap confidence intervals at $\tau_{99}$ . . | 150 |
| 30 | Methodological comparison between the present dissertation and the ICEnet<br>of Richman and Wüthrich (2024). . . . .                                               | 168 |
| 31 | Reference categories used in dummy encoding. . . . .                                                                                                               | 183 |
| 32 | Marginal distributions used for Monte Carlo profile generation. . . . .                                                                                            | 187 |
| 33 | Dummy-to-raw variable aggregation mapping. . . . .                                                                                                                 | 191 |

---

|    |                                                                           |     |
|----|---------------------------------------------------------------------------|-----|
| 34 | Full five-phase hyperparameter comparison results. . . . .                | 195 |
| 35 | Best hyperparameter configuration selected for the final DNN model. . . . | 196 |
| 36 | Python package versions used in the dissertation. . . . .                 | 197 |

# List of Equations

|        |                                                             |    |
|--------|-------------------------------------------------------------|----|
| 2.2.1  | Dataset of input-output pairs . . . . .                     | 41 |
| 2.2.2  | Regression formulation . . . . .                            | 42 |
| 2.2.3  | Fitted DNN prediction function . . . . .                    | 42 |
| 2.2.4  | Training attribute vectors . . . . .                        | 42 |
| 2.2.5  | Input distribution as product of fitted marginals . . . . . | 43 |
| 2.2.6  | Monte Carlo sample from input distribution . . . . .        | 43 |
| 2.2.7  | Fitted predictions for Monte Carlo samples . . . . .        | 43 |
| 2.2.8  | Set of input features . . . . .                             | 44 |
| 2.2.9  | Shapley value function . . . . .                            | 44 |
| 2.2.10 | Exact Shapley value of feature $j$ . . . . .                | 44 |
| 2.2.11 | Shapley decomposition of fitted prediction . . . . .        | 44 |
| 2.2.12 | Permutation-sampling Shapley estimator . . . . .            | 45 |
| 2.2.13 | High-cost threshold . . . . .                               | 45 |
| 2.2.14 | Simulated high-cost set . . . . .                           | 45 |
| 2.2.15 | Proportion exceeding high-cost threshold . . . . .          | 45 |
| 2.2.16 | Signed Shapley contribution in high-cost region . . . . .   | 45 |
| 2.2.17 | Absolute Shapley contribution in high-cost region . . . . . | 45 |
| 2.3.1  | Mean squared error . . . . .                                | 47 |
| 2.3.2  | Root mean squared error . . . . .                           | 47 |
| 2.3.3  | Mean absolute error . . . . .                               | 47 |
| 2.3.4  | Coefficient of determination . . . . .                      | 47 |
| 2.3.5  | Linear predictor . . . . .                                  | 47 |
| 2.3.6  | Conditional mean function . . . . .                         | 48 |
| 2.3.7  | Generalised linear model link function . . . . .            | 48 |
| 2.4.1  | Input layer . . . . .                                       | 51 |
| 2.4.2  | Pre-activation vector . . . . .                             | 51 |
| 2.4.3  | Activation with nonlinear function . . . . .                | 51 |
| 2.4.4  | Fitted DNN prediction for observation $i$ . . . . .         | 51 |
| 2.4.5  | Parameter vector . . . . .                                  | 52 |
| 2.4.6  | Matrix form of layer recursion . . . . .                    | 52 |
| 2.4.7  | Hyperbolic tangent activation . . . . .                     | 52 |
| 2.4.8  | Rectified linear unit . . . . .                             | 52 |
| 2.4.9  | CELU activation function . . . . .                          | 53 |
| 2.4.10 | GELU activation function . . . . .                          | 53 |
| 2.4.11 | Mean squared error loss . . . . .                           | 53 |
| 2.4.12 | Fitted parameters by loss minimisation . . . . .            | 54 |
| 2.4.13 | Layer recursion without bias . . . . .                      | 57 |

|        |                                                                  |    |
|--------|------------------------------------------------------------------|----|
| 2.4.14 | Training, validation and test partition . . . . .                | 57 |
| 2.4.15 | Training loss at epoch $e$ . . . . .                             | 57 |
| 2.4.16 | Validation loss at epoch $e$ . . . . .                           | 57 |
| 2.4.17 | Optimal epoch selection . . . . .                                | 57 |
| 2.4.18 | Population regression formulation . . . . .                      | 58 |
| 2.4.19 | Pointwise bias-variance decomposition . . . . .                  | 59 |
| 2.4.20 | Population expected prediction error . . . . .                   | 60 |
| 2.7.1  | Empirical row resampling . . . . .                               | 67 |
| 2.7.2  | Maximum likelihood for marginal parameters . . . . .             | 67 |
| 2.7.3  | Draw from fitted marginal distribution . . . . .                 | 67 |
| 2.8.1  | Target quantity as expectation . . . . .                         | 68 |
| 2.8.2  | IID sample from distribution $F$ . . . . .                       | 69 |
| 2.8.3  | Crude Monte Carlo estimator . . . . .                            | 69 |
| 2.8.4  | Law of large numbers for Monte Carlo . . . . .                   | 69 |
| 2.8.5  | Variance condition for $g(X)$ . . . . .                          | 69 |
| 2.8.6  | Variance of Monte Carlo estimator . . . . .                      | 69 |
| 2.8.7  | Monte Carlo standard error . . . . .                             | 69 |
| 2.8.8  | Sample variance for MCSE . . . . .                               | 69 |
| 2.8.9  | Central limit theorem for Monte Carlo . . . . .                  | 69 |
| 2.8.10 | Monte Carlo confidence interval . . . . .                        | 69 |
| 3.2.1  | Input standardisation of numerical predictors . . . . .          | 79 |
| 3.2.2  | Response standardisation . . . . .                               | 79 |
| 3.2.3  | Response de-normalisation to monetary scale . . . . .            | 79 |
| 3.4.1  | DNN prediction on standardised scale . . . . .                   | 89 |
| 3.4.2  | DNN total trainable parameter count . . . . .                    | 90 |
| 3.4.8  | Validation $R^2$ epoch selection criterion . . . . .             | 91 |
| 3.4.9  | GLM specification with gamma distribution and log link . . . . . | 94 |
| 3.4.10 | Gamma GLM log-likelihood . . . . .                               | 95 |
| 3.4.11 | GLM score equations . . . . .                                    | 95 |
| 3.4.12 | Random Forest ensemble prediction . . . . .                      | 96 |
| 3.4.13 | Random Forest split criterion . . . . .                          | 96 |
| 3.4.14 | Random Forest variance reduction . . . . .                       | 97 |
| 3.4.15 | XGBoost additive ensemble model . . . . .                        | 98 |
| 3.4.16 | XGBoost regularised objective . . . . .                          | 98 |
| 3.4.17 | XGBoost tree complexity penalty . . . . .                        | 99 |
| 3.4.18 | XGBoost gradient and Hessian statistics . . . . .                | 99 |
| 3.4.19 | XGBoost second-order approximate objective . . . . .             | 99 |
| 3.4.20 | XGBoost optimal leaf weight and objective . . . . .              | 99 |
| 3.4.21 | XGBoost split gain criterion . . . . .                           | 99 |

|       |                                                                        |     |
|-------|------------------------------------------------------------------------|-----|
| 3.5.1 | Residual definition . . . . .                                          | 101 |
| 3.5.2 | PCA sample covariance matrix . . . . .                                 | 106 |
| 3.5.3 | PCA eigenvalue decomposition . . . . .                                 | 106 |
| 3.5.4 | PCA principal component scores . . . . .                               | 106 |
| 3.6.1 | Product-marginal Monte Carlo input distribution . . . . .              | 111 |
| 3.6.2 | Categorical probability estimator from training data . . . . .         | 112 |
| 3.6.3 | Base Monte Carlo DNN prediction . . . . .                              | 114 |
| 3.7.1 | Correlation matrix difference (training minus MC) . . . . .            | 115 |
| 3.7.2 | Bootstrap Monte Carlo resampling from empirical distribution . . . . . | 117 |
| 3.7.3 | Spearman rank correlation for Shapley invariance . . . . .             | 118 |
| 3.8.1 | Centred fitted residuals . . . . .                                     | 120 |
| 3.8.2 | Residual standard deviation . . . . .                                  | 120 |
| 3.8.3 | Residual draw from empirical distribution . . . . .                    | 120 |
| 3.8.4 | Residual-augmented predicted cost . . . . .                            | 120 |
| 3.8.5 | Fixed residual perturbation sensitivity band . . . . .                 | 120 |
| 3.9.1 | Raw-variable Shapley aggregation from encoded inputs . . . . .         | 123 |
| 3.9.2 | Mean signed Shapley contribution . . . . .                             | 123 |
| 3.9.3 | Mean absolute Shapley contribution (feature importance) . . . . .      | 123 |

## List of Acronyms / Abbreviations

| Acronym        | Meaning                                             |
|----------------|-----------------------------------------------------|
| <b>ANN</b>     | Artificial Neural Networks                          |
| <b>BMI</b>     | Body Mass Index                                     |
| <b>CELU</b>    | Continuously Differentiable Exponential Linear Unit |
| <b>DNN</b>     | Deep Neural Network                                 |
| <b>GELU</b>    | Gaussian Error Linear Unit                          |
| <b>GLM</b>     | Generalised Linear Model                            |
| <b>ICE</b>     | Individual Conditional Expectation                  |
| <b>LOOCV</b>   | Leave-One-Out Cross-Validation                      |
| <b>MAE</b>     | Mean Absolute Error                                 |
| <b>MAPE</b>    | Mean Absolute Percentage Error                      |
| <b>MCSE</b>    | Monte Carlo Standard Error                          |
| <b>MSE</b>     | Mean Squared Error                                  |
| <b>PCA</b>     | Principal Component Analysis                        |
| $R^2$          | Coefficient of Determination                        |
| <b>ReLU</b>    | Rectified Linear Unit                               |
| <b>RF</b>      | Random Forest                                       |
| <b>RMSE</b>    | Root Mean Squared Error                             |
| <b>SHAP</b>    | SHapley Additive exPlanations                       |
| <b>XGBoost</b> | eXtreme Gradient Boosting                           |

# Contents

|                                                                                               |           |
|-----------------------------------------------------------------------------------------------|-----------|
| Originality Declaration                                                                       | 2         |
| Declaration on the Use of Generative AI                                                       | 3         |
| Acknowledgements                                                                              | 4         |
| Abstract                                                                                      | 5         |
| List of Figures                                                                               | 6         |
| List of Tables                                                                                | 9         |
| List of Equations                                                                             | 11        |
| List of Acronyms / Abbreviations                                                              | 14        |
| <b>1 Introduction</b>                                                                         | <b>19</b> |
| 1.1 Background and Problem Statement . . . . .                                                | 19        |
| 1.1.1 Classical actuarial pricing models . . . . .                                            | 19        |
| 1.1.2 Deep neural networks for insurance regression . . . . .                                 | 21        |
| 1.1.3 Beyond point prediction: distributional analysis and feature attri-<br>bution . . . . . | 21        |
| 1.2 Positioning within Mathematical Statistics . . . . .                                      | 22        |
| 1.2.1 Supervised regression and function approximation . . . . .                              | 22        |
| 1.2.2 Monte Carlo simulation in statistical inference . . . . .                               | 23        |
| 1.2.3 Shapley values and cooperative game theory . . . . .                                    | 23        |
| 1.2.4 The integration as the statistical contribution . . . . .                               | 24        |
| 1.3 Conceptual Framework . . . . .                                                            | 24        |
| 1.4 Significance of the Study . . . . .                                                       | 25        |
| 1.5 Research Questions . . . . .                                                              | 26        |
| 1.6 Research Objectives . . . . .                                                             | 27        |
| 1.7 Research Gap . . . . .                                                                    | 28        |
| 1.8 Methodological Overview . . . . .                                                         | 29        |
| 1.8.1 Prediction stage . . . . .                                                              | 29        |
| 1.8.2 Simulation stage . . . . .                                                              | 29        |
| 1.8.3 Explanation stage . . . . .                                                             | 30        |
| 1.8.4 The benchmark dataset . . . . .                                                         | 30        |
| 1.9 Validation and Robustness Strategy . . . . .                                              | 30        |
| 1.10 Scope and Delimitations . . . . .                                                        | 31        |
| 1.11 Ethical Considerations . . . . .                                                         | 33        |

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| 1.12     | Definition of Key Terms . . . . .                                         | 33        |
| 1.13     | Dissertation Outline . . . . .                                            | 35        |
| <b>2</b> | <b>Literature Review</b>                                                  | <b>37</b> |
| 2.1      | Introduction . . . . .                                                    | 37        |
| 2.2      | DNN-Monte Carlo-Shapley Framework . . . . .                               | 37        |
| 2.2.1    | Predictive Modelling . . . . .                                            | 41        |
| 2.2.2    | Monte Carlo Simulation . . . . .                                          | 42        |
| 2.2.3    | Shapley attribution . . . . .                                             | 43        |
| 2.2.4    | High-cost profiles . . . . .                                              | 45        |
| 2.3      | Statistical Learning and Supervised Regression . . . . .                  | 46        |
| 2.4      | Deep Neural Networks for Regression . . . . .                             | 51        |
| 2.4.1    | Activation Functions . . . . .                                            | 52        |
| 2.4.2    | Loss and Optimisation . . . . .                                           | 53        |
| 2.4.3    | Generalisation, Validation Analysis and Bias-Variance Trade-off . . . . . | 58        |
| 2.5      | Health Insurance Pricing and Cost Prediction . . . . .                    | 62        |
| 2.6      | Input Distribution Diagnostics for Simulation . . . . .                   | 65        |
| 2.7      | Specification of the Input Distribution . . . . .                         | 66        |
| 2.8      | Foundations of Monte Carlo . . . . .                                      | 68        |
| 2.9      | Variance Reduction and Tail Simulation . . . . .                          | 70        |
| 2.10     | Explainability, Shapley Values and Conditional Interpretation . . . . .   | 72        |
| 2.11     | Research Gap . . . . .                                                    | 74        |
| <b>3</b> | <b>Methods and Data</b>                                                   | <b>76</b> |
| 3.1      | Data Source and Variable Structure . . . . .                              | 76        |
| 3.2      | Data Audit, Partitioning and Preprocessing . . . . .                      | 78        |
| 3.2.1    | Data audit . . . . .                                                      | 78        |
| 3.2.2    | Training, validation and test partition . . . . .                         | 78        |
| 3.2.3    | Encoding and standardisation . . . . .                                    | 78        |
| 3.2.4    | Data-leakage prevention . . . . .                                         | 80        |
| 3.3      | Exploratory Diagnostics . . . . .                                         | 80        |
| 3.3.1    | Numerical variables and the response . . . . .                            | 80        |
| 3.3.2    | Dependence structure . . . . .                                            | 82        |
| 3.3.3    | Categorical predictors and encoded features . . . . .                     | 84        |
| 3.3.4    | Partition consistency . . . . .                                           | 88        |
| 3.3.5    | Decisions from the exploratory diagnostics . . . . .                      | 88        |
| 3.4      | Network Specification and Training . . . . .                              | 89        |
| 3.4.1    | Architecture . . . . .                                                    | 89        |
| 3.4.2    | Forward and Backward Propagation . . . . .                                | 90        |
| 3.4.3    | Training configuration . . . . .                                          | 91        |

|          |                                                                         |            |
|----------|-------------------------------------------------------------------------|------------|
| 3.4.4    | Hyperparameter comparison . . . . .                                     | 92         |
| 3.4.5    | Benchmark Generalised Linear Model . . . . .                            | 94         |
| 3.4.6    | Tree-Based Benchmarks . . . . .                                         | 96         |
| 3.5      | Predictive and Structural Diagnostics . . . . .                         | 101        |
| 3.5.1    | Predictive error assessment . . . . .                                   | 102        |
| 3.5.2    | Actual versus predicted diagnostics . . . . .                           | 103        |
| 3.5.3    | Residual diagnostics . . . . .                                          | 104        |
| 3.5.4    | Principal component structural diagnostics . . . . .                    | 106        |
| 3.5.5    | Correlation heatmap diagnostics . . . . .                               | 109        |
| 3.6      | Parametric Monte Carlo Simulation Design . . . . .                      | 111        |
| 3.6.1    | Joint input distribution . . . . .                                      | 111        |
| 3.6.2    | Fitted marginal distributions . . . . .                                 | 111        |
| 3.6.3    | Goodness-of-fit and support checks . . . . .                            | 113        |
| 3.6.4    | Sample size and convergence monitoring . . . . .                        | 113        |
| 3.6.5    | Base DNN Monte Carlo scoring . . . . .                                  | 114        |
| 3.7      | Validation of the Monte Carlo Input Distribution . . . . .              | 114        |
| 3.7.1    | Joint distribution diagnostics . . . . .                                | 115        |
| 3.7.2    | Bootstrap Monte Carlo resampling . . . . .                              | 117        |
| 3.7.3    | Shapley invariance assessment . . . . .                                 | 118        |
| 3.7.4    | Interpretation and conclusion . . . . .                                 | 119        |
| 3.8      | Residual-Augmented Monte Carlo Sensitivity Analysis . . . . .           | 119        |
| 3.8.1    | Fitted residual distribution . . . . .                                  | 119        |
| 3.8.2    | Residual-augmented simulation . . . . .                                 | 120        |
| 3.8.3    | Fixed residual perturbation check . . . . .                             | 120        |
| 3.9      | Shapley-Value Attribution for Simulated Profiles . . . . .              | 121        |
| 3.9.1    | Prediction function and Shapley decomposition . . . . .                 | 121        |
| 3.9.2    | Implementation settings . . . . .                                       | 122        |
| 3.9.3    | Aggregation from encoded inputs to raw variables . . . . .              | 123        |
| 3.9.4    | Global Shapley summaries . . . . .                                      | 123        |
| 3.9.5    | High-cost region and conditional Shapley summaries . . . . .            | 123        |
| 3.9.6    | Bootstrap confidence intervals for conditional Shapley values . . . . . | 124        |
| 3.10     | Modelling Assumptions . . . . .                                         | 125        |
| <b>4</b> | <b>Results</b>                                                          | <b>127</b> |
| 4.1      | Hyperparameter Comparison Results . . . . .                             | 127        |
| 4.1.1    | Phase 1: Activation function . . . . .                                  | 127        |
| 4.1.2    | Phase 2: Learning rate . . . . .                                        | 128        |
| 4.1.3    | Phase 3: Batch size . . . . .                                           | 128        |
| 4.1.4    | Phase 4: Optimiser . . . . .                                            | 129        |

|          |                                                                           |            |
|----------|---------------------------------------------------------------------------|------------|
| 4.1.5    | Phase 5: Momentum parameters . . . . .                                    | 129        |
| 4.1.6    | Hyperparameter comparison overview . . . . .                              | 130        |
| 4.1.7    | Final model predictive performance . . . . .                              | 131        |
| 4.1.8    | Baseline comparison: GLM, RF, XGBoost and DNN . . . . .                   | 132        |
| 4.2      | Monte Carlo Predicted-Cost Distribution . . . . .                         | 134        |
| 4.2.1    | Simulated input validation . . . . .                                      | 135        |
| 4.2.2    | Model-implied cost distribution . . . . .                                 | 140        |
| 4.2.3    | Convergence diagnostics . . . . .                                         | 141        |
| 4.2.4    | Comparison with observed training charges . . . . .                       | 143        |
| 4.3      | Residual-Augmented Monte Carlo Sensitivity . . . . .                      | 143        |
| 4.4      | Shapley-Value Attribution Results . . . . .                               | 145        |
| 4.4.1    | Global feature importance . . . . .                                       | 145        |
| 4.4.2    | Feature-level SHAP beeswarm . . . . .                                     | 147        |
| 4.4.3    | Training versus Monte Carlo attribution invariance . . . . .              | 148        |
| 4.5      | High-Cost Conditional Shapley Analysis . . . . .                          | 148        |
| 4.5.1    | Conditional versus global importance . . . . .                            | 149        |
| 4.5.2    | Threshold stability of the importance ranking . . . . .                   | 151        |
| 4.5.3    | High-cost profile characteristics . . . . .                               | 152        |
| 4.5.4    | Representative high-cost waterfall decomposition . . . . .                | 153        |
| 4.6      | Integration of Monte Carlo and Shapley Results . . . . .                  | 154        |
| <b>5</b> | <b>Conclusion</b>                                                         | <b>156</b> |
| 5.1      | Review of the Dissertation . . . . .                                      | 156        |
| 5.2      | Addressing the Research Questions . . . . .                               | 158        |
| 5.3      | Contributions . . . . .                                                   | 161        |
| 5.4      | Strengths of the Study . . . . .                                          | 163        |
| 5.5      | Limitations . . . . .                                                     | 164        |
| 5.6      | Implications for Actuarial Practice and Statistical Methodology . . . . . | 166        |
| 5.7      | Comparison with Constrained Neural-Network Approaches . . . . .           | 168        |
| 5.8      | Recommendations for Future Research . . . . .                             | 169        |
| 5.9      | Concluding Remark . . . . .                                               | 172        |
|          | <b>References</b>                                                         | <b>174</b> |
|          | <b>Annexures</b>                                                          | <b>179</b> |

# 1 Introduction

## 1.1 Background and Problem Statement

Health insurance cost prediction lies at the intersection of statistical learning, actuarial modelling and explainable machine learning. The empirical task involves estimating a continuous monetary outcome from policyholder characteristics such as age, body-mass index, smoking status, coverage level and medical history. In the insurance industry, these estimates underpin individual pricing, reserve adequacy and portfolio risk management. A single fitted value or prediction-error metric, however, does not fully characterise the behaviour of the fitted model. Understanding how predicted costs vary across a simulated population and identifying which input variables contribute most to high predicted costs are questions that require methods beyond point prediction.

It is therefore important to establish the modelling landscape within which this dissertation operates: the classical actuarial framework based on generalised linear models, the tree-based ensemble methods that occupy an intermediate position between linear and fully flexible models, the introduction of deep neural networks into insurance pricing, and the growing demand for post-hoc model explanation. Each of these themes is introduced below, before the specific research problem and analytical framework are stated.

### 1.1.1 Classical actuarial pricing models

The statistical modelling of insurance costs has a long history. Generalised linear models (GLMs) have served as the standard framework for actuarial pricing since the seminal work of [Nelder and Wedderburn \(1972\)](#) and their subsequent adoption by the insurance profession. In health insurance, the Gamma distribution with a logarithmic link function is commonly used to model the right-skewed, positive-valued distribution of claim amounts. Under this specification, the conditional mean of the response is given by

$$\log \mu_i = \mathbf{x}_i^\top \boldsymbol{\beta}, \quad (1.1.1)$$

where  $\mu_i = E[Y_i \mid \mathbf{x}_i]$  is the conditional mean cost for policyholder  $i$ ,  $\mathbf{x}_i$  is the vector of covariate values and  $\boldsymbol{\beta}$  is the coefficient vector. The multiplicative structure of the log-link allows each coefficient to be interpreted as a relative risk factor: exponentiating  $\beta_j$  yields the multiplicative effect of a unit increase in the  $j$ -th covariate on the expected cost, holding all other covariates constant. This interpretive simplicity has made GLMs the dominant modelling tool in actuarial practice for several decades.

Despite their advantages, GLMs impose a pre-specified linear predictor structure that

restricts the functional form of the relationship between covariates and the response. Interaction effects must be specified manually, and nonlinear relationships can only be captured through explicit basis expansions or spline terms. When the true relationship between policyholder attributes and cost is complex, involving higher-order interactions among multiple categorical and continuous variables, these structural constraints limit the predictive performance of the model. The gap between what a linear predictor can represent and what the data contain motivates the exploration of more flexible model classes.

Moreover, the actuarial convention of using the Gamma distribution assumes a specific variance-mean relationship ( $\text{Var}(Y_i) \propto \mu_i^2$ ), which may not hold uniformly across all segments of the policyholder population. While robust standard errors and alternative distributional assumptions can mitigate this concern, the fundamental limitation of the linear predictor structure remains. As [Goodfellow et al. \(2016\)](#) note, the capacity of a model class to approximate a target function is determined by the richness of the hypothesis space, and the hypothesis space of a GLM is inherently constrained by its additive structure.

Between the constrained linear predictor of the GLM and the fully flexible hidden-layer representations of a deep neural network lie tree-based ensemble methods, which approximate nonlinear relationships through collections of decision trees. Random forests ([Breiman, 2001](#)) aggregate predictions across independently grown trees, each trained on a bootstrap sample with a random feature subset at each split, reducing variance while capturing interactions that the GLM cannot represent. Gradient-boosted trees, exemplified by the XGBoost implementation ([Chen and Guestrin, 2016](#)), construct an additive ensemble of shallow trees sequentially, with each new tree fitted to the residuals of the preceding ensemble. Both methods offer greater flexibility than the GLM but remain fundamentally piecewise-constant approximators: unlike neural networks, they do not learn smooth, continuous mappings through differentiable transformations. Including a random forest and an XGBoost benchmark alongside the GLM and DNN provides a graded comparison across model classes of increasing flexibility. The core comparison remains the actuarial-standard GLM against the DNN, since this dissertation is situated within an actuarial modelling context where the GLM is the established industry baseline and the DNN represents the class of model whose behaviour the Monte Carlo and Shapley analyses are designed to examine. The tree-based benchmarks complement this pairing by establishing where piecewise-constant nonlinear methods fall on the performance spectrum.

### 1.1.2 Deep neural networks for insurance regression

Deep neural networks provide a flexible alternative to the GLM framework. The universal approximation theorem (Hornik et al., 1989; Cybenko, 1989) establishes that a sufficiently wide feed-forward network with a non-polynomial activation function can approximate any continuous function on a compact set to arbitrary precision. In formal terms, for any continuous  $f : [0, 1]^p \rightarrow \mathbb{R}$  and any  $\varepsilon > 0$ , there exists a single-hidden-layer network  $F$  such that  $\sup_{\mathbf{x}} |F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$ . In practice, modern deep networks with multiple hidden layers and nonlinear activations learn hierarchical representations that capture complex interactions without requiring the modeller to enumerate them in advance (Goodfellow et al., 2016).

This capacity for automatic feature interaction has led to increasing adoption of neural networks in insurance applications. Studies by Richman (2021) and Kuo (2019) have demonstrated that neural networks can achieve predictive improvements over classical actuarial models on insurance cost prediction tasks, particularly when the data contain non-linear covariate-response relationships that the GLM cannot capture. The feed-forward architecture, in which each layer computes  $\mathbf{h}^{(\ell)} = \sigma(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)})$  for an activation function  $\sigma$ , weight matrix  $\mathbf{W}^{(\ell)}$  and bias vector  $\mathbf{b}^{(\ell)}$ , is the most commonly used architecture for tabular regression problems.

The flexibility of deep neural networks, however, comes at the expense of direct interpretability. The parameters of a trained network, which may number in the tens of thousands, do not admit the same coefficient-level reading as a linear model. A policyholder with specific attributes receives a predicted cost, but the relative contribution of each attribute to that prediction is not immediately transparent from the network weights. This lack of transparency is a practical concern in regulated industries such as insurance, where stakeholders require not only accurate predictions but also explanations of the factors driving those predictions. The tension between predictive flexibility and model transparency has motivated a substantial body of research on post-hoc explanation methods, of which Shapley-value attribution is one of the most principled.

### 1.1.3 Beyond point prediction: distributional analysis and feature attribution

Prediction accuracy alone provides an incomplete picture of model behaviour. A test-set metric such as  $R^2$  or RMSE summarises average predictive performance but does not reveal how the predicted cost varies across the input space, what the distributional shape of predicted costs looks like under population-level input variation, or which variables are most important in specific regions of the output distribution. Answering these ques-

tions requires supplementary analyses that go beyond the standard train-test evaluation paradigm.

The challenge is compounded by the heterogeneity of policyholder populations. An insurance portfolio typically contains a large number of low-cost policyholders and a smaller number of high-cost policyholders whose claims dominate the aggregate loss. From a risk-management perspective, understanding the drivers of predicted costs in this upper tail is at least as important as understanding the average prediction. Global feature-importance summaries, which aggregate attributions across all predictions, may obscure the factors that are most relevant in the high-cost region. A conditional analysis that focuses specifically on the upper tail of the predicted-cost distribution is therefore needed to complement the global view.

This tension between predictive flexibility and interpretability, and between point-prediction accuracy and distributional understanding, motivates the present study. A framework is needed that combines flexible prediction with structured distributional analysis and principled feature attribution, so that the fitted model can be examined not only for its accuracy but also for its behaviour under simulated input variation and for the variables to which it assigns large contributions in the upper tail of predicted costs.

## 1.2 Positioning within Mathematical Statistics

This dissertation is positioned within statistical learning, simulation-based inference, model-implied predictive distributions, and explainable statistical modelling. The statistical contribution is not only the fitting of a predictive model, but the construction of a model-implied distribution under an explicit input law and the conditional analysis of feature attributions in the upper tail of that distribution. Each of the three methodological pillars is discussed below to situate the work within the broader mathematical-statistics tradition.

### 1.2.1 Supervised regression and function approximation

The predictive component draws on the theory of supervised regression and neural-network function approximation. Supervised regression seeks a function  $\hat{f} : \mathbb{R}^p \rightarrow \mathbb{R}$  that minimises a loss functional, typically the expected squared error  $E[(Y - \hat{f}(\mathbf{X}))^2]$ , over a distribution of input-output pairs  $(\mathbf{X}, Y)$ . The deep neural network is one particular parameterisation of this function class, distinguished by its capacity to represent highly nonlinear mappings through compositions of affine transformations and element-wise activation functions.

The bias-variance decomposition provides the theoretical framework for understanding the

trade-off between model complexity and generalisation performance. A model that is too simple (high bias) will underfit the data, while a model that is too complex (high variance) will overfit. The five-phase hyperparameter comparison employed in this dissertation is designed to navigate this trade-off by selecting the configuration that minimises validation-set error, while the held-out test set provides an unbiased estimate of the generalisation performance.

### 1.2.2 Monte Carlo simulation in statistical inference

The simulation component uses parametric Monte Carlo methods, one of the foundational tools of computational statistics. Monte Carlo simulation allows the analyst to explore the distribution of a quantity of interest by drawing random samples from a specified input distribution and propagating them through a deterministic or stochastic transformation. In the classical setting, Monte Carlo methods are used to study the sampling distribution of estimators, the power of test statistics and the coverage properties of confidence intervals.

In this dissertation, the transformation is the trained neural network prediction function  $\hat{f}$ , and the input distribution is a product of parametric marginals fitted to the training data. The resulting distribution of predicted costs is a model-implied distribution that characterises the range, central tendency, dispersion and shape of the network's output under the specified input law. Formally, if  $\mathbf{X} \sim F_{\mathbf{X}}$  denotes the input distribution and  $\hat{Y} = \hat{f}(\mathbf{X})$  denotes the predicted cost, then the distribution of  $\hat{Y}$  under  $F_{\mathbf{X}}$  is the object of inferential interest. This construction connects the neural network to the simulation-based inference tradition within mathematical statistics, where the behaviour of estimators, test statistics and predictive quantities is routinely studied through Monte Carlo experiments.

### 1.2.3 Shapley values and cooperative game theory

The explanation component applies Shapley-value decomposition, a concept originating in cooperative game theory (Shapley, 1953), to attribute fitted predictions to individual input variables. In a cooperative game with  $p$  players and a characteristic function  $v$  that maps each coalition  $S \subseteq \{1, \dots, p\}$  to a real-valued payoff, the Shapley value of player  $j$  is defined as

$$\phi_j(v) = \sum_{S \subseteq \{1, \dots, p\} \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [v(S \cup \{j\}) - v(S)]. \quad (1.2.1)$$

The Shapley value provides the unique additive decomposition of the game's total payoff that satisfies the axioms of efficiency, symmetry, linearity and the null-player property.

In the machine-learning interpretability literature, this decomposition has been operationalised through the SHAP framework (Lundberg and Lee, 2017), which interprets each prediction as a cooperative game in which the “players” are input features and the “payoff” is the predicted value. The gradient-based implementation used in this dissertation exploits the differentiability of the neural network to obtain Shapley-value approximations efficiently for large samples.

#### 1.2.4 The integration as the statistical contribution

The choice to combine these three components reflects a broader methodological perspective: that a fitted prediction model is a statistical object whose properties extend beyond its point-prediction accuracy. Just as the sampling distribution of an estimator is studied through Monte Carlo experiments to understand its variability, the distribution of a prediction function’s outputs under a specified input law can be studied to understand the range and shape of its predictions. And just as the influence of individual observations on an estimator can be decomposed through influence functions, the contribution of individual input variables to a prediction can be decomposed through Shapley values.

Each of these components has a well-established theoretical basis in mathematical statistics. The contribution of this dissertation lies not in advancing any single component, but in their integration within a single empirical workflow applied to health insurance cost prediction. The Monte Carlo distribution defines the regions of interest for the Shapley analysis, and the conditional Shapley analysis decomposes the model behaviour within those regions. This integration produces insight that none of the three components could provide in isolation.

### 1.3 Conceptual Framework

The analytical workflow of this dissertation is organised around three sequentially connected stages: prediction, simulation and explanation. Each stage produces outputs that serve as inputs to the next, creating an integrated framework in which the fitted model, the simulated input population and the feature attributions are jointly analysed.

In the first stage, a deep neural network regression model is trained on the benchmark dataset using a systematic hyperparameter comparison. The trained network defines a prediction function  $\hat{f} : \mathbb{R}^p \rightarrow \mathbb{R}$  that maps encoded policyholder attributes to predicted costs. A Gamma generalised linear model with a logarithmic link, a random forest regressor and an XGBoost gradient-boosted tree regressor are fitted alongside the network to provide benchmarks spanning three alternative model classes. The primary comparison is between the actuarial-standard GLM and the DNN, with the tree-based models serving as

intermediate reference points. The output of this stage is a validated prediction function with documented out-of-sample performance across all four models.

In the second stage, parametric distributions are fitted to each input variable in the training partition, and a Monte Carlo sample of  $B = 100\,000$  synthetic policyholder profiles is drawn from the product of these marginals. Each profile is encoded and passed through the trained prediction function, yielding a sample of  $B$  predicted costs. This sample constitutes a model-implied predicted-cost distribution whose properties, including mean, variance, skewness and tail behaviour, are analysed through summary statistics, density estimation and convergence diagnostics. Joint-distribution validation and residual-augmented sensitivity analysis confirm the adequacy of the input distribution and the robustness of the results.

In the third stage, gradient-based Shapley values are computed for each of the  $B$  simulated profiles. These values decompose each predicted cost into additive contributions from the  $p$  input variables. The Shapley values are summarised at the global level (across all profiles) and conditionally within high-cost regions defined by upper-tail thresholds at the 90th, 95th and 99th percentiles. The conditional analysis reveals how the feature-importance ranking changes as the focus shifts from average to extreme predicted costs.

This three-stage structure ensures that the dissertation answers not only “How well does the model predict?” but also “What does the model’s output distribution look like under population-level input variation?” and “Which variables matter most, and does their relative importance change in the tail?”

## 1.4 Significance of the Study

The study contributes to several intersecting areas within mathematical statistics and actuarial science.

From a methodological perspective, the integration of deep neural network prediction, parametric Monte Carlo simulation and Shapley-value attribution within a single reproducible framework demonstrates a workflow that is applicable to any regression problem where distributional analysis and feature attribution in specific output regions are of interest. While each component is individually well established, their combination in a sequential framework, where the simulation defines the regions and the Shapley analysis explains the model behaviour within those regions, is not standard in the literature.

From a practical perspective, the conditional Shapley analysis at upper-tail thresholds addresses a question of direct relevance to actuarial practice: which variables drive the most expensive predicted claims? Traditional global feature-importance summaries may mask the fact that the drivers of average-cost predictions differ from the drivers of extreme-

cost predictions. By computing conditional Shapley summaries at multiple thresholds, the study provides a graded view of how feature importance evolves into the tail. This information is relevant to risk selection, pricing segmentation and reinsurance analysis, where the focus is often on the upper tail of the cost distribution.

From a statistical-learning perspective, the construction of a model-implied predicted-cost distribution under an explicit input law connects the neural network prediction function to the broader tradition of simulation-based inference. Rather than evaluating the model only at observed data points, the Monte Carlo component generates a distribution of predicted costs that reveals the range and shape of the model’s output landscape. This approach treats the prediction function as a statistical object whose distributional properties are themselves worthy of analysis, consistent with the mathematical-statistics emphasis on characterising estimators and prediction rules through their sampling distributions.

Finally, from an explainability perspective, the study demonstrates that Shapley-value attribution, often presented as a single global summary, can be extended to conditional summaries that provide region-specific explanations. The ranking reversal observed in the upper tail of the predicted-cost distribution illustrates that a single global importance ranking may not adequately represent the model’s behaviour in all regions of the output space. This finding contributes to the growing literature on context-dependent model explanation.

The South African insurance market, in which this dissertation is situated, is subject to regulatory frameworks that increasingly require transparency in pricing and underwriting decisions. While the present study uses a publicly available benchmark dataset rather than proprietary South African data, the methodological framework developed here is designed to be transferable to any insurance pricing context where prediction, distributional analysis and feature attribution are jointly required.

## 1.5 Research Questions

The background and problem statement outlined above motivate a set of research questions that span the three stages of the analytical framework. These questions are formulated to be answerable within the scope of the benchmark data and the fitted model, without requiring causal interpretation. Each question addresses a specific aspect of the prediction, simulation and explanation framework.

The dissertation addresses the following research questions:

1. How does a deep neural network regression model compare with a classical generalised linear model, a random forest and an XGBoost gradient-boosted tree in predicting health insurance costs on a benchmark dataset?

2. What is the shape and distributional behaviour of the model-implied predicted-cost distribution obtained by propagating parametric Monte Carlo input profiles through the trained network?
3. Is the product-marginal Monte Carlo input distribution a defensible approximation of the training-data joint distribution for the purpose of generating simulated profiles?
4. How sensitive is the predicted-cost distribution to residual perturbation from the fitted model?
5. Which input variables receive the largest Shapley-value attributions globally, and how does the feature-importance ranking change conditionally within high-cost regions of the predicted-cost distribution?
6. What explains any differences between the global and conditional feature-importance rankings in the upper tail?

Research Question 1 addresses the predictive stage. Research Questions 2 to 4 address the simulation stage, covering the distributional characterisation, the input-distribution validity and the residual-perturbation robustness respectively. Research Questions 5 and 6 address the explanation stage, covering both the global and conditional Shapley analyses and the interpretation of any observed ranking changes.

## 1.6 Research Objectives

The research questions stated above are operationalised through the following objectives, each of which maps directly to one or more research questions:

1. To fit a deep neural network regression model to a health insurance benchmark dataset and to evaluate its predictive performance against a classical generalised linear model, a random forest and an XGBoost gradient-boosted tree benchmark.
2. To design and implement a parametric Monte Carlo simulation that generates synthetic policyholder profiles from fitted marginal distributions, propagates them through the trained network, and produces a model-implied distribution of predicted costs.
3. To validate the Monte Carlo input distribution against the observed training data using joint-distribution diagnostics, bootstrap resampling, and a Shapley attribution invariance check.

4. To conduct a residual-augmented Monte Carlo sensitivity analysis that examines the effect of residual perturbation on the predicted-cost distribution.
5. To compute Shapley-value attributions for simulated policyholder profiles and to summarise feature importance at the global level and conditionally within high-cost regions defined by upper-tail thresholds.
6. To examine how the conditional feature-importance ranking in high-cost regions differs from the global ranking, and to interpret these differences within the context of the fitted model and benchmark data.

## 1.7 Research Gap

The health insurance prediction literature has grown substantially in recent years, driven by the increasing availability of large datasets and the development of flexible machine-learning models. Several studies have demonstrated that neural networks and tree-based ensembles can outperform classical actuarial models on insurance cost prediction tasks (Richman, 2021; Kuo, 2019). These studies typically focus on predictive accuracy metrics such as RMSE, MAE and  $R^2$ , comparing multiple model classes on test-set performance. While predictive performance is an important criterion, the literature has paid less attention to the distributional properties of predicted costs under population-level input variation.

A separate stream of research has applied Monte Carlo simulation methods to insurance and actuarial problems, including loss reserving, ruin probability estimation and aggregate claims modelling. However, the use of Monte Carlo simulation to construct a model-implied predicted-cost distribution, in which synthetic input profiles are generated from fitted marginals and propagated through a trained prediction function, has not been presented as a standard component of the insurance prediction workflow. The distinction is important: rather than reporting predictions at observed data points only, this approach generates a distribution of predicted costs under an explicit input law, connecting the prediction model to the simulation-based inference tradition.

The explainability literature has similarly expanded, with Shapley-value methods becoming a widely adopted tool for model interpretation (Lundberg and Lee, 2017). Most applications report global feature-importance summaries, typically beeswarm plots or bar charts of mean absolute Shapley values, across the full prediction set. Fewer studies examine how feature importance changes conditionally within specific regions of the output distribution. The question of whether the variables that drive average predicted costs are the same as those that drive extreme predicted costs has not been systematically addressed in the health insurance context.

The combination of a fitted neural network, Monte Carlo simulation under an explicit input distribution, and conditional Shapley summaries in high-cost regions has not been presented as an integrated workflow in the health insurance literature. This gap is developed through the reviewed studies in Section 2 and stated formally in Section 2.11.

## 1.8 Methodological Overview

The empirical analysis follows a three-stage workflow, each stage building on the outputs of the preceding one. The stages, and their principal components, are described below.

### 1.8.1 Prediction stage

A feed-forward deep neural network with a funnel architecture (see Table 6) is trained on a publicly available health insurance benchmark dataset. The network uses CELU activation, no bias terms and NAdam optimisation. A five-phase sequential hyperparameter comparison identifies the best configuration from among 16 candidate settings, evaluating activation functions, learning rates, batch sizes, optimiser algorithms and momentum parameters in turn. Each phase fixes the winner from the preceding phase and varies the next hyperparameter, so that the search space is explored sequentially rather than through a full grid.

Three benchmark models are fitted to contextualise the network’s predictive performance: a Gamma generalised linear model with a logarithmic link function, a random forest regressor with 500 trees, and an XGBoost gradient-boosted tree regressor with 1 000 boosting rounds and early stopping. The GLM serves as the actuarial-standard baseline, while the two tree-based models occupy an intermediate position between the linear GLM and the fully flexible DNN. Hyperparameters for the GLM and tree-based models are set to standard defaults rather than tuned, so that the comparison isolates model-class differences. All four models are evaluated on a held-out test set that is never used during training or hyperparameter selection, and data-leakage prevention protocols ensure the integrity of all reported metrics.

### 1.8.2 Simulation stage

Parametric distributions are fitted to each input variable in the training partition. Continuous variables are modelled with appropriate distributions (e.g. truncated normal for age, log-normal for BMI), while categorical variables are sampled from their observed relative frequencies. A Monte Carlo sample of  $B = 100\,000$  synthetic policyholder profiles is drawn from the product of these independent marginals. Each profile is encoded using

the same preprocessing procedure applied to the training data and passed through the trained network, yielding a sample of predicted costs.

The resulting model-implied predicted-cost distribution is characterised through summary statistics, density estimation, convergence diagnostics and tail-behaviour analysis. Joint-distribution validation, including pairwise correlation comparisons, categorical cross-tabulation comparisons, PCA projection overlays and a formal energy distance permutation test, establishes whether the independent-marginal assumption introduces material distributional distortion. A residual-augmented sensitivity analysis examines the robustness of the predicted-cost distribution to fitted residual perturbation.

### 1.8.3 Explanation stage

Gradient-based Shapley values are computed for each of the  $B$  simulated profiles using a background sample of  $m_0 = 200$  training observations. The Shapley values decompose each predicted cost into additive contributions from the  $p = 22$  encoded input features. Global feature importance is summarised through mean absolute Shapley values, beeswarm plots and bar charts. Conditional Shapley summaries are computed at three upper-tail thresholds:  $\tau_{90}$ ,  $\tau_{95}$  and  $\tau_{99}$ . The conditional analysis identifies the variables that drive extreme predicted costs and examines whether and how the feature-importance ranking differs from the global ranking. A Shapley attribution invariance check compares the rankings obtained from training-data and Monte Carlo input regimes to confirm consistency.

### 1.8.4 The benchmark dataset

The dataset used is the Kaggle Insurance Data for Machine Learning benchmark, containing one million policyholder records with eleven predictor variables and a continuous monetary response. The dataset is partitioned into training (60%,  $n = 524\,288$ ), validation ( $n = 237\,856$ ) and test ( $n = 237\,856$ ) subsets using a fixed random seed for reproducibility. The dataset is treated as a methodological benchmark rather than a representative underwriting portfolio from a specific insurer. All conclusions are internal to the modelling workflow and the benchmark data.

## 1.9 Validation and Robustness Strategy

A central concern in any modelling study is whether the reported results are genuine reflections of the model's behaviour or artefacts of data contamination, distributional mismatch or estimation instability. This dissertation addresses this concern through a

multi-layered validation strategy that spans all three stages of the analytical framework.

At the prediction stage, the validation strategy consists of three elements. First, a strict data-leakage prevention protocol ensures that all preprocessing parameters, including means, standard deviations, category mappings and normalisation constants, are computed exclusively on the training partition and applied without refitting to the validation and test partitions. This guarantees that the model has no access to information from the evaluation data during training. Second, the five-phase hyperparameter comparison uses validation-set performance as the selection criterion, while the final performance metrics are reported on the held-out test set that is never used for model selection. Third, the comparison of training-set and test-set performance metrics for both the DNN and the GLM provides evidence on the presence or absence of overfitting.

At the simulation stage, the validation strategy addresses the central question of whether the independent-marginal Monte Carlo input distribution is a defensible approximation of the training-data joint distribution. This is assessed through four complementary diagnostics: pairwise correlation comparisons, categorical cross-tabulation comparisons, PCA projection overlays, and a formal energy distance permutation test. In addition, convergence diagnostics monitor the stability of the predicted-cost distribution summary statistics as the simulation size increases, confirming that  $B = 100\,000$  is sufficient for the level of Monte Carlo precision required. The residual-augmented sensitivity analysis adds a further robustness layer by examining whether the distributional conclusions change when scaled fitted residuals are incorporated into the predicted costs.

At the explanation stage, the validation strategy uses a Shapley attribution invariance check. This diagnostic compares the feature-importance rankings obtained when Shapley values are computed on training-data input profiles with those obtained on Monte Carlo input profiles. Agreement between the two regimes confirms that the Shapley results are not sensitive to the choice of input regime and that the Monte Carlo input distribution produces attributions consistent with those obtained from observed data.

The details of each validation component are documented in the methods chapter (Chapter 3), and the corresponding results are reported in Chapter 4.

## 1.10 Scope and Delimitations

The following delimitations define the boundaries of the study:

- **Benchmark data.** The analysis uses a publicly available synthetic insurance dataset. The results describe relationships learned by the fitted model within the benchmark data and should not be interpreted as causal determinants of health-care costs or as direct evidence of real-world pricing practice. The unusually high

predictive performance observed on this dataset reflects its simulated structure and should not be extrapolated to real-world insurance portfolios, where data are noisier and covariate-response relationships are more complex.

- **Focused model comparison.** Three benchmark models (a Gamma GLM, a random forest and an XGBoost gradient-boosted tree) are included to contextualise the predictive performance of the deep neural network across model classes of increasing flexibility. However, the dissertation does not attempt an exhaustive model-selection study. Bayesian methods, support vector machines, other neural-network architectures and stacked-ensemble approaches are not evaluated. The comparison is designed to position the DNN within a spectrum from linear to piecewise-constant to smooth nonlinear models, rather than to identify the globally optimal model class.
- **Single fitted network.** The Monte Carlo and Shapley analyses are applied to a single trained network with a fixed random seed. The predicted-cost distribution and feature attributions are conditional on this particular fitted model and its estimated parameters. Variability across random initialisations is not quantified.
- **Independent marginals.** The Monte Carlo input distribution is constructed as a product of independent fitted marginals. This is a deliberate modelling choice that broadens the simulated input space but does not reproduce the dependence structure of the observed data. Copula-based joint distributions are not considered.
- **Approximate Shapley values.** The reported attributions are gradient-based SHAP approximations, not exact game-theoretic Shapley values. They are conditional on the fitted network, the background sample, and the explainer settings. Exact Shapley computation is infeasible for the number of features and the sample size considered.
- **No causal inference.** The Shapley decomposition identifies variables to which the fitted model assigns large contributions. It does not establish causal relationships between input variables and healthcare expenditure. All attributions are internal to the fitted model and the benchmark data.
- **No temporal dynamics.** The benchmark dataset is cross-sectional. Longitudinal effects, claims development patterns and time-varying risk factors are not considered.
- **No regulatory or pricing application.** The study does not produce premium rates, regulatory capital estimates or underwriting rules. The results are intended as a methodological demonstration rather than a deployable pricing model.

## 1.11 Ethical Considerations

The analysis is conducted on the Kaggle Insurance Data for Machine Learning benchmark dataset, which is publicly available and contains no personally identifiable information. The dataset is synthetic in nature and does not represent actual policyholder records from any insurance company. No data collection involving human participants was conducted for this study, and no ethical clearance from a research ethics committee was required.

Notwithstanding the use of synthetic data, the dissertation acknowledges the broader ethical considerations associated with the use of machine learning in insurance pricing. Predictive models that use sensitive attributes such as gender, age or health status as input variables may raise fairness concerns if deployed in production pricing systems. The Shapley-value analysis presented in this dissertation quantifies the contribution of each variable to the fitted model's predictions, which could in principle inform fairness audits by identifying variables that exert disproportionate influence on predicted costs. However, the study does not conduct a formal fairness analysis, and the Shapley attributions should not be interpreted as evidence of discriminatory pricing.

All modelling code, trained model artefacts and simulation outputs are documented to support reproducibility. The use of a fixed random seed throughout the framework ensures that all results can be replicated by independent researchers with access to the same dataset and software environment.

## 1.12 Definition of Key Terms

The following definitions establish the meaning of key terms as they are used throughout this dissertation. Formal mathematical definitions are deferred to the relevant chapters.

**Deep neural network (DNN).** A feed-forward artificial neural network with multiple hidden layers, each consisting of an affine transformation followed by a nonlinear activation function. In this dissertation, the DNN is used as a regression model to predict continuous insurance costs from policyholder attributes.

**Funnel architecture.** A network architecture in which the number of neurons decreases monotonically from the input layer to the output layer. The specific architecture used in this study has layer widths  $256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16$ .

**Generalised linear model (GLM).** A class of statistical models that generalise ordinary linear regression by allowing the response variable to follow any distribution in the exponential family and by relating the mean of the response to the linear predictor through a link function. In this dissertation, a Gamma GLM with a logarithmic link function serves as the actuarial benchmark.

**Random forest.** An ensemble learning method that constructs a collection of decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split. The final prediction is the average of the individual tree predictions. In this dissertation, a random forest regressor serves as one of three benchmark models against which the DNN is compared.

**Gradient-boosted trees (XGBoost).** An ensemble method that builds an additive sequence of shallow decision trees, with each new tree fitted to the residuals of the preceding ensemble. XGBoost ([Chen and Guestrin, 2016](#)) is a widely used implementation that incorporates regularisation, approximate histogram-based splitting and early stopping. In this dissertation, an XGBoost regressor serves as one of three benchmark models.

**Monte Carlo simulation.** A computational method that uses repeated random sampling from a specified probability distribution to estimate the properties of a quantity of interest. In this study, Monte Carlo simulation generates synthetic policyholder profiles from fitted parametric marginals and propagates them through the trained network to obtain a predicted-cost distribution.

**Model-implied predicted-cost distribution.** The distribution of predicted costs obtained by evaluating the trained prediction function at a large sample of input profiles drawn from a specified input distribution. This distribution characterises the range, central tendency, dispersion and shape of the model’s output under population-level input variation.

**Shapley value.** A concept from cooperative game theory that assigns to each player in a coalition game a unique payoff satisfying the axioms of efficiency, symmetry, linearity and the null-player property. In the context of model interpretation, the “players” are input features and the “payoff” is the prediction for a given observation. The Shapley value quantifies the average marginal contribution of each feature across all possible feature coalitions.

**SHAP (SHapley Additive exPlanations).** A framework that operationalises the computation of approximate Shapley values for machine-learning models. The gradient-based variant used in this dissertation exploits the differentiability of the neural network to compute Shapley-value approximations efficiently.

**Conditional Shapley analysis.** The computation and summarisation of Shapley values within a subset of predictions that exceed a specified threshold. In this dissertation, conditional Shapley summaries are computed at the 90th, 95th and 99th percentiles of the predicted-cost distribution.

**Feature-importance ranking reversal.** The phenomenon whereby the ordering of features by their mean absolute Shapley values differs between the global summary and the conditional summary at a specific threshold. A ranking reversal indicates that the variables driving predictions in the specified region differ from those that dominate on average.

**Data-leakage prevention.** The set of protocols that ensure no information from the validation or test partitions contaminates the training process. In this dissertation, all preprocessing parameters are computed exclusively on the training partition and applied to the validation and test partitions without refitting.

**Residual-augmented sensitivity analysis.** A diagnostic procedure that adds scaled fitted residuals to the Monte Carlo predicted costs to examine whether the distributional conclusions are robust to the residual structure of the fitted model.

## 1.13 Dissertation Outline

The remainder of this dissertation is organised as follows.

### Chapter 2: Literature Review

This chapter reviews the theoretical and empirical foundations of the DNN, Monte Carlo and Shapley framework. The review covers statistical learning theory and supervised regression, deep neural network architectures and their approximation properties, generalised linear models as actuarial baselines, health insurance cost prediction with machine learning, Monte Carlo simulation methodology, Shapley-value attribution theory and the SHAP framework, and conditional feature-importance analysis. The chapter concludes with a formal identification of the research gap that motivates the present study.

### Chapter 3: Methods and Data

This chapter describes the benchmark dataset, the data preprocessing and encoding procedure, the exploratory data analysis, the network architecture and training protocol, the five-phase hyperparameter comparison, the predictive and structural diagnostics, the GLM and tree-based benchmark specifications, the Monte Carlo simulation design and input-distribution validation, the residual-augmented sensitivity analysis, and the Shapley-value implementation. All methodological choices are documented with their statistical justification, and a consolidated table of modelling assumptions is provided.

## Chapter 4: Results

This chapter presents the empirical results of the three-stage workflow. The chapter reports the hyperparameter comparison outcomes, the four-model benchmark comparison (GLM, random forest, XGBoost and DNN), the Monte Carlo predicted-cost distribution and its convergence diagnostics, the joint-distribution validation results, the residual-augmented sensitivity analysis, the global and conditional Shapley-value attributions, the feature-importance ranking reversal in the upper tail, and the Shapley attribution invariance check.

## Chapter 5: Conclusion

The final chapter summarises the key findings in relation to each research question, states the contributions and strengths of the study, discusses limitations, examines implications for actuarial practice and statistical methodology, and identifies directions for future research.

## 2 Literature Review

### 2.1 Introduction

Chapter 1 introduced the tension between predictive flexibility and interpretability in health insurance cost prediction and outlined a three-stage framework that combines deep neural network regression, Monte Carlo simulation and Shapley-value attribution. This chapter develops the conceptual and theoretical basis for that framework by reviewing the relevant literature in a sequence that mirrors the framework itself: from prediction to simulation and then to explanation.

The review begins with the integrated DNN, Monte Carlo and Shapley framework (Section 2.2), establishing why prediction accuracy alone provides only a partial account of model behaviour and why distributional analysis and feature-level attribution are needed as complementary tools. Subsequent sections develop each component in detail: supervised regression and neural-network approximation theory supply the predictive formulation; Monte Carlo methods extend the fitted prediction function from individual point estimates to a model-implied cost distribution; and Shapley-value decomposition attributes individual predictions to input variables, with particular attention to profiles in the upper tail of the predicted-cost distribution.

The research gap that motivates the dissertation follows from the limited integration of these components in the existing literature. This gap is developed through the reviewed studies in Section 2.2 and stated formally in Section 2.11.

### 2.2 DNN-Monte Carlo-Shapley Framework

Monte Carlo simulation is a long-established framework for analysing the behaviour of complex functions by generating samples from a specified input distribution and evaluating the function at each sampled point (Glasserman, 2004; Rubinstein and Kroese, 2016; Owen, 2013). Its central advantage is that direct analytical evaluation can be replaced with repeated numerical evaluation over a large synthetic sample, so that the function is studied indirectly through its simulated outputs. The resulting outputs form an empirical distribution that can be summarised using descriptive quantities such as the mean, variance, quantiles and upper-tail probabilities. This approach is particularly useful when the function of interest has no convenient closed-form representation, as is often the case for nonlinear fitted models such as deep neural networks (Goodfellow et al., 2016; Hornik et al., 1989). Although such models define an explicit computational mapping from inputs to outputs, their fitted response surfaces are typically too complex to be examined using

closed-form analytical arguments.

Shapley values provide the attribution layer for the simulated output space. Originally introduced in cooperative game theory by [Shapley \(1953\)](#) and subsequently adapted to the explanation of machine-learning predictions ([Lundberg and Lee, 2017](#); [Rozemberczki et al., 2022](#)), Shapley values decompose an individual fitted prediction into additive contributions attributed to each input variable. This decomposition is particularly valuable when the fitted model is too complex for its parameters to admit direct interpretation. In a linear regression, individual coefficients describe the contribution of each predictor to the fitted response. In a nonlinear model such as a DNN, the relationship between inputs and predictions is distributed across many layers and parameters ([Goodfellow et al., 2016](#)), so that no single parameter captures the role of a given input variable. Shapley values address this difficulty by providing a model-agnostic attribution that satisfies formal fairness axioms<sup>[1](#)</sup> regardless of the internal structure of the fitted function ([Lundberg and Lee, 2017](#); [Rozemberczki et al., 2022](#)). Monte Carlo simulation therefore describes the distributional behaviour of the fitted function, while Shapley values describe the variables associated with particular fitted predictions. Used jointly, the two methods allow both the output distribution and the feature-level drivers of selected regions of that distribution to be examined.

Health insurance cost prediction provides a natural setting in which to combine these ideas. The empirical task can be framed as a supervised statistical-learning problem in which a vector of policyholder attributes is used to approximate a continuous monetary outcome ([Hastie et al., 2009](#); [James et al., 2021](#)). A DNN supplies a flexible nonlinear regression function for modelling the relationship between policyholder attributes and cost ([Goodfellow et al., 2016](#); [Hornik et al., 1989](#); [Richman, 2021](#); [Samiuddin et al., 2023](#)). The use of a DNN in this role is motivated by the universal approximation property: feedforward neural networks with suitable activation functions can approximate broad classes of continuous functions ([Hornik et al., 1989](#); [Goodfellow et al., 2016](#)), so that the nonlinear relationship between policyholder attributes and cost does not need to be specified in advance by the researcher. The formal statement of this property and its conditions is presented in Chapter 3. After the network has been fitted, Monte Carlo simulation is used to generate a large synthetic sample of policyholder profiles from parametric distributions fitted to the observed training attributes. The fitted network is then evaluated at each simulated profile to obtain a model-implied distribution of predicted costs. Shapley values are subsequently applied to the simulated predictions, with particular attention given to profiles in the high-cost region of that distribution, namely profiles whose fitted predictions exceed a chosen upper-tail threshold, as formalised in Section 2.2.4. The

---

<sup>[1](#)</sup>The four axioms are efficiency, symmetry, dummy and additivity ([Shapley, 1953](#); [Lundberg and Lee, 2017](#)).

precise distributional choices and simulation procedure are specified in Chapter 3. The three components are hierarchically justified: the DNN provides the fitted prediction surface, Monte Carlo simulation examines its behaviour empirically where closed-form analysis is unavailable, and Shapley values attribute predictions to input features where the network’s parameters cannot be interpreted directly.

Predictive accuracy provides only a partial description of model behaviour. Measures such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE) and Coefficient of Determination ( $R^2$ ) are informative about point-prediction performance, but they do not show how fitted costs vary across a simulated population or which variables are associated with high predicted costs. A broader assessment requires both distributional analysis of fitted predictions and feature-level attribution of those predictions.

At the broader actuarial level, Richman (2021) surveys recent developments in artificial intelligence and deep learning in actuarial science and concludes that neural-network methods are increasingly being adopted across pricing, reserving and risk-classification tasks, typically supplementing rather than replacing the established generalised linear modelling tradition. The review supports the choice of a deep neural network as the prediction function in the framework adopted here, but it is positioned at the level of a methodological survey and does not prescribe how a fitted network should be examined once it has been trained. In particular, it does not consider the combination of simulation-based exploration of the fitted prediction surface with feature-level attribution as a single actuarial workflow, which is the methodological gap that the proposed framework addresses.

A closer comparison is provided by Orji and Ukwandu (2024), who apply machine-learning regressors together with SHapley Additive exPlanations (SHAP) and Individual Conditional Expectation (ICE) plots to predict medical insurance premiums on the same family of policyholder attributes used here and identify smoker status, age and Body Mass Index (BMI) as the strongest contributors to the predicted premium. Their use of SHAP for feature-level attribution is consistent with the attribution layer adopted here, and their finding that smoker status and age dominate the explanation of high premiums is in line with what would be expected for the same family of inputs in the present setting. The relevant difference lies in the input space over which attribution is performed. Their analysis is conducted on the observed training and test data alone, so the resulting attributions describe the fitted model’s behaviour on profiles that actually appear in the data. The framework adopted here retains SHAP for attribution but applies it to a larger synthetic policyholder population generated by Monte Carlo simulation, which allows attribution to be examined at policyholder configurations that are not individually present in the training data, and in particular within the high-cost region of the simulated predicted-cost distribution.

The closest deep-learning-based precedent in health insurance cost prediction is [Samiudin et al. \(2023\)](#), who fit [Artificial Neural Networks \(ANN\)](#) and [DNN](#) models to the same family of attributes used here, namely age, gender, [BMI](#), number of children, smoking behaviour and region, and report predictive performance comparable to benchmark regression models on held-out data. Their results support the use of a feed-forward neural network as a viable prediction function for health insurance cost on policyholder-level attributes, which is consistent with the role assigned to the [DNN](#) in the predictive layer of the framework adopted here. The point of departure is the scope of the analysis. Their study is restricted to the point-prediction stage of the workflow: the fitted network is evaluated against held-out data using standard predictive metrics, but no simulation is conducted over a synthetic input space and no feature-level attribution of the fitted predictions is reported. The framework developed here builds on their use of a [DNN](#) for cost prediction and extends it with the simulation and attribution stages that their study does not consider.

Taken together, these studies establish that [DNN](#)-based prediction and [SHAP](#)-based attribution are individually well supported in the health insurance cost-prediction literature, and that the choice of a [DNN](#) for the prediction layer is justified by recent evidence. What is not yet standard, however, is the integrated treatment of a fitted neural network, Monte Carlo simulation over a synthetic policyholder population and Shapley attribution at the level of high-cost simulated profiles within a single workflow, and it is this combination that the proposed framework places at the centre of the analysis.

A related three-stage structure combining Monte Carlo simulation, machine-learning prediction and Shapley-based attribution has been proposed in other applied domains. [Santos et al. \(2023\)](#) implement such a sequence in an earned-value project-control setting: Monte Carlo simulation is used to generate a large synthetic sample of input configurations representing plausible project states, a machine-learning model is fitted to approximate the relationship between those inputs and a continuous project-performance outcome, and Shapley-based explanations attribute the resulting predictions to individual input variables. Although their empirical context is project management rather than insurance pricing, the workflow is methodologically similar: simulation over a synthetic input space, nonlinear prediction, and post-hoc feature-level attribution. The framework adopted here applies the same three-stage logic to health insurance cost prediction: Monte Carlo simulation generates synthetic policyholder profiles from parametric distributions fitted to the observed training attributes, the fitted [DNN](#) maps each profile to a predicted cost, and Shapley values decompose those predictions into feature-level contributions, with particular attention given to the high-cost region of the simulated predicted-cost distribution. This adaptation remains subject to the benchmark-data limitations stated above, namely that the conclusions are internal to the modelling workflow and are not direct evidence

of real-world pricing practice.

Figure 1 summarises the framework. The predictive layer fits the DNN to the preprocessed benchmark data. The distributional layer generates simulated policyholder profiles and propagates them through the fitted network. The attributional layer decomposes fitted predictions into Shapley contributions. These layers converge in the high-cost analysis, where simulated upper-tail predictions are interpreted through conditional Shapley summaries.

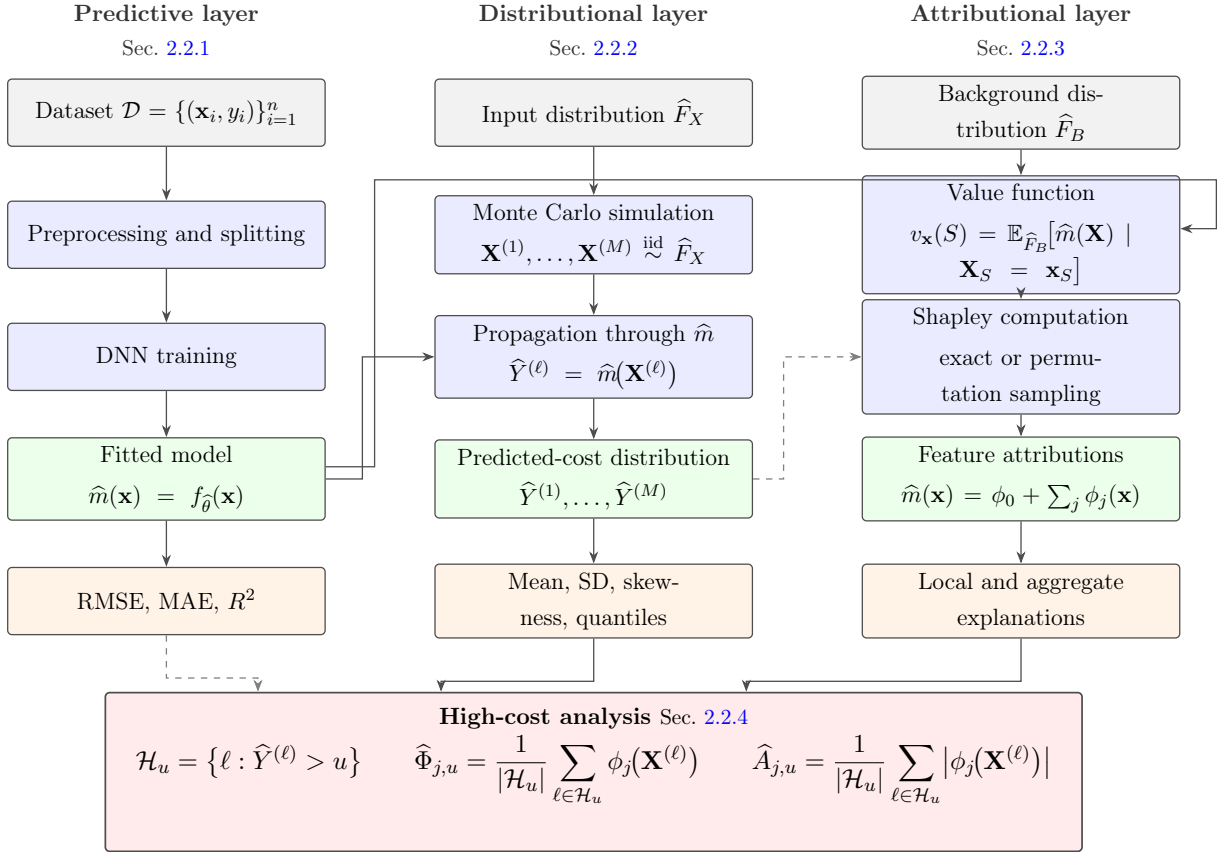


Figure 1: Prediction, Simulation and Explanation framework.

### 2.2.1 Predictive Modelling

Let

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n \quad (2.2.1)$$

denote the preprocessed insurance benchmark dataset, where  $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})^\top \in \mathbb{R}^p$  is the vector of policyholder attributes for observation  $i$ , and  $y_i \geq 0$  is the corresponding insurance cost, charge or premium. The regression formulation is

$$y_i = m(\mathbf{x}_i) + \varepsilon_i, \quad \mathbb{E}(\varepsilon_i \mid \mathbf{x}_i) = 0, \quad (2.2.2)$$

where  $m(\cdot)$  denotes the unknown regression function and  $\varepsilon_i$  represents variation not explained by the observed predictors (Hastie et al., 2009; James et al., 2021). A DNN with trainable parameter vector  $\theta$  is fitted to approximate  $m(\cdot)$ , yielding

$$\hat{m}(\mathbf{x}) = f_{\hat{\theta}}(\mathbf{x}). \quad (2.2.3)$$

For a feed-forward network with  $L$  hidden layers, the parameter vector  $\theta$  collects all weight matrices  $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L+1)}$  and, where included, bias vectors  $\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L+1)}$  across the layers (Goodfellow et al., 2016). The total number of trainable parameters depends on the layer widths and on whether bias terms are included. Under a bias-free architecture,  $\theta$  reduces to the weight matrices alone and does not include additive layer-bias parameters. This architectural meaning of bias is distinct from statistical bias in the bias-variance trade-off (Hastie et al., 2009) and from data bias arising from the dataset, sampling mechanism or preprocessing procedure.

The fitted function  $\hat{m}$  maps a policyholder profile to a predicted cost. It is interpreted as a statistical learning function fitted to the preprocessed benchmark data, not as a causal model of healthcare expenditure and not as evidence of real-world insurer pricing rules. A DNN is useful in this setting because it can represent nonlinearities and interactions among policyholder attributes without requiring all interaction terms to be specified manually, as would be necessary in a generalised linear model or polynomial regression (Goodfellow et al., 2016; Hastie et al., 2009). Such flexibility is relevant where predicted costs depend on combinations of demographic, behavioural and health-related variables rather than on isolated predictors alone (Orji and Ukwandu, 2024; Samiuddin et al., 2023). However, the use of a DNN does not imply that neural networks are automatically superior to generalised linear models, tree-based models or other actuarial models for tabular insurance data (Richman, 2021). Its role is to provide a nonlinear fitted prediction surface that can be evaluated, simulated and interpreted within the proposed framework.

### 2.2.2 Monte Carlo Simulation

The Monte Carlo stage studies how the fitted prediction surface  $\hat{m}(\mathbf{x})$  defined in equation (2.2.3) responds to variation in the input attributes. Let  $\mathcal{D}_{\text{train}}$  denote the training partition of the preprocessed benchmark data and define

$$\mathcal{X}_{\text{train}} = \{\mathbf{x}_i : (\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{train}}\} \quad (2.2.4)$$

as the corresponding set of training attribute vectors. To simulate over the input space, a probability distribution must be specified for generating synthetic policyholder profiles. This distribution is denoted  $\hat{F}_X$  and is estimated from  $\mathcal{X}_{\text{train}}$  and is constructed as a

product of independent fitted marginals,

$$\widehat{F}_X = \prod_{j=1}^p \widehat{F}_{X_j}, \quad (2.2.5)$$

where each  $\widehat{F}_{X_j}$  is a parametric distribution fitted to the observed values of the  $j$ -th attribute in  $\mathcal{X}_{\text{train}}$ . The independence assumption in equation (2.2.5) is a deliberate modelling choice: the purpose of the simulation is to explore how the fitted network responds across a broad range of covariate combinations, including combinations that may not appear in  $\mathcal{X}_{\text{train}}$ , rather than to reproduce the observed joint distribution of policyholders.  $\widehat{F}_X$  is not assumed to coincide with the true population distribution of policyholders; it is the modelling object that defines the simulated input space. The distributional family, fitted parameter values, support restrictions and encoding rules for each attribute are specified in Chapter 3.

Given  $\widehat{F}_X$ , the procedure draws  $M$  synthetic policyholder profiles,

$$\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(M)} \stackrel{\text{iid}}{\sim} \widehat{F}_X, \quad (2.2.6)$$

and evaluates the fitted network at each profile to obtain the corresponding predicted costs,

$$\widehat{Y}^{(\ell)} = \widehat{m}(\mathbf{X}^{(\ell)}), \quad \ell = 1, \dots, M. \quad (2.2.7)$$

The collection  $\{\widehat{Y}^{(\ell)}\}_{\ell=1}^M$  is the model-implied predicted-cost distribution under  $\widehat{F}_X$  (Glasserman, 2004; Rubinstein and Kroese, 2016; Owen, 2013).

Two points of interpretation follow directly from the construction above. First, the output of the simulation is conditional on both  $\widehat{m}$  and  $\widehat{F}_X$ . The simulation does not generate realised healthcare costs; it generates fitted predictions obtained by evaluating a deterministic trained network at synthetic input profiles. Variation in  $\widehat{Y}^{(1)}, \dots, \widehat{Y}^{(M)}$  therefore reflects variation in the generated attributes and the nonlinear response of  $\widehat{m}$  to those attributes. It does not represent residual variation around the fitted surface or parameter uncertainty in  $\widehat{\theta}$ . Second, because  $\widehat{F}_X$  is constructed under the independence assumption in equation (2.2.5), the simulated profiles populate the input space more broadly than the training data, so  $\mathcal{X}_{\text{sim}}$  will contain attribute combinations that  $\widehat{m}$  did not encounter during training. Residual-augmented Monte Carlo and predictive-uncertainty analyses that extend beyond the fitted surface are developed in Chapter 3.

### 2.2.3 Shapley attribution

Monte Carlo input simulation must be distinguished from Monte Carlo Shapley estimation. Whereas input simulation studies the predicted-cost distribution under  $\widehat{F}_X$ , Shapley

estimation is a computational device for approximating exact Shapley values when the required summation over all  $2^p$  feature coalitions is infeasible. Because the exact calculation is equivalent to averaging over all  $p!$  feature orderings, sampling-based estimators are often employed. [Castro et al. \(2009\)](#) proposed a polynomial-time scheme based on randomly sampled permutations, and [Maleki et al. \(2013\)](#) derived confidence bounds for such estimators.

Let

$$\mathcal{P} = \{1, \dots, p\} \quad (2.2.8)$$

denote the set of input features. For an observed or simulated profile  $\mathbf{x}$ , the Shapley framework defines a value function  $v_{\mathbf{x}}(S)$  for each coalition  $S \subseteq \mathcal{P}$ . In a model-prediction setting,  $v_{\mathbf{x}}(S)$  represents the fitted prediction available when the feature values in  $S$  are fixed at their values in  $\mathbf{x}$ , while the remaining features are integrated over or replaced using a background distribution. Let  $\hat{F}_B$  denote the background distribution used for Shapley-value evaluation. A conditional form of the valuefunction is

$$v_{\mathbf{x}}(S) = \mathbb{E}_{\hat{F}_B}[\hat{m}(\mathbf{X}) \mid \mathbf{X}_S = \mathbf{x}_S]. \quad (2.2.9)$$

This notation separates the input distribution used for Monte Carlo simulation,  $\hat{F}_X$ , from the background distribution used for Shapley evaluation,  $\hat{F}_B$ . These distributions may coincide, but they need not be identical. The empirical implementation therefore reports the background data and the SHAP convention used.

The exact Shapley contribution of feature  $j$  for profile  $\mathbf{x}$  is

$$\phi_j(\mathbf{x}) = \sum_{S \subseteq \mathcal{P} \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [v_{\mathbf{x}}(S \cup \{j\}) - v_{\mathbf{x}}(S)]. \quad (2.2.10)$$

The fitted prediction can then be expressed as

$$\hat{m}(\mathbf{x}) = \phi_0 + \sum_{j=1}^p \phi_j(\mathbf{x}), \quad (2.2.11)$$

where  $\phi_0$  is the baseline prediction determined by the chosen background distribution, commonly corresponding to  $v_{\mathbf{x}}(\emptyset)$ , and  $\phi_j(\mathbf{x})$  is the contribution attributed to feature  $j$  for that profile.

When exact Shapley values are not computed, a permutation-based Monte Carlo ap-

proximation may be used. Let  $\pi^{(1)}, \dots, \pi^{(R)}$  be sampled permutations of  $\mathcal{P}$ , and let  $S_j(\pi^{(r)})$  denote the set of features appearing before feature  $j$  in permutation  $\pi^{(r)}$ . The permutation-sampling estimator is

$$\hat{\phi}_{j,R}(\mathbf{x}) = \frac{1}{R} \sum_{r=1}^R [v_{\mathbf{x}}(S_j(\pi^{(r)}) \cup \{j\}) - v_{\mathbf{x}}(S_j(\pi^{(r)}))] . \quad (2.2.12)$$

Increasing  $R$  generally improves the stability of the approximation but also increases computational cost. Whenever sampling-based explanations are used, the number of Shapley samples, the background dataset, the random seed or replication strategy, and any stability checks are reported to support reproducibility.

### 2.2.4 High-cost profiles

Distributional and attributional results meet in the analysis of high predicted costs. Let  $u$  denote a high-cost threshold. The threshold may be specified as a fixed monetary amount or as an empirical upper-tail quantile of the simulated predicted-cost distribution, for example

$$u = q_{\tau}(\hat{Y}^{(1)}, \dots, \hat{Y}^{(M)}) , \quad 0 < \tau < 1. \quad (2.2.13)$$

The simulated high-cost set is

$$\mathcal{H}_u = \left\{ \ell \in \{1, \dots, M\} : \hat{Y}^{(\ell)} > u \right\} . \quad (2.2.14)$$

The relative size of this set,

$$\hat{p}_u = \frac{|\mathcal{H}_u|}{M} , \quad (2.2.15)$$

summarises the simulated proportion of profiles whose fitted predictions exceed the chosen threshold. Conditional on  $|\mathcal{H}_u| > 0$ , the average signed Shapley contribution of feature  $j$  in the high-cost region is

$$\hat{\Phi}_{j,u} = \frac{1}{|\mathcal{H}_u|} \sum_{\ell \in \mathcal{H}_u} \phi_j(\mathbf{X}^{(\ell)}) , \quad (2.2.16)$$

and the corresponding average absolute contribution is

$$\hat{A}_{j,u} = \frac{1}{|\mathcal{H}_u|} \sum_{\ell \in \mathcal{H}_u} |\phi_j(\mathbf{X}^{(\ell)})| . \quad (2.2.17)$$

The signed quantity  $\hat{\Phi}_{j,u}$  indicates whether feature  $j$  tends to raise or lower predictions within the high-cost subset. The magnitude-based quantity  $\hat{A}_{j,u}$  measures the strength of the feature's contribution regardless of direction. Together, these summaries identify which variables the fitted model attributes to high predicted costs among simulated profiles.

The interpretation of high-cost Shapley summaries remains model-based. A large value of  $\hat{A}_{j,u}$  means that feature  $j$  has a large average model-attributed contribution among simulated profiles satisfying  $\hat{Y}^{(\ell)} > u$ . It does not mean that the variable causally determines real-world healthcare expenditure, nor that it is necessarily used in actual insurance underwriting. Similarly, a positive value of  $\hat{\Phi}_{j,u}$  indicates that the fitted model tends to associate the feature with higher predictions in the simulated high-cost region, not that the feature is a causal pricing factor.

The scope of the conclusions is determined by the data source, fitted model, input distribution and Shapley background distribution. Data bias may therefore affect both the simulated predicted-cost distribution and the Shapley summaries. The framework provides a coherent route from health insurance cost prediction to distributional and attributional understanding of fitted predictions, while remaining bounded by the benchmark nature of the data.

## 2.3 Statistical Learning and Supervised Regression

The predictive component of the DNN-Monte Carlo-Shapley workflow is a supervised statistical-learning problem. Statistical learning concerns the estimation of relationships between observed variables for purposes such as prediction, interpretation and model assessment (James et al., 2021; Hastie et al., 2009). In supervised learning, each observation contains both predictors and an observed response. In unsupervised learning, by contrast, the aim is to identify structure in the predictor space without an observed response (James et al., 2021). The insurance data considered in the empirical analysis pair policyholder attributes  $\mathbf{x}_i$  with an observed cost outcome  $y_i$ , placing the prediction task within the supervised regression setting.

The fitted value for observation  $i$  is  $\hat{y}_i = \hat{m}(\mathbf{x}_i)$ , following the regression formulation in equations (2.2.2) and (2.2.3). The response  $y_i$  is a continuous monetary outcome, and the problem is therefore a regression problem rather than a classification problem. Classification methods are designed for categorical responses and are commonly assessed using quantities such as accuracy, sensitivity, specificity or cross-entropy. Regression methods are evaluated on a continuous scale using prediction-error measures (James et al., 2021; Hastie et al., 2009). The specific measures employed in this dissertation are defined below.

For a fitted regression function  $\hat{m}$  evaluated on an evaluation sample  $\mathcal{I}_{\text{eval}}$  of size  $n_{\text{eval}}$ , the mean squared error is

$$\text{MSE} = \frac{1}{n_{\text{eval}}} \sum_{i \in \mathcal{I}_{\text{eval}}} (y_i - \hat{m}(\mathbf{x}_i))^2. \quad (2.3.1)$$

The root mean squared error is the square root of the mean squared error,

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{eval}}} \sum_{i \in \mathcal{I}_{\text{eval}}} (y_i - \hat{m}(\mathbf{x}_i))^2} = \sqrt{\text{MSE}}. \quad (2.3.2)$$

Thus, [RMSE](#) measures the typical size of the prediction error after squaring errors, averaging them and returning the result to the original scale of the response variable. The mean absolute error is

$$\text{MAE} = \frac{1}{n_{\text{eval}}} \sum_{i \in \mathcal{I}_{\text{eval}}} |y_i - \hat{m}(\mathbf{x}_i)|. \quad (2.3.3)$$

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_{i \in \mathcal{I}_{\text{eval}}} (y_i - \hat{m}(\mathbf{x}_i))^2}{\sum_{i \in \mathcal{I}_{\text{eval}}} (y_i - \bar{y}_{\text{eval}})^2}, \quad \bar{y}_{\text{eval}} = \frac{1}{n_{\text{eval}}} \sum_{i \in \mathcal{I}_{\text{eval}}} y_i. \quad (2.3.4)$$

The  $R^2$  statistic measures the relative reduction in squared prediction error compared with the mean-only baseline  $\bar{y}_{\text{eval}}$  on the same evaluation sample. On held-out data,  $R^2$  may be negative if the fitted model performs worse than this baseline. For this reason, [RMSE](#), [MAE](#) and  $R^2$  are reported together rather than treated as interchangeable summaries of predictive performance ([James et al., 2021](#); [Hastie et al., 2009](#)).

Out-of-sample evaluation is central to statistical learning. A flexible model may achieve low training error while performing poorly on unseen observations, which corresponds to overfitting. Conversely, a model that is too restrictive may have high training and test error, which corresponds to underfitting. The bias-variance trade-off formalises this tension between systematic approximation error and sensitivity to the training sample ([James et al., 2021](#); [Hastie et al., 2009](#)). Classical regression models provide important reference points for actuarial and insurance applications. Ordinary least squares regression represents the conditional mean through a linear predictor,

$$\hat{m}_{\text{lin}}(\mathbf{x}_i) = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j x_{ij}. \quad (2.3.5)$$

Generalised linear models extend this formulation by specifying a distributional family for the response and linking the conditional mean

$$\mu_i = \mathbb{E}(Y_i \mid \mathbf{X}_i = \mathbf{x}_i) \quad (2.3.6)$$

to a linear predictor through a link function:

$$g(\mu_i) = \beta_0 + \sum_{j=1}^p \beta_j x_{ij}. \quad (2.3.7)$$

These models remain attractive in actuarial work because their assumptions, coefficients and diagnostics are relatively transparent, and because generalised linear modelling has a long history in insurance pricing and risk modelling (Dobson and Barnett, 2008; Denuit et al., 2007). A purely linear predictor, however, cannot represent nonlinear predictor effects without explicitly specifying transformations or basis expansions in advance.

Because GLMs remain the industry-standard framework for health insurance pricing, a gamma GLM with a logarithmic link function serves as a natural benchmark against which the predictive performance of the more flexible DNN can be compared. The benchmark is intended to quantify the gain in accuracy achieved by replacing a purely linear predictor with a non-linear neural network, and to provide a familiar reference point for readers accustomed to classical actuarial modelling. The specification of the benchmark model is given in Chapter 3.

Machine-learning methods offer a more flexible way to approximate complex predictor-response relationships. In actuarial contexts, Richman (2021) notes that neural-network methods can capture nonlinearities and interactions in risk relationships, learn internal feature representations from mixed-type inputs, and reduce the need for manual specification of predictor transformations. These properties are relevant when predictor effects are complex and involve both continuous and categorical variables, as is the case in the health insurance cost data considered here. Related evidence from population health-cost prediction indicates that flexible models can be useful when healthcare expenditure depends on nonlinear covariate relationships (Drewe-Boss et al., 2022). The DNN serves as one such flexible estimator of  $m(\cdot)$ .

The methodological role of the [DNN](#) rests on two properties. First, multilayer feedforward neural networks have strong function-approximation guarantees ([Hornik et al., 1989](#)), and modern deep-learning methods learn nonlinear mappings through compositions of transformations and activation functions ([Goodfellow et al., 2016](#)). Secondly, neural networks have been used in actuarial science as efficient surrogate models for computationally demanding valuation and prediction problems ([Hejazi and Jackson, 2016](#)). These properties make a [DNN](#) suitable for estimating a fitted prediction function  $\hat{m}$  that can be evaluated repeatedly in the Monte Carlo and Shapley-value stages. Under smooth activation choices, the fitted function is differentiable; under common piecewise-linear activations, it is piecewise differentiable. In either case, it provides a flexible parametric prediction surface compatible with the simulation and interpretation workflow.

The use of a [DNN](#) should not be read as a dismissal of competing tabular-data methods. Tree-based models and ensemble methods such as [RF](#) and gradient-boosted trees can also capture nonlinearities and interactions, and empirical work on structured datasets shows that they are often strong competitors ([Grinsztajn et al., 2022](#); [Orji and Ukwandu, 2024](#)). The [DNN](#) is used because it provides a fitted nonlinear prediction function that can be repeatedly evaluated and then connected to Monte Carlo input simulation and Shapley-value explanation. Tree-based models remain useful benchmarks for contextualising empirical performance.

The supervised regression formulation also clarifies the roles of the Monte Carlo and Shapley stages. The learning task is to estimate  $m(\cdot)$  from labelled observations in  $\mathcal{D}$ , producing the fitted prediction function  $\hat{m}(\cdot)$ . Monte Carlo simulation subsequently studies the distribution of fitted predictions under input variation. Shapley values then attribute fitted predictions to input variables. Predictive accuracy, distributional behaviour and attributional interpretation therefore address related but distinct aspects of model behaviour.

The resulting taxonomy is summarised in [Figure 2](#). The empirical problem belongs to machine learning, falls under supervised learning because responses are observed, belongs to regression because the response is continuous, and is implemented through [DNN](#) regression for the fitted prediction component.

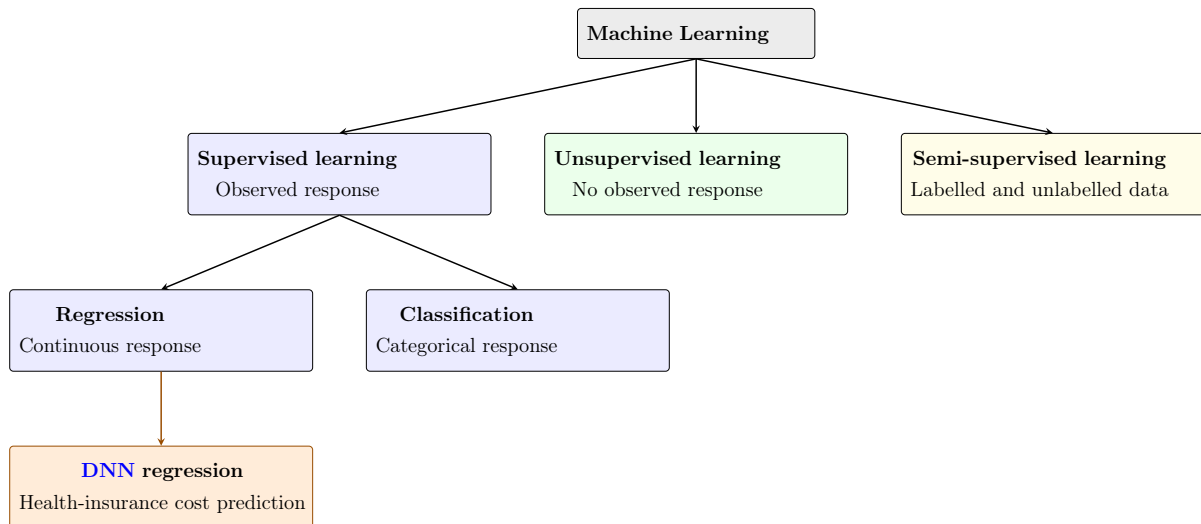


Figure 2: Position of the health-insurance cost prediction task within a standard machine-learning taxonomy.

The central empirical task is not unsupervised because the objective is not to discover clusters or latent structure without a response variable. A semi-supervised formulation is also unnecessary because the training observations contain both policyholder attributes and cost outcomes. Supervised regression is therefore the appropriate learning formulation, with the fitted **DNN** serving as the prediction function to be evaluated, simulated and interpreted.

## 2.4 Deep Neural Networks for Regression

A deep neural network is a composition of layers that maps an input vector to an output prediction. In regression problems, the aim is to approximate an unknown function that maps explanatory variables to a continuous response (Goodfellow et al., 2016; Bishop, 2006). In the empirical application, the input vector consists of encoded policyholder characteristics and the response is a health insurance charge or premium.

Let  $\mathbf{x}_i \in \mathbb{R}^p$  denote the preprocessed input vector for observation  $i$ , and let  $y_i \in \mathbb{R}_+$  denote the corresponding observed insurance cost. A feed-forward neural network constructs a sequence of layer activations from this input. Let  $d_0 = p$  denote the input dimension, let  $d_l$  denote the number of nodes in layer  $l$ , and let  $d_L = 1$  for a single continuous regression output. For observation  $i$ , the input layer is defined as

$$\mathbf{a}_i^{(0)} = \mathbf{x}_i \in \mathbb{R}^{d_0}. \quad (2.4.1)$$

For layers  $l = 1, \dots, L$ , the pre-activation vector and activation vector are defined by

$$\mathbf{z}_i^{(l)} = (W^{(l)})^\top \mathbf{a}_i^{(l-1)} + \mathbf{b}^{(l)}, \quad (2.4.2)$$

and

$$\mathbf{a}_i^{(l)} = \sigma_l(\mathbf{z}_i^{(l)}), \quad (2.4.3)$$

where

$$W^{(l)} \in \mathbb{R}^{d_{l-1} \times d_l}, \quad \mathbf{b}^{(l)} \in \mathbb{R}^{d_l},$$

and therefore

$$\mathbf{z}_i^{(l)}, \mathbf{a}_i^{(l)} \in \mathbb{R}^{d_l}.$$

The activation function  $\sigma_l(\cdot)$  is applied componentwise. For the hidden layers,  $\sigma_l(\cdot)$  is nonlinear. For the regression output layer, a linear activation is used, so that

$$\sigma_L(z) = z.$$

The fitted prediction for observation  $i$  is therefore

$$\hat{y}_i = f_\theta(\mathbf{x}_i) = a_{i1}^{(L)}, \quad (2.4.4)$$

where  $a_{i1}^{(L)}$  is the single output activation. A linear output is appropriate because the response is a continuous monetary variable rather than a probability or class label. Classification tasks would instead use logistic or softmax output functions.

The network parameters are collected in

$$\theta = \{W^{(1)}, \mathbf{b}^{(1)}, \dots, W^{(L)}, \mathbf{b}^{(L)}\}. \quad (2.4.5)$$

When the network is evaluated for all training observations simultaneously, let

$$A^{(0)} = [\mathbf{a}_1^{(0)}, \dots, \mathbf{a}_n^{(0)}] = X^\top \in \mathbb{R}^{d_0 \times n},$$

where  $X \in \mathbb{R}^{n \times p}$  is the model-ready design matrix. The matrix form of the layer recursion is

$$Z^{(l)} = (W^{(l)})^\top A^{(l-1)} + \mathbf{b}^{(l)} \mathbf{1}_n^\top, \quad A^{(l)} = \sigma_l(Z^{(l)}), \quad (2.4.6)$$

where

$$Z^{(l)}, A^{(l)} \in \mathbb{R}^{d_l \times n}, \quad \mathbf{1}_n \in \mathbb{R}^n.$$

This notation makes clear that the network is evaluated recursively from the input activations  $\mathbf{a}_i^{(0)}$  to the output activation  $\mathbf{a}_i^{(L)}$ , which supplies the fitted insurance-cost prediction.

### 2.4.1 Activation Functions

The choice of activation function affects both the expressive capacity and the optimisation dynamics of the network. Activation functions introduce nonlinearity into the layer recursion; without them, a composition of affine layers would reduce to a single affine transformation. Several activation functions may be considered for the hidden layers, including the hyperbolic tangent function, the rectified linear unit, the continuously differentiable exponential linear unit and the Gaussian error linear unit ([Goodfellow et al., 2016](#); [Lee and Wu, 2023](#)).

For a scalar input  $z \in \mathbb{R}$ , the hyperbolic tangent activation is defined by

$$\tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}. \quad (2.4.7)$$

The tanh function is smooth and bounded between  $-1$  and  $1$ . Its centred range may be useful in some network architectures, although its saturation for large positive or negative inputs can lead to small gradients in deeper networks.

The rectified linear unit is defined by

$$\text{ReLU}(z) = \max\{0, z\}. \quad (2.4.8)$$

The [Rectified Linear Unit \(ReLU\)](#) is widely used in feed-forward neural networks because it is simple, computationally efficient and less prone to the saturation behaviour associated

with some bounded activation functions (Nair and Hinton, 2010; Goodfellow et al., 2016). Its limitation is that negative inputs are mapped to zero, so some units may become inactive during training.

The continuously differentiable exponential linear unit with parameter  $\alpha > 0$  is defined by

$$\text{CELU}_\alpha(z) = \begin{cases} z, & z \geq 0, \\ \alpha \{\exp(z/\alpha) - 1\}, & z < 0. \end{cases} \quad (2.4.9)$$

**Continuously Differentiable Exponential Linear Unit (CELU)** is a smooth rectifier-type activation. It retains the linear positive branch of ReLU while assigning a smooth nonzero transformation to negative inputs (Barron, 2017).

The Gaussian error linear unit is defined by

$$\text{GELU}(z) = z\Phi(z), \quad (2.4.10)$$

where  $\Phi(\cdot)$  is the cumulative distribution function of a standard normal random variable. A commonly used approximation is

$$\text{GELU}(z) \approx \frac{1}{2}z \left[ 1 + \tanh \left\{ \sqrt{\frac{2}{\pi}} (z + 0.044715z^3) \right\} \right].$$

**Gaussian Error Linear Unit (GELU)** is smooth and differentiable, and may be viewed as a soft probabilistic gating alternative to a hard rectifier (Hendrycks and Gimpel, 2016; Lee and Wu, 2023). These activation functions are introduced as candidate nonlinear transformations. Their empirical suitability for health insurance cost prediction is assessed through the modelling procedure rather than assumed from their theoretical definitions alone.

### 2.4.2 Loss and Optimisation

The DNN is trained by minimising a loss function. For continuous cost prediction, mean squared error is a natural objective:

$$\mathcal{L}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i \in \mathcal{I}_{\text{train}}} (y_i - f_\theta(\mathbf{x}_i))^2, \quad (2.4.11)$$

where  $\mathcal{I}_{\text{train}}$  is the training index set and  $n_{\text{train}} = |\mathcal{I}_{\text{train}}|$ . The fitted parameters are obtained by solving

$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}(\theta). \quad (2.4.12)$$

For multilayer neural networks, this optimisation problem is generally nonconvex. Closed-form estimation is therefore not available in the usual case, and iterative gradient-based methods are used to search for a set of parameters that gives good predictive performance.

Gradient descent provides the conceptual basis for this training procedure. At iteration  $t$ , the full-batch gradient descent update is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}(\theta^{(t)}),$$

where  $\eta > 0$  is the learning rate. The gradient points in the direction of steepest increase of the loss, so subtracting the gradient moves the parameter vector in a direction intended to reduce the loss. In neural-network training, this gradient is computed by backpropagation, which applies the chain rule through the sequence of layer transformations.

In practice, mini-batch stochastic gradient methods are often preferred over full-batch gradient descent because computing the full gradient at every iteration can be computationally expensive. If  $\mathcal{B}_t$  denotes the mini-batch used at iteration  $t$ , the mini-batch gradient is

$$\mathbf{g}_t = \nabla_{\theta} \mathcal{L}_{\mathcal{B}_t}(\theta^{(t)}).$$

Stochastic gradient descent updates the parameters directly in the negative mini-batch gradient direction:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \mathbf{g}_t.$$

The use of mini-batches reduces computational cost and introduces stochasticity into the optimisation path. This is useful for training flexible models, but it also means that convergence behaviour must be monitored carefully.

Momentum methods modify stochastic gradient descent by using an exponentially weighted moving average of previous gradients:

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1 - \beta) \mathbf{g}_t, \quad 0 < \beta < 1,$$

followed by the update

$$\theta^{(t+1)} = \theta^{(t)} - \eta \mathbf{m}_t.$$

The moving-average term can reduce the effect of isolated noisy gradients and can accelerate movement in directions that remain persistent across iterations.

Adaptive gradient methods rescale parameter updates using information from previous gradients. RMSProp uses an exponentially weighted average of squared gradients,

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1 - \rho) \mathbf{g}_t^2, \quad 0 < \rho < 1,$$

and updates the parameters according to

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon},$$

where the square and square root are applied componentwise and  $\epsilon > 0$  is included for numerical stability. RMSProp is useful when different parameters require different effective learning-rate scales (Ruder, 2016).

Adam combines a momentum-like first-moment estimate with an adaptive second-moment estimate (Kingma and Ba, 2015). It updates

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2,$$

with bias-corrected estimates

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}.$$

The Adam update is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}.$$

Adam is widely used in applied deep learning because it combines adaptive parameter-wise scaling with a first-moment estimate of the gradient.

Nesterov Adaptive Moment Estimation, or NAdam, modifies Adam by incorporating a Nesterov-type momentum correction into the first-moment component (Dozat, 2016). A common form replaces the Adam numerator by

$$\tilde{\mathbf{m}}_t = \beta_1 \hat{\mathbf{m}}_t + \frac{1 - \beta_1}{1 - \beta_1^t} \mathbf{g}_t,$$

so that

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{\tilde{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}.$$

NAdam therefore retains Adam's adaptive second-moment scaling while modifying the momentum component through a Nesterov-style lookahead correction. These optimisers are introduced as candidate training mechanisms. The final optimiser choice is an empirical modelling decision and is justified in the methodology chapter using validation performance, convergence behaviour and held-out test results.

The optimiser is not a distributional assumption about insurance costs. It is the numerical mechanism used to estimate  $\hat{\theta}$ . This distinction is important because the subsequent Monte Carlo and Shapley analyses are applied to the fitted function  $f_{\hat{\theta}}$ . The stability and predictive quality of the fitted DNN therefore affect not only point prediction, but

also the model-implied predicted-cost distribution and the feature-attribution summaries derived later in the framework.

The empirical application considers a feed-forward DNN architecture for health insurance cost prediction. After categorical variables are encoded and numerical variables are standardised, the model-ready input dimension is  $d_0 = p$ . Under the preprocessing design used here, this dimension is  $p = 22$ . A candidate funnel architecture is shown in Figure 3.

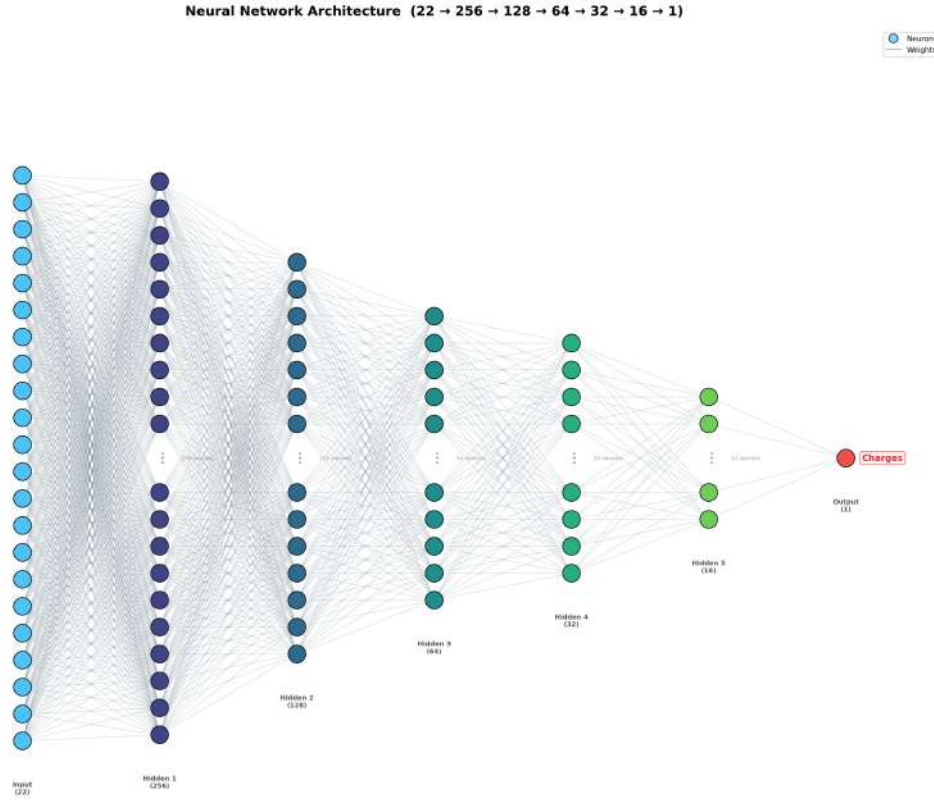


Figure 3: Candidate feed-forward neural network architecture for health insurance cost prediction.

The final output neuron represents the predicted health insurance charge or premium. This architecture starts with a wide hidden layer and progressively compresses the representation through successive layers, allowing the network to learn nonlinear representations before reducing them to a scalar prediction.

The architecture used in the empirical chapter may be implemented without explicit bias vectors, depending on the final modelling choice confirmed for the study. In that case, the layer recursion in equations (2.4.2)-(2.4.3) simplifies to

$$\mathbf{z}_i^{(l)} = (W^{(l)})^\top \mathbf{a}_i^{(l-1)}, \quad \mathbf{a}_i^{(l)} = \sigma_l(\mathbf{z}_i^{(l)}), \quad l = 1, \dots, L, \quad (2.4.13)$$

with  $\mathbf{a}_i^{(0)} = \mathbf{x}_i$  as in equation (2.4.1). The output prediction is then

$$f_{\theta}(\mathbf{x}_i) = a_{i1}^{(L)}.$$

This bias-free specification removes additive layer-bias parameters from the network architecture; the distinction between architectural, statistical and data-related meanings of bias is noted above. Its adequacy is assessed empirically through training, validation and test performance, and where appropriate through comparison with a bias-inclusive network.

The final architecture should not be selected purely by intuition; it should be determined using validation performance and evaluated on a held-out test set. This is necessary because neural networks can be highly flexible. A network that is too small may underfit, while a network that is too large may fit the training data closely without generalising to unseen observations. The DNN is therefore treated as a statistical learning model whose usefulness is judged by out-of-sample prediction performance and by the quality of the subsequent simulation and interpretability analysis (James et al., 2021; Hastie et al., 2009).

The dataset is partitioned as

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}, \quad (2.4.14)$$

where the three components correspond to a 60/20/20 split. An epoch is one complete pass through the training data. For a fitted parameter vector  $\hat{\theta}^{(e)}$  after epoch  $e$ , the training and validation losses are

$$\mathcal{L}_{\text{train}}^{(e)} = \frac{1}{n_{\text{train}}} \sum_{i \in \mathcal{I}_{\text{train}}} (y_i - f_{\hat{\theta}^{(e)}}(\mathbf{x}_i))^2 \quad (2.4.15)$$

and

$$\mathcal{L}_{\text{val}}^{(e)} = \frac{1}{n_{\text{val}}} \sum_{i \in \mathcal{I}_{\text{val}}} (y_i - f_{\hat{\theta}^{(e)}}(\mathbf{x}_i))^2. \quad (2.4.16)$$

The candidate number of epochs is selected by comparing validation loss over the epoch grid. If  $\mathcal{E}$  denotes the set of candidate epoch values, the selected training duration is

$$e^* = \arg \min_{e \in \mathcal{E}} \mathcal{L}_{\text{val}}^{(e)}. \quad (2.4.17)$$

This validation step is necessary because training loss typically decreases as the optimiser continues updating the model, but validation loss may eventually increase if the network begins to adapt too closely to the training sample. A decreasing training loss with a rising validation loss is a common indication of overfitting, while weak training and validation

performance may indicate underfitting or unsuitable training settings.

**Leave-One-Out Cross-Validation (LOOCV)** is not adopted as the primary validation strategy. It would require refitting the neural network once for each observation. This is computationally inefficient for a **DNN**, since each fit involves iterative nonconvex optimisation rather than a closed-form estimation step. In addition, the training sets used in **LOOCV** differ by only one observation, so the resulting fitted models are highly overlapping and the error estimates may be strongly correlated (James et al., 2021; Hastie et al., 2009).

Although  $K$ -fold cross-validation is often useful when a separate validation set is unavailable or when dependence on a single random split must be reduced, it is not required for the main analysis because the data are already supplied with the fixed partition in equation (2.4.14). Applying  $K$ -fold cross-validation to the full dataset would mix observations assigned to validation and test components into the training process, weakening the interpretation of the final test performance. Applying  $K$ -fold cross-validation only to the training set would avoid this leakage, but would require repeated **DNN** training across folds and candidate epoch values, while the supplied validation set already provides the intended mechanism for selecting the training duration. The validation analysis therefore uses the pre-specified validation set to select  $e^*$ , and the test set is used only once for final model assessment.

### 2.4.3 Generalisation, Validation Analysis and Bias-Variance Trade-off

The tension between overfitting and underfitting is particularly acute for flexible models such as **DNNs**, which may contain many trainable parameters and can represent complex nonlinear relationships. The formal bias-variance decomposition below makes this trade-off precise.

Expressing the regression formulation in equation (2.2.2) in population-level notation,

$$Y = f(\mathbf{X}) + \varepsilon, \quad (2.4.18)$$

where  $f(\mathbf{X})$  denotes the true regression function  $m(\cdot)$  from equation (2.2.2) and  $\varepsilon$  represents irreducible noise with  $\mathbb{E}(\varepsilon \mid \mathbf{X}) = 0$ . In the homoscedastic case, the irreducible variation is denoted  $\sigma_\varepsilon^2$ .

If  $\hat{f}_{\mathcal{D}}$  is a model trained on a dataset  $\mathcal{D}$ , then prediction error on a new observation depends not only on how well the model fits the observed data, but also on how stable the fitted function is across different possible training samples. This motivates the bias-variance trade-off in statistical learning (Hastie et al., 2009; James et al., 2021; Bishop, 2006).

For squared-error loss, the expected prediction error at a point  $\mathbf{x}$  can be decomposed into

three components:

$$\mathbb{E}_{\mathcal{D}, Y} \left[ \left( Y - \hat{f}_{\mathcal{D}}(\mathbf{x}) \right)^2 \right] = \underbrace{\left( f(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} \left[ \hat{f}_{\mathcal{D}}(\mathbf{x}) \right] \right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[ \left( \hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}} \left[ \hat{f}_{\mathcal{D}}(\mathbf{x}) \right] \right)^2 \right]}_{\text{Variance}} + \underbrace{\sigma_{\varepsilon}^2}_{\text{Noise}}. \quad (2.4.19)$$

The bias term measures the systematic error introduced by the choice of model class, while the variance term measures the instability of the fitted model across different training samples. The noise term  $\sigma_{\varepsilon}^2$  represents irreducible variation in the response that cannot be removed by any model based on the available predictors.

The bias-variance trade-off can therefore be summarised as

$$\text{Expected test error} = \text{Variance} + \text{Bias}^2 + \text{Irreducible noise}.$$

Equivalently, in terms of model complexity,

$$\text{Model complexity} \uparrow \implies \text{Bias}^2 \downarrow, \quad \text{Variance} \uparrow,$$

whereas

$$\text{Model complexity} \downarrow \implies \text{Bias}^2 \uparrow, \quad \text{Variance} \downarrow.$$

This relationship explains why a highly flexible model may fit the training data closely but perform poorly on unseen observations, while an overly simple model may fail to capture the underlying pattern in both training and unseen data. The aim is not to minimise training error alone, but to find a level of model complexity that gives a favourable balance between bias and variance.

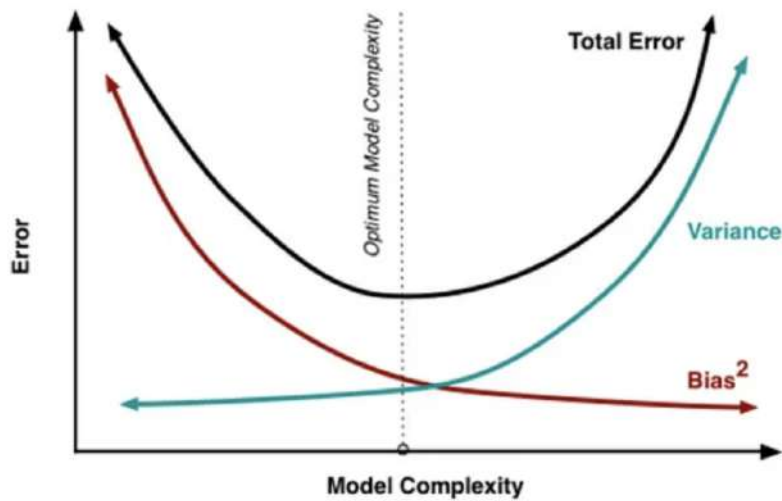


Figure 4: Illustration of the bias-variance trade-off. Adapted from (James et al., 2021).

Figure 4 provides a schematic illustration of this relationship. The diagram emphasises that neither a very simple model nor an excessively flexible model is generally desirable for prediction. Instead, the preferred model complexity is the one that achieves the best out-of-sample performance by balancing bias and variance. This principle motivates the epoch-selection criterion defined in equation (2.4.17).

The pointwise squared-error decomposition in equation (2.4.19) separates the expected prediction error at a fixed input value  $\mathbf{x}$  into bias, variance and irreducible noise. The same principle can also be expressed at the population level by considering expected out-of-sample error. Let  $g^{(\mathcal{D})}$  denote a fitted model obtained from a training sample  $\mathcal{D}$ , and define the average fitted model by

$$\bar{g}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[g^{(\mathcal{D})}(\mathbf{x})] .$$

Also let

$$\bar{Y} = f(\mathbf{X}) = \mathbb{E}(Y \mid \mathbf{X})$$

denote the systematic component of the response. The expected out-of-sample error under squared loss can then be written as

$$\begin{aligned} \mathbb{E}_{\mathcal{D}}[E_{\text{out}}(g^{(\mathcal{D})})] &= \mathbb{E}_{\mathcal{D}}[\mathbb{E}_{\mathbf{X}, Y}[(g^{(\mathcal{D})}(\mathbf{X}) - Y)^2]] \\ &= \mathbb{E}_{\mathbf{X}}[\underbrace{\mathbb{E}_{\mathcal{D}}[(g^{(\mathcal{D})}(\mathbf{X}) - \bar{g}(\mathbf{X}))^2]}_{\text{Model variance}}] \\ &\quad + \underbrace{\mathbb{E}_{\mathbf{X}}[(\bar{g}(\mathbf{X}) - f(\mathbf{X}))^2]}_{\text{Model bias}} + \underbrace{\mathbb{E}_{\mathbf{X}}[\mathbb{E}_{Y|\mathbf{X}}[(\bar{Y} - Y)^2]]}_{\text{Noise}} . \end{aligned} \tag{2.4.20}$$

This form is useful because it links model complexity directly to expected out-of-sample performance, which is the quantity approximated by the validation and test-set analysis. Complex models may reduce model bias by representing richer patterns, but they may also increase model variance by becoming sensitive to the particular training sample. Simpler models may have lower variance, but they may miss important nonlinear structure. The practical aim is therefore to choose a model that captures stable structure without merely recalling noise in the training data.

The statistical notion of bias in this decomposition is distinct from the trainable bias parameter in a neural-network layer and from data bias in the benchmark dataset. The bias-variance trade-off is therefore used as a general learning principle and not as a justification for including or omitting bias parameters in the DNN architecture.

The training partition  $\mathcal{D}_{\text{train}}$  (equation (2.4.14)) is used to estimate model parameters through gradient-based optimisation. The validation partition  $\mathcal{D}_{\text{val}}$  guides the choice of

training duration via the epoch-selection criterion in equation (2.4.17). The test partition  $\mathcal{D}_{\text{test}}$  is reserved for final out-of-sample evaluation after the model has been selected and its architecture fixed. Using the test set for any model-selection decision invalidates its role as an honest estimate of generalisation performance (Hastie et al., 2009; Abu-Mostafa et al., 2012).

Generalisation is assessed by comparing training, validation and test performance using RMSE, MAE and  $R^2$ . If training performance is substantially better than validation or test performance, this may indicate overfitting. If all three are weak, this may indicate underfitting. If training, validation and test results are similar and sufficiently strong, this supports the conclusion that the fitted model generalises reasonably well within the available dataset.

The use of a benchmark health insurance dataset introduces an additional qualification. A model may generalise well within the dataset’s distributional domain, but this does not necessarily imply validity for real-world health insurance populations with different risk profiles, benefit structures or underlying demographics. Internal predictive validity on the available dataset must therefore be distinguished from external validity for real insurance settings. The analysis is therefore framed as a methodological study of DNN prediction, Monte Carlo input simulation and Shapley-value interpretation on a structured health insurance benchmark, with any claims about real-world insurance populations requiring further validation.

## 2.5 Health Insurance Pricing and Cost Prediction

The supervised-regression formulation gives the statistical form of the empirical problem: the observed response  $y_i$  is a continuous monetary outcome, and the fitted function  $\hat{m}(\mathbf{x})$  maps policyholder attributes to a predicted cost. The health-insurance setting gives this regression problem its actuarial meaning. A predicted monetary outcome is not merely an abstract response variable; it is connected to premium rating, claim-cost estimation and risk segmentation. Classical actuarial pricing often relies on transparent regression-based models, especially generalised linear models, while the DNN considered here plays the related role of a flexible nonlinear estimator of the cost surface rather than a replacement for actuarial reasoning.

Recent actuarial work has demonstrated that neural networks can be incorporated into actuarial pricing structures without abandoning the underlying frequency-severity logic. Laporta et al. (2025) propose neural-network models for pricing correlated health risks by embedding negative multinomial neural networks for claim frequency and gamma neural networks for claim severity within a classical frequency-severity structure. Their empirical results on real-world health insurance data show that neural-network models achieve lower out-of-sample deviance and better risk diversification than established regression methods such as generalised linear models, as measured by the ABC lift metric. The relevance of this work is that neural networks need not be treated as generic black-box alternatives; they can be incorporated within actuarial pricing logic to improve both accuracy and risk segmentation.

A complementary approach to neural-network pricing is to impose explicit shape constraints during training. Richman and Wüthrich (2024) develop the ICEnet, which adds smoothness and monotonicity penalty terms to the loss function so that the network's Individual Conditional Expectation outputs vary in a manner consistent with actuarial domain knowledge. Applied to claims frequency prediction on the French motor third-party liability dataset, the ICEnet achieves a test-set Poisson deviance within 0.0001 of the unconstrained network, demonstrating that actuarial reasonableness can be enforced with negligible accuracy cost. Because the ICEnet targets count data under a Poisson deviance criterion, its results are not directly comparable with the continuous monetary cost predictions in the present dissertation; a methodological comparison is presented in Section 5.7.

Alongside neural-network approaches, tree-based ensemble methods have become widely used for insurance cost prediction on structured tabular data. Orji and Ukwandu (2024) apply random forests and gradient-boosted trees to health insurance cost data and report predictive performance comparable to, and in some configurations exceeding, that of neural networks. More broadly, Grinsztajn et al. (2022) provide systematic evidence that

tree-based models, particularly gradient-boosted implementations such as [XGBoost](#) ([Chen and Guestrin, 2016](#)), frequently match or outperform deep neural networks on medium-sized tabular datasets of the kind encountered in actuarial practice. Because insurance cost prediction is fundamentally an actuarial problem defined on structured policyholder attributes rather than on unstructured data such as images or text, tree-based methods represent a credible alternative to the [DNN](#) and may in some settings be the preferred model class. The present dissertation nonetheless selects the [DNN](#) as the primary model for the Monte Carlo and Shapley workflow because its smooth, differentiable prediction surface is directly compatible with gradient-based Shapley computation and continuous forward-pass evaluation over large simulated input samples. A [RF](#) and an [XGBoost](#) regressor are fitted as benchmarks to contextualise the [DNN](#)'s predictive performance within this spectrum of model classes.

The available Kaggle insurance data do not provide separate claim-frequency and claim-severity processes. A separate actuarial claims model is therefore not fitted. Instead, the observed monetary response is treated as an aggregate health-insurance charge or premium proxy, and cost prediction is formulated as a supervised regression problem. This gives a narrower empirical objective: to study how a fitted nonlinear prediction surface behaves on a benchmark health-insurance dataset, without claiming to reproduce the complete pricing tariff or underwriting rules of a specific insurer.

The objective is not merely to obtain a point estimate of cost for each policyholder, but to understand the distribution and drivers of predicted costs under varying input profiles. This is important because actuarial modelling is concerned not only with average prediction, but also with uncertainty, high-cost scenarios and the interpretation of risk drivers ([Denuit et al., 2007](#)). In this sense, the fitted regression function  $\hat{m}$  is the starting point for a broader analysis of predicted-cost behaviour rather than the end of the modelling exercise.

Health expense projection literature provides a further actuarial motivation for studying predicted costs distributionally. [Christiansen et al. \(2018\)](#) develop practical models for projecting health insurance expenditures over short time horizons, which supports the view that health cost analysis should consider the behaviour of cost outcomes beyond isolated point estimates. The present dissertation differs from traditional health expense projection because it does not construct a demographic, utilisation or claims-process model. Instead, it trains a [DNN](#) on policyholder-level attributes and uses Monte Carlo input simulation to examine the distribution of costs implied by the fitted prediction function. This distinction keeps the empirical claim modest: the analysis concerns model-implied predicted costs under the selected benchmark-data and simulation design, rather than realised expenditure dynamics in an insurer's full portfolio.

Ordinary cost prediction is therefore extended in two ways. First, after fitting the [DNN](#),

Monte Carlo input simulation is used to generate many possible policyholder profiles and examine the distribution of model-implied predicted costs. Second, Shapley-value interpretation is used to identify the variables that drive individual predictions and high-cost simulated profiles. This connects health insurance cost prediction with simulation-based risk analysis and explainable machine learning. The resulting analysis asks not only how accurately the fitted model predicts costs, but also how predicted costs vary across simulated profiles and which variables the model attributes to the upper tail of the predicted-cost distribution.

The interpretation remains model-based. The Monte Carlo distribution describes variation in fitted predictions under the selected input distribution, not the full population distribution of realised healthcare expenditure. Similarly, Shapley values explain the fitted [DNN](#)'s predictions relative to the chosen background distribution; they do not establish causal determinants of healthcare costs or provide evidence of actual insurer pricing rules. The health insurance application is therefore used as an actuarially motivated setting in which prediction, simulation and explanation can be combined while keeping the empirical claims within the limits of the data.

## 2.6 Input Distribution Diagnostics for Simulation

Validation analysis addresses whether the fitted [DNN](#) generalises within the supplied training, validation and test structure, but Monte Carlo input simulation requires an additional diagnostic step. Even when a fitted prediction function  $f_{\hat{\theta}}$  has acceptable validation and test performance, the simulation results may be misleading if the input profiles are generated from an unrealistic or poorly specified input distribution. The diagnostic concern therefore shifts from the fitted model itself to the probability law used to generate simulated policyholder profiles.

A trained model  $f_{\hat{\theta}}$  can be evaluated at any admissible input vector, but simulation-based analysis requires a probability law for the random input vector  $\mathbf{X}$ . The distribution of simulated predictions depends not only on the fitted [DNN](#), but also on the assumed or estimated distribution of policyholder attributes. Input distribution diagnostics therefore provide the empirical basis for specifying the input distribution  $\hat{F}_{\mathbf{X}}$  used later in the Monte Carlo component of the framework.

Let

$$\mathbf{X} = (X_1, \dots, X_p)^\top$$

denote the vector of input variables before final model encoding, and let  $\tilde{\mathbf{X}}$  denote the corresponding model-ready vector after encoding and standardisation. Where no ambiguity arises,  $\mathbf{X}$  is used as the generic random input vector. Before specifying an input distribution for  $\mathbf{X}$ , the empirical distribution of each variable must be examined. For numerical variables, this includes measures of location, scale, skewness, tail behaviour and possible outliers. For categorical variables, this includes category frequencies, rare levels and possible class imbalance.

For a numerical variable  $X_j$ , the sample mean and variance are

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{ij}, \quad s_j^2 = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2.$$

Skewness can be estimated by

$$\hat{\gamma}_{1j} = \frac{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^3}{\left[ \frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2 \right]^{3/2}},$$

and kurtosis by

$$\hat{\gamma}_{2j} = \frac{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^4}{\left[ \frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2 \right]^2}.$$

Values of  $\hat{\gamma}_{1j} > 0$  indicate right-skewness, while values of  $\hat{\gamma}_{2j}$  substantially above 3 indicate heavier-than-normal tails. These quantities help identify variables that are asymmetric,

heavy-tailed or potentially unsuitable for simple symmetric distributional assumptions.

The response variable charges is also subject to diagnostic analysis, even though Monte Carlo input simulation focuses primarily on the predictors. Insurance cost variables often display right-skewness, heterogeneity and upper-tail behaviour (Denuit et al., 2007). If the response is strongly skewed, a log transformation of the form

$$Y_i^* = \log(1 + Y_i)$$

may be considered during model training. If such a transformation is used, final predictions and performance metrics must be back-transformed and reported on the original monetary scale.

For categorical variables, the relevant diagnostic is the empirical frequency distribution. If  $X_j$  takes values in the finite set  $\{c_1, \dots, c_{K_j}\}$ , the empirical proportion of category  $c_k$  is

$$\hat{p}_{jk} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}\{X_{ij} = c_k\}.$$

Rare categories may be poorly represented in the training set and may lead to unstable predictions or unreliable Shapley summaries. In the health insurance setting, categorical variables such as region, smoking status, medical history, exercise frequency, occupation and coverage level may all influence the structure learned by the fitted model.

Input diagnostics also inform preprocessing. Numerical variables are standardised before training a DNN to support numerical stability in gradient-based optimisation. The standardisation parameters must be computed from the training set only and then applied to the validation and test sets using those same training-set values, so that information from held-out observations does not leak into the training procedure. The specific standardisation formula and its implementation are detailed in Chapter 3.

Validation analysis assesses whether  $f_{\hat{\theta}}$  performs adequately on held-out observations from the available data distribution. Input distribution diagnostics assess whether the profiles generated for Monte Carlo simulation remain compatible with that same empirical domain. Without both checks, the subsequent predicted-cost distribution and high-cost Shapley summaries could reflect unrealistic simulated profiles rather than meaningful behaviour of the fitted model.

## 2.7 Specification of the Input Distribution

The input distribution  $\hat{F}_X$  was defined as a product of independent fitted marginals (equation (2.2.5)), with the sampling and propagation procedure given by equations (2.2.6)

and (2.2.7). Three main approaches are available for constructing  $\hat{F}_X$ .

The simplest approach is empirical row resampling, in which complete observed covariate vectors are sampled from the dataset with replacement (Efron and Tibshirani, 1993):

$$\mathbf{X}^{(\ell)} \in \{\mathbf{x}_1, \dots, \mathbf{x}_n\}, \quad \ell = 1, \dots, M. \quad (2.7.1)$$

Because the full covariate vector is sampled as a unit, empirical row resampling retains whatever joint structure is present in the observed data. Its limitation is that it cannot generate policyholder profiles outside the observed data range.

A second approach is marginal parametric simulation, in which each input variable is assigned a fitted marginal distribution (Rubinstein and Kroese, 2016; Glasserman, 2004). For a numerical variable  $X_j$ , a parametric family  $F_j(\cdot; \psi_j)$  may be fitted by maximum likelihood:

$$\hat{\psi}_j = \arg \max_{\psi_j} \sum_{i=1}^n \log f_j(x_{ij}; \psi_j), \quad (2.7.2)$$

where  $f_j(\cdot; \psi_j)$  is the corresponding density or probability mass function. Simulated values are then drawn as

$$X_j^{(\ell)} \sim F_j(\cdot; \hat{\psi}_j), \quad \ell = 1, \dots, M. \quad (2.7.3)$$

This approach is convenient and can generate values beyond the observed sample. It is the approach adopted in this dissertation, where independent fitted marginals are combined as specified in equation (2.2.5). The independence assumption is a deliberate modelling choice, as discussed in connection with equation (2.2.5).

A third approach is nonparametric or semi-parametric simulation, using bootstrap-type resampling or smoothed empirical distributions (Efron and Tibshirani, 1993). These methods avoid imposing a strong parametric form on the marginal distributions, although they remain limited by the information contained in the observed sample. Their usefulness depends on whether the empirical data provide sufficient coverage of the regions of the input space that are relevant for the simulation exercise.

The choice of specification method determines the input profiles passed through the fitted prediction function. A poorly specified input distribution may create high-cost profiles that are artefacts of the simulation design rather than meaningful evaluations of the fitted model. The specific distributional families, fitted parameter values, support restrictions and encoding rules for each attribute are detailed in Chapter 3.

Figure 5 illustrates the marginal parametric construction adopted here. Each column of  $\mathcal{X}_{\text{train}}$  is fitted independently to a parametric marginal distribution, and the product of these marginals defines  $\hat{F}_X$ , from which  $M$  synthetic input vectors are drawn to form the

simulation design matrix.

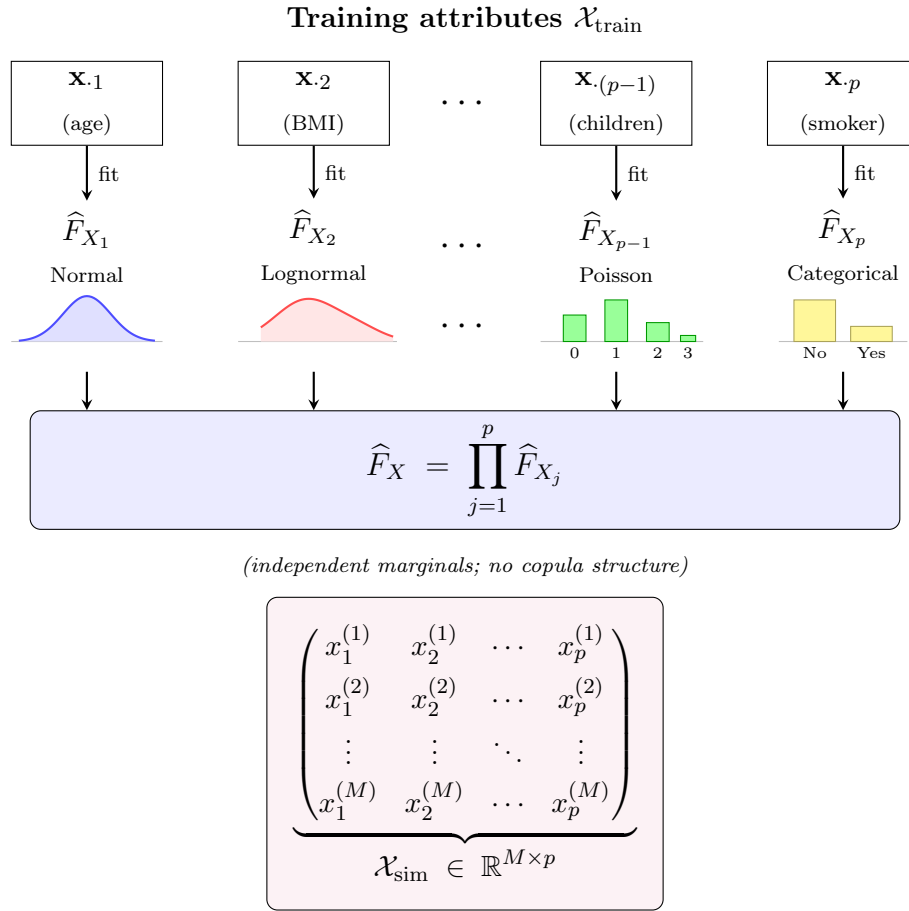


Figure 5: Construction of the input distribution  $\hat{F}_X$  (equation (2.2.5)).

Each column of  $\mathcal{X}_{\text{train}}$  is fitted independently to a parametric marginal distribution, whose shape is illustrated beneath the corresponding label in Figure 5. The product of these marginals defines  $\hat{F}_X$ , from which  $M$  synthetic input vectors are drawn to form the simulation design matrix  $\mathcal{X}_{\text{sim}} \in \mathbb{R}^{M \times p}$ .

## 2.8 Foundations of Monte Carlo

Monte Carlo simulation rests on a small number of foundational results concerning the crude Monte Carlo estimator, its convergence properties and the precision measures that underpin the analysis (Glasserman, 2004; Rubinstein and Kroese, 2016).

Let  $X$  be a random variable with distribution  $F$ , and suppose that the quantity of interest is

$$\theta = \mathbb{E}[g(X)]. \quad (2.8.1)$$

If

$$X^{(1)}, \dots, X^{(M)} \stackrel{\text{iid}}{\sim} F, \quad (2.8.2)$$

then the crude Monte Carlo estimator of  $\theta$  is

$$\hat{\theta}_M = \frac{1}{M} \sum_{\ell=1}^M g(X^{(\ell)}) . \quad (2.8.3)$$

Under standard regularity conditions, the law of large numbers implies

$$\hat{\theta}_M \longrightarrow \theta \quad \text{as} \quad M \longrightarrow \infty . \quad (2.8.4)$$

If

$$\sigma_g^2 = \text{Var}\{g(X)\} < \infty , \quad (2.8.5)$$

then

$$\text{Var}(\hat{\theta}_M) = \frac{\sigma_g^2}{M} , \quad (2.8.6)$$

which yields the familiar  $M^{-1/2}$  convergence rate of ordinary Monte Carlo simulation (Caffisch, 1998; Glasserman, 2004).

The Monte Carlo standard error quantifies the simulation error associated with using a finite number of random draws. It is estimated by

$$\text{MCSE}(\hat{\theta}_M) = \frac{s_M}{\sqrt{M}} , \quad (2.8.7)$$

where

$$s_M^2 = \frac{1}{M-1} \sum_{\ell=1}^M \left[ g(X^{(\ell)}) - \hat{\theta}_M \right]^2 . \quad (2.8.8)$$

By the central limit theorem,

$$\frac{\hat{\theta}_M - \theta}{s_M/\sqrt{M}} \xrightarrow{d} N(0, 1) , \quad (2.8.9)$$

giving the approximate  $100(1 - \alpha)\%$  Monte Carlo confidence interval

$$\hat{\theta}_M \pm z_{1-\alpha/2} \frac{s_M}{\sqrt{M}} . \quad (2.8.10)$$

This interval describes uncertainty due to the finite number of Monte Carlo draws. It does not, by itself, represent all sources of statistical uncertainty in the fitted DNN, the input distribution or the realised insurance cost.

In the health insurance framework, the function  $g$  is the fitted prediction function  $\hat{m}$ , the distribution  $F$  is the fitted input distribution  $\hat{F}_X$ , and the target  $\theta$  corresponds to summaries of the model-implied predicted-cost distribution such as the mean, variance and upper-tail probabilities. The convergence, variance and confidence-interval results

above therefore apply directly to those summaries.

## 2.9 Variance Reduction and Tail Simulation

For many quantities, ordinary Monte Carlo simulation is sufficient when the simulation budget is large and the Monte Carlo standard error is reported. However, when the quantity of interest is a tail probability, a high quantile or a rare high-cost event, crude Monte Carlo can become inefficient because few simulated draws fall in the region of interest. This issue is directly relevant to insurance because high-cost events may be financially important even when they are relatively rare ([Glasserman, 2004](#)).

Let  $u$  denote a high-cost threshold, and suppose that the model-implied exceedance probability is

$$p_u = \mathbb{P}(\hat{Y} > u),$$

where

$$\hat{Y} = \hat{m}(\mathbf{X}), \quad \mathbf{X} \sim \hat{F}_X.$$

Given simulated predictions  $\hat{Y}^{(1)}, \dots, \hat{Y}^{(M)}$ , the crude Monte Carlo estimator of  $p_u$  is

$$\hat{p}_u = \frac{1}{M} \sum_{\ell=1}^M \mathbb{I}\{\hat{Y}^{(\ell)} > u\}.$$

Its estimated standard error is

$$\widehat{\text{SE}}(\hat{p}_u) = \sqrt{\frac{\hat{p}_u(1 - \hat{p}_u)}{M}}.$$

The relative standard error is approximately

$$\frac{\widehat{\text{SE}}(\hat{p}_u)}{\hat{p}_u} \approx \sqrt{\frac{1 - \hat{p}_u}{M\hat{p}_u}}.$$

When  $p_u$  is small, the expected number of exceedances,  $Mp_u$ , may remain small unless  $M$  is large. This explains why rare-event estimation can be difficult under crude Monte Carlo sampling.

Importance sampling is one classical variance-reduction method. Suppose that samples are drawn from an alternative density  $q$  rather than the original density  $f$ . For a target expectation

$$\theta = \int g(x)f(x) dx,$$

the same quantity may be written as

$$\theta = \int g(x) \frac{f(x)}{q(x)} q(x) dx.$$

The corresponding importance-sampling estimator is

$$\hat{\theta}_{\text{IS}} = \frac{1}{M} \sum_{\ell=1}^M g(X^{(\ell)}) w(X^{(\ell)}), \quad X^{(\ell)} \sim q,$$

where

$$w(x) = \frac{f(x)}{q(x)}$$

is the likelihood ratio, also called the importance weight. A well-chosen proposal distribution  $q$  concentrates simulation effort in the region that contributes most to the quantity being estimated, and can substantially reduce estimator variance (Glasserman, 2004; Rubinstein and Kroese, 2016). A poor choice of  $q$ , however, may increase variance rather than reduce it, so the proposal distribution must be selected carefully.

Stratified sampling is another variance-reduction approach. Suppose the input space is partitioned into disjoint strata

$$A_1, \dots, A_H,$$

with

$$p_h = \mathbb{P}(\mathbf{X} \in A_h).$$

If  $M_h$  samples are drawn from stratum  $h$ , the stratified estimator of  $\theta = \mathbb{E}\{g(\mathbf{X})\}$  is

$$\hat{\theta}_{\text{strat}} = \sum_{h=1}^H p_h \bar{g}_h,$$

where  $\bar{g}_h$  is the sample mean within stratum  $h$ . Its variance is

$$\text{Var}(\hat{\theta}_{\text{strat}}) = \sum_{h=1}^H \frac{p_h^2 \sigma_h^2}{M_h},$$

where  $\sigma_h^2$  is the conditional variance of  $g(\mathbf{X})$  within stratum  $h$ . In a health insurance application, natural strata may include smoking status, age band, BMI category, medical history group or coverage level. Stratification is useful when these groups are expected to contribute differently to the predicted-cost distribution (Rubinstein and Kroese, 2016).

Quasi-Monte Carlo methods replace independent random points with low-discrepancy sequences designed to cover the unit cube more evenly. If

$$\mathbf{u}^{(1)}, \dots, \mathbf{u}^{(M)} \in [0, 1]^p$$

are low-discrepancy points and  $T$  maps uniform vectors to input profiles, the quasi-Monte Carlo estimator is

$$\hat{\theta}_{\text{QMC}} = \frac{1}{M} \sum_{\ell=1}^M g(T(\mathbf{u}^{(\ell)})) .$$

For sufficiently smooth integrands and moderate effective dimension, quasi-Monte Carlo methods can improve convergence relative to ordinary Monte Carlo, with rates closer to

$$O(M^{-1}(\log M)^p)$$

than the usual

$$O(M^{-1/2})$$

Monte Carlo rate ([Caffisch, 1998](#); [Lemieux, 2009](#); [Owen, 2013](#)).

In the present framework, variance-reduction methods are primarily relevant for interpreting high-cost simulated profiles. The main simulation exercise may use crude Monte Carlo or empirical resampling, but the variance-reduction literature clarifies why tail quantities require care when reporting precision. If high-cost probabilities, upper quantiles or conditional Shapley summaries for extreme predicted costs are estimated, the simulation size and precision of those estimates should be reported and justified.

A practical criterion for selecting a simulation size is a pilot-based calculation. If  $s_0^2$  is a pilot estimate of the variance of the simulated quantity and an absolute half-width  $\epsilon$  is desired for an approximate confidence interval, then a rough simulation-size requirement is

$$M \gtrsim \left( \frac{z_{1-\alpha/2} s_0}{\epsilon} \right)^2 .$$

This expression connects the simulation budget to the desired numerical precision of the reported Monte Carlo summaries.

Tail simulation also connects to Shapley-value attribution. The simulated high-cost set  $\mathcal{H}_u$  (equation (2.2.14)) contains the profiles for which feature-attribution summaries are most relevant to the upper tail of the fitted model. Shapley values can then be used to identify which variables receive large model-attributed contributions among simulated profiles whose fitted predictions exceed  $u$ . In this way, tail simulation provides the subset of simulated profiles on which the high-cost interpretability analysis is based.

## 2.10 Explainability, Shapley Values and Conditional Interpretation

Deep neural networks are flexible nonlinear prediction models, but their flexibility often comes at the cost of interpretability. In insurance applications, this is a serious practical

and regulatory concern. A model may produce accurate predictions, but insurers, regulators and policyholders may still require an explanation of why a particular profile receives a high predicted cost. This motivates the use of explainable artificial intelligence methods (Lundberg and Lee, 2017).

Shapley values originate in cooperative game theory and provide a principled axiomatic method for allocating a total payoff among players (Shapley, 1953). In machine learning, the players are the input variables and the payoff is the model prediction. The Shapley value of feature  $j$  measures the average marginal contribution of that feature across all possible subsets of features, as formalised in equation (2.2.10). The Shapley value is the unique solution satisfying four axioms: efficiency (contributions sum to the total prediction shift), symmetry (identically contributing features receive identical values), dummy (a feature that never changes the value receives zero), and additivity (Shapley, 1953).

The SHAP framework connects Shapley values to machine learning model interpretation Lundberg and Lee (2017). The additive decomposition in equation (2.2.11) provides a local explanation of an individual prediction. Lundberg and Lee (2017) proved that this is the only additive feature attribution method satisfying efficiency, dummy and symmetry simultaneously, which is why it constitutes the theoretical foundation for the SHAP computational framework.

Shapley-based explanations can be interpreted locally or globally. A local explanation focuses on a single policyholder profile and explains why the model assigned a particular predicted cost. A global explanation summarises feature contributions across many observations. A common global importance measure is the mean absolute Shapley value:

$$I_j = \frac{1}{n} \sum_{i=1}^n |\phi_j(\mathbf{x}_i)|.$$

Large values of  $I_j$  indicate that feature  $j$  tends to contribute strongly to predictions across the dataset.

In health insurance cost prediction, Shapley values can help identify whether variables such as smoking status, age, BMI, medical history, occupation, region or coverage level are driving predicted charges. The model-based interpretation caveat stated in Section 2.2.4 applies equally here: Shapley contributions reflect the fitted model's attributions, not causal effects. This distinction is particularly important when working with generated or synthetic health insurance data, where Shapley values explain the relationships learned from the dataset's generating structure rather than from real-world pricing processes.

The dissertation extends ordinary Shapley analysis by connecting it to Monte Carlo simulation. Instead of computing Shapley values only on the observed test set, the dissertation

also examines Shapley values for simulated policyholder profiles, concentrating on the high-cost region of the model-implied prediction distribution. Let  $\hat{Y} = \hat{m}(\mathbf{X})$  denote the predicted cost for a simulated input profile and let  $\phi_j(\mathbf{X})$  denote the Shapley contribution of feature  $j$ . For a high-cost threshold  $u$ , the conditional Shapley contribution of feature  $j$  is

$$\mathbb{E}[\phi_j(\mathbf{X}) \mid \hat{Y} > u].$$

This quantity answers a targeted actuarial question: among the profiles that the model regards as high-cost, which variables contribute most strongly to those predictions?

Using Monte Carlo samples  $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(M)}$ , the conditional Shapley contribution is estimated by the signed average  $\hat{\Phi}_{j,u}$  and the magnitude-based version  $\hat{A}_{j,u}$  defined in equations (2.2.16) and (2.2.17). This conditional interpretation moves beyond ordinary global feature ranking by focusing on the part of the prediction distribution that is most relevant for insurance risk analysis.

## 2.11 Research Gap

Supervised regression provides the predictive formulation, the [DNN](#) supplies a fitted non-linear prediction surface, Monte Carlo simulation propagates variation in the input profiles, and Shapley values provide feature-level attribution for fitted predictions. These components are not new methods in isolation.

Monte Carlo simulation has a long history in actuarial science and quantitative finance, where it is used primarily for loss reserving, aggregate loss distributions, solvency capital estimation and derivative pricing ([Glasserman, 2004](#)). In these applications the simulation target is typically a financial or insurance outcome variable, and the objective is to estimate a distributional quantity such as a quantile or expected shortfall. The use of Monte Carlo simulation to generate a synthetic input space and propagate it through a fitted prediction function, so that the model-implied predicted-cost distribution can be examined, is less standard in the health-insurance cost-prediction literature.

Shapley-based explanations have become widespread in applied machine learning ([Rozemberczki et al., 2022](#)), including insurance applications where [SHAP](#) is used to rank feature importance on observed test data ([Kaushik et al., 2022](#)). Conditional Shapley analysis, in which attributions are computed for profiles within a specific region of the predicted-cost distribution such as the simulated high-cost tail, is not a routine component of these analyses. The survey by [Rozemberczki et al. \(2022\)](#) catalogues a wide range of Shapley-value applications in machine learning but does not discuss conditioning Shapley attribution on simulated prediction regions.

At the prediction stage, several studies fit deep neural networks or comparable machine-

learning models to health-insurance cost data and evaluate predictive performance using standard regression metrics (Samiuddin et al., 2023; Drewe-Boss et al., 2022; Kaushik et al., 2022). These studies demonstrate that neural networks and related estimators can approximate the cost surface on policyholder-level attributes, but the analysis typically concludes at the point-prediction stage. Neither simulation over a synthetic input space nor conditional feature attribution beyond global importance is reported.

The research gap is therefore the limited integration of these three components for health insurance cost prediction. The literature supports each component individually but gives less attention to the combined question of how a fitted prediction model behaves across many simulated policyholder profiles, how the resulting model-implied predicted-cost distribution behaves, and which variables the fitted model attributes to high-cost simulated profiles.

The framework outlined in Section 1.8 addresses this gap by connecting the three components within a single empirical workflow (Figure 1). The concrete methodological workflow, distributional choices and implementation details are specified in Chapter 3.

The conceptual framework, theoretical foundations and research gap developed in this chapter provide the basis for the empirical methodology that follows. Chapter 3 translates these components into a concrete methodological workflow, specifying the benchmark dataset, preprocessing protocol, network architecture, Monte Carlo simulation design and Shapley-value implementation.

### 3 Methods and Data

This chapter translates the conceptual DNN, Monte Carlo and Shapley framework of Chapter 2 into a concrete empirical workflow. The analysis uses a publicly available health insurance benchmark dataset containing one million policyholder records, eleven raw predictors, and a continuous response variable, **charges**. The chapter proceeds sequentially: it first describes the data and the exploratory diagnostics that motivate the main modelling choices; it then specifies the architecture and training protocol of the feed-forward deep neural network that serves as the prediction function; and it presents the predictive and structural checks that must be satisfied before the network is used in the Monte Carlo and Shapley-value analyses. The Monte Carlo simulation design, a validation of the resulting input distribution that diagnoses its joint-distribution difference from the training data, a residual-augmented sensitivity analysis, and the Shapley-value implementation are treated in the sections that follow. The chapter closes with a formal statement of the modelling assumptions and interpretive boundaries.

#### 3.1 Data Source and Variable Structure

The raw dataset, taken from Kaggle under the title Insurance Data for Machine Learning ([Kaggle.com](https://www.kaggle.com/ahmedmohamed2020/insurance), 2023), contains  $n = 1\,000\,000$  observations and 12 original columns. Eleven of these are predictors; the remaining column, **charges**, is the continuous monetary outcome. Reference-category dummy encoding expands the categorical predictors, yielding a final model-ready input dimension of  $p = 22$ .

Let  $Y = \text{charges}$  denote the response, and let  $\mathbf{X}_{\text{raw}}$  be the vector of raw predictors before encoding. The predictor set consists of three numerical variables (**age**, **bmi**, **children**), two binary variables (**gender**, **smoker**), and six multi-level categorical variables (**region**, **medical\_history**, **family\_medical\_history**, **exercise\_frequency**, **occupation**, **coverage\_level**). Table 1 summarises the role, measurement scale, and model-ready representation of each variable. All categorical levels are well represented; none require merging.

Table 1: Variable dictionary and model-ready representation.

| Variable               | Role and type          | Description                                 | Model-ready representation                                 |
|------------------------|------------------------|---------------------------------------------|------------------------------------------------------------|
| charges                | Response, continuous   | Insurance charge; positive monetary outcome | Standardised during training; de-normalised for evaluation |
| age                    | Predictor, numerical   | Age (years), 18 to 65                       | One numerical input, standardised                          |
| bmi                    | Predictor, numerical   | Body mass index, 18.00 to 50.00             | One numerical input, standardised                          |
| children               | Predictor, count       | Number of dependants, 0 to 5                | One numerical input, standardised                          |
| gender                 | Predictor, binary      | Policyholder gender                         | One binary indicator (reference: female)                   |
| smoker                 | Predictor, binary      | Smoking status (yes/no)                     | One binary indicator (reference: no)                       |
| region                 | Predictor, categorical | Geographical region (4 levels)              | Three dummy indicators (reference: northeast)              |
| medical_history        | Predictor, categorical | Personal medical history                    | Three dummy indicators (reference: none / no condition)    |
| family_medical_history | Predictor, categorical | Family medical history                      | Three dummy indicators (reference: none / no condition)    |
| exercise_frequency     | Predictor, categorical | Exercise frequency (4 levels)               | Three dummy indicators (reference: rarely)                 |
| occupation             | Predictor, categorical | Occupation (4 levels)                       | Three dummy indicators (reference: unemployed)             |
| coverage_level         | Predictor, categorical | Coverage tier (3 levels)                    | Two dummy indicators (reference: basic)                    |

## 3.2 Data Audit, Partitioning and Preprocessing

### 3.2.1 Data audit

A comprehensive audit was performed before modelling. The variables `medical_history` and `family_medical_history` contain 250 762 and 250 404 missing values, respectively; all other variables are complete. Missing entries are treated as the reference category (“no reported condition”), and no observations are discarded. No exact duplicate records exist, all numerical values fall within plausible empirical ranges, and every categorical level is adequately represented.

### 3.2.2 Training, validation and test partition

The model-ready dataset is split into three mutually exclusive subsets according to the partition defined in equation (2.4.14). The split was generated by the SamplingForML platform [Blomerus \(2026\)](#). The training set contains 524 288 observations, the validation set contains 237 856 observations, and the test set contains 237 856 observations. The role of each partition is as described in Section 2.4.3.

### 3.2.3 Encoding and standardisation

Categorical variables are converted to numerical form using reference-category dummy encoding, as introduced in equation (2.4.2). For a categorical variable  $C$  with levels  $c_1, \dots, c_K$ , the dummy for level  $c_k$  is  $Z_{ik} = \mathbb{I}\{C_i = c_k\}$ ,  $k = 1, \dots, K-1$ , with  $c_K$  as the reference. Table 2 records the full encoding scheme.

Table 2: Variable encoding conventions and reference categories.

| Variable               | Encoded columns                              | Reference category           |
|------------------------|----------------------------------------------|------------------------------|
| gender                 | One indicator (male = 1)                     | Female                       |
| smoker                 | One indicator (yes = 1)                      | No                           |
| region                 | southwest, northwest, southeast              | Northeast                    |
| medical_history        | Heart_disease, High_blood_pressure, Diabetes | None / no reported condition |
| family_medical_history | Heart_disease, High_blood_pressure, Diabetes | None / no reported condition |
| exercise_frequency     | Occasionally, Frequently, Never              | Rarely                       |
| occupation             | Student, Blue_collar, White_collar           | Unemployed                   |
| coverage_level         | Standard, Premium                            | Basic                        |

All numerical predictors are standardised using the training-set means  $\hat{\mu}_{j,\text{train}}$  and standard deviations  $\hat{\sigma}_{j,\text{train}}$ :

$$\tilde{x}_{ij} = \frac{x_{ij} - \hat{\mu}_{j,\text{train}}}{\hat{\sigma}_{j,\text{train}}}. \quad (3.2.1)$$

For binary dummy variables that have zero variance on the training set, the standard deviation is set to one so that the column retains its original 0/1 coding. The response is standardised analogously during training:

$$\tilde{y}_i = \frac{y_i - \hat{\mu}_{Y,\text{train}}}{\hat{\sigma}_{Y,\text{train}}}, \quad (3.2.2)$$

where  $\hat{\mu}_{Y,\text{train}} \approx 16\,740$  and  $\hat{\sigma}_{Y,\text{train}} \approx 4\,416$ . Predictions are mapped back to the original monetary scale by

$$\hat{y}_i = \hat{\mu}_{Y,\text{train}} + \hat{\sigma}_{Y,\text{train}} \hat{\tilde{y}}_i. \quad (3.2.3)$$

Every preprocessing parameter is estimated from the training partition only and applied unchanged to the validation set, the test set, and the Monte Carlo simulated profiles. We denote this complete preprocessing procedure by  $T(\cdot)$ . The treatment of categorical variables during Monte Carlo simulation, including the requirement to simulate at the raw level before encoding, is described in Section 3.6.

### 3.2.4 Data-leakage prevention

Data leakage occurs when information from outside the training partition influences the model fitting or evaluation process, leading to overly optimistic performance estimates that do not generalise to new data. Three safeguards are enforced throughout the analysis to prevent leakage:

1. **Split before preprocessing.** The training, validation and test partitions are fixed before any preprocessing is applied. Standardisation parameters ( $\hat{\mu}_{j,\text{train}}, \hat{\sigma}_{j,\text{train}}$ ) and encoding schemes are estimated from the training set only.
2. **Training-only standardisation.** The validation and test sets are transformed using the training-set means and standard deviations (equations (3.2.1) and (3.2.2)). Monte Carlo simulated profiles are likewise preprocessed through the same training-derived transformation  $T(\cdot)$ .
3. **No test-set feedback.** The test partition is never consulted during hyperparameter selection or model training. It is evaluated only once, after the final network configuration is selected.

Additionally, no variable derived from **charges** is used as an input feature, preventing target leakage. Duplicate observations were checked during the data audit (Section 3.2), and no exact duplicate records were found. The unusually high predictive performance ( $R^2 = 0.9957$ ) is attributable to the strong simulated structure of the Kaggle benchmark dataset rather than to any leakage pathway. These protocols ensure that every reported performance metric reflects genuine out-of-sample predictive ability. The preprocessing procedure is audited in Annexure G.

## 3.3 Exploratory Diagnostics

Exploratory diagnostics are used to guide the preprocessing, modelling and simulation choices. The analysis focuses on four aspects: the marginal behaviour of the numerical variables, the balance of categorical variables, the empirical distribution of the response, and the dependence structure among the numerical predictors.

### 3.3.1 Numerical variables and the response

Table 3 reports descriptive statistics for the three numerical predictors (**age**, **bmi**, **children**) and the response variable (**charges**).

Table 3: Descriptive statistics and distributional shape for numerical variables.

| Variable        | Mean   | SD    | Min   | Q1     | Median | Q3     | Max    | $\hat{\gamma}_1$ | $\hat{\gamma}_2$ |
|-----------------|--------|-------|-------|--------|--------|--------|--------|------------------|------------------|
| <b>age</b>      | 41.50  | 13.86 | 18    | 29     | 41     | 53     | 65     | 0.001            | -1.201           |
| <b>bmi</b>      | 34.00  | 9.23  | 18.00 | 26.02  | 34.00  | 41.99  | 50.00  | -0.000           | -1.198           |
| <b>children</b> | 2.50   | 1.71  | 0     | 1      | 2      | 4      | 5      | 0.000            | -1.268           |
| <b>charges</b>  | 16 735 | 4 416 | 3 445 | 13 600 | 16 622 | 19 781 | 32 562 | 0.130            | -0.349           |

The three numerical predictors are essentially symmetric, with skewness values close to zero and negative excess kurtosis. This platykurtic behaviour is consistent with bounded, uniform-type distributions: the observations are spread relatively evenly across their supports rather than concentrated around their means. This directly motivates the use of uniform marginal distributions in the Monte Carlo simulation design. The empirical bounds **age**  $\in [18, 65]$ , **bmi**  $\in [18, 50]$ , and **children**  $\in \{0, \dots, 5\}$  also provide the support restrictions that will be enforced when synthetic profiles are generated.

The response variable **charges** has mean 16 735 and median 16 622; their close agreement indicates that the centre of the distribution is not strongly affected by extreme observations. The skewness of 0.130 indicates mild positive skewness, while the excess kurtosis of -0.349 confirms that the response is not heavy-tailed. No logarithmic transformation is therefore applied; the response is standardised during training for numerical stability and de-normalised for evaluation.

Figure 6 displays the empirical histograms. **age** and **bmi** are approximately flat across their bounded ranges, with mean and median nearly coincident. **children** is discrete and approximately balanced across its six integer values. **charges** is unimodal with the mean lying slightly to the right of the median, reflecting the mild positive skewness.

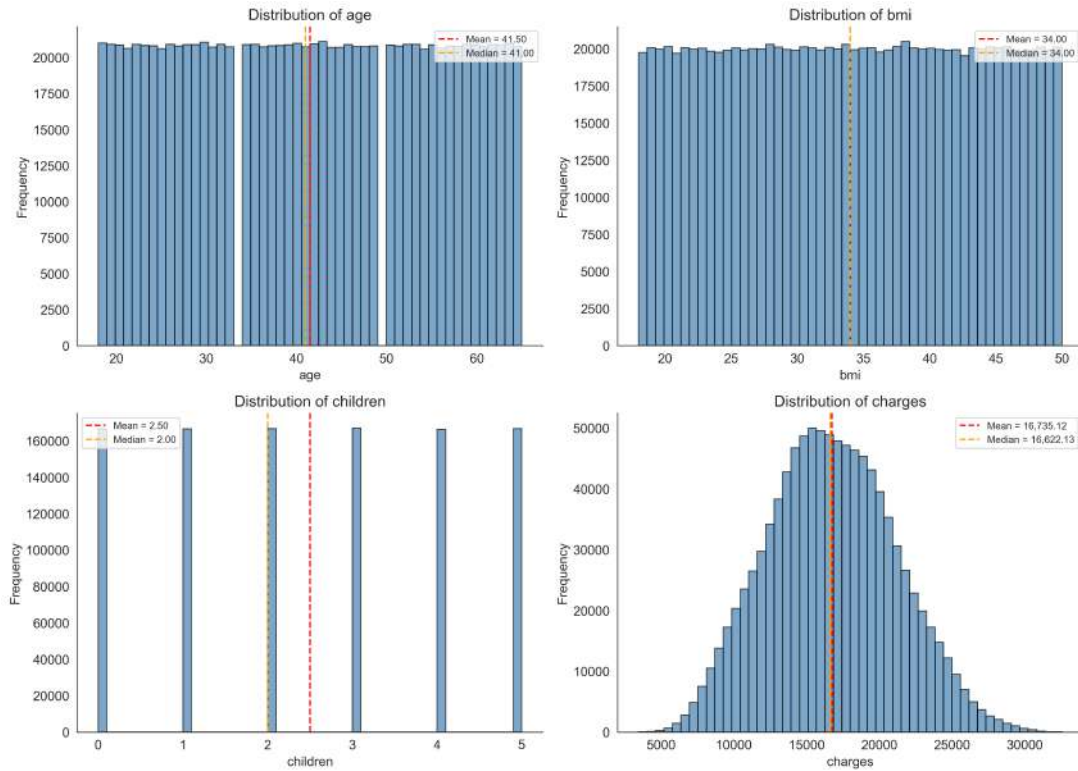


Figure 6: Empirical distributions of the numerical variables.

Selected percentiles of `charges` are reported in Table 4. The interquartile range is approximately 6 181, and the 90th, 95th, and 99th percentiles are 22 591, 24 224, and 27 016, respectively. These values serve as empirical scale references; in the later Monte Carlo and Shapley stages, high-cost profiles are defined using upper-tail quantiles of the simulated predicted-cost distribution, not directly from these observed percentiles.

Table 4: Selected percentiles of the response variable `charges`.

| Percentile | P1    | P5    | P10    | P25    | P50    | P75    | P90    | P95    | P99    |
|------------|-------|-------|--------|--------|--------|--------|--------|--------|--------|
| Value      | 7 520 | 9 558 | 10 947 | 13 600 | 16 622 | 19 781 | 22 591 | 24 224 | 27 016 |

### 3.3.2 Dependence structure

The Pearson correlation matrix for the four numerical variables is given in Table 5.

Table 5: Pearson correlation matrix for the numerical variables.

|          | age    | bmi    | children | charges |
|----------|--------|--------|----------|---------|
| age      | 1.000  | 0.001  | −0.001   | 0.063   |
| bmi      | 0.001  | 1.000  | −0.002   | 0.104   |
| children | −0.001 | −0.002 | 1.000    | 0.077   |
| charges  | 0.063  | 0.104  | 0.077    | 1.000   |

Pairwise correlations among the predictors are negligible ( $|r| \leq 0.002$ ), and the marginal correlations with **charges** are weak. This does not imply that the predictors are unimportant, the deep network can learn complex nonlinear interactions invisible to a linear correlation matrix, but it does provide empirical support for the product-marginal independence assumption used in the Monte Carlo simulation. The correlation heatmap in Figure 7 confirms the uniformly pale off-diagonal shading.

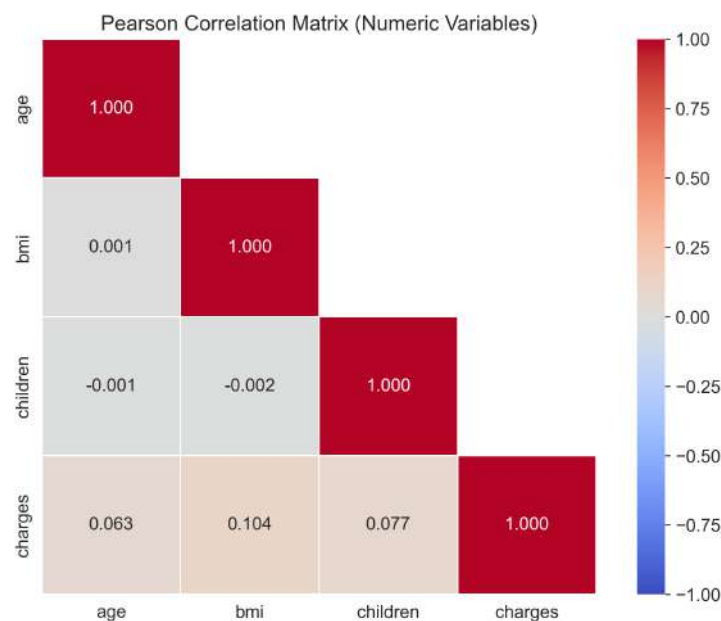
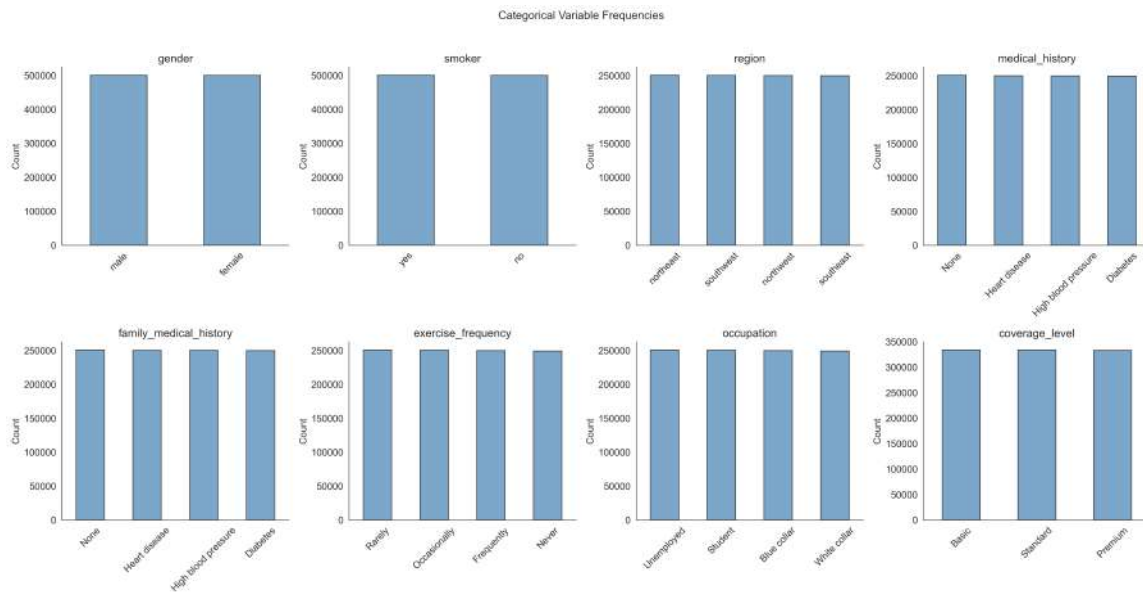


Figure 7: Pearson correlation heatmap for the numerical variables.

## 3.3.3 Categorical predictors and encoded features



(a) Frequency distributions of the eight categorical and binary predictor variables.

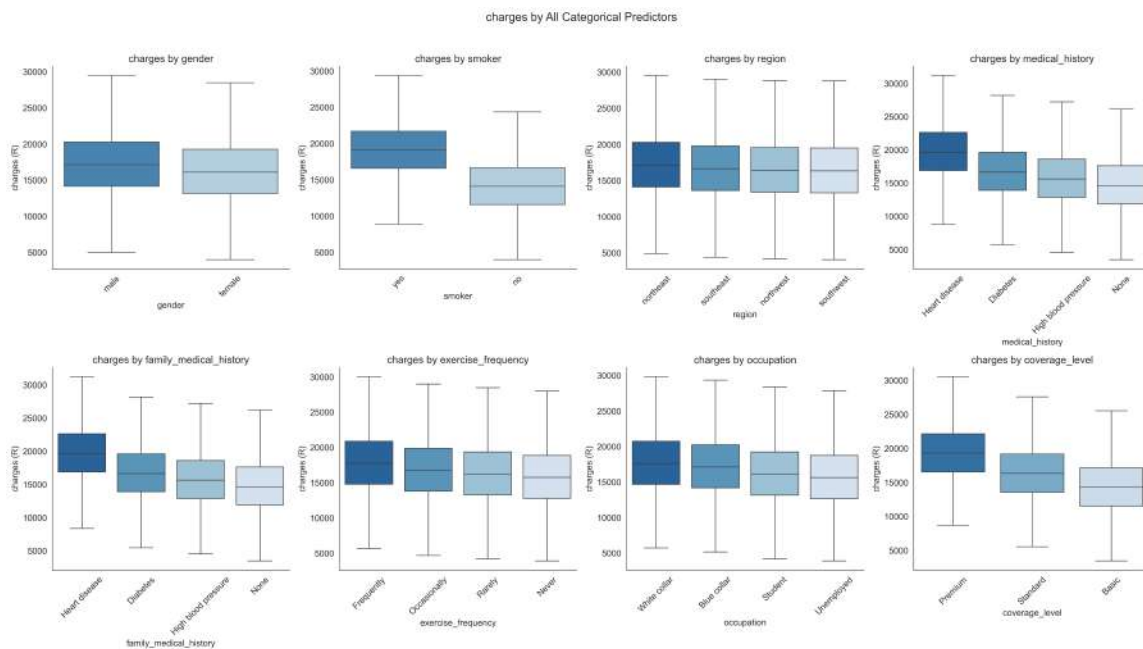
(b) Boxplots of **charges** conditioned on each categorical predictor variable.

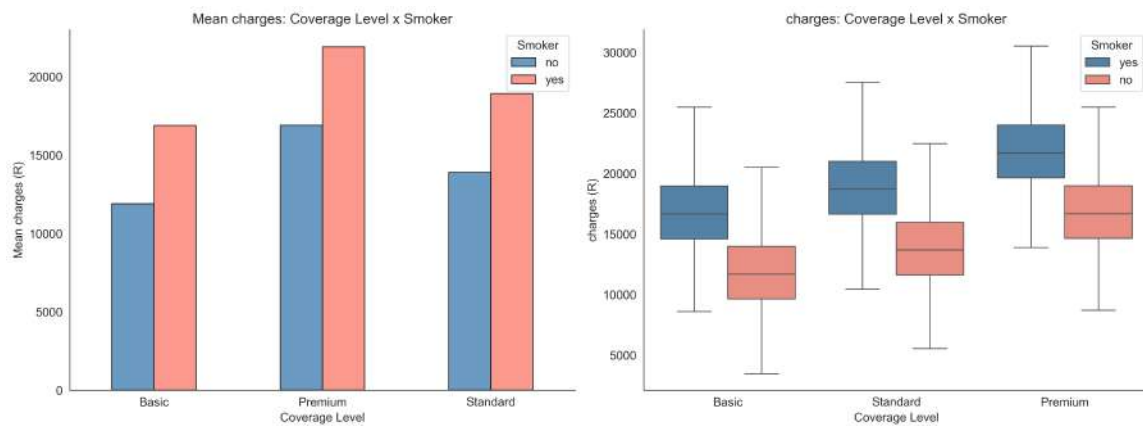
Figure 8: Categorical predictor diagnostics: frequency distributions and conditional boxplots.

Figure 8a confirms that every categorical and binary predictor is approximately uniformly distributed across its levels. The two binary variables (**gender** and **smoker**) each have close to 500 000 observations per level, the four-level variables (**region**, **medical\_history**, **family\_medical\_history**, **exercise\_frequency** and **occupation**) each have approximately 250 000 observations per level, and the three-level **coverage\_level** variable has

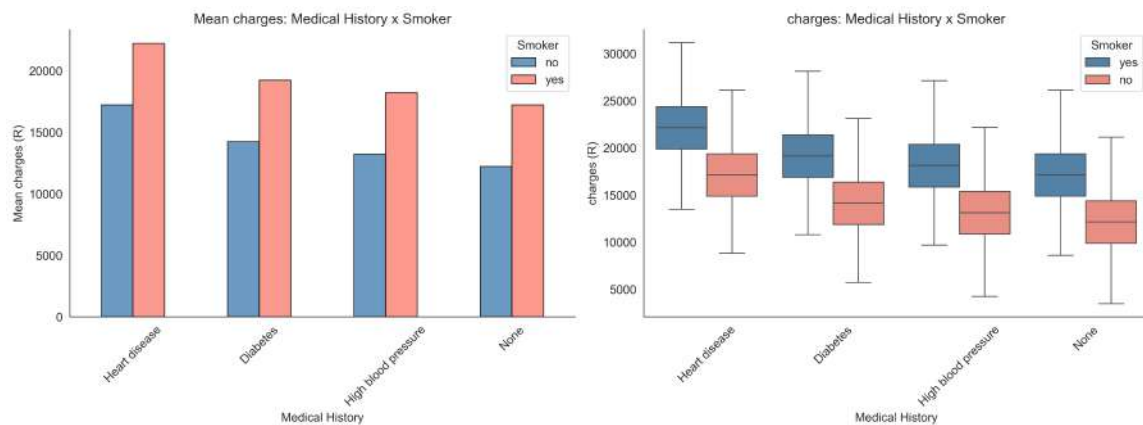
approximately 333 000 observations per tier. These balanced frequencies ensure that no category is underrepresented in the training data and that the fitted category proportions used in the Monte Carlo simulation are estimated with high precision.

Figure 8b displays the conditional distribution of charges for each categorical predictor. The **smoker** panel shows the clearest separation: smokers have a median near R19 000 compared to approximately R14 000 for non-smokers. The **coverage\_level** panel reveals a monotonic tier structure, with Premium policyholders having the highest median charges (approximately R20 000), followed by Standard (approximately R17 000) and Basic (approximately R15 000). Among **medical\_history** categories, Heart disease is associated with the highest median charges (approximately R20 000), followed by Diabetes, High blood pressure and None. The **family\_medical\_history** panel shows the same ordering, with Heart disease again yielding the highest charges. The **exercise\_frequency** panel shows a modest gradient, with Frequently exercising policyholders having marginally higher median charges than those who Never exercise. The **gender**, **region** and **occupation** panels display smaller unconditional differences, though males tend to have slightly higher charges than females across all panels.

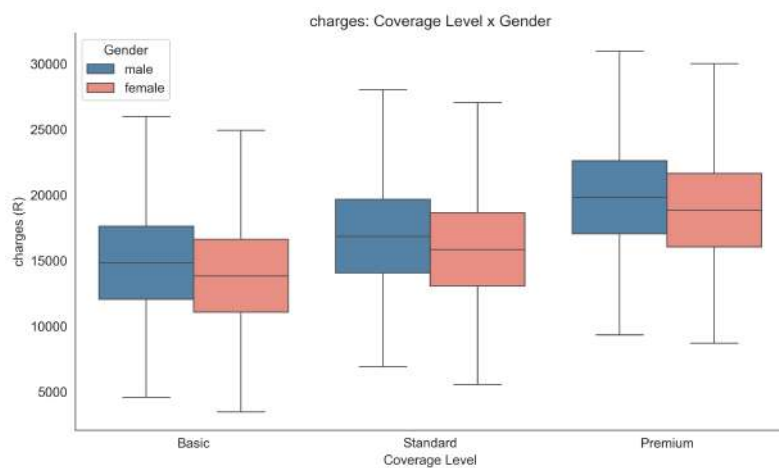
Two-way interactions were examined for **smoker**  $\times$  **coverage\_level**, **smoker**  $\times$  **medical\_history** and **coverage\_level**  $\times$  **gender** to assess whether the effect of one variable on charges depends on the level of another.



(a) `smoker × coverage_level`: mean charges (left) and boxplots (right).



(b) `smoker × medical_history`: mean charges (left) and boxplots (right).



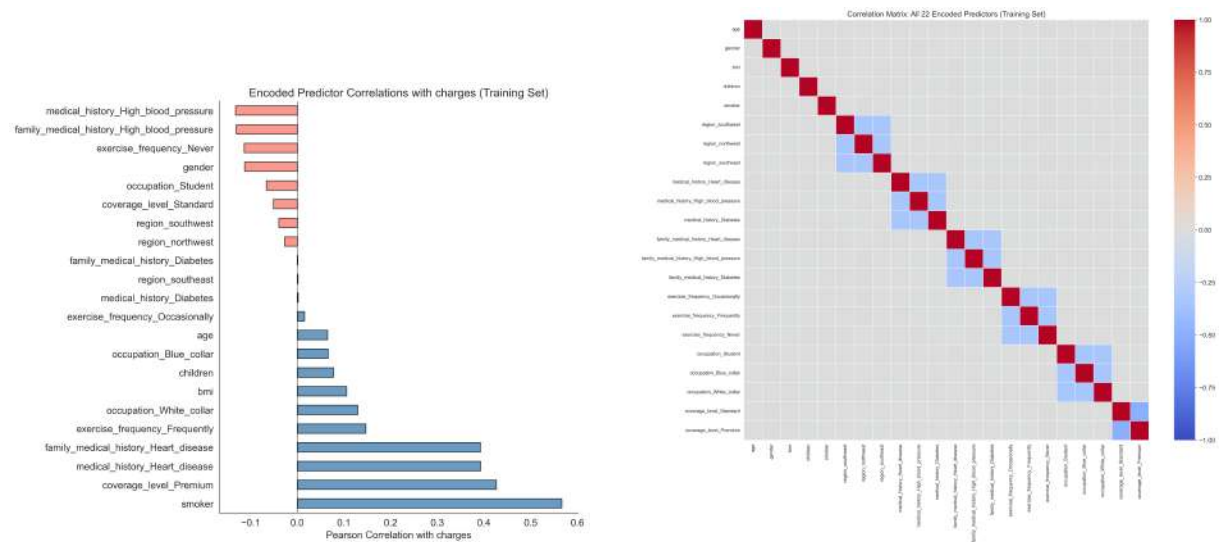
(c) `coverage_level × gender`: boxplots of charges.

Figure 9: Two-way interaction effects on `charges`.

Figure 9a shows that the smoker effect is present at every coverage tier but is amplified at higher coverage levels. At the Basic tier, the smoker, non-smoker difference in mean charges is approximately R5 000 (R17 000 versus R12 000), whereas at the Premium tier the difference widens to approximately R5 000-R6 000 on a higher base (R22 000 versus

R17 000). The boxplots confirm that smokers consistently occupy a higher distributional range within each tier. Figure 9b reveals a consistent smoker premium across all medical history categories. The largest combined effect occurs for smokers with Heart disease (mean charges approximately R22 500), while the lowest occurs for non-smokers with no reported condition (approximately R12 500). The smoker effect (approximately R5 000) is additive with the medical history effect, with the gap remaining relatively stable across all four levels. Figure 9c shows that males have slightly higher median charges than females at every coverage tier, with the gender gap widening modestly from Basic (approximately R1 000) to Premium (approximately R1 500). The interaction effects visible in Figure 9 support the use of a flexible DNN architecture capable of capturing nonlinear and interaction-based relationships.

After encoding, the Pearson correlations between the 22 encoded predictors and **charges** were computed on the training partition. The strongest linear associations are **smoker** ( $r = 0.567$ ), **coverage\_level\_Premium** ( $r = 0.427$ ), **medical\_history\_Heart\_disease** ( $r = 0.393$ ) and **family\_medical\_history\_Heart\_disease** ( $r = 0.393$ ).



(a) Pearson correlations of the 22 encoded predictors with **charges**, sorted by magnitude.

(b) Full  $22 \times 22$  Pearson correlation matrix of the encoded predictors.

Figure 10: Encoded predictor correlation analysis on the training partition.

Figure 10a ranks all 22 encoded predictors by their Pearson correlation with **charges**. The four strongest positive associations are **smoker** ( $r = 0.567$ ), **coverage\_level\_Premium** ( $r \approx 0.43$ ), **medical\_history\_Heart\_disease** ( $r \approx 0.39$ ) and **family\_medical\_history\_Heart\_disease** ( $r \approx 0.39$ ). A second tier of moderate positive associations includes **exercise\_frequency\_Frequently** ( $r \approx 0.15$ ), **occupation\_White-collar** ( $r \approx 0.12$ ), **bmi** ( $r \approx 0.10$ ), **children** ( $r \approx 0.08$ ) and **age** ( $r \approx 0.06$ ). On the negative side, the largest associations are **medical\_history\_High\_blood\_pressure** ( $r \approx -0.11$ ), **family\_medical\_history\_High\_blood\_pressure** ( $r \approx -0.10$ ), **exercise\_frequency\_Never** ( $r \approx -0.08$ ), and **gender** ( $r \approx -0.06$ ).

( $r \approx -0.09$ ) and **gender** ( $r \approx -0.08$ ). These negative values reflect the reference-category encoding: for example, a negative correlation for **medical\_history\_High\_blood\_pressure** indicates lower charges relative to the omitted reference category rather than a protective effect. Figure 10b shows the full inter-predictor correlation structure. The dominant feature is the block of negative correlations within each dummy group: dummy indicators derived from the same raw categorical variable are mechanically negatively correlated because only one indicator can be active at a time. Outside these within-group blocks, the off-diagonal correlations are uniformly near zero (grey shading), confirming the absence of problematic multicollinearity among predictors from different raw variables.

Split consistency was verified by overlaying kernel density estimates of **charges** and each numerical predictor across the training, validation and test partitions. Dummy-variable proportions were also compared across the three splits. In all cases the distributional characteristics are consistent, supporting the view that the partitioning procedure has not introduced systematic distributional shifts.

### 3.3.4 Partition consistency

Kernel density estimates of **charges** and of each numerical predictor were overlaid for the training, validation, and test partitions; the distributions are virtually indistinguishable. Proportions of each dummy variable were likewise compared, and no systematic shifts are present. The partitioning procedure therefore preserves the distributional characteristics of the data.

### 3.3.5 Decisions from the exploratory diagnostics

Four implementation decisions follow directly from the EDA:

1. Bounded numerical predictors are simulated under support restrictions that maintain their observed ranges.
2. Categorical variables are simulated at the raw-category level and only afterwards encoded.
3. The weak linear correlations among the numerical predictors, together with their uniform-like marginal shapes, support the product-marginal independence assumption in the Monte Carlo design as a faithful reflection of the benchmark's data-generating structure.
4. The response is standardised during training and de-normalised for evaluation; no further transformation is needed.

### 3.4 Network Specification and Training

The predictive component is a feed-forward **DNN** for supervised regression, following the formulation of equations (2.2.2) and (2.2.3). The network maps the model-ready input vector  $\mathbf{z}_i \in \mathbb{R}^{22}$  to a scalar prediction of the standardised response,

$$\hat{\tilde{y}}_i = f_{\hat{\theta}}(\mathbf{z}_i), \quad (3.4.1)$$

which is subsequently de-normalised to the original monetary scale via equation (3.2.3).

#### 3.4.1 Architecture

The network adopts the contracting funnel topology

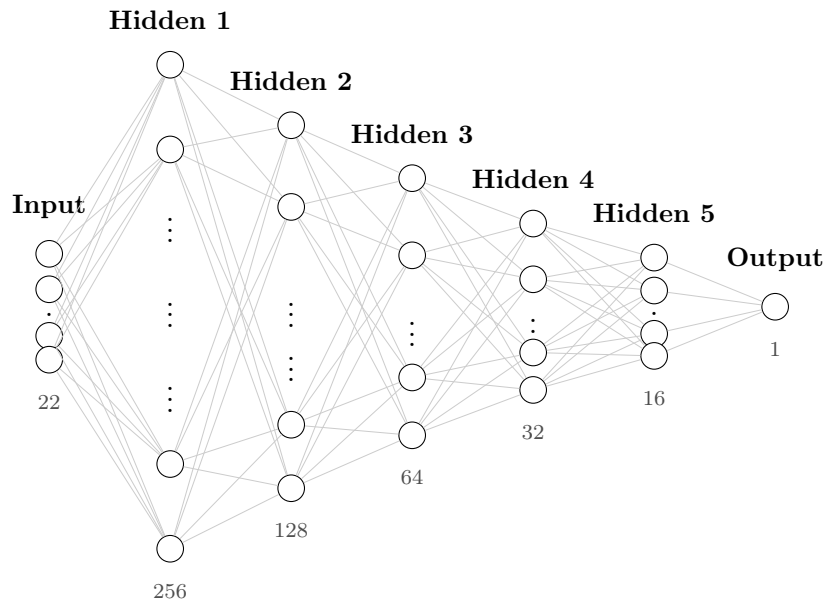


Figure 11: Funnel architecture of the deep neural network.

with the layer-wise recursion given by equation (2.4.2). All hidden layers use the **CELU** activation function (equation (2.4.9)); the output layer uses an identity mapping, as required for continuous regression. The full five-hidden-layer depth, including the final 16-unit layer, is retained because the validation diagnostics (Section 3.5) showed no evidence that removing it would improve out-of-sample performance.

The network is constructed without bias vectors. As discussed in Chapter 2, when all numerical inputs and the response are standardised the wide first hidden layer can approximate any necessary additive shift through the weight parameters alone, and the bias-free

specification is therefore not restrictive. The total number of trainable parameters is

$$\begin{aligned} N_{\text{par}} &= (22)(256) + (256)(128) + (128)(64) + (64)(32) + (32)(16) + (16)(1) \\ &= 49\,168. \end{aligned} \quad (3.4.2)$$

### 3.4.2 Forward and Backward Propagation

The operation of the contracting funnel specified in figure 11 can be described compactly by the forward- and backward-pass equations that underpin all gradient-based training. For a single input vector  $\mathbf{x}$  (with  $\mathbf{a}^{(0)} = \mathbf{x}$ ), the forward computation of layer  $l$  is

$$\mathbf{z}^{(l)} = (W^{(l)})^\top \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad W^{(l)} \in \mathbb{R}^{d_{l-1} \times d_l}, \quad \mathbf{b}^{(l)} \in \mathbb{R}^{d_l}, \quad (3.4.3)$$

$$\mathbf{a}^{(l)} = \sigma_l(\mathbf{z}^{(l)}), \quad (3.4.4)$$

where  $d_{l-1}$  and  $d_l$  are the widths of the adjacent layers. Because the input and response are standardised and the architecture omits bias vectors (Section 3.4), the bias term is not used; the forward pass simplifies to equation (2.4.13).

The network is trained by minimising the mean squared error loss  $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$ . Defining the gradient of the loss with respect to the output pre-activation as

$$\delta^{(L)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} = \hat{y} - y, \quad (3.4.5)$$

this error signal is propagated backward through the hidden layers. For layer  $l = L - 1, L - 2, \dots, 1$ , the recursion is

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = (W^{(l+1)})^\top \delta^{(l+1)} \odot \sigma'_l(\mathbf{z}^{(l)}), \quad (3.4.6)$$

where  $\odot$  denotes element-wise multiplication and  $\sigma'_l$  is the derivative of the activation function. For the CELU activation employed in every hidden layer (equation (2.4.9)), the derivative is smooth and non-zero for all inputs, ensuring stable gradient flow.

The gradient of the loss with respect to the weight matrix of layer  $l$  is

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \mathbf{a}^{(l-1)} (\delta^{(l)})^\top. \quad (3.4.7)$$

Because bias vectors are omitted, no bias-update term appears. The weight matrices are then updated by the NAdam optimiser (Section 3.4.3), which combines Nesterov-accelerated momentum with adaptive learning rates.

These equations are standard in the deep learning literature (Goodfellow et al., 2016) and are reproduced here for completeness; they make the training procedure explicit and justify the bias-free simplification. In practice, all gradients are computed automatically by the deep learning framework (PyTorch) through reverse-mode automatic differentiation, so the closed-form expressions are never coded directly. Their inclusion ensures that the reader can relate the chosen architecture, loss function, and activation to the mechanics of gradient-based optimisation.

### 3.4.3 Training configuration

The network is trained by minimising the mean squared error on the standardised response scale (equation (2.4.11)). Weights are updated with the NAdam optimiser (Dozat, 2016), described in Section 2.4.2. Validation performance is evaluated every  $v = 5$  epochs. For a fixed validation set, minimising the validation Mean Squared Error (MSE) is equivalent to maximising the validation  $R^2$ ; the dot-product  $R^2$  is therefore used as the operational model-selection criterion. The selected epoch is

$$e^* = \arg \max_{e \in \mathcal{E}_{\text{val}}} R^2_{\text{val}}(e), \quad \mathcal{E}_{\text{val}} = \{v, 2v, 3v, \dots\} \cap \{1, \dots, E_{\text{max}}\}. \quad (3.4.8)$$

Training stops early if the validation  $R^2$  does not improve for  $P = 10$  consecutive validation checks; the retained model is the checkpoint with the highest validation  $R^2$  observed. The test set is reserved exclusively for final out-of-sample evaluation.

On the original monetary scale, the standard  $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$  (equation (2.3.4)) is reported alongside the MAE and RMSE.

Table 6 summarises the complete architecture and training configuration.

Table 6: Feed-forward DNN architecture and training configuration.

| Component                    | Specification                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------|
| Input dimension              | $p = 22$ encoded predictor variables                                                                          |
| Hidden layer widths          | 256, 128, 64, 32, 16                                                                                          |
| Output dimension             | One scalar regression output                                                                                  |
| Architecture type            | Fully connected feed-forward network                                                                          |
| Network shape                | Contracting representation from 256 to 16 hidden units                                                        |
| Bias vectors                 | Omitted                                                                                                       |
| Hidden activation            | CELU (equation (2.4.9))                                                                                       |
| Output activation            | Identity mapping                                                                                              |
| Training response            | Standardised <b>charges</b>                                                                                   |
| Prediction scale             | De-normalised to the original monetary scale                                                                  |
| Training objective           | Mean squared error on the standardised response scale                                                         |
| Secondary evaluation metrics | MAE, RMSE and $R^2$ on the original response scale                                                            |
| Optimiser                    | NAdam (Dozat, 2016)                                                                                           |
| Momentum parameters          | $(\beta_1, \beta_2)$ selected by validation comparison from $\{(0.90, 0.999), (0.95, 0.999), (0.99, 0.999)\}$ |
| Learning rate                | Selected by validation comparison from $\{0.01, 0.001, 0.0005, 0.0001\}$                                      |
| Batch size                   | Selected by validation comparison from $\{64, 128, 256, 512\}$                                                |
| Maximum epochs               | $E_{\max} = 200$                                                                                              |
| Validation frequency         | Every $v = 5$ epochs                                                                                          |
| Model-selection criterion    | Dot-product validation $R^2$                                                                                  |
| Early stopping               | Patience $P = 10$ validation checks without $R^2$ improvement                                                 |
| Explicit regularisation      | Not used                                                                                                      |
| Dropout                      | Not used                                                                                                      |
| Random seed                  | $s = 42$ , fixed for reproducibility                                                                          |
| Trainable parameters         | 49 168 weights                                                                                                |

#### 3.4.4 Hyperparameter comparison

The main training hyperparameters that materially affect the fitted prediction surface are compared sequentially:

1. **Hidden activation:** CELU, GELU and Tanh are evaluated with learning rate 0.001, batch size 256, and Adam held fixed.
2. **Learning rate:** Values  $\{0.01, 0.001, 0.0005, 0.0001\}$  are tested with the best activation carried forward.
3. **Batch size:** Sizes  $\{64, 128, 256, 512\}$  are tested with the best activation and learning rate.
4. **Optimiser:** Adam and NAdam are compared with all previous best choices carried forward.
5. **Momentum parameters:**  $(\beta_1, \beta_2)$ . Since NAdam is selected in the previous step, its Nesterov-accelerated momentum parameters are compared over  $\{(0.90, 0.999), (0.95, 0.999), (0.99, 0.999)\}$ . The first coefficient  $\beta_1$  controls the decay of the gradient mean estimate; the second,  $\beta_2$ , controls the decay of the uncentred variance estimate (Dozat, 2016).

At each stage the best choice is carried forward and all other factors are held constant. Explicit  $\mathcal{L}_1$ ,  $\mathcal{L}_2$ , and dropout regularisation are not included; the adequacy of this design is assessed through validation curves, out-of-sample error metrics, and structural diagnostics<sup>[2]</sup>.

Algorithm 7 formalises the training procedure.

---

<sup>[2]</sup>In this context,  $\mathcal{L}_1$  and  $\mathcal{L}_2$  regularisation refer to adding penalty terms to the training objective. If  $\mathcal{L}_0(\theta)$  denotes the original training loss and  $\theta = (\theta_1, \dots, \theta_d)$  denotes the vector of trainable parameters, then  $\mathcal{L}_1$  regularisation uses

$$\mathcal{L}_1(\theta) = \mathcal{L}_0(\theta) + \lambda \sum_{j=1}^d |\theta_j|,$$

whereas  $\mathcal{L}_2$  regularisation uses

$$\mathcal{L}_2(\theta) = \mathcal{L}_0(\theta) + \lambda \sum_{j=1}^d \theta_j^2.$$

Here,  $\lambda \geq 0$  controls the strength of the penalty. The  $\mathcal{L}_1$  penalty encourages sparse parameter estimates, while the  $\mathcal{L}_2$  penalty shrinks parameters towards zero without necessarily setting them exactly equal to zero.

Table 7: DNN training algorithm.

**Algorithm 3.0: Feed-forward DNN training and model selection**

**Input:** model-ready training data  $\{(\mathbf{z}_i, \tilde{y}_i)\}_{i \in \mathcal{D}_{\text{train}}}$ , model-ready validation data  $\{(\mathbf{z}_i, \tilde{y}_i)\}_{i \in \mathcal{D}_{\text{val}}}$ , architecture specification  $(d_0, d_1, \dots, d_L)$ , activation function  $g(\cdot)$ , loss function  $\mathcal{L}_{\text{MSE}}$ , optimiser, learning rate  $\eta$ , momentum parameters  $(\beta_1, \beta_2)$ , batch size  $|\mathcal{B}|$ , maximum epochs  $E_{\text{max}} = 200$ , validation frequency  $v = 5$ , early stopping patience  $P = 10$ , random seed  $s = 42$ .

**Output:** fitted network weights  $\hat{\theta}$ , training history, selected epoch  $e^*$ .

1. Set the random seed to  $s$ .
2. Initialise network weights  $\theta^{(0)}$  using Kaiming uniform initialisation (He et al., 2015).
3. For epoch  $e = 1, \dots, E_{\text{max}}$ :
  - (a) Partition  $\mathcal{D}_{\text{train}}$  into random mini-batches of size  $|\mathcal{B}|$ .
  - (b) For each mini-batch  $\mathcal{B}$ , compute  $\mathcal{L}_{\text{MSE}}(\theta; \mathcal{B})$  and update  $\theta$  using the optimiser.
  - (c) Record training loss  $\mathcal{L}_{\text{train}}(e)$ .
  - (d) If  $e \bmod v = 0$ : evaluate validation loss  $\mathcal{L}_{\text{val}}(e)$  and dot-product  $R^2$  on both partitions. If validation  $R^2$  improves, checkpoint the model weights; otherwise, increment the patience counter.
  - (e) If the patience counter reaches  $P$ , terminate training.
4. Select  $e^* = \arg \max_{e \in \mathcal{E}_{\text{val}}} R_{\text{val}}^2(e)$ .
5. Retain the network weights  $\hat{\theta} = \theta^{(e^*)}$ .
6. De-normalise predictions and evaluate on the held-out test set using  $R^2$ , RMSE and MAE on the original monetary scale.
7. Save  $\hat{\theta}$ , architecture specification, standardisation parameters, training history, and all evaluation outputs.

**3.4.5 Benchmark Generalised Linear Model**

A classical actuarial benchmark is provided by a generalised linear model (GLM) with a gamma distribution and logarithmic link function (Dobson and Barnett, 2008; Denuit et al., 2007). For a policyholder with model-ready input vector  $\mathbf{z}_i \in \mathbb{R}^{22}$  (the same encoded predictors used by the DNN), the GLM assumes

$$Y_i \sim \text{Gamma}(\mu_i, \phi), \quad \log(\mu_i) = \beta_0 + \sum_{j=1}^{22} \beta_j z_{ij}, \quad (3.4.9)$$

where  $\mu_i = \mathbb{E}[Y_i \mid \mathbf{z}_i]$  and  $\phi$  is the dispersion parameter. The gamma distribution is a natural choice for positive, continuous, right-skewed monetary outcomes such as

insurance charges, and the logarithmic link function ensures that the predicted mean  $\mu_i = \exp(\beta_0 + \sum_j \beta_j z_{ij})$  is always positive (Denuit et al., 2007).

The model includes no interaction terms or non-linear transformations. It is fitted to the training partition by maximum likelihood estimation. The log-likelihood for the gamma GLM is

$$\ell(\boldsymbol{\beta}, \phi) = \sum_{i=1}^n \left[ \frac{1}{\phi} \left( -\frac{y_i}{\mu_i} - \log \mu_i \right) + \left( \frac{1}{\phi} - 1 \right) \log y_i - \log \Gamma\left(\frac{1}{\phi}\right) - \frac{1}{\phi} \log \phi \right], \quad (3.4.10)$$

where  $\mu_i = \exp(\mathbf{z}_i^\top \boldsymbol{\beta})$  and  $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_{22})^\top$ . The parameter vector  $\boldsymbol{\beta}$  is estimated by iteratively reweighted least squares (IRLS), which solves the score equations

$$\sum_{i=1}^n \frac{(y_i - \mu_i)}{\text{Var}(Y_i)} \frac{\partial \mu_i}{\partial \eta_i} z_{ij} = 0, \quad j = 0, 1, \dots, 22, \quad (3.4.11)$$

where  $\eta_i = \log(\mu_i)$  is the linear predictor and  $\text{Var}(Y_i) = \phi \mu_i^2$  under the gamma family. Each IRLS iteration computes a weighted least squares regression of an adjusted dependent variable on the design matrix, with weights inversely proportional to the variance function evaluated at the current fitted values (Dobson and Barnett, 2008).

Algorithm 8 formalises the fitting procedure.

Table 8: GLM fitting algorithm.

---

**Algorithm 3.0c: Gamma GLM with log link**

---

**Input:** model-ready training data  $\{(\mathbf{z}_i, y_i)\}_{i \in \mathcal{D}_{\text{train}}}$ , distribution family (gamma), link function (log).

**Output:** fitted coefficient vector  $\hat{\boldsymbol{\beta}}$ , dispersion estimate  $\hat{\phi}$ .

1. Initialise the linear predictor:  $\eta_i^{(0)} = \log(\bar{y}_{\text{train}})$  for all  $i$ .
  2. Repeat until convergence:
    - (a) Compute fitted values:  $\mu_i^{(t)} = \exp(\eta_i^{(t)})$ .
    - (b) Compute working weights:  $w_i^{(t)} = \mu_i^{(t)2} / (\phi \mu_i^{(t)2}) = 1/\phi$ .
    - (c) Compute the adjusted dependent variable:  $\tilde{y}_i^{(t)} = \eta_i^{(t)} + (y_i - \mu_i^{(t)})/\mu_i^{(t)}$ .
    - (d) Solve the weighted least squares problem:  $\boldsymbol{\beta}^{(t+1)} = (\mathbf{Z}^\top \mathbf{W}^{(t)} \mathbf{Z})^{-1} \mathbf{Z}^\top \mathbf{W}^{(t)} \tilde{\mathbf{y}}^{(t)}$ .
    - (e) Update the linear predictor:  $\eta_i^{(t+1)} = \mathbf{z}_i^\top \boldsymbol{\beta}^{(t+1)}$ .
  3. Estimate the dispersion parameter:  $\hat{\phi} = \frac{1}{n-p-1} \sum_{i=1}^n ((y_i - \hat{\mu}_i)/\hat{\mu}_i)^2$ .
  4. Predict on the original monetary scale:  $\hat{y}_i = \exp(\mathbf{z}_i^\top \hat{\boldsymbol{\beta}})$ .
  5. Evaluate  $R^2$ , RMSE and MAE on the training, validation and test partitions.
- 

No hyperparameter tuning is applied to the GLM; its role as a benchmark was motivated

in Section 2.3.

### 3.4.6 Tree-Based Benchmarks

Two tree-based ensemble methods are included alongside the GLM to provide a more comprehensive model-class comparison. Both models use the same 22 encoded features and 60/20/20 partition as the DNN and GLM.

#### Random Forest

A RF regressor (Breiman, 2001) is fitted with 500 trees, maximum depth unrestricted, minimum samples per leaf of 5, and `random_state=42`. RF aggregates predictions across decorrelated decision trees, reducing variance through bootstrap aggregation (bagging) while maintaining low bias.

The RF constructs an ensemble of  $B$  regression trees, each grown on a bootstrap sample of the training data. At every candidate split the algorithm considers only a random subset of  $m$  out of  $p$  predictors, thereby decorrelating the individual trees and reducing the variance of the averaged prediction. For a regression tree  $T_b$  grown on bootstrap sample  $\mathcal{D}_b^*$ , the prediction for input  $\mathbf{z}$  is  $\hat{y}_b(\mathbf{z}) = T_b(\mathbf{z})$ . The RF prediction is the average across all  $B$  trees:

$$\hat{f}_{\text{RF}}(\mathbf{z}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{z}). \quad (3.4.12)$$

Each tree  $T_b$  is grown by recursively partitioning the feature space. At each internal node, the algorithm selects a random subset of  $m = \lfloor p/3 \rfloor$  predictors (the default for regression) and chooses the split variable  $j^*$  and split point  $s^*$  that minimise the weighted sum of squared residuals in the resulting child nodes:

$$(j^*, s^*) = \arg \min_{j \in \mathcal{S}, s} \left[ \sum_{i: z_{ij} \leq s} (y_i - \bar{y}_L)^2 + \sum_{i: z_{ij} > s} (y_i - \bar{y}_R)^2 \right], \quad (3.4.13)$$

where  $\mathcal{S} \subset \{1, \dots, p\}$  with  $|\mathcal{S}| = m$  is the randomly selected predictor subset, and  $\bar{y}_L$  and  $\bar{y}_R$  denote the sample means in the left and right child nodes respectively. Trees are grown until each terminal node contains at most 5 observations.

The variance reduction mechanism of the RF can be understood through the decomposition of the variance of the ensemble average. If individual tree predictions have variance  $\sigma^2$  and pairwise correlation  $\rho$ , then

$$\text{Var}(\hat{f}_{\text{RF}}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2. \quad (3.4.14)$$

As  $B$  increases, the second term vanishes, and the ensemble variance is governed by  $\rho\sigma^2$ . By restricting splits to a random predictor subset, the RF reduces  $\rho$  below the value obtained by standard bagging (which uses all  $p$  predictors at every split), yielding a lower ensemble variance (Hastie et al., 2009).

Algorithm 9 formalises the training procedure.

Table 9: Random Forest training algorithm.

---

**Algorithm 3.0a: Random Forest regression**

---

**Input:** model-ready training data  $\{(\mathbf{z}_i, y_i)\}_{i \in \mathcal{D}_{\text{train}}}$ , number of trees  $B = 500$ , number of candidate predictors per split  $m = \lfloor p/3 \rfloor$ , minimum samples per leaf  $n_{\min} = 5$ , random seed  $s = 42$ .

**Output:** ensemble of fitted trees  $\{T_1, \dots, T_B\}$ .

1. Set the random seed to  $s$ .
  2. For  $b = 1, \dots, B$ :
    - (a) Draw a bootstrap sample  $\mathcal{D}_b^*$  of size  $n_{\text{train}}$  with replacement from  $\mathcal{D}_{\text{train}}$ .
    - (b) Grow an unpruned regression tree  $T_b$  on  $\mathcal{D}_b^*$ :
      - (i) At each node, select a random subset  $\mathcal{S} \subset \{1, \dots, p\}$  with  $|\mathcal{S}| = m$ .
      - (ii) Choose the split variable  $j^*$  and split point  $s^*$  from  $\mathcal{S}$  that minimise the weighted residual sum of squares (equation (3.4.13)).
      - (iii) Split the node into two child nodes and repeat until each terminal node contains at most  $n_{\min}$  observations.
    - (c) Assign the sample mean of the response values in each terminal node as the node prediction.
  3. For a new input  $\mathbf{z}$ , predict  $\hat{f}_{\text{RF}}(\mathbf{z}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{z})$ .
  4. Evaluate  $R^2$ , RMSE and MAE on the training, validation and test partitions on the original monetary scale.
- 

Table 10 summarises the RF configuration.

Table 10: Random Forest configuration.

| Component                              | Specification                                      |
|----------------------------------------|----------------------------------------------------|
| Number of trees ( $B$ )                | 500                                                |
| Candidate predictors per split ( $m$ ) | $\lfloor p/3 \rfloor = 7$                          |
| Maximum depth                          | Unrestricted                                       |
| Minimum samples per leaf               | 5                                                  |
| Bootstrap sampling                     | With replacement, sample size = $n_{\text{train}}$ |
| Split criterion                        | Mean squared error (equation (3.4.13))             |
| Hyperparameter tuning                  | Not applied (default values)                       |
| Random seed                            | 42                                                 |
| Implementation                         | <code>scikit-learn RandomForestRegressor</code>    |

## XGBoost

A gradient-boosted tree regressor is fitted using the [XGBoost](#) implementation ([Chen and Guestrin, 2016](#)) with 1000 boosting rounds, a learning rate of 0.05, maximum depth of 8, and early stopping on the validation set with a patience of 50 rounds. [XGBoost](#) constructs an additive ensemble of shallow trees via gradient boosting, using approximate histogram-based splitting for computational efficiency.

Gradient boosting builds an additive model in a stage-wise fashion. The ensemble prediction after  $M$  boosting rounds is

$$\hat{f}_M(\mathbf{z}) = \sum_{m=0}^M \eta h_m(\mathbf{z}), \quad (3.4.15)$$

where  $h_0(\mathbf{z}) = \bar{y}$  is the initial constant prediction (the training-set mean),  $h_m$  is the regression tree fitted at round  $m$ , and  $\eta \in (0, 1]$  is the learning rate (shrinkage parameter) that scales each tree's contribution.

At each boosting round  $m$ , [XGBoost](#) fits a new tree  $h_m$  to minimise a regularised objective. For the squared-error loss used in regression, the objective at round  $m$  is

$$\mathcal{O}^{(m)} = \sum_{i=1}^n \ell(y_i, \hat{f}_{m-1}(\mathbf{z}_i) + h_m(\mathbf{z}_i)) + \Omega(h_m), \quad (3.4.16)$$

where  $\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$  is the squared-error loss and  $\Omega(h_m)$  is a regularisation penalty

on the complexity of tree  $h_m$ . The regularisation term is defined as

$$\Omega(h) = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2, \quad (3.4.17)$$

where  $J$  is the number of terminal nodes (leaves),  $w_j$  is the predicted value in leaf  $j$ ,  $\gamma \geq 0$  penalises additional leaves, and  $\lambda \geq 0$  applies  $\mathcal{L}_2$  shrinkage to the leaf weights.

The key computational innovation of **XGBoost** is the use of a second-order Taylor expansion of the loss around the current prediction  $\hat{f}_{m-1}(\mathbf{z}_i)$ . Define the first- and second-order gradient statistics for observation  $i$ :

$$g_i = \left. \frac{\partial \ell(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=\hat{f}_{m-1}(\mathbf{z}_i)}, \quad h_i = \left. \frac{\partial^2 \ell(y_i, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=\hat{f}_{m-1}(\mathbf{z}_i)}. \quad (3.4.18)$$

For the squared-error loss,  $g_i = \hat{f}_{m-1}(\mathbf{z}_i) - y_i$  (the negative residual) and  $h_i = 1$ . With these gradient statistics, the approximate objective simplifies to

$$\tilde{\mathcal{O}}^{(m)} = \sum_{j=1}^J \left[ G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma J, \quad (3.4.19)$$

where  $G_j = \sum_{i \in I_j} g_i$  and  $H_j = \sum_{i \in I_j} h_i$  are the sums of gradient and Hessian statistics over the observations  $I_j$  assigned to leaf  $j$ . The optimal leaf weight and the corresponding optimal objective value are

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \tilde{\mathcal{O}}^* = -\frac{1}{2} \sum_{j=1}^J \frac{G_j^2}{H_j + \lambda} + \gamma J. \quad (3.4.20)$$

To evaluate a candidate split that partitions observations in a node into left ( $I_L$ ) and right ( $I_R$ ) child nodes, **XGBoost** computes the gain:

$$\text{Gain} = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma, \quad (3.4.21)$$

where  $G_L = \sum_{i \in I_L} g_i$ ,  $G_R = \sum_{i \in I_R} g_i$ ,  $H_L = \sum_{i \in I_L} h_i$ , and  $H_R = \sum_{i \in I_R} h_i$ . A split is made only if  $\text{Gain} > 0$ ; the regularisation parameter  $\gamma$  acts as a minimum-gain threshold that prunes unprofitable splits ([Chen and Guestrin, 2016](#)).

Algorithm 11 formalises the training procedure.

Table 11: XGBoost training algorithm.

**Algorithm 3.0b: XGBoost gradient-boosted tree regression**

**Input:** model-ready training data  $\{(\mathbf{z}_i, y_i)\}_{i \in \mathcal{D}_{\text{train}}}$ , model-ready validation data  $\{(\mathbf{z}_i, y_i)\}_{i \in \mathcal{D}_{\text{val}}}$ , maximum boosting rounds  $M = 1000$ , learning rate  $\eta = 0.05$ , maximum tree depth  $d_{\text{max}} = 8$ ,  $\mathcal{L}_2$  regularisation  $\lambda$ , minimum split gain  $\gamma$ , early stopping patience  $P = 50$  rounds, random seed  $s = 42$ .

**Output:** fitted additive ensemble  $\hat{f}_{M^*}(\mathbf{z})$ , selected round  $M^*$ .

1. Set the random seed to  $s$ .
2. Initialise the prediction:  $\hat{f}_0(\mathbf{z}_i) = \bar{y}_{\text{train}}$  for all  $i$ .
3. For boosting round  $m = 1, \dots, M$ :
  - (a) Compute gradient statistics  $g_i$  and  $h_i$  for each training observation using equation (3.4.18).
  - (b) Grow a regression tree  $h_m$  of maximum depth  $d_{\text{max}}$ :
    - (i) At each node, evaluate all candidate split points across all  $p$  predictors.
    - (ii) Select the split  $(j^*, s^*)$  that maximises the gain (equation (3.4.21)).
    - (iii) Split only if  $\text{Gain} > 0$ ; otherwise the node becomes a leaf.
    - (iv) Assign optimal leaf weights  $w_j^* = -G_j / (H_j + \lambda)$  (equation (3.4.20)).
  - (c) Update predictions:  $\hat{f}_m(\mathbf{z}_i) = \hat{f}_{m-1}(\mathbf{z}_i) + \eta h_m(\mathbf{z}_i)$ .
  - (d) Evaluate validation loss  $\mathcal{L}_{\text{val}}(m)$ . If the validation loss has not improved for  $P$  consecutive rounds, terminate training.
4. Select the round  $M^* = \arg \min_{m \in \{1, \dots, M\}} \mathcal{L}_{\text{val}}(m)$  and retain the ensemble up to that round.
5. Evaluate  $R^2$ , [RMSE](#) and [MAE](#) on the training, validation and test partitions on the original monetary scale.

Table 12 summarises the [XGBoost](#) configuration.

Table 12: XGBoost configuration.

| Component                                    | Specification                                  |
|----------------------------------------------|------------------------------------------------|
| Maximum boosting rounds ( $M$ )              | 1 000                                          |
| Learning rate ( $\eta$ )                     | 0.05                                           |
| Maximum tree depth ( $d_{\max}$ )            | 8                                              |
| Booster                                      | <code>gbtree</code> (tree-based)               |
| Objective                                    | <code>reg:squarederror</code>                  |
| Split evaluation                             | Approximate histogram-based                    |
| $\mathcal{L}_2$ regularisation ( $\lambda$ ) | Default (1.0)                                  |
| Minimum split gain ( $\gamma$ )              | Default (0.0)                                  |
| Early stopping patience                      | 50 rounds on validation loss                   |
| Hyperparameter tuning                        | Not applied (default values)                   |
| Random seed                                  | 42                                             |
| Implementation                               | <code>xgboost</code> <code>XGBRegressor</code> |

Hyperparameters for both tree-based models are set to commonly recommended defaults rather than tuned via cross-validation, paralleling the no-tuning treatment of the [GLM](#). This ensures that the comparison isolates model-class differences rather than tuning effort. All four models are evaluated using  $R^2$ , [RMSE](#) and [MAE](#) on the original monetary scale across all three partitions.

### 3.5 Predictive and Structural Diagnostics

The predictive and structural diagnostics described below serve a single purpose: to verify that the fitted [DNN](#) is sufficiently stable and well-behaved to act as the prediction surface in the subsequent Monte Carlo and Shapley-value analyses.

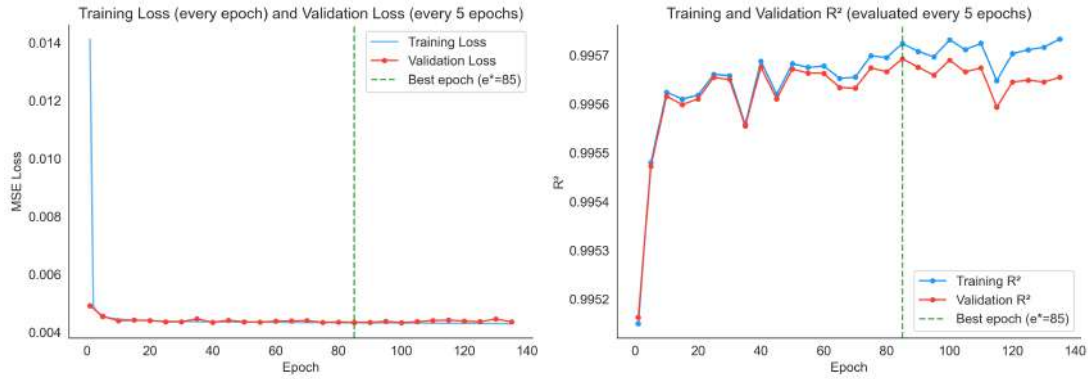
All predictive performance metrics are computed on the original monetary scale after de-normalising the fitted predictions. The metrics are the [MAE](#), [RMSE](#) and  $R^2$  defined in equations (2.3.3), (2.3.2) and (2.3.4). For observation  $i$  the fitted prediction is denoted  $\hat{y}_i$  and the residual is

$$r_i = y_i - \hat{y}_i. \quad (3.5.1)$$

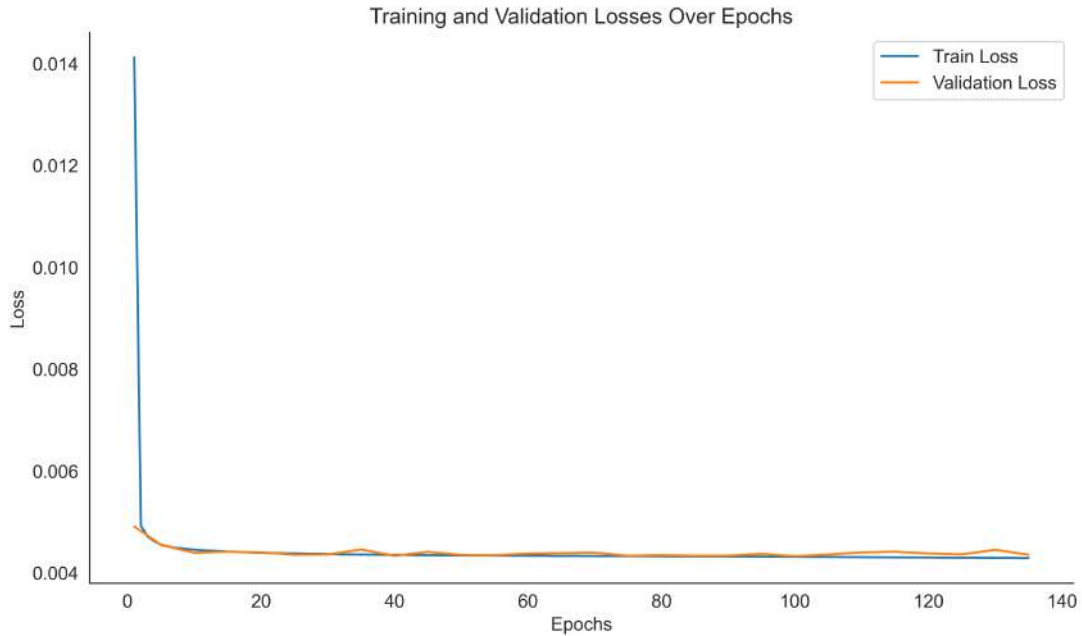
Residuals are computed separately for the training, validation and test partitions.

### 3.5.1 Predictive error assessment

Predictive error is assessed by comparing observed charges with fitted predictions on the original response scale. The training partition is used to evaluate in-sample fit, the validation partition to guide model-selection decisions, and the test partition to provide a final out-of-sample evaluation. The three scalar metrics are interpreted jointly: a suitable model should exhibit consistent performance across the validation and test partitions, without a large deterioration relative to the training partition.



(a) Training loss (every epoch) and validation loss and  $R^2$  (every 5 epochs).



(b) Training and validation MSE losses over epochs for the final model.

Figure 12: Learning curves for the fitted DNN.

Figure 12a presents the learning curves for the fitted DNN. In the left panel, the training MSE loss drops sharply from approximately 0.014 at the first epoch to roughly 0.005 by epoch 5, after which it decreases gradually and stabilises near 0.0043. The validation MSE loss tracks the training loss closely throughout, with no visible divergence between

the two curves. The selected epoch  $e^* = 85$  (green dashed line) falls in the region where both losses have plateaued. In the right panel, both training and validation  $R^2$  values rise rapidly from approximately 0.9951 to a plateau near 0.9957, with the gap between them remaining negligible. Figure 12b provides a complementary view: the training loss (blue) exhibits a single spike at epoch 1, consistent with random weight initialisation, before converging rapidly, and the validation loss (orange) overlaps with it from epoch 20 onwards. The absence of any sustained divergence confirms stable convergence without overfitting.

### 3.5.2 Actual versus predicted diagnostics

Two graphical summaries are used to compare observed charges with fitted predictions. First, the observed values  $y_i$  are plotted directly against the fitted values  $\hat{y}_i$  with a  $45^\circ$  reference line. Second, observations are sorted by their observed charge, and the actual and predicted values are displayed as overlaid curves; a well-fitted model produces a predicted curve that closely tracks the actual curve across the full range of the response. Both diagnostics are produced separately for the training, validation and test partitions, and together they indicate whether the fitted model behaves differently in low-, central- and high-cost regions.

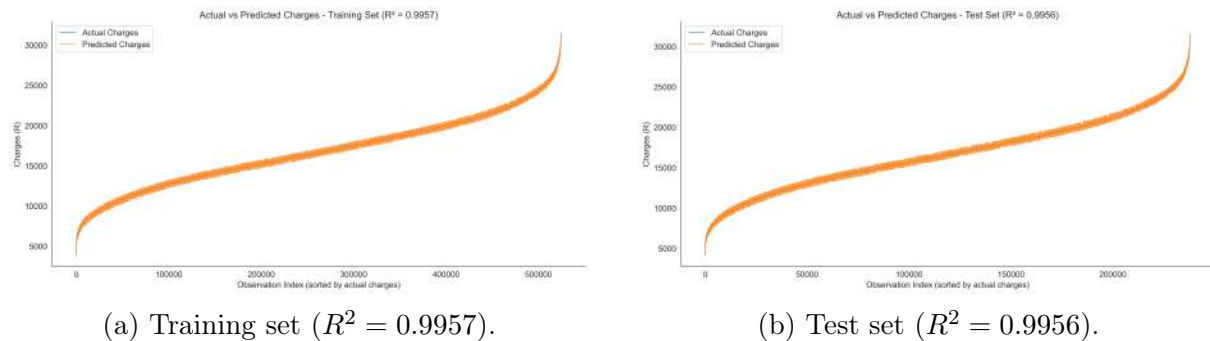


Figure 13: Sorted actual versus predicted insurance charges on the training and test sets.

Figure 13 shows the sorted actual versus predicted charges on both partitions. In the training set (Figure 13a), the predicted line (orange) tracks the actual line (blue) so closely that the two are nearly indistinguishable across the full range of the response, from approximately R4000 at the lower end to R32000 at the upper end. A modest increase in prediction scatter is visible in the upper tail above R25000, but the deviations remain small relative to the response scale. The test set (Figure 13b) displays a virtually identical pattern. The negligible drop from training  $R^2 = 0.9957$  to test  $R^2 = 0.9956$  confirms that the model generalises to unseen observations without meaningful performance degradation.

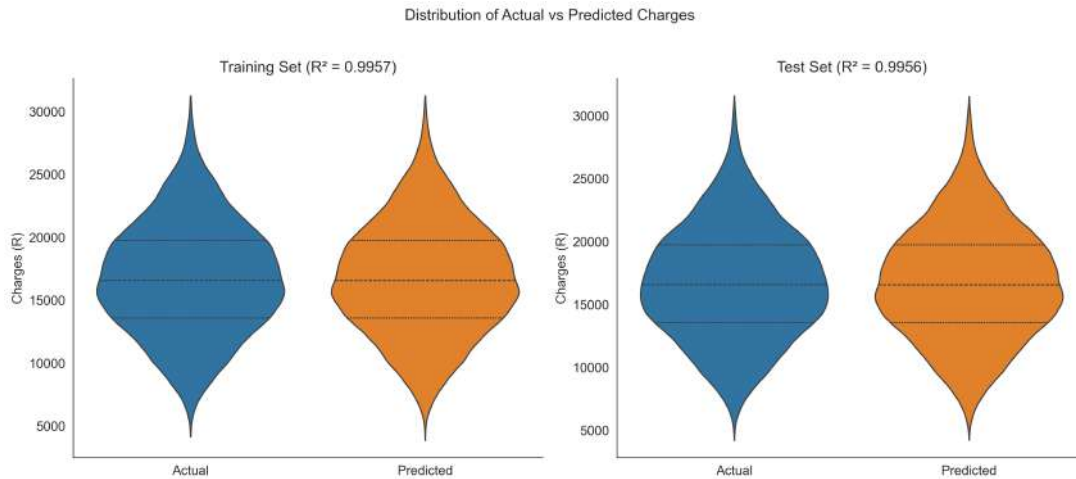


Figure 14: Violin plots comparing the distribution of actual and predicted charges on the training and test sets.

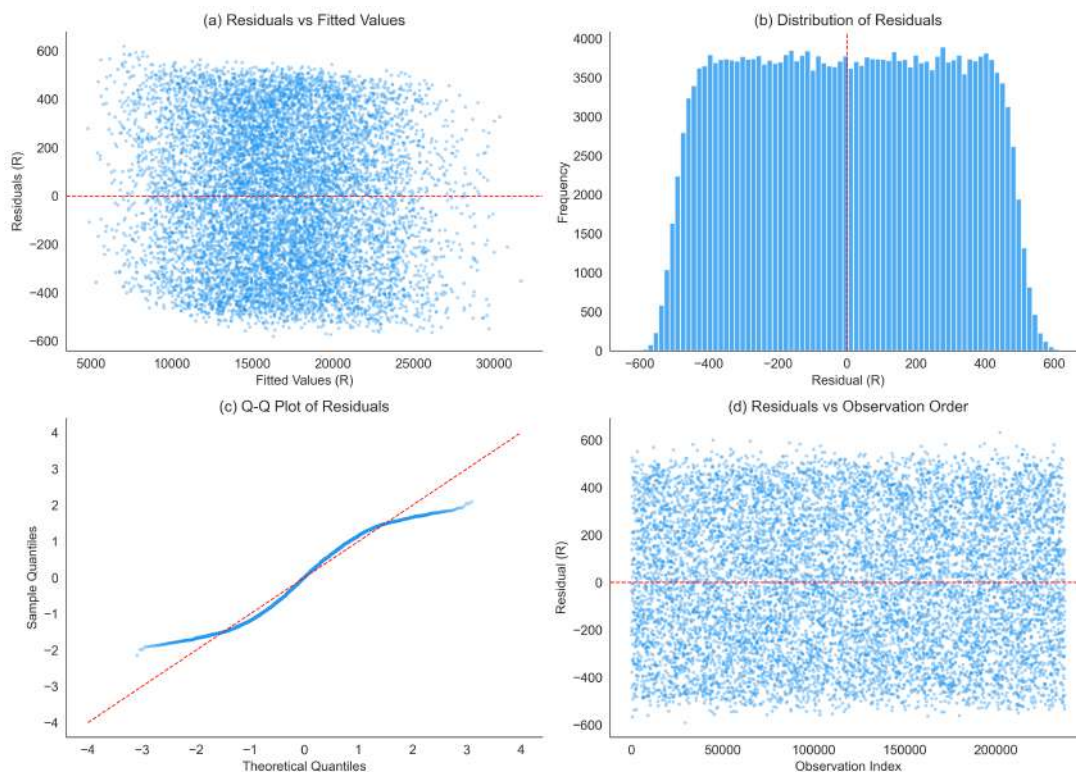
Figure 14 compares the distributional profiles of actual and predicted charges on both partitions. In the training set (left panel,  $R^2 = 0.9957$ ), the actual (blue) and predicted (orange) violins exhibit closely matching shapes: both are unimodal with the bulk of density concentrated between approximately R10 000 and R25 000, and the dashed median lines fall at approximately the same level near R16 700. The predicted violin is marginally narrower at the extreme tails, indicating that the DNN compresses the most extreme values slightly, which is consistent with the smoothing behaviour of a regression model. The test set (right panel,  $R^2 = 0.9956$ ) displays the same pattern, with the actual and predicted distributions virtually mirroring each other. The consistency between training and test panels confirms that the model’s distributional calibration is preserved on unseen data.

### 3.5.3 Residual diagnostics

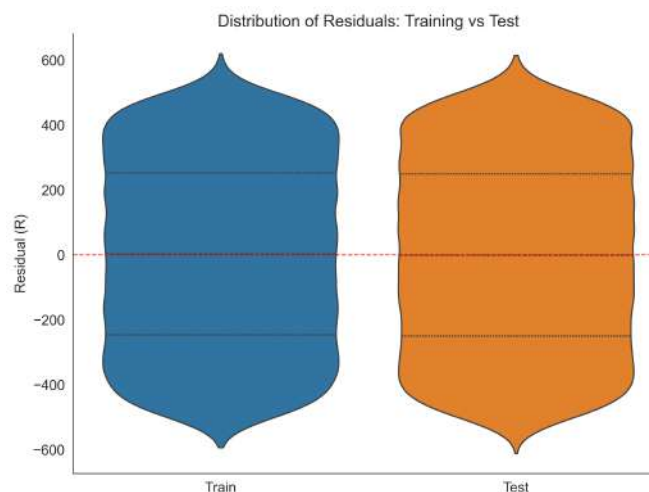
Residual diagnostics examine whether prediction errors contain systematic structure. Using the residuals defined in equation (3.5.1), the following plots are produced:

1. residuals versus fitted values, to detect systematic bias or changes in spread;
2. a residual histogram, to assess the empirical shape and centring of the prediction errors;
3. a  $Q$ - $Q$  plot of standardised residuals, to inspect tail behaviour;
4. residuals versus observation index, to identify possible ordering or sequential patterns;
5. residual distribution comparisons across the training, validation and test partitions.

The DNN does not require normally distributed residuals; these checks are purely descriptive. The fitted residuals are also retained for the residual-augmented Monte Carlo sensitivity analysis described later.



(a) Residual diagnostic panels for the fitted DNN.



(b) Violin plots of residuals on the training and test sets.

Figure 15: Residual diagnostics for the fitted DNN.

Figure 15a presents the four residual diagnostic panels for the test set. Panel (a) shows residuals versus fitted values: the residuals are centred at zero across the full range of fitted charges (approximately R5 000 to R30 000), with no visible funnel shape or system-

atic trend. The residual magnitudes range from approximately  $-600$  to  $+600$ , which is small relative to the response scale. Panel (b) displays the residual histogram, which is approximately symmetric and bell-shaped, centred at zero. Panel (c) shows the Q-Q plot of standardised residuals. The central portion follows the theoretical reference line closely, while both tails exhibit slight compression, indicating that the residual distribution has slightly lighter tails than a Gaussian distribution. This mild platykurtic behaviour is not a concern for the DNN, which does not assume normality of residuals. Panel (d) plots residuals against observation index: the points are scattered randomly around zero with no discernible sequential pattern. Figure 15b compares the residual distributions across partitions. Both the training and test violins are centred at zero and are nearly identical in width, spread and symmetry, confirming that the residual error does not systematically differ between the training and test data and that the fitted DNN does not overfit the training partition.

### 3.5.4 Principal component structural diagnostics

**Principal Component Analysis (PCA)** is employed as a structural diagnostic of the standardised, model-ready input space. It is not used for dimension reduction, and no predictors are removed on the basis of the principal components. The purpose is twofold: to assess whether the data partition preserves the structure of the input space, and to check whether the fitted predictions reproduce the dominant target pattern when displayed on the same low-dimensional projection.

Let  $\tilde{\mathbf{X}} \in \mathbb{R}^{n_{\text{train}} \times p}$  denote the column-standardised training input matrix ( $p = 22$ ). The sample covariance matrix is

$$\mathbf{S} = \frac{1}{n_{\text{train}} - 1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}. \quad (3.5.2)$$

Its spectral decomposition is

$$\mathbf{S} = \mathbf{W} \mathbf{\Lambda} \mathbf{W}^\top, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_p), \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0, \quad (3.5.3)$$

where  $\mathbf{W} = [\mathbf{w}_1 \dots \mathbf{w}_p]$  contains the principal loading vectors. The score of observation  $i$  on component  $k$  is

$$z_{ik} = \tilde{\mathbf{x}}_i^\top \mathbf{w}_k, \quad k = 1, \dots, p. \quad (3.5.4)$$

The PCA decomposition is computed from the training partition only; the resulting eigenvectors are then used to project the validation and test observations, so that all sets are displayed in a common coordinate system. The first two principal components provide the two-dimensional diagnostic projection.

Two PCA comparisons are performed. First, the training, validation and test point clouds are overlaid; similar structure across partitions supports the view that the data split has

not introduced a major structural shift. Second, the same PCA coordinates are coloured by observed charges and by fitted predictions; similar colour gradients indicate that the fitted DNN has captured the dominant target structure visible in the input space.

The loading vectors are inspected to verify that the first two principal components are not driven by a single variable and that they account for a moderate share of the total variance, so that the diagnostic is not dominated by one narrow direction (Hastie et al., 2009). This inspection is descriptive and is not used to modify the fitted model.

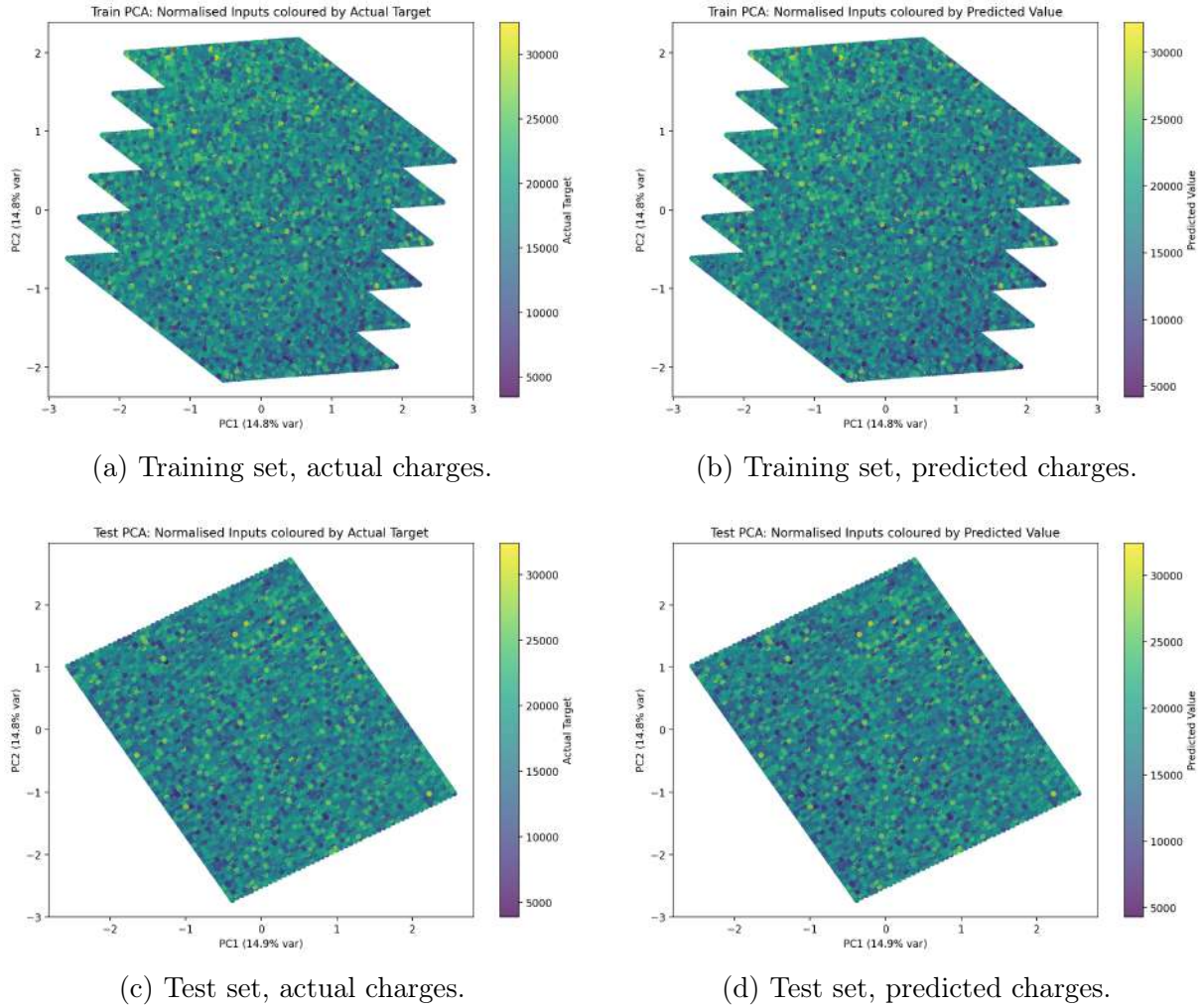


Figure 16: PCA projections coloured by actual and predicted charges on training and test sets.

Figure 16 displays the PCA diagnostic scatter plots. In the training set (Figures 16a and 16b), the first two principal components each explain approximately 14.8% of the total variance. The point cloud exhibits a layered structure with several horizontal shelves, reflecting the discrete levels of the encoded categorical predictors projected onto the PC axes. The colour gradient, representing charges from approximately R5 000 (purple) to R30 000 (yellow), is broadly mixed across the point cloud, with higher charges (yellow-

green) tending to appear at specific regions within the layered structure. The predicted-target colouring (Figure 16b) is virtually indistinguishable from the actual-target version, confirming that the fitted DNN has captured the dominant target structure visible in the first two principal component directions. The test set (Figures 16c and 16d) shows the same point-cloud shape and colour distribution as the training set (PC1: 14.9%, PC2: 14.8%), and the actual and predicted colourings again match closely. This consistency across partitions and between actual and predicted targets jointly satisfies both the partition consistency and model capture diagnostic criteria.

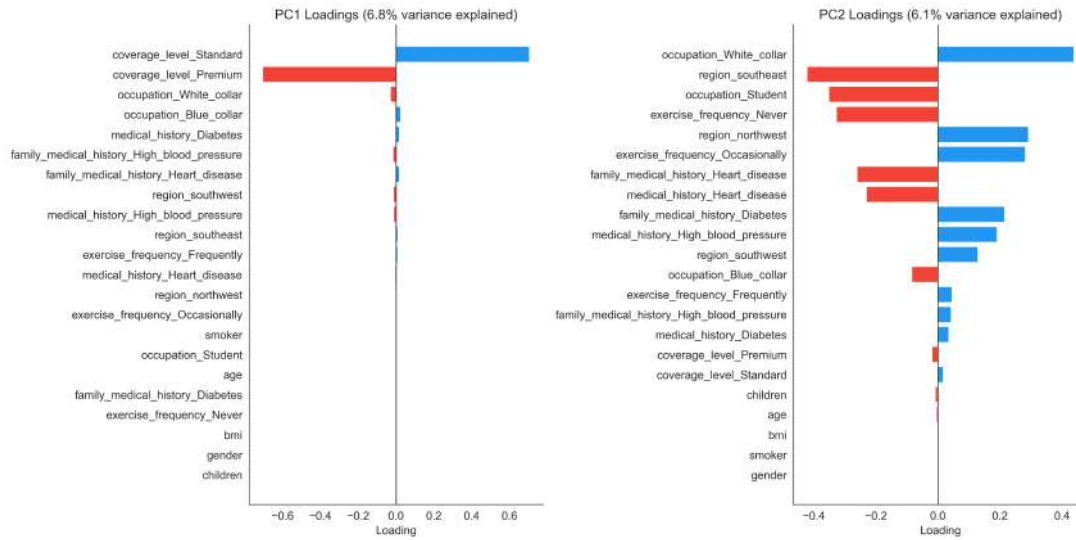


Figure 17: PCA loading vectors for all 22 encoded predictors.

Figure 17 displays the loading vectors for the first two principal components. In the left panel, PC1 (6.8% of variance) is dominated by `coverage_level_Standard` (positive loading  $\approx 0.71$ ) and `coverage_level_Premium` (negative loading  $\approx -0.71$ ), with all other loadings substantially smaller in magnitude. This confirms that the layered structure observed in the scatter plots is driven by the `coverage_level` variable. In the right panel, PC2 (6.1% of variance) has loadings spread across multiple categorical predictors: `occupation_White_collar` ( $\approx 0.44$ ), `region_northwest` ( $\approx 0.33$ ) and `exercise_frequency_Never` ( $\approx 0.30$ ) contribute positively, while `region_southeast` ( $\approx -0.42$ ) and `medical_history_Heart_disease` ( $\approx -0.20$ ) contribute negatively. The more distributed loading structure of PC2 indicates that no single variable dominates the second component. Although PC1 is driven by a single raw variable, the first two components together account for only 13% of total variance, so the diagnostic is not determined by a single narrow direction in the 22-dimensional input space.

### 3.5.5 Correlation heatmap diagnostics

Correlation heatmaps provide an additional structural check. The full Pearson correlation matrix is computed for the standardised encoded predictors together with either the observed response or the fitted predictions, yielding two views:

1. correlations between predictors and observed charges;
2. correlations between predictors and fitted predictions.

Comparing these views indicates whether the fitted [DNN](#) has preserved the main linear feature-target relationships present in the data. The heatmaps are also produced for the training, validation and test partitions to assess whether the data split preserves the linear dependence structure of the model-ready input space. These heatmaps are purely diagnostic; they do not capture the nonlinear relationships learned by the network, nor do they constitute a proof of model correctness.

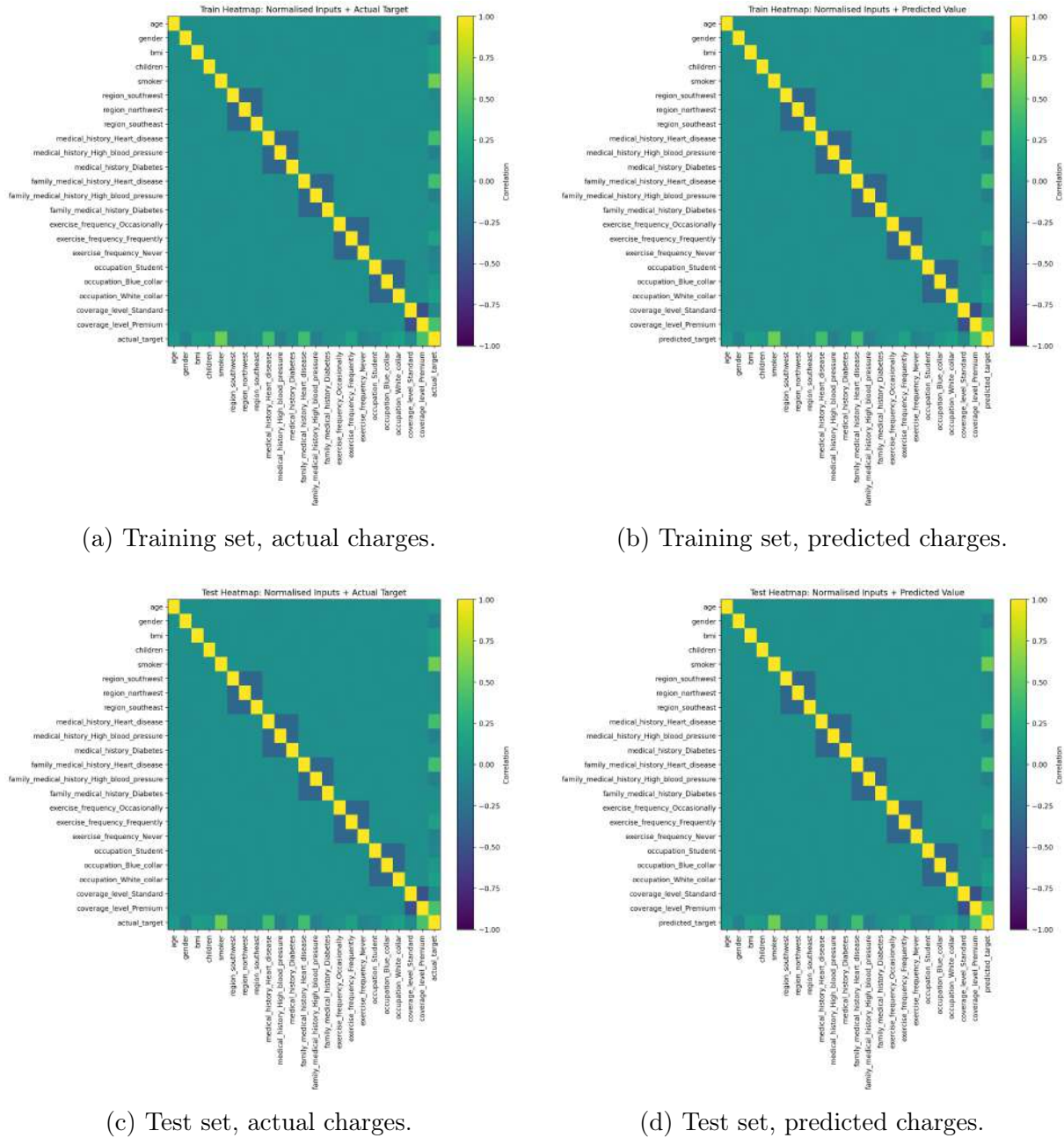


Figure 18: Input-target correlation heatmaps for actual and predicted charges.

Figure 18 presents the four correlation heatmaps. The inter-feature correlations are predominantly near zero (teal shading), with the expected negative correlations visible within dummy groups belonging to the same raw categorical variable (e.g. the `medical_history`, `region`, `occupation` and `coverage_level` dummy blocks). The bottom row of each panel shows the feature-target correlations. In the training set (Figure 18a), `smoker` shows the largest positive correlation with actual charges (bright green-yellow), followed by `coverage_level_Premium`, `medical_history_Heart_disease` and `bmi`. The predicted version (Figure 18b) displays a virtually identical target row, confirming that the fitted DNN has preserved the linear association structure. The test set (Figures 18c and 18d)

reproduces the same patterns, with the target-correlation rows near-identical across both the actual and predicted versions. These consistency findings complete the diagnostic verification; the fitted DNN is therefore ready to serve as the prediction surface for the Monte Carlo and Shapley-value analyses that follow.

### 3.6 Parametric Monte Carlo Simulation Design

The Monte Carlo stage generates a synthetic policyholder input space and the corresponding model-implied distribution of predicted costs. Each raw predictor variable is assigned a fitted simulation distribution, synthetic policyholder profiles are drawn from these distributions, and the fitted DNN computes predicted costs for the generated profiles. The simulation is performed at the raw-variable level: categorical variables are simulated as valid categories first and are only then transformed into the same encoding used during model training. Dummy variables are never simulated independently, since that could produce impossible category combinations.

#### 3.6.1 Joint input distribution

The joint simulation distribution is constructed as a product of fitted marginal distributions, following the principle of equation (2.2.5). Let  $\mathbf{X}_{\text{raw}} = (X_1, X_2, \dots, X_q)$  denote the vector of raw predictor variables, where  $q = 11$ . Applying the product-marginal construction of equation (2.2.5) to the raw variables before encoding yields

$$\hat{F}_{\mathbf{X}, \text{raw}} = \prod_{j=1}^q \hat{F}_{X_j}. \quad (3.6.1)$$

As discussed in Section 2.2.2, the product-marginal construction is a modelling choice adopted for simulation purposes. Two empirical findings from the exploratory analysis support it in this dataset: the weak pairwise correlations among the numerical predictors reported in Table 5, and the observation that `age`, `bmi`, and `children` are closely consistent with independent uniform draws over their observed ranges (Section 3.3). The product-marginal construction is therefore not merely a computational convenience but a faithful reflection of the benchmark data’s apparent generating design.

#### 3.6.2 Fitted marginal distributions

The fitted marginal distributions are selected using the empirical summaries from the exploratory diagnostics. All distributional parameters, including the support bounds for the uniform distributions, are estimated from  $\mathcal{D}_{\text{train}}$  only, ensuring that no validation-

or test-set information enters the simulation design. Bounded numerical variables are assigned distributions that respect their observed support. Binary variables are generated from Bernoulli distributions. Multi-level categorical variables are generated using the observed training-set proportions. Table 13 specifies the distributional assumptions for each raw variable.

Table 13: Monte Carlo input distributions for raw policyholder variables.

| Variable               | Type               | Distribution used  | Parameter estimation                                       | Support             |
|------------------------|--------------------|--------------------|------------------------------------------------------------|---------------------|
| age                    | Bounded integer    | Discrete uniform   | $\hat{a} = 18, \hat{b} = 65$                               | $\{18, \dots, 65\}$ |
| bmi                    | Bounded continuous | Continuous uniform | $\hat{a} = 18, \hat{b} = 50$                               | $[18, 50]$          |
| children               | Bounded count      | Discrete uniform   | $\hat{a} = 0, \hat{b} = 5$                                 | $\{0, \dots, 5\}$   |
| gender                 | Binary             | Bernoulli          | $\hat{p}_{\text{male}} = n_{\text{male}}/n_{\text{train}}$ | $\{0, 1\}$          |
| smoker                 | Binary             | Bernoulli          | $\hat{p}_{\text{yes}} = n_{\text{yes}}/n_{\text{train}}$   | $\{0, 1\}$          |
| region                 | Categorical        | Categorical        | $\hat{p}_k = n_k/n_{\text{train}}$                         | Valid regions       |
| medical_history        | Categorical        | Categorical        | $\hat{p}_k = n_k/n_{\text{train}}$                         | Valid categories    |
| family_medical_history | Categorical        | Categorical        | $\hat{p}_k = n_k/n_{\text{train}}$                         | Valid categories    |
| exercise_frequency     | Categorical        | Categorical        | $\hat{p}_k = n_k/n_{\text{train}}$                         | Valid categories    |
| occupation             | Categorical        | Categorical        | $\hat{p}_k = n_k/n_{\text{train}}$                         | Valid categories    |
| coverage_level         | Categorical        | Categorical        | $\hat{p}_k = n_k/n_{\text{train}}$                         | Valid tiers         |

The fitted probabilities for categorical variables are estimated from the training partition only, following the marginal parametric approach in equation (2.7.2). If  $C$  is a categorical variable with levels  $c_1, \dots, c_K$ , then

$$\hat{p}_k = \frac{1}{n_{\text{train}}} \sum_{i \in \mathcal{D}_{\text{train}}} \mathbb{I}\{C_i = c_k\}, \quad k = 1, \dots, K. \quad (3.6.2)$$

A simulated categorical value is drawn as  $C^{(b)} \sim \text{Categorical}(\hat{p}_1, \dots, \hat{p}_K)$  and subsequently encoded using the reference-category scheme.

### 3.6.3 Goodness-of-fit and support checks

For continuous variables, empirical histograms and density overlays are inspected, and Kolmogorov-Smirnov (K-S) and Anderson-Darling (A-D) statistics are recorded. These two tests were selected because they are among the most powerful omnibus goodness-of-fit tests for the continuous distributions under consideration: the K-S statistic measures the maximum absolute deviation between the empirical and fitted cumulative distribution functions, while the A-D statistic places greater weight on the tails, making it particularly sensitive to distributional departures in the regions most relevant to the high-cost analysis. For discrete and categorical variables, observed and fitted category probabilities are compared using frequency tables and chi-squared summaries. Because the sample size is large, formal tests are interpreted cautiously: very small distributional differences may be statistically significant without being practically important for simulation. Support restrictions are treated as compulsory: simulated ages remain between 18 and 65, `bmi` values between 18 and 50, `children` values between 0 and 5, and categorical variables take only valid observed levels.

### 3.6.4 Sample size and convergence monitoring

The base simulation uses  $B = 100\,000$  generated policyholder profiles. A pilot convergence check is conducted by repeating the simulation for increasing values of  $B \in \{1\,000, 2\,500, 5\,000, 10\,000, 25\,000, 50\,000, 100\,000\}$ . For each  $B$ , the running mean, standard deviation, and selected quantiles of the simulated predicted costs are recorded. The Monte Carlo standard error is estimated using equation (2.8.7), with  $s_B$  denoting the simulated standard deviation. The selected simulation size is retained when running summaries are stable relative to the scale of the response variable.

Table 14: Monte Carlo convergence quantities recorded during the pilot run.

| Quantity                   | Purpose                                                  |
|----------------------------|----------------------------------------------------------|
| Running mean               | Checks stability of the average simulated predicted cost |
| Running standard deviation | Checks stability of simulated output spread              |
| Median                     | Checks central distributional stability                  |
| 90th percentile            | Checks upper-tail stability                              |
| 95th percentile            | Candidate high-cost threshold                            |
| 99th percentile            | Extreme upper-tail stability check                       |
| Monte Carlo standard error | Quantifies simulation precision for selected summaries   |

### 3.6.5 Base DNN Monte Carlo scoring

For each simulated raw profile, the preprocessing map  $T(\cdot)$  produces the model-ready input vector  $\mathbf{z}^{(b)}$ . The fitted DNN then calculates the base Monte Carlo prediction, following the formulation in equation (2.2.7):

$$\hat{Y}^{(b)} = \hat{m}(\mathbf{z}^{(b)}), \quad b = 1, \dots, B. \quad (3.6.3)$$

The resulting collection  $\{\hat{Y}^{(1)}, \dots, \hat{Y}^{(B)}\}$  forms the base model-implied predicted-cost distribution introduced in equation (2.2.7). The generated input profiles are also summarised before interpreting the predicted-cost outputs, to confirm what type of simulated input space produced the DNN predictions.

Table 15: Base Monte Carlo simulation algorithm.

---

**Algorithm 3.1: Base parametric Monte Carlo simulation**

---

**Input:** fitted raw-variable distributions  $\{\hat{F}_{X_j}\}_{j=1}^q$ , fitted model  $\hat{m}(\cdot)$ , preprocessing map  $T(\cdot)$ , number of replications  $B$ , random seed  $s$ .

**Output:** simulated raw profiles  $\{\mathbf{X}_{\text{raw}}^{(b)}\}_{b=1}^B$ , model-ready profiles  $\{\mathbf{z}^{(b)}\}_{b=1}^B$ , and base predicted costs  $\{\hat{Y}^{(b)}\}_{b=1}^B$ .

1. Set the random seed to  $s$ .
  2. For  $b = 1, \dots, B$ , simulate each raw variable from its fitted marginal distribution.
  3. For categorical variables, simulate a valid category before applying reference-category encoding.
  4. Apply the training-set preprocessing map  $T(\cdot)$  to obtain  $\mathbf{z}^{(b)}$ .
  5. Calculate  $\hat{Y}^{(b)} = \hat{m}(\mathbf{z}^{(b)})$ .
  6. Transform predictions back to the original monetary scale if the network output is on the standardised scale.
  7. Store the simulated inputs, encoded inputs and predicted costs.
- 

## 3.7 Validation of the Monte Carlo Input Distribution

The product-marginal simulation law in equation (3.6.1) constructs the joint input distribution as a product of independent fitted marginals. Even if every marginal distribution matches the training data closely, the joint distribution of the simulated profiles is structurally different from the joint distribution of the training data because all cross-variable dependencies are set to zero by construction. This section diagnoses the magnitude of that joint-distribution difference and then assesses whether it affects the Shapley-value attributions derived from the simulated profiles.

### 3.7.1 Joint distribution diagnostics

#### Correlation matrix difference

The Pearson correlation matrix is computed separately for the  $q = 11$  raw predictor variables on the training partition and on the parametric Monte Carlo sample. The element-wise difference

$$\Delta_{jk} = r_{jk}^{(\text{train})} - r_{jk}^{(\text{MC})}, \quad j, k = 1, \dots, q, \quad (3.7.1)$$

quantifies how the pairwise linear association structure of the simulated profiles deviates from that of the training data. Because the parametric Monte Carlo sample draws each variable independently, the off-diagonal entries of the MC correlation matrix are approximately zero by construction, so  $\Delta_{jk} \approx r_{jk}^{(\text{train})}$  for  $j \neq k$ . The resulting difference heatmap therefore reveals which variable pairs carry the strongest dependencies in the training data that are absent from the simulation.

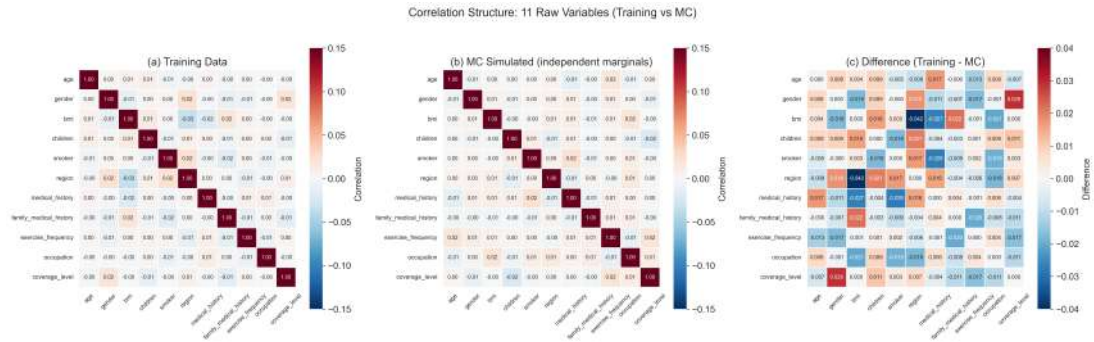


Figure 19: Correlation structure comparison: training data versus parametric MC sample.

#### Cramér's $V$ pairwise association for categorical variables

For multi-level categorical predictors the strength of association is measured using Cramér's  $V$  statistic (Cramér, 1946). For each pair of categorical variables,  $V$  is computed on the training partition and on the parametric Monte Carlo sample. All pairwise values of  $V$  are compared to confirm that cross-variable associations are below a weak-association threshold in both datasets and that the parametric MC sample, which has exactly zero association by construction, does not materially differ from the training data.

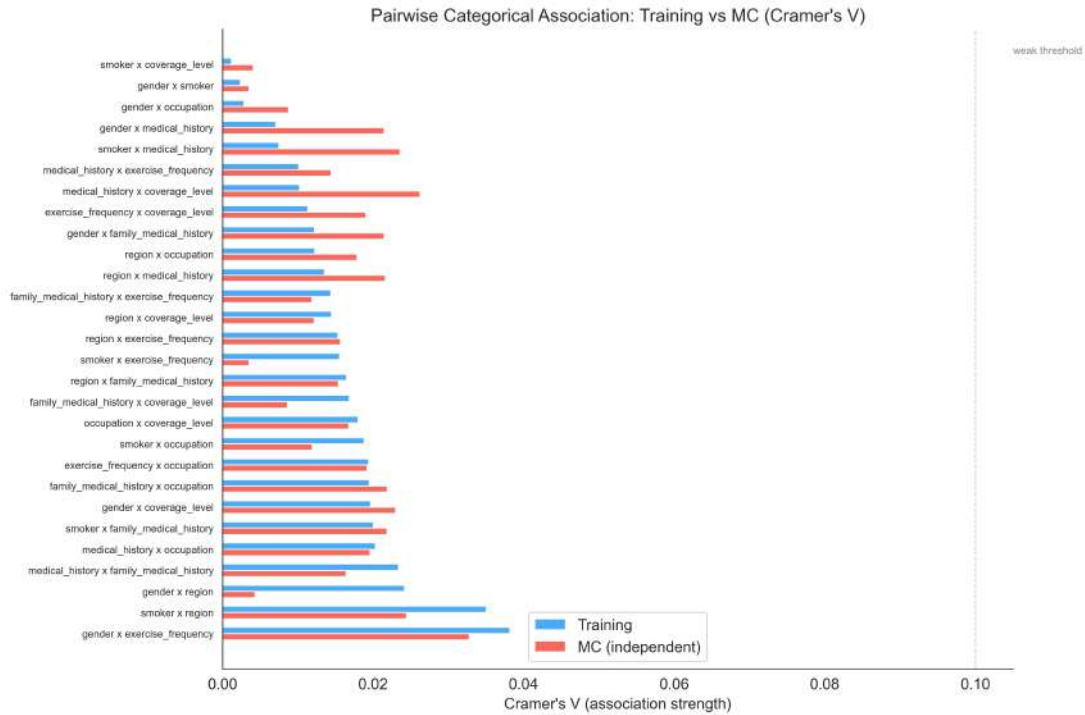


Figure 20: Cramér's  $V$  comparison: training versus parametric MC sample.

### PCA overlay of the joint input space

A principal component projection provides a visual comparison of the joint input structure. The first two principal components are computed from the standardised training inputs (using the eigenvectors from Section 3.5.4), and both the training set and the parametric MC sample are projected onto these components. The two point clouds are overlaid with distinct colours. If the independence assumption substantially alters the joint structure, the MC cloud will appear more spread out or exhibit less internal clustering than the training cloud. The loadings of the first two principal components (Figure 17) show that PC1 (6.9% of total variance) is driven primarily by `coverage_level_Standard` (loading 0.706) and `coverage_level_Premium` (−0.706), while PC2 (6.4%) has loadings spread across multiple categorical predictors. Although PC1 is dominated by a single raw variable, the first two components together account for only 13.3% of total variance, indicating that no single direction dominates the 22-dimensional input space.

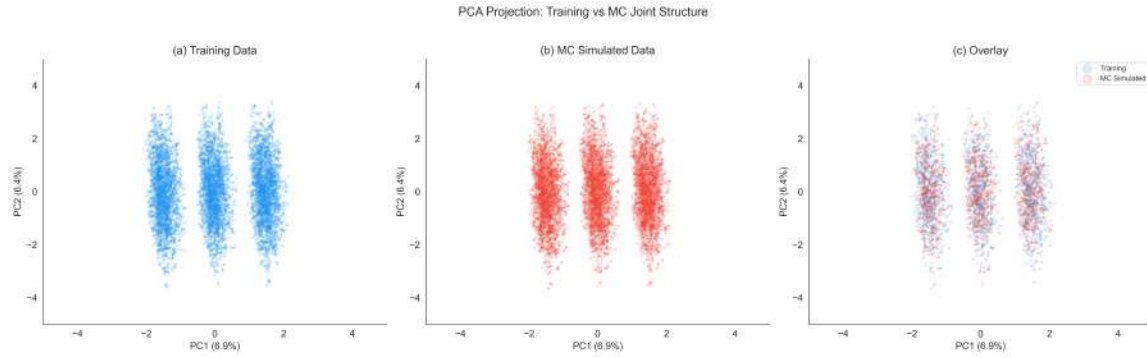


Figure 21: PCA projection of the joint input structure: (a) training data, (b) parametric MC sample, and (c) overlay of both sets.

The Monte Carlo sample populates the same region of principal component space as the training data. The vertical banding visible in both panels reflects the discrete structure introduced by the categorical predictors. The overlay in panel (c) confirms that the two point clouds are broadly co-located, with the MC cloud exhibiting a marginally wider spread consistent with the removal of the weak dependencies identified in the correlation analysis.

### 3.7.2 Bootstrap Monte Carlo resampling

To separate the effect of the independence assumption from the effect of marginal distribution choice, a second Monte Carlo sample of size  $B$  is drawn via bootstrap resampling with replacement from the training set:

$$\mathbf{X}_{\text{raw}}^{(b, \text{boot})} \sim \hat{F}_{X, \text{emp}}, \quad b = 1, \dots, B, \quad (3.7.2)$$

where  $\hat{F}_{X, \text{emp}}$  is the empirical distribution of the training data. This sample preserves the empirical joint distribution exactly, including all cross-variable dependencies, while the parametric MC sample uses independent marginals. The bootstrap profiles are preprocessed and scored through the fitted DNN in the same manner as the parametric profiles. Shapley values are then computed for both sets of profiles using identical explainer settings, allowing a direct comparison of the attribution results under the two input regimes.

Table 16: Comparison of the two Monte Carlo input regimes.

| Property        | Parametric MC                                                 | Bootstrap MC                                                |
|-----------------|---------------------------------------------------------------|-------------------------------------------------------------|
| Marginals       | Fitted parametric (uniform, Bernoulli, categorical)           | Empirical (training-set resampled)                          |
| Joint structure | Independent (product of marginals)                            | Empirical (preserves all dependencies)                      |
| Sample size     | $B = 100,000$                                                 | $B = 100,000$                                               |
| Purpose         | Primary simulation for cost distribution and Shapley analysis | Validation check for Shapley invariance to input dependence |

### 3.7.3 Shapley invariance assessment

The mean absolute Shapley values  $I_g$  are aggregated to the raw-variable level ( $q = 11$  variables) and compared between the parametric MC and bootstrap MC profiles. If the Shapley attributions are driven by the fitted DNN’s internal importance structure rather than by the particular dependence pattern in the input profiles, the two sets of attributions should be nearly identical.

The comparison is summarised by the Spearman rank correlation between the two feature-importance vectors:

$$\rho_S = \text{Spearman}(\{I_g^{(\text{param})}\}_{g=1}^q, \{I_g^{(\text{boot})}\}_{g=1}^q). \quad (3.7.3)$$

A rank correlation close to one indicates that the feature-importance ordering is preserved regardless of whether the input profiles carry training-data dependencies or assume full independence.

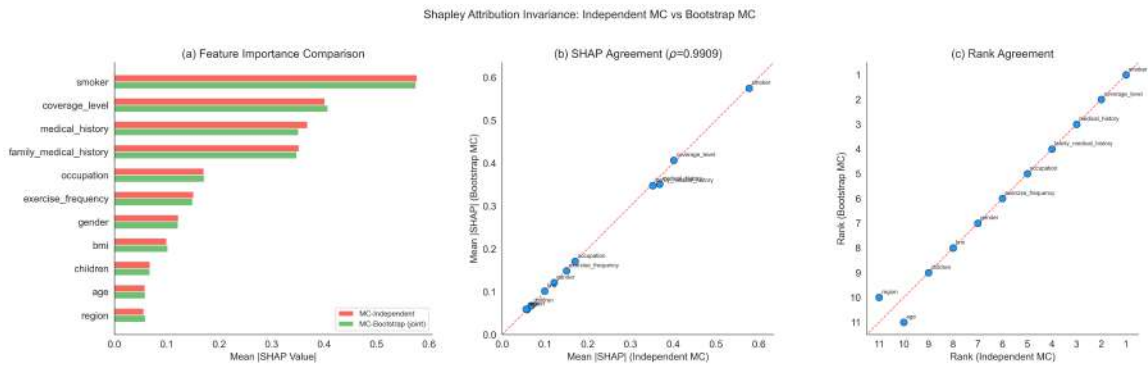


Figure 22: Shapley attribution invariance: parametric MC versus bootstrap MC input regimes.

### 3.7.4 Interpretation and conclusion

The joint-distribution diagnostics confirm that the parametric MC sample differs from the training data in its dependence structure: the parametric sample has zero cross-variable association by construction, whereas the training data carries weak but non-zero dependencies. However, the magnitude of these differences is small. The correlation matrix differences do not exceed 0.042 (Figure 19), all Cramér’s  $V$  values fall below 0.04 (Figure 20), and the PCA overlays show that both datasets populate the same region of principal component space (Figure 21). The Shapley invariance comparison yields a Spearman rank correlation of  $\rho_s = 0.991$  between the parametric and bootstrap attribution vectors (Figure 22), with the feature-importance ordering preserved for all but the two weakest contributors.

Because the joint-distribution differences are small and the Shapley attributions are invariant to the input dependence structure, the product-marginal independence assumption in Section 3.6 is a defensible modelling choice. It does not introduce meaningful bias into the subsequent high-cost conditional Shapley analysis. The fitted DNN’s internal importance structure, rather than the dependence pattern of the input profiles, determines the attribution results.

## 3.8 Residual-Augmented Monte Carlo Sensitivity Analysis

A second Monte Carlo design assesses sensitivity to fitted residual variation. The base Monte Carlo design produces deterministic fitted predictions. The residual-augmented design adds a residual component to the predicted costs to examine how the simulated output distribution changes when fitted model error is introduced. The perturbation operates in the output space (predicted costs) rather than the input space (policyholder covariates). This is appropriate because the primary concern is whether the distributional conclusions drawn from the base Monte Carlo analysis are sensitive to the magnitude of the fitted model’s residual error, not to misspecification of the input distributions.

### 3.8.1 Fitted residual distribution

Let  $e_i = y_i - \hat{y}_i$  denote the fitted residual on the original response scale for observations in the residual estimation set  $\mathcal{S}_\epsilon$ , taken as the training partition. Using the training set for residual estimation is appropriate in this context because the residuals serve only to characterise the magnitude and distributional shape of fitted prediction error for the sensitivity analysis, not to assess out-of-sample generalisation performance. The centred

residuals are

$$e_i^c = e_i - \bar{e}, \quad \bar{e} = \frac{1}{|\mathcal{S}_\varepsilon|} \sum_{i \in \mathcal{S}_\varepsilon} e_i. \quad (3.8.1)$$

The empirical residual distribution is  $\hat{F}_\varepsilon = |\mathcal{S}_\varepsilon|^{-1} \sum_{i \in \mathcal{S}_\varepsilon} \delta_{e_i^c}$ , where  $\delta_{e_i^c}$  denotes a point mass at  $e_i^c$ . The residual standard deviation is

$$\hat{\sigma}_\varepsilon = \sqrt{\frac{1}{|\mathcal{S}_\varepsilon| - 1} \sum_{i \in \mathcal{S}_\varepsilon} (e_i - \bar{e})^2}. \quad (3.8.2)$$

### 3.8.2 Residual-augmented simulation

For each base Monte Carlo prediction  $\hat{Y}^{(b)}$ , a residual draw is sampled:

$$\varepsilon^{(b)} \sim \hat{F}_\varepsilon, \quad b = 1, \dots, B. \quad (3.8.3)$$

The residual-augmented predicted cost is

$$Y_{\text{res}}^{(b)} = \hat{Y}^{(b)} + \varepsilon^{(b)}, \quad b = 1, \dots, B. \quad (3.8.4)$$

Negative values are truncated at zero:  $Y_{\text{res},+}^{(b)} = \max\{0, Y_{\text{res}}^{(b)}\}$ .

### 3.8.3 Fixed residual perturbation check

A fixed perturbation check assesses sensitivity to the magnitude of residual augmentation:

$$Y_+^{(b)} = \hat{Y}^{(b)} + 0.5 \hat{\sigma}_\varepsilon, \quad Y_-^{(b)} = \max\left\{0, \hat{Y}^{(b)} - 0.5 \hat{\sigma}_\varepsilon\right\}. \quad (3.8.5)$$

This is a deterministic sensitivity band around the fitted DNN predictions, not a probabilistic uncertainty interval. The residual-augmented simulation is interpreted as a sensitivity analysis based on fitted residual variability. It does not account for parameter uncertainty, model uncertainty or data-generating uncertainty.

Table 17: Residual-augmented Monte Carlo sensitivity design.

| Component                 | Specification                                              |
|---------------------------|------------------------------------------------------------|
| Base prediction           | $\hat{Y}^{(b)} = \hat{m}(\mathbf{z}^{(b)})$                |
| Residual source           | Fitted residuals from the training partition               |
| Residual centring         | Residuals centred before resampling                        |
| Main residual design      | $\varepsilon^{(b)} \sim \hat{F}_\varepsilon$               |
| Residual-augmented output | $Y_{\text{res}}^{(b)} = \hat{Y}^{(b)} + \varepsilon^{(b)}$ |
| Fixed perturbation check  | $\hat{Y}^{(b)} \pm 0.5 \hat{\sigma}_\varepsilon$           |
| Non-negativity rule       | Negative simulated costs truncated at zero                 |

Table 18: Residual-augmented Monte Carlo algorithm.

---

**Algorithm 3.2: Residual-augmented Monte Carlo sensitivity simulation**


---

**Input:** base Monte Carlo predictions  $\{\hat{Y}^{(b)}\}_{b=1}^B$ , fitted residual distribution  $\hat{F}_\varepsilon$ , residual standard deviation  $\hat{\sigma}_\varepsilon$ , random seed  $s$ .

**Output:** residual-augmented outputs  $\{Y_{\text{res}}^{(b)}\}_{b=1}^B$ , upper perturbation outputs  $\{Y_+^{(b)}\}_{b=1}^B$ , and lower perturbation outputs  $\{Y_-^{(b)}\}_{b=1}^B$ .

1. Set the random seed to  $s$ .
  2. Estimate residuals  $e_i = y_i - \hat{y}_i$  on the residual estimation set.
  3. Centre the residuals and construct  $\hat{F}_\varepsilon$ .
  4. For  $b = 1, \dots, B$ , draw  $\varepsilon^{(b)} \sim \hat{F}_\varepsilon$ .
  5. Calculate  $Y_{\text{res}}^{(b)} = \hat{Y}^{(b)} + \varepsilon^{(b)}$ .
  6. Apply the non-negativity rule if required.
  7. Calculate the fixed perturbation outputs  $Y_+^{(b)}$  and  $Y_-^{(b)}$ .
  8. Store all outputs for comparison with the base Monte Carlo distribution.
- 

### 3.9 Shapley-Value Attribution for Simulated Profiles

Shapley-value methods are used to explain the fitted [DNN](#) predictions calculated for simulated policyholder profiles. The attribution analysis is connected to the Monte Carlo predicted-cost distribution, with particular attention to the upper tail.

#### 3.9.1 Prediction function and Shapley decomposition

The prediction function supplied to [SHAP](#) is the fitted [DNN](#) together with the response de-normalisation step. Shapley values are therefore reported on the original monetary scale

of **charges**. For a simulated model-ready profile  $\mathbf{z}^{(b)}$ , the fitted prediction is decomposed following equation (2.2.11), with  $p = 22$  encoded features. The baseline prediction  $\phi_0$  and the feature-level contributions  $\phi_j(\mathbf{z}^{(b)})$  are defined exactly as in Section 2.10. As discussed there, Shapley values are model attributions rather than causal effects.

### 3.9.2 Implementation settings

The background data are sampled from the model-ready training partition. The primary implementation uses a gradient-based SHAP explainer for the differentiable DNN. A model-agnostic KernelSHAP calculation may be applied to a small validation subset as a computational check if required.

It is important to note that the gradient-based SHAP explainer computes approximate Shapley values, not exact game-theoretic Shapley values. Exact computation requires evaluating the prediction function at all  $2^p$  feature subsets, which is computationally infeasible for  $p = 22$  encoded features ( $2^{22} = 4\,194\,304$  coalition evaluations per profile). The gradient-based approximation leverages the differentiability of the fitted DNN to estimate feature contributions efficiently, but the resulting attributions are conditional on the background sample, the explainer algorithm, and the fitted network parameters. All reported Shapley values should therefore be interpreted as model-specific, approximate attributions rather than exact cooperative-game solutions.

Table 19: SHAP implementation settings.

| Item                  | Specification                                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Prediction function   | Fitted DNN with response de-normalisation                                                                                        |
| Output scale          | Original monetary scale of <b>charges</b>                                                                                        |
| Explainer             | Gradient-based SHAP explainer                                                                                                    |
| Background data       | Sample from the model-ready training partition                                                                                   |
| Background size       | $m_0 = 200$ observations, selected as the largest size that maintains computational feasibility for the gradient-based explainer |
| Explained profiles    | Monte Carlo simulated profiles $\mathbf{z}^{(b)}$                                                                                |
| Primary summaries     | Mean signed and mean absolute Shapley values                                                                                     |
| Conditional summaries | Shapley summaries for upper-tail simulated profiles                                                                              |
| Categorical treatment | Dummy-level attributions aggregated to raw-variable level                                                                        |

### 3.9.3 Aggregation from encoded inputs to raw variables

The fitted [DNN](#) receives  $p = 22$  encoded features, but some columns are dummy variables derived from a single raw categorical predictor. If  $G_g$  denotes the set of encoded columns associated with raw variable  $g$ , the raw-variable Shapley contribution is

$$\Phi_g(\mathbf{z}^{(b)}) = \sum_{j \in G_g} \phi_j(\mathbf{z}^{(b)}). \quad (3.9.1)$$

This aggregation prevents the interpretation from being fragmented across several dummy columns belonging to the same original variable.

The four Shapley axioms that uniquely characterise the Shapley value ([Shapley, 1953](#)) were discussed in Section [2.10](#). In the present context, the efficiency, symmetry, dummy, and additivity properties ensure that the aggregation from encoded features to raw variables is a linear repartitioning that preserves the complete decomposition. The raw-variable contributions  $\Phi_g$  therefore retain the same axiomatic guarantees as the underlying feature-level values  $\phi_j$ , and the total prediction remains fully decomposed into  $q = 11$  raw-variable contributions without remainder.

### 3.9.4 Global Shapley summaries

Shapley values are computed for all  $B$  simulated profiles generated in the base Monte Carlo simulation, so  $B_{\text{shap}} = B$ . Global feature importance  $I_j$  follows the mean absolute Shapley value defined in Section [2.10](#), computed over the simulated profiles. A mean signed summary is also reported. The mean signed Shapley contribution for feature  $j$  is

$$\bar{\phi}_j = \frac{1}{B_{\text{shap}}} \sum_{b=1}^{B_{\text{shap}}} \phi_j(\mathbf{z}^{(b)}), \quad (3.9.2)$$

and the mean absolute Shapley contribution is

$$I_j = \frac{1}{B_{\text{shap}}} \sum_{b=1}^{B_{\text{shap}}} |\phi_j(\mathbf{z}^{(b)})|. \quad (3.9.3)$$

The signed summary describes the average direction of the contribution; the absolute summary measures average magnitude.

### 3.9.5 High-cost region and conditional Shapley summaries

The high-cost region is defined using the quantile-based threshold  $u_\tau$  from equation [\(2.2.13\)](#) and the high-cost set  $\mathcal{H}_\tau$  from equation [\(2.2.14\)](#), applied to the  $B$  simulated profiles with

$\tau \in \{0.90, 0.95, 0.99\}$ . The 95th percentile is used as the main high-cost threshold because it balances two competing requirements: it retains a sufficient number of upper-tail profiles ( $|H_{0.95}| = 0.05B$ ) for stable conditional Shapley averages, while still isolating profiles that represent genuinely high predicted costs. The 90th and 99th percentiles are retained as sensitivity thresholds to assess whether the attribution patterns are robust to the choice of cutoff. The conditional mean signed contribution  $\bar{\phi}_{j,\tau}$  and the conditional mean absolute contribution  $I_{j,\tau}$  are computed as in equations (2.2.16) and (2.2.17), with the high-cost set  $\mathcal{H}_\tau$  replacing the generic conditioning set and the summation taken over the  $B$  simulated profiles. These conditional summaries are also aggregated to the raw-variable level using equation (3.9.1). Comparing  $I_j$  with  $I_{j,\tau}$  shows whether a feature is generally important across all simulated profiles or specifically important in the high-cost region. For the residual-augmented simulation, a separate high-cost set can be defined using the residual-augmented outputs to examine whether the profiles classified as high cost change when fitted residual variability is introduced. Since the residual component is added after the fitted DNN prediction, Shapley values explain the fitted DNN component  $\hat{m}(\mathbf{z}^{(b)})$  rather than the residual addition.

### 3.9.6 Bootstrap confidence intervals for conditional Shapley values

At  $\tau_{99}$  the conditioning set contains only  $|\mathcal{H}_{0.99}| = 0.01B$  profiles, so the conditional mean absolute Shapley values are estimated from a small effective sample. To quantify estimation uncertainty, a nonparametric bootstrap is applied. Let  $\Phi_\tau \in \mathbb{R}^{|\mathcal{H}_\tau| \times q}$  denote the matrix of raw-variable Shapley values for profiles in  $\mathcal{H}_\tau$ . The bootstrap procedure is:

1. Draw  $R = 10\,000$  bootstrap resamples  $\Phi_\tau^{*(r)}$ , each of size  $|\mathcal{H}_\tau|$ , with replacement from  $\Phi_\tau$ .
2. For each resample  $r$ , compute the mean absolute Shapley value for each variable:

$$I_{j,\tau}^{*(r)} = \frac{1}{|\mathcal{H}_\tau|} \sum_{i=1}^{|\mathcal{H}_\tau|} |\phi_{ij}^{*(r)}|, \quad j = 1, \dots, q.$$

3. The 95% bootstrap confidence interval for  $I_{j,\tau}$  is the interval  $[I_{j,\tau}^{*(0.025)}, I_{j,\tau}^{*(0.975)}]$ , where  $I_{j,\tau}^{*(\alpha)}$  is the  $\alpha$ -quantile of the bootstrap distribution  $\{I_{j,\tau}^{*(1)}, \dots, I_{j,\tau}^{*(R)}\}$ .

This percentile bootstrap is reported alongside the conditional importance values at  $\tau_{99}$  to assess the reliability of the ranking reversal discussed in Chapter 4.

Table 20: Shapley summaries calculated for simulated profiles.

| Summary                               | Purpose                                                           |
|---------------------------------------|-------------------------------------------------------------------|
| Mean signed Shapley value             | Average directional effect on fitted predictions                  |
| Mean absolute Shapley value           | Average contribution magnitude across simulated profiles          |
| Raw-variable aggregated Shapley value | Combines dummy-level attributions into original variables         |
| High-cost conditional Shapley value   | Identifies drivers of upper-tail predicted costs                  |
| Threshold comparison                  | Compares attribution at 90th, 95th and 99th percentiles           |
| Base versus residual high-cost sets   | Checks whether residual augmentation changes high-cost membership |

The attribution analysis is interpreted jointly with the Monte Carlo distribution. Monte Carlo simulation identifies the distributional regions of interest, and Shapley values decompose the fitted model behaviour within those regions.

### 3.10 Modelling Assumptions

The empirical framework depends on the modelling assumptions stated in Table 21. These assumptions define the boundary of the empirical claims: the analysis can support statements about how the fitted DNN behaves on the benchmark dataset and under the stated Monte Carlo input assumptions, but cannot, without external validation, support causal claims about healthcare expenditure.

Table 21: Summary of modelling assumptions and interpretive boundaries.

| No. | Assumption                          | Interpretation                                                                                                                                                                                              |
|-----|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A1  | Benchmark data                      | The dataset is a public benchmark and not a representative insurer portfolio                                                                                                                                |
| A2  | Training-set preprocessing          | Encoding, numerical scaling and response scaling are estimated from the training set and applied unchanged elsewhere                                                                                        |
| A3  | Bias-free DNN                       | The constrained architecture is assessed empirically through validation and test diagnostics                                                                                                                |
| A4  | No explicit regularisation          | Generalisation is assessed using validation behaviour, held-out performance and structural diagnostics                                                                                                      |
| A5  | Raw-variable MC simulation          | Synthetic profiles are generated at the raw variable level and only then encoded                                                                                                                            |
| A6  | Product-marginal input distribution | The joint Monte Carlo input law is a practical simulation assumption and not a proof of covariate independence; validated by joint-distribution diagnostics and Shapley invariance comparison (Section 3.7) |
| A7  | Joint-distribution validation       | The Shapley attributions are shown to be invariant to whether input profiles preserve training-data dependencies (bootstrap MC) or assume full independence (parametric MC)                                 |
| A8  | Base MC interpretation              | The base simulated output distribution represents DNN fitted predictions under the stated input assumptions, not realised healthcare costs                                                                  |
| A9  | Residual augmentation               | Residual-augmented simulation is a sensitivity analysis and not a complete predictive uncertainty model                                                                                                     |
| A10 | SHAP interpretation                 | Shapley values explain the fitted DNN predictions and are not causal feature effects                                                                                                                        |
| A11 | High-cost definition                | High-cost profiles are defined by empirical upper-tail quantiles of the simulated predicted-cost distribution                                                                                               |

With the data, architecture, simulation design, attribution method and modelling assumptions now fully specified, the empirical results of the three-stage framework are presented in Chapter 4.

## 4 Results

This chapter presents the empirical results of the three-component methodological framework specified in Chapter 3. The results are organised as follows: Section 4.1 reports the sequential hyperparameter comparison that produced the final DNN training configuration, together with the predictive performance of the retained model and a baseline comparison with the actuarial-standard GLM; Section 4.2 presents the Monte Carlo predicted-cost distribution, its joint-distribution input validation, and convergence properties; Section 4.3 evaluates the residual-augmented sensitivity analysis; Section 4.4 reports the Shapley-value attribution results at the global and high-cost conditional levels; and Section 4.6 examines the correspondence between the Monte Carlo distributional findings and the Shapley attribution patterns.

### 4.1 Hyperparameter Comparison Results

The five-phase sequential hyperparameter comparison (Section 3.4) was carried out on the architecture specified in Figure 11. The results of each phase are presented below.

#### 4.1.1 Phase 1: Activation function

Three candidate activation functions were compared: CELU (equation (2.4.9)), GELU and Tanh, with all other hyperparameters fixed at learning rate 0.001, batch size 256 and Adam.

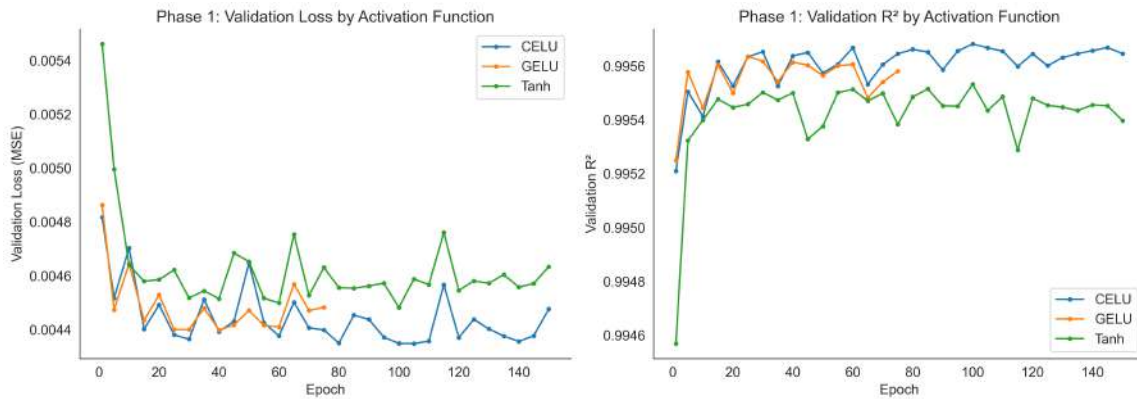


Figure 23: Phase 1: validation MSE loss (left) and validation  $R^2$  (right) by activation function over training epochs.

Figure 23 shows that CELU achieves the lowest validation loss (approximately 0.00435) and the highest validation  $R^2$  (approximately 0.9957) from epoch 60 onwards. GELU tracks CELU closely throughout training but settles at a marginally higher loss plateau.

Tanh converges more slowly and exhibits greater epoch-to-epoch volatility, reaching a best validation  $R^2$  of approximately 0.9954. The CELU activation was therefore retained.

#### 4.1.2 Phase 2: Learning rate

Four learning rates were evaluated:  $\eta \in \{0.01, 0.001, 0.0005, 0.0001\}$ , with CELU carried forward.

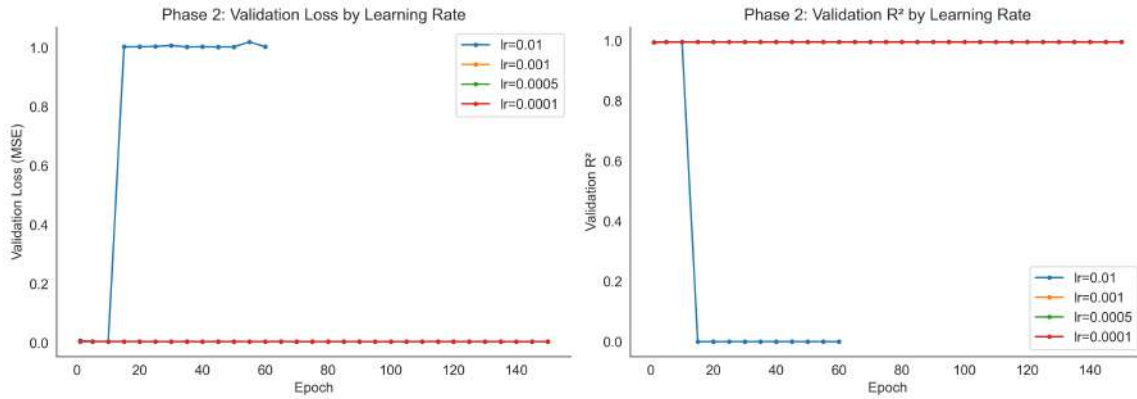


Figure 24: Phase 2: validation MSE loss (left) and validation  $R^2$  (right) by learning rate.

Figure 24 reveals that  $\eta = 0.01$  produces catastrophic divergence: the validation loss exceeds 1.0 and the validation  $R^2$  drops to near zero, indicating that the gradient updates overshoot the loss surface at this step size. The three remaining learning rates (0.001, 0.0005 and 0.0001) all converge to validation  $R^2 \approx 1.00$  on the standardised scale, with their loss curves overlapping on the visible scale. Among these,  $\eta = 0.0005$  achieves the lowest best validation loss and was retained.

#### 4.1.3 Phase 3: Batch size

Four batch sizes were compared:  $|\mathcal{B}| \in \{64, 128, 256, 512\}$ , with CELU and  $\eta = 0.0005$  fixed.

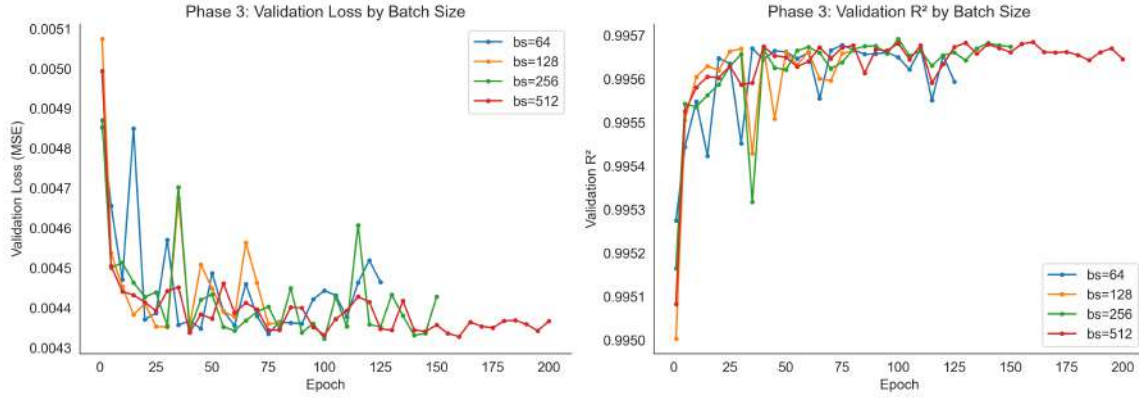


Figure 25: Phase 3: validation MSE loss (left) and validation  $R^2$  (right) by batch size.

Figure 25 shows that all four batch sizes converge to similar validation performance, with best validation  $R^2$  values between 0.9956 and 0.9957. The batch size  $|\mathcal{B}| = 256$  was retained, as it achieves among the lowest validation loss values while maintaining stable convergence behaviour.

#### 4.1.4 Phase 4: Optimiser

Adam and NAdam were compared with all previous best choices carried forward.

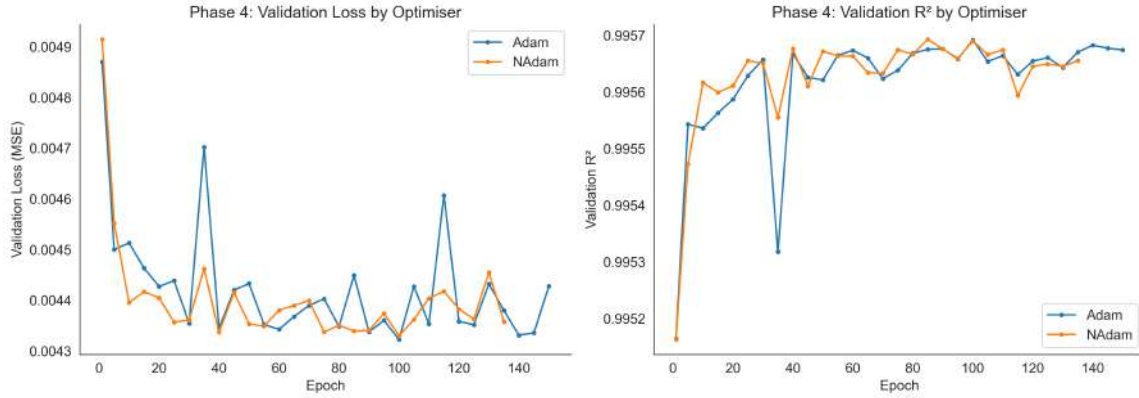


Figure 26: Phase 4: validation MSE loss (left) and validation  $R^2$  (right) by optimiser.

Figure 26 shows that NAdam converges faster than Adam in the initial epochs and exhibits smoother validation loss trajectories throughout training. From approximately epoch 40 onwards, NAdam consistently achieves lower validation loss and higher validation  $R^2$  than Adam, with fewer loss spikes. NAdam was therefore retained.

#### 4.1.5 Phase 5: Momentum parameters

Three momentum configurations were compared for the NAdam optimiser:  $(\beta_1, \beta_2) \in \{(0.90, 0.999), (0.95, 0.999), (0.99, 0.999)\}$ .

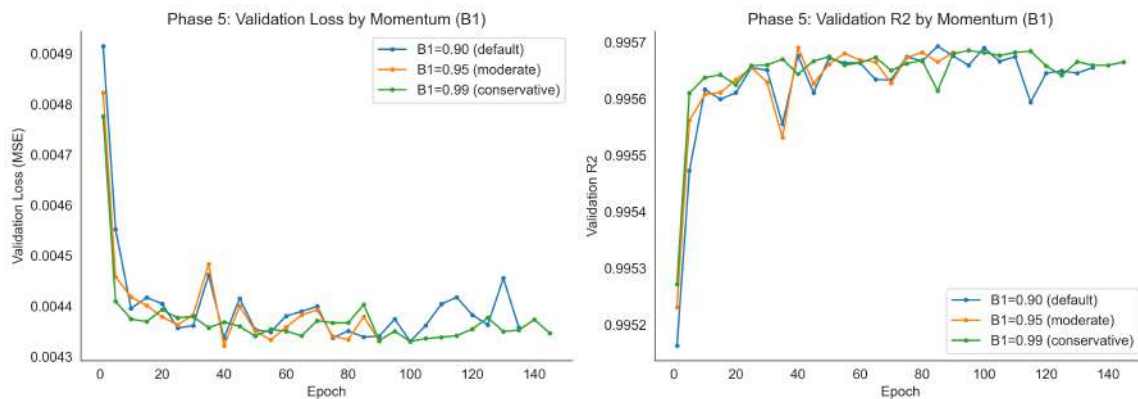


Figure 27: Phase 5: validation MSE loss (left) and validation  $R^2$  (right) by first momentum coefficient  $\beta_1$ .

Figure 27 shows that all three momentum configurations converge to similar final validation performance. The  $\beta_1 = 0.95$  configuration achieves marginally lower and more stable validation loss from epoch 40 onwards, with the best validation  $R^2$  reaching approximately 0.9957. The conservative setting  $\beta_1 = 0.99$  converges slightly faster in early epochs but exhibits comparable final performance. The configuration  $(\beta_1, \beta_2) = (0.95, 0.999)$  was retained.

#### 4.1.6 Hyperparameter comparison overview

Figure 28 presents the best validation loss achieved by each configuration across all five phases.

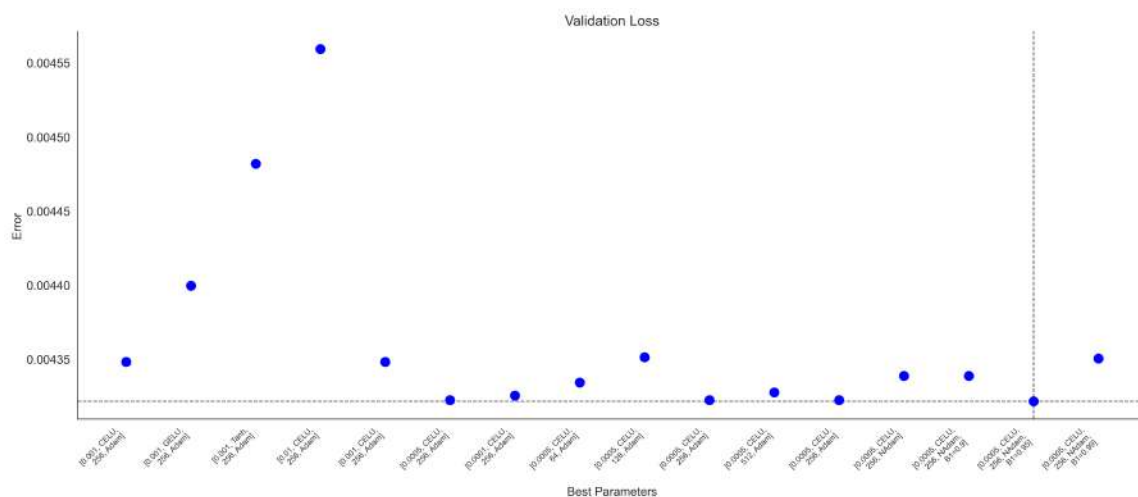


Figure 28: Best validation MSE loss across all hyperparameter configurations evaluated in Phases 1-5.

The selected configuration (CELU,  $\eta = 0.0005$ ,  $|\mathcal{B}| = 256$ , NAdam,  $\beta_1 = 0.95$ ) achieves the lowest validation loss among all evaluated configurations. Table 22 records the final

training configuration.

Table 22: Selected hyperparameters from the sequential comparison.

| Phase                            | Candidates                                  | Selected      |
|----------------------------------|---------------------------------------------|---------------|
| 1. Activation function           | CELU, GELU, Tanh                            | CELU          |
| 2. Learning rate                 | 0.01, 0.001, 0.0005, 0.0001                 | 0.0005        |
| 3. Batch size                    | 64, 128, 256, 512                           | 256           |
| 4. Optimiser                     | Adam, NAdam                                 | NAdam         |
| 5. Momentum $(\beta_1, \beta_2)$ | (0.90, 0.999), (0.95, 0.999), (0.99, 0.999) | (0.95, 0.999) |

#### 4.1.7 Final model predictive performance

The final DNN was retrained with the selected configuration and early stopping at epoch  $e^* = 85$  (the checkpoint with the highest validation  $R^2$ ). Table 23 reports the predictive performance metrics computed on the original monetary scale after de-normalising the fitted predictions, using the MAE (equation (2.3.3)), RMSE (equation (2.3.2)), and  $R^2$  (equation (2.3.4)).

Table 23: Predictive performance of the final DNN on the original monetary scale.

| Partition                   | MAE (R) | RMSE (R) | $R^2$  |
|-----------------------------|---------|----------|--------|
| Training ( $n = 524\,288$ ) | 251     | 291      | 0.9957 |
| Test ( $n = 237\,856$ )     | 251     | 290      | 0.9957 |

The near-identical performance across partitions (MAE = R251 on both; RMSE within R290 to R291;  $R^2 = 0.9957$  on both) confirms that the fitted model generalises to unseen data without overfitting. The training-set response statistics used for standardisation are  $\bar{y}_{\text{train}} = \text{R}16\,744$  and  $s_{y,\text{train}} = \text{R}4\,419$ . On the standardised scale, the validation MSE at the selected epoch  $e^* = 85$  is 0.00432 (Figure 28), and the original-scale residuals range from approximately  $-600$  to  $+600$  as characterised in the residual diagnostics of Section 3.5.

The model's predictive performance supports its use in the subsequent simulation and attribution analyses.

#### 4.1.8 Baseline comparison: GLM, RF, XGBoost and DNN

The [GLM](#) benchmark specified in Section 3.4.5 and the two tree-based benchmarks specified in Section 3.4.6 were evaluated alongside the [DNN](#). All four models were fitted on the same training partition ( $n = 524\,288$ ) using the same encoded feature set and the same train/validation/test split, ensuring that the comparison reflects genuine model-class differences rather than data-handling artefacts. Table 24 reports the performance of all four models across training, validation and test partitions.

Table 24: Predictive performance comparison across four model classes on the original monetary scale.

| Model                                 | Partition  | MAE (R) | RMSE (R) | $R^2$  |
|---------------------------------------|------------|---------|----------|--------|
| <a href="#">GLM</a> (Gamma, log-link) | Training   | 623     | 911      | 0.9574 |
|                                       | Validation | 622     | 913      | 0.9574 |
|                                       | Test       | 624     | 913      | 0.9571 |
| <a href="#">RF</a>                    | Training   | 261     | 324      | 0.9946 |
|                                       | Validation | 359     | 443      | 0.9900 |
|                                       | Test       | 358     | 443      | 0.9899 |
| <a href="#">XGBoost</a>               | Training   | 241     | 281      | 0.9959 |
|                                       | Validation | 255     | 297      | 0.9955 |
|                                       | Test       | 255     | 297      | 0.9955 |
| <a href="#">DNN</a> (Funnel)          | Training   | 251     | 291      | 0.9957 |
|                                       | Validation | 251     | 291      | 0.9957 |
|                                       | Test       | 251     | 290      | 0.9957 |

The [DNN](#) outperforms the [GLM](#) on every metric. On the test set, the [DNN](#) achieves an  $R^2$  of 0.9957 compared with 0.9571 for the [GLM](#), an improvement of 3.86 percentage points. The test [RMSE](#) decreases from R913 to R290 (a 68.2% reduction), and the test [MAE](#) decreases from R624 to R251 (a 59.8% reduction).

The two tree-based benchmarks narrow the gap considerably. The [RF](#) achieves a test  $R^2$  of 0.9899, reducing the [GLM](#)'s test [RMSE](#) from R913 to R443 (51.5%) and its [MAE](#) from R624 to R358 (42.6%). However, its training  $R^2$  of 0.9946 versus its test  $R^2$  of 0.9899 indicates a degree of overfitting despite the `max_depth = 30` constraint, suggesting that the ensemble memorises some training-set noise. [XGBoost](#) performs substantially better, achieving a test  $R^2$  of 0.9955 with a test [RMSE](#) of R297 and a test [MAE](#) of R255. Its training-to-test gap is negligible ( $R^2_{\text{train}} = 0.9959$  versus  $R^2_{\text{test}} = 0.9955$ ), indicating

effective regularisation from the learning rate ( $\eta = 0.05$ ) and subsampling parameters. The **DNN** marginally outperforms **XGBoost** on all three test metrics ( $R^2$ : 0.9957 versus 0.9955; **RMSE**: R290 versus R297; **MAE**: R251 versus R255), while showing perfect training-to-test agreement. The **DNN**'s advantage over the tree-based models, although modest in absolute terms, is consistent with its ability to learn smooth, continuous mappings through backpropagation, whereas tree-based models approximate these mappings through piecewise-constant splits.

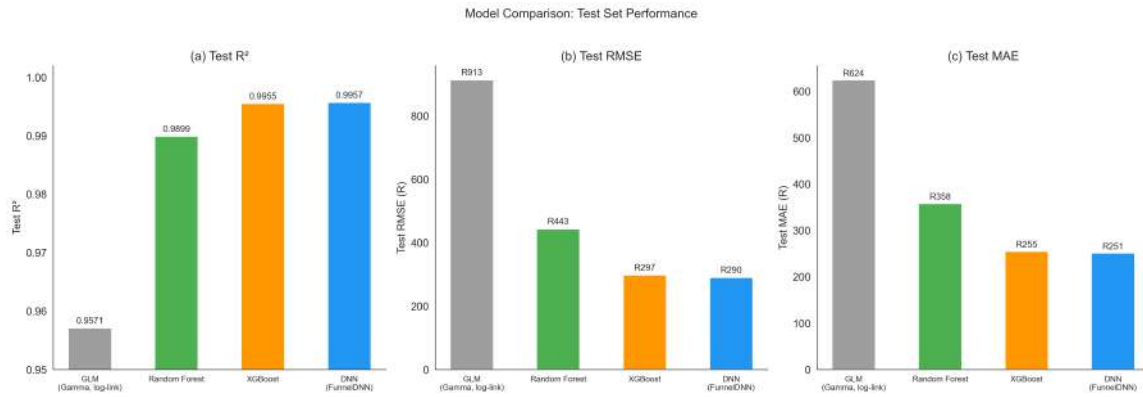


Figure 29: Test-set performance comparison across all four model classes: **GLM** (Gamma, log-link), **RF**, **XGBoost** and **DNN**. (a) Test  $R^2$ ; (b) test RMSE; (c) test MAE.

Figure 29 visualises the test-set comparison across all four model classes. The **GLM**'s inferior fit is attributable to the log-link's multiplicative mean structure, which does not match the additive data-generating process underlying this dataset. The charges variable is constructed as an approximately linear function of the input features plus noise; the **GLM**'s exponential inverse-link therefore introduces systematic model misspecification. The **RF** and **XGBoost** close the gap substantially, with **XGBoost** nearly matching the **DNN**. The **DNN** learns the additive structure directly through its flexible hidden-layer representations and achieves the highest test  $R^2$  and lowest test RMSE and MAE.

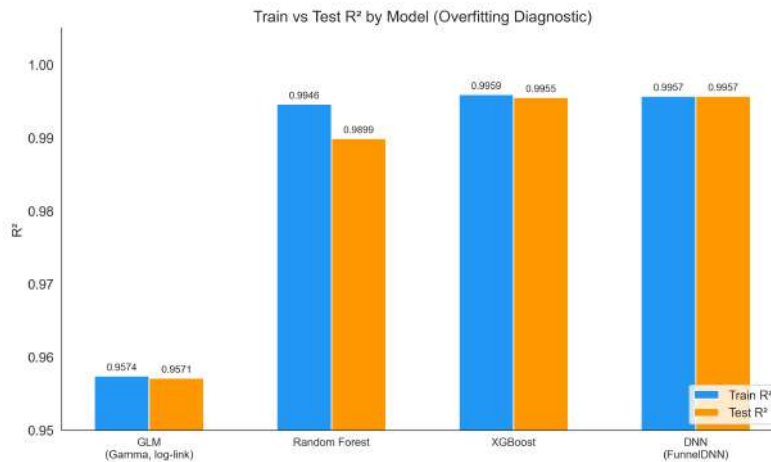


Figure 30: Overfitting diagnostic: training versus test  $R^2$  for all four model classes.

Figure 30 compares the training and test  $R^2$  for all four models. The GLM shows negligible generalisation loss ( $R^2_{\text{train}} = 0.9574$  versus  $R^2_{\text{test}} = 0.9571$ ), as expected for a parametric model with far fewer parameters than observations. The RF exhibits the largest train-test gap ( $R^2_{\text{train}} = 0.9946$  versus  $R^2_{\text{test}} = 0.9899$ ), indicating a degree of overfitting despite the depth constraint. XGBoost shows a negligible gap ( $R^2_{\text{train}} = 0.9959$  versus  $R^2_{\text{test}} = 0.9955$ ), confirming that the learning rate and subsampling regularisation are effective. The DNN likewise shows no evidence of overfitting ( $R^2_{\text{train}} = 0.9957$  versus  $R^2_{\text{test}} = 0.9957$ ), confirming that the early stopping criterion (Section 3.4) effectively regularises the 49 168 trainable parameters.

The benchmark comparison serves two purposes. First, it establishes that the DNN’s high  $R^2$  reflects genuine predictive superiority over the actuarial-standard GLM and competitive performance relative to the tree-based alternatives, not merely an artefact of the dataset’s synthetic regularity. Second, because the Gamma/log-link GLM is the model an actuary would deploy as a first approach and XGBoost represents the current state of the art for tabular data, the comparison provides practical benchmark context that situates the DNN within the actuarial modelling landscape. The DNN is therefore retained for the subsequent analyses on the grounds of its superior accuracy and its differentiability for gradient-based attribution.

The finding that a deep neural network outperforms a classical GLM on this benchmark is consistent with the broader health-cost prediction literature. Drewe-Boss et al. (2022) report that deep-learning models achieve lower prediction error than standard actuarial models on claims data, while Samiuddin et al. (2023) and Kaushik et al. (2022) similarly observe that neural-network architectures improve upon traditional regression baselines for insurance cost estimation. As noted in Section 2.2, however, this superiority is not universal across all tabular insurance datasets (Richman, 2021), and the present comparison is conducted on a benchmark dataset with a strong synthetic structure.

A related line of actuarial research applies explicit shape constraints to neural networks during training. Richman and Wüthrich (2024) develop the ICEnet, which adds smoothness and monotonicity penalties to the loss function so that Individual Conditional Expectation outputs remain actuarially reasonable. Because that study targets claims frequency under a Poisson deviance criterion rather than continuous monetary costs, a direct numerical comparison with the present results is not possible; the methodological differences are discussed in Section 5.7.

## 4.2 Monte Carlo Predicted-Cost Distribution

The base parametric Monte Carlo simulation (Algorithm 3.1) was executed with  $B = 100\,000$  synthetic policyholder profiles drawn from the product-marginal input distribution

specified in equation (3.6.1). Each simulated profile was preprocessed, scored through the fitted DNN, and de-normalised to the original monetary scale.

### 4.2.1 Simulated input validation

The simulated inputs were validated following the joint-distribution diagnostic protocol in Section 3.7.

### Joint distribution validation

Three complementary diagnostics compare the bivariate and multivariate dependence structure of the training data with that of the Monte Carlo sample.

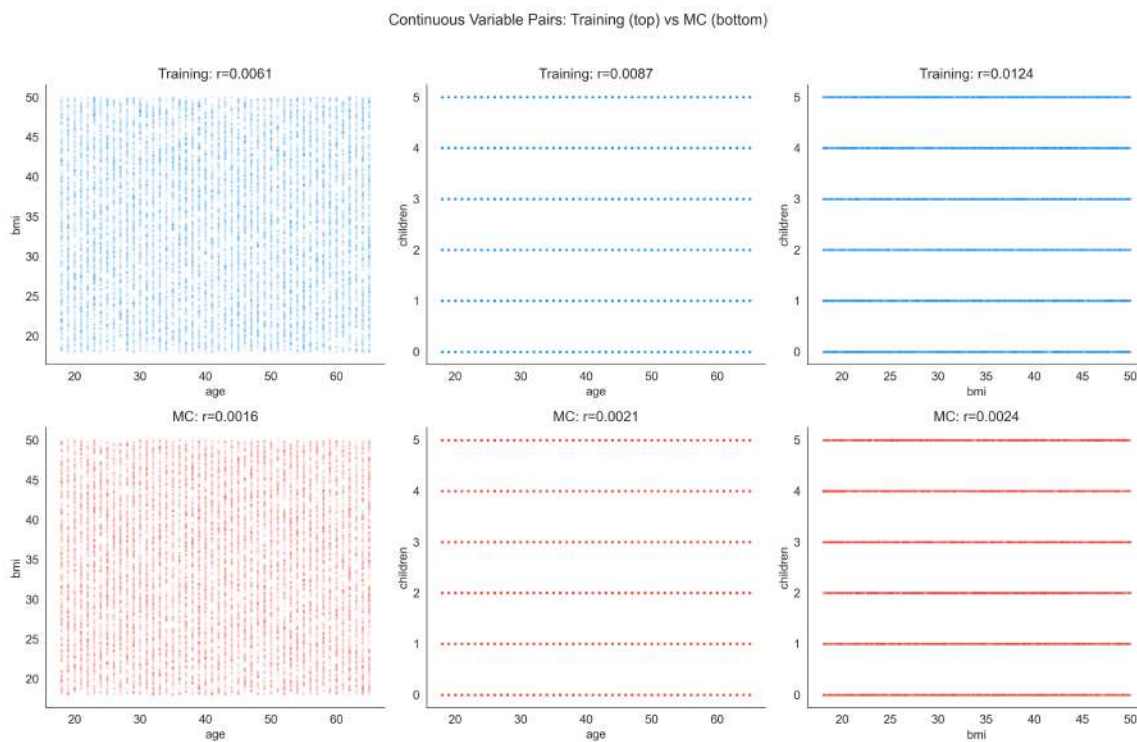


Figure 31: Continuous variable pairs: training (top row, blue) versus Monte Carlo (bottom row, red).

Figure 31 displays pairwise scatter plots for the three continuous variables (age, bmi, children). In the training data, all pairwise Pearson correlations are negligible ( $r = 0.006, 0.009, 0.012$ ). The Monte Carlo sample shows even weaker correlations ( $r = 0.002, 0.002, 0.002$ ), as expected under independence. The near-zero training correlations confirm that the product-marginal simulation does not discard meaningful continuous dependence.

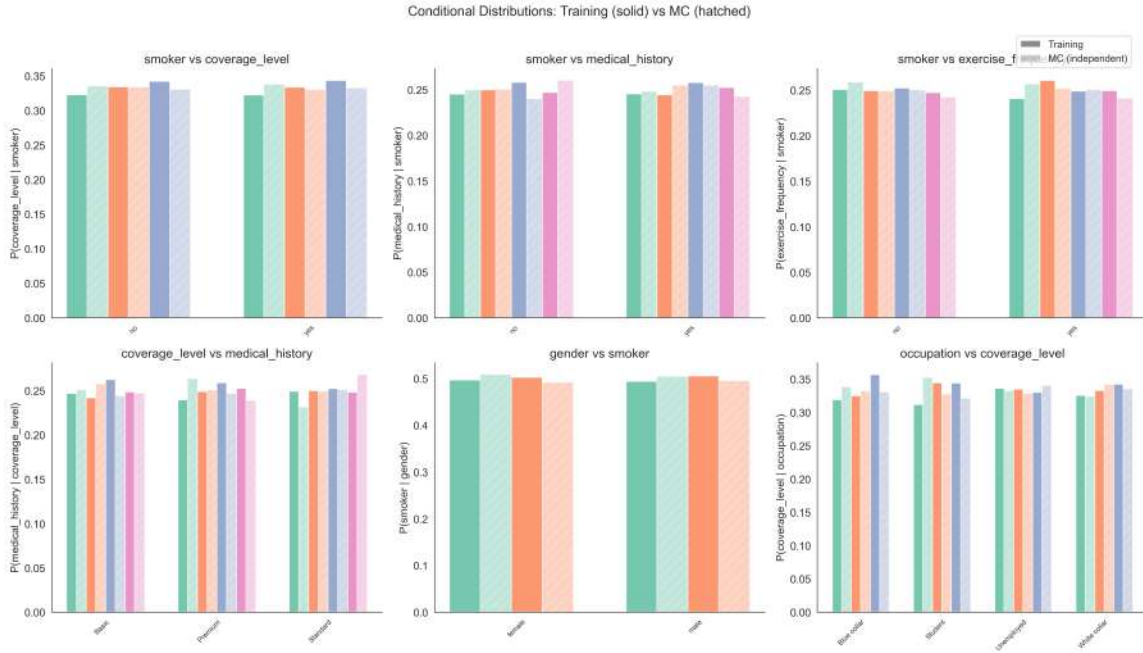


Figure 32: Conditional distributions of selected categorical variable pairs: training (solid bars) versus Monte Carlo (hatched bars).

Figure 32 compares the conditional distributions of six key categorical variable pairs. In each panel, the conditional proportions are approximately uniform across the categories of the conditioning variable, both in the training data and in the Monte Carlo sample. For instance, the distribution of `coverage_level` given `smoker` is near-uniform in both regimes, and the distribution of `medical_history` given `smoker` is similarly uniform. These cross-tabulations indicate that the training-data categoricals are effectively independent of one another, and the product-marginal simulation preserves this structure.

These bivariate diagnostics complement the correlation-matrix and Cramér's  $V$  comparisons reported in Section 3.7, which showed that the Pearson correlation structure and categorical association structure of the training data are closely reproduced by the parametric Monte Carlo sample.

### Higher-order interaction check

To confirm that the negligible dependence extends beyond pairwise associations, three-way joint proportions were computed for the four categorical variables most relevant to the high-cost analysis (`smoker`, `coverage_level`, `medical_history`, `family_medical_history`).

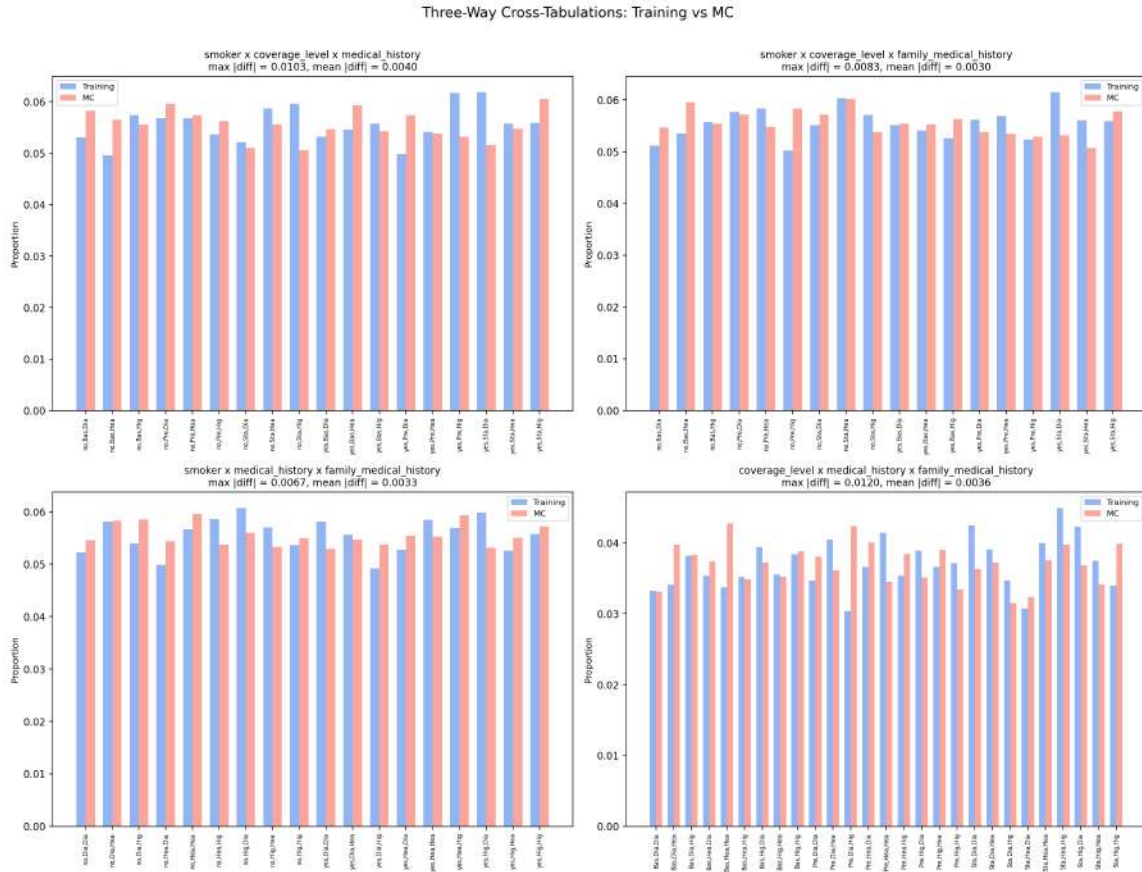


Figure 33: Three-way cross-tabulations of the four key categorical variables: training (blue) versus Monte Carlo (red).

Figure 33 presents four three-way cross-tabulations. Across all triplets, the maximum absolute difference between the training and Monte Carlo joint proportions is 0.012 (`coverage_level`  $\times$  `medical_history`  $\times$  `family_medical_history`), and the mean absolute differences are between 0.003 and 0.004. The near-uniform joint proportions in the training data confirm that higher-order interactions among the categorical variables are negligible, and the product-marginal simulation does not discard meaningful multi-way dependence.

### Multivariate two-sample test

As a formal test of the overall joint-distribution similarity, the energy distance between the training and Monte Carlo samples was computed.

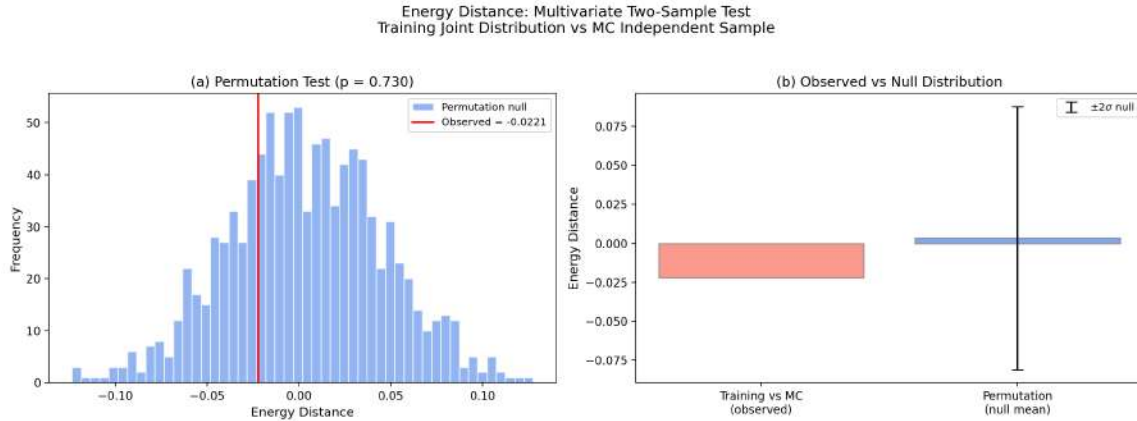


Figure 34: Energy distance permutation test: null distribution and observed statistic.

Figure 34 reports the result of a permutation-based energy distance test with 1 000 permutations. The observed energy distance is  $-0.022$ , which falls well within the permutation null distribution (panel a). The permutation  $p$ -value is  $0.730$ , indicating that the observed discrepancy between the training and Monte Carlo joint distributions is not statistically significant. The observed energy distance is smaller in magnitude than the null standard deviation ( $0.042$ ), confirming that the joint distributions are indistinguishable by this non-parametric criterion.

### Non-linear manifold overlay

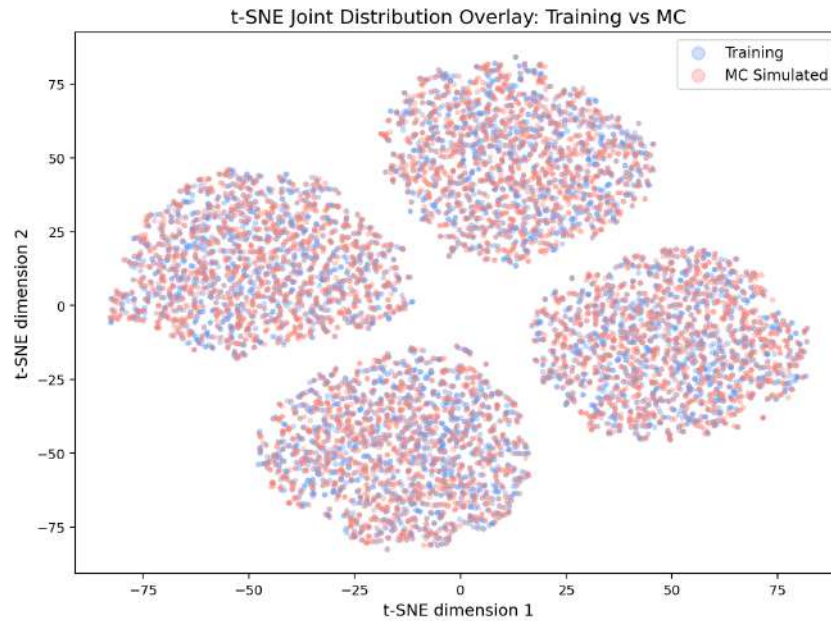


Figure 35: t-SNE projection of training versus Monte Carlo joint distributions.

Figure 35 provides a nonlinear complement to the linear PCA overlay reported in Section 3.7. The t-SNE projection maps both the training and Monte Carlo profiles into two dimensions, preserving local neighbourhood structure. The resulting point clouds are thoroughly intermixed across all identified clusters. No region of the manifold is dominated by one sample, confirming that the Monte Carlo profiles do not occupy novel regions of the feature space that are unrepresented in the training data. This result, combined with the PCA overlay, the energy distance test ( $p = 0.730$ ), and the three-way cross-tabulations, provides converging evidence that the product-marginal independence assumption does not materially distort the joint distributional structure of the training data.

### Marginal distributions

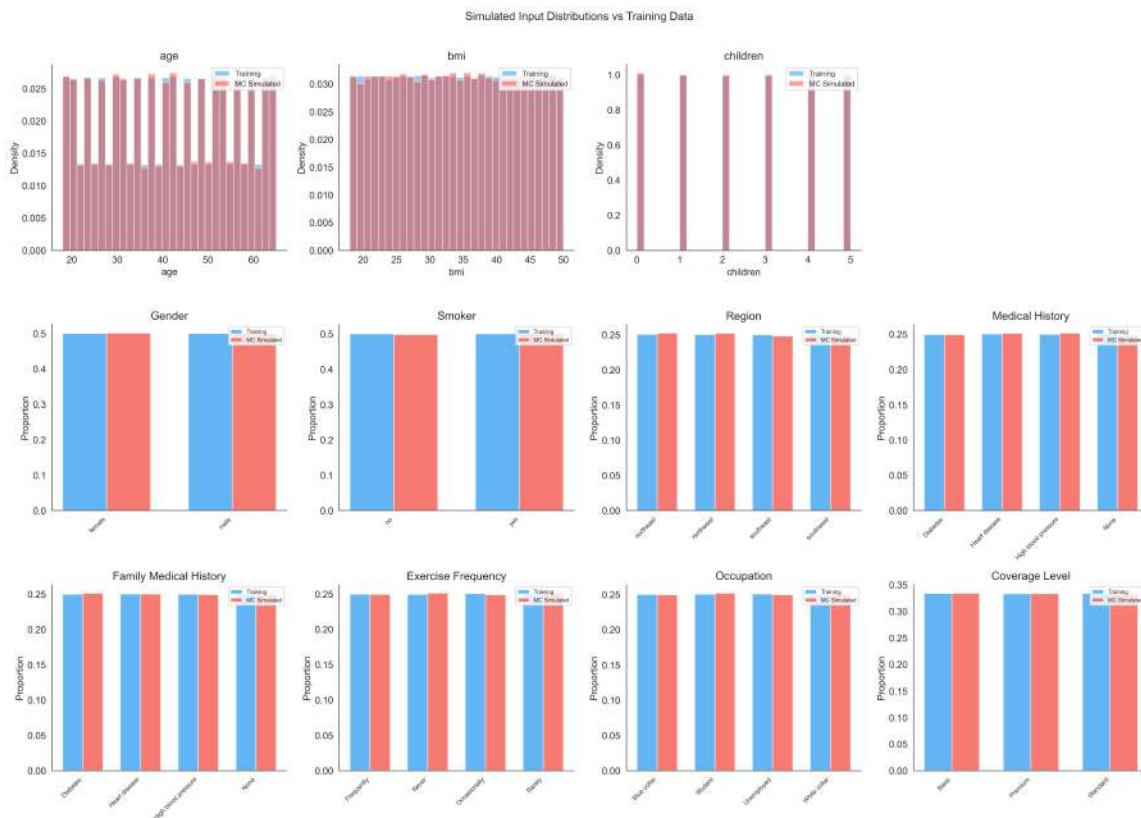


Figure 36: Marginal distributions of all 11 raw predictor variables: training versus Monte Carlo.

Figure 36 confirms that the simulated marginals closely match the training-data marginals for all 11 variables. The continuous variables (`age`, `bmi`) show near-uniform histograms in both samples; the discrete count variable (`children`) reproduces the approximately equal proportions across levels 0 to 5. For all binary and categorical variables, the simulated category proportions closely match the training-set proportions estimated via equa-

tion (3.6.2). The parametric simulation therefore faithfully implements the distributional assumptions stated in Table 13, at both the joint and marginal levels.

### 4.2.2 Model-implied cost distribution

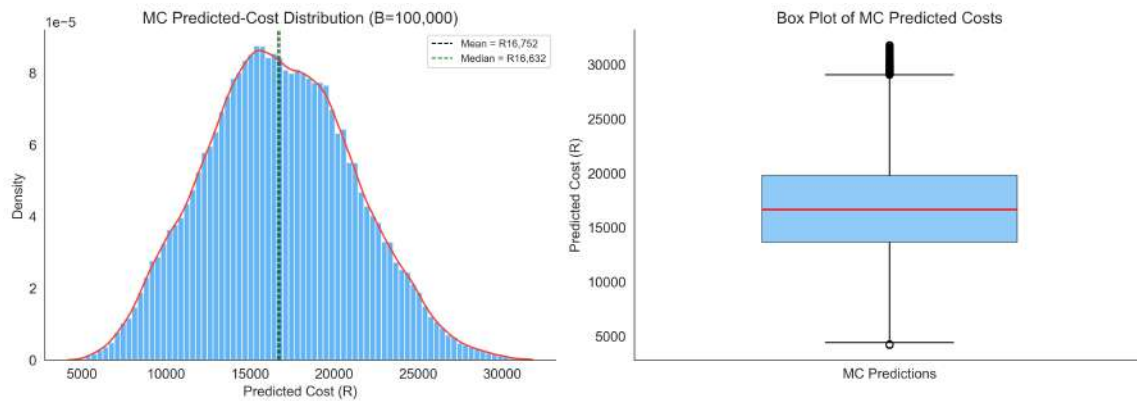


Figure 37: Monte Carlo predicted-cost distribution ( $B = 100\,000$ ): histogram with kernel density overlay (left) and box plot (right).

Figure 37 displays the model-implied distribution of predicted insurance costs. Table 25 reports the distributional summary statistics.

Table 25: Distributional summary of the Monte Carlo predicted costs ( $B = 100\,000$ ).

| Statistic          | Value   |
|--------------------|---------|
| Mean               | R16 752 |
| Standard deviation | R4 417  |
| Skewness           | 0.132   |
| Kurtosis           | −0.337  |
| Minimum            | R4 172  |
| 25th percentile    | R13 621 |
| Median             | R16 632 |
| 75th percentile    | R19 792 |
| 90th percentile    | R22 585 |
| 95th percentile    | R24 249 |
| 99th percentile    | R27 049 |
| Maximum            | R31 781 |

The distribution is approximately symmetric (skewness = 0.132) and slightly platykurtic (kurtosis = −0.337), with a mean of R16 752 and a standard deviation of R4 417. The

interquartile range spans R13 621 to R19 792, and the upper tail extends to a maximum predicted cost of R31 781. The near-zero skewness and the close agreement between the mean and median (R16 752 versus R16 632) indicate that the predicted-cost distribution is centred, without heavy-tailed distortion.

### 4.2.3 Convergence diagnostics

The Monte Carlo standard error and running distributional summaries were recorded at increasing simulation sizes  $B \in \{1\,000, 2\,500, 5\,000, 10\,000, 25\,000, 50\,000, 100\,000\}$  following the convergence monitoring procedure in Section 3.6.

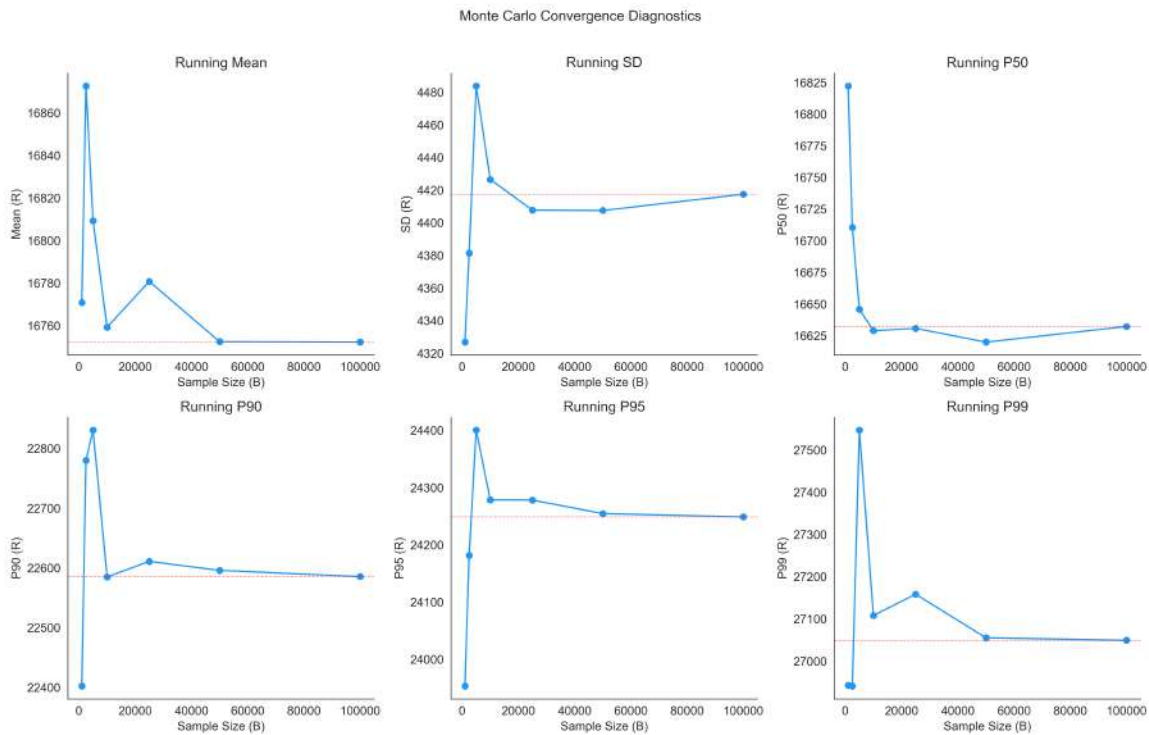


Figure 38: Monte Carlo convergence diagnostics by simulation size  $B$ .

Figure 38 shows that the running mean stabilises within a narrow band (R16 752 to R16 873) across all simulation sizes. The running standard deviation converges from R4 327 at  $B = 1\,000$  to R4 417 at  $B = 100\,000$ . The median and upper-tail quantiles exhibit similar stability, with the 95th percentile ranging from R23 953 to R24 400 across all sizes. Table 26 reports the [Monte Carlo Standard Error \(MCSE\)](#) at each simulation size.

Table 26: Monte Carlo convergence: running summaries and MCSE by simulation size.

| $B$     | Mean   | SD    | Median | P95    | P99    | MCSE  |
|---------|--------|-------|--------|--------|--------|-------|
| 1 000   | 16 771 | 4 327 | 16 822 | 23 953 | 26 942 | 136.8 |
| 2 500   | 16 873 | 4 381 | 16 711 | 24 181 | 26 941 | 87.6  |
| 5 000   | 16 809 | 4 484 | 16 646 | 24 400 | 27 548 | 63.4  |
| 10 000  | 16 759 | 4 427 | 16 629 | 24 278 | 27 108 | 44.3  |
| 25 000  | 16 781 | 4 408 | 16 631 | 24 278 | 27 158 | 27.9  |
| 50 000  | 16 752 | 4 408 | 16 620 | 24 255 | 27 055 | 19.7  |
| 100 000 | 16 752 | 4 417 | 16 632 | 24 249 | 27 049 | 14.0  |

The [MCSE](#) decreases from R136.8 at  $B = 1\,000$  to R14.0 at  $B = 100\,000$ , representing 0.08% of the mean predicted cost. At the selected simulation size of  $B = 100\,000$ , the running summaries have stabilised and the simulation precision is adequate for the subsequent distributional and attribution analyses.

The theoretical [MCSE](#) for the sample mean of  $B$  independent draws from a distribution with standard deviation  $\sigma$  is  $\sigma/\sqrt{B}$ , which decreases at rate  $\mathcal{O}(B^{-1/2})$ . The empirical convergence rate in [Table 26](#) closely follows this theoretical law: the ratio  $\text{MCSE}(B=1\,000) / \text{MCSE}(B=10\,000) = 136.8/44.3 = 3.09$  matches the theoretical ratio  $\sqrt{10\,000/1\,000} = \sqrt{10} \approx 3.16$ , and  $\text{MCSE}(B=1\,000) / \text{MCSE}(B=100\,000) = 136.8/14.0 = 9.77$  matches  $\sqrt{100} = 10.0$ . The simulation therefore exhibits the canonical Monte Carlo convergence behaviour, confirming that no pathological variance inflation or serial dependence is present in the simulated profiles.

The running upper-tail percentiles in [Table 26](#) provide an empirical assessment of quantile estimation precision. The 95th percentile varies within a band of R447 across simulation sizes (R23 953 to R24 400), and the 99th percentile within R606 (R26 941 to R27 548). The progressively wider variation at higher quantiles reflects the decreasing effective sample size in the tail: at  $\tau_{99}$  with  $B = 100\,000$ , approximately 1 000 observations determine the percentile estimate, compared with 5 000 at  $\tau_{95}$ . The running 99th percentile has stabilised to within  $\pm 0.4\%$  of its final value from  $B = 25\,000$  onwards, indicating that the tail percentile estimates are sufficiently precise for the conditional Shapley analysis that follows.

#### 4.2.4 Comparison with observed training charges

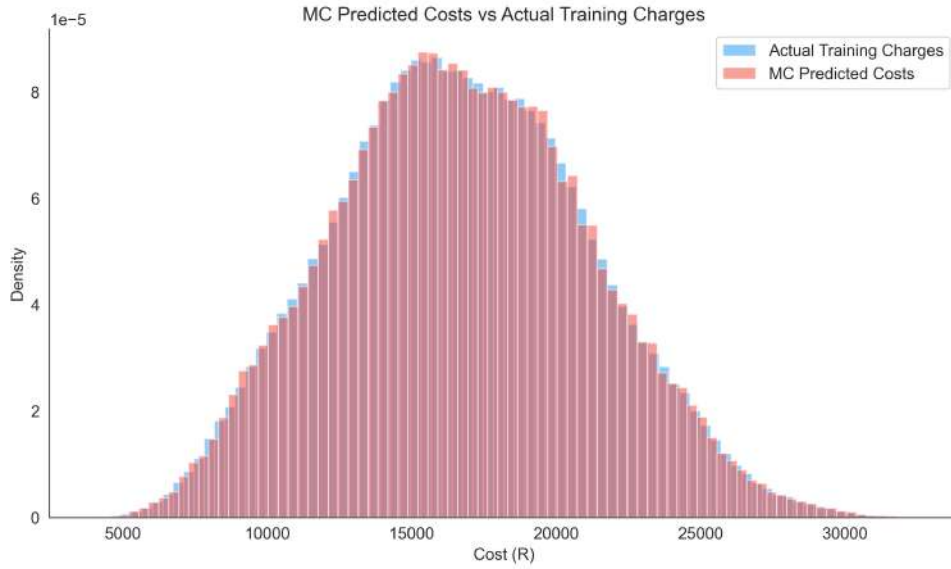


Figure 39: Overlay of the Monte Carlo predicted-cost distribution (red) and the observed training-set charge distribution (blue).

Figure 39 overlays the Monte Carlo predicted-cost histogram with the observed training-set charge distribution. The two distributions share similar location, spread and shape: both are unimodal, roughly symmetric, and span the range from approximately R5 000 to R32 000. The close agreement confirms that the fitted DNN, when evaluated on profiles drawn from the parametric input distribution, produces a predicted-cost landscape that is consistent with the distributional characteristics of the observed training data. Minor differences in the tails reflect the smoothing behaviour of the regression model and the slight mismatch between the product-marginal simulation law and the empirical joint distribution.

### 4.3 Residual-Augmented Monte Carlo Sensitivity

The residual-augmented sensitivity analysis (Section 3.8) was applied to the  $B = 100\,000$  base Monte Carlo predictions. Centred fitted residuals were drawn from the empirical residual distribution  $\hat{F}_\varepsilon$  (equation (3.8.3)), and a fixed perturbation band of  $\pm 0.5 \hat{\sigma}_\varepsilon$  was computed following equation (3.8.5).

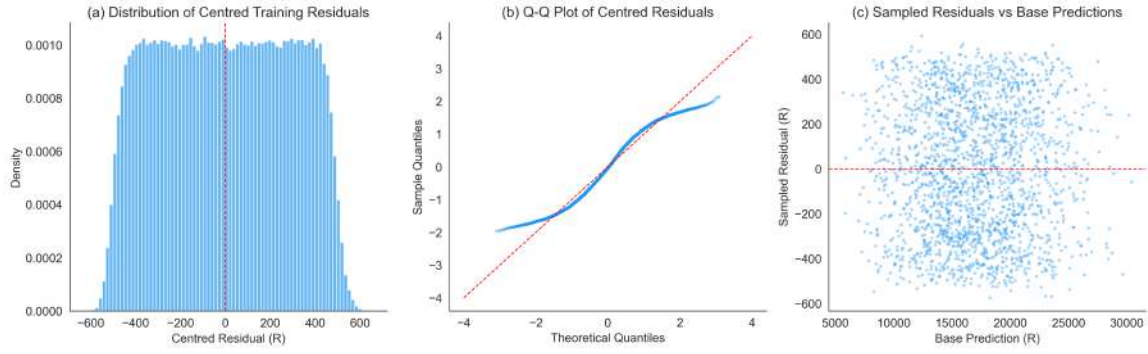


Figure 40: Centred training residual diagnostics for the augmented simulation.

Figure 40 characterises the centred training residuals from which the augmented draws are sampled. The histogram in panel (a) is symmetric and bell-shaped, confirming that the residual distribution is well-centred and does not contain systematic bias. The Q-Q plot in panel (b) shows slight tail compression relative to a Gaussian reference, consistent with the mild platykurtic behaviour noted in the model diagnostics of Section 3.5. Panel (c) plots the sampled residuals against the base predictions: the scatter is random and centred at zero across the full prediction range (R5 000 to R30 000), indicating that the residual magnitude does not depend on the predicted cost level. These properties confirm that the empirical residual distribution is suitable for the perturbation analysis.

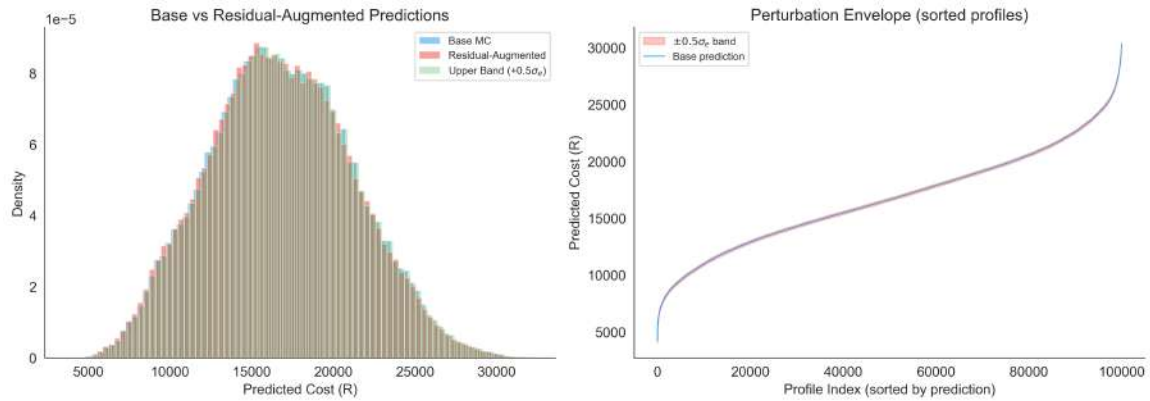


Figure 41: Residual-augmented Monte Carlo sensitivity analysis.

Figure 41 (left panel) shows that the residual-augmented distribution closely overlaps the base Monte Carlo distribution. The residual perturbation does not shift the central tendency or materially alter the distributional shape: the mode, spread and tail behaviour of all three distributions remain aligned. The right panel displays the sorted base predictions with the  $\pm 0.5\hat{\sigma}_\epsilon$  perturbation envelope. The envelope forms a narrow, uniform band around the base prediction curve across the full range of predicted costs, from approximately R5 000 at the lower end to R30 000 at the upper end. The width of the band remains constant relative to the prediction magnitude, confirming that the fitted residual

variability is small and homogeneous across the predicted-cost range.

These results indicate that the distributional conclusions drawn from the base Monte Carlo analysis are not sensitive to the magnitude of the fitted model’s residual error. The base predicted-cost distribution is therefore a stable foundation for the Shapley-value attribution analysis that follows.

## 4.4 Shapley-Value Attribution Results

Shapley values were computed for all  $B = 100\,000$  simulated profiles using the gradient-based SHAP explainer described in Table 19. Dummy-level attributions were aggregated to the raw-variable level ( $q = 11$ ) using equation (3.9.1).

### 4.4.1 Global feature importance

Global feature importance was assessed using the mean absolute Shapley value  $I_g$  (equation (3.9.3)) and the mean signed Shapley value  $\bar{\phi}_g$  (equation (3.9.2)), computed over all  $B$  simulated profiles.

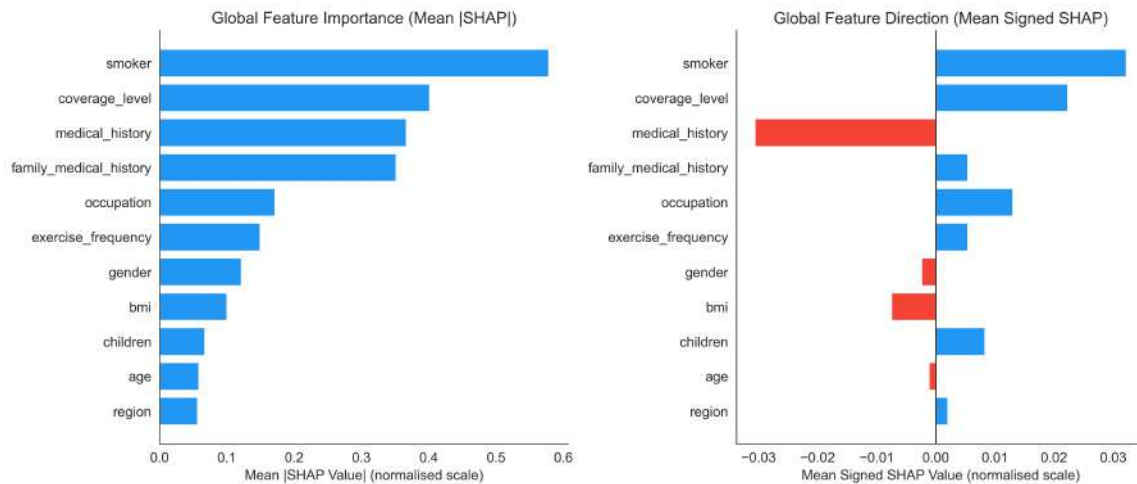


Figure 42: Global Shapley feature importance for all 11 raw variables.

Figure 42 displays the global feature-importance ranking. Table 27 reports the numerical values.

Table 27: Global Shapley feature importance: mean signed and mean absolute values for all 11 raw variables on the normalised scale.

| Variable               | Mean signed SHAP | Mean  SHAP |
|------------------------|------------------|------------|
| smoker                 | +0.032           | 0.578      |
| coverage_level         | +0.022           | 0.401      |
| medical_history        | −0.031           | 0.367      |
| family_medical_history | +0.005           | 0.351      |
| occupation             | +0.013           | 0.171      |
| exercise_frequency     | +0.005           | 0.149      |
| gender                 | −0.002           | 0.122      |
| bmi                    | −0.007           | 0.100      |
| children               | +0.008           | 0.067      |
| age                    | −0.001           | 0.059      |
| region                 | +0.002           | 0.057      |

The four most important variables form a clear top tier: **smoker** ( $I_g = 0.578$ ), **coverage\_level** (0.401), **medical\_history** (0.367) and **family\_medical\_history** (0.351). These four variables together account for the large majority of the fitted model’s attribution magnitude. A second tier of moderate contributors includes **occupation** (0.171) and **exercise\_frequency** (0.149). The remaining five variables (**gender**, **bmi**, **children**, **age** and **region**) each have mean |SHAP| values below 0.13.

The mean signed values are close to zero for all variables, indicating that the positive and negative Shapley contributions approximately cancel out across the simulated profiles. The largest signed value in absolute terms is **medical\_history** (−0.031), reflecting a mild directional asymmetry in how the model attributes cost to different medical history categories.

## 4.4.2 Feature-level SHAP beeswarm

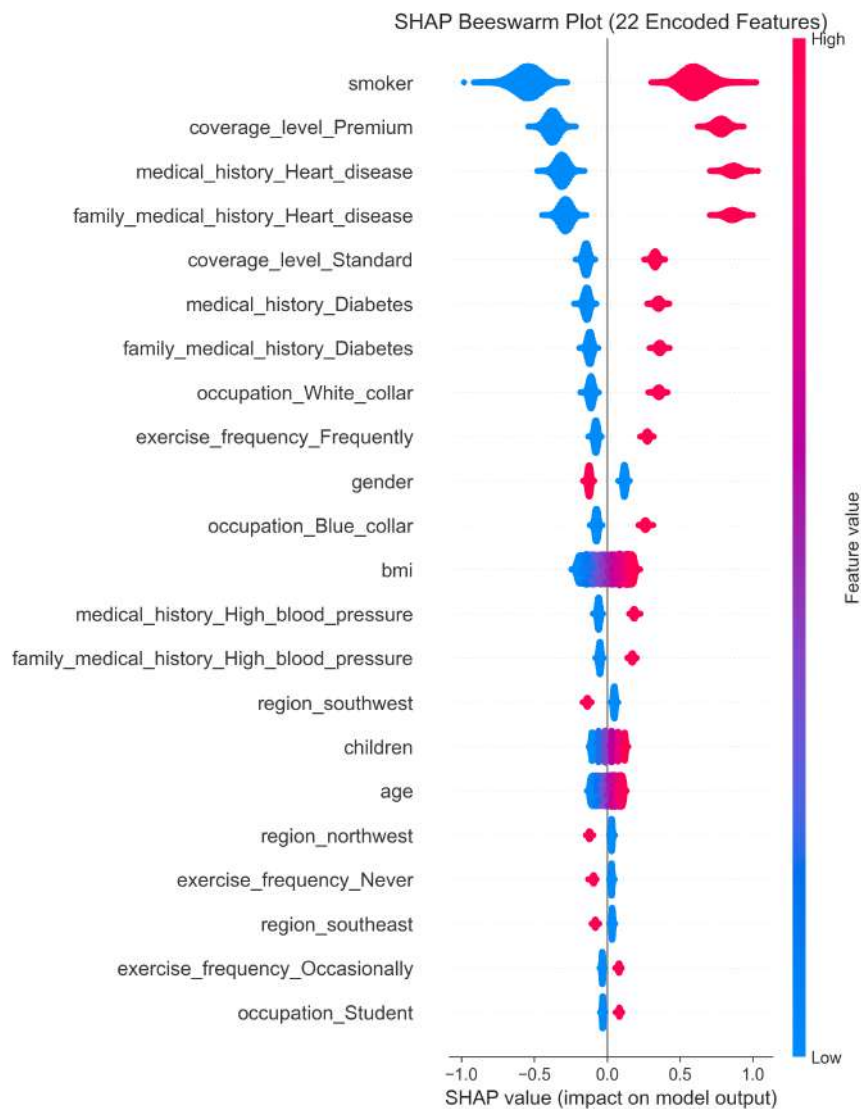


Figure 43: SHAP beeswarm plot for all 22 encoded features.

Figure 43 provides a feature-level view of the Shapley attributions across all 22 encoded inputs. Several patterns are visible:

1. **Smoker:** The binary `smoker` variable produces a clear bimodal separation. High feature values (red, `smoker = yes`) yield large positive SHAP values (approximately  $+0.5$  to  $+1.0$ ), while low feature values (blue, `smoker = no`) yield large negative SHAP values (approximately  $-0.5$  to  $-0.7$ ). This confirms that smoking status is the single largest determinant of fitted predicted costs.
2. **Coverage level:** The `coverage_level_Premium` dummy exhibits a similar bimodal pattern: premium coverage pushes predictions upward ( $+0.3$  to  $+0.7$ ), while its

absence pulls predictions downward. The `coverage_level_Standard` dummy shows a complementary pattern of smaller magnitude.

3. **Medical and family medical history:** The Heart disease dummies for both `medical_history` and `family_medical_history` produce the next largest attributions. High feature values (Heart disease present) consistently increase predicted costs by approximately +0.3 to +0.8.
4. **Continuous variables:** `bmi`, `children` and `age` display more diffuse SHAP distributions centred near zero, consistent with their smaller global importance. The `bmi` swarm shows a broad purple-to-pink gradient, reflecting its continuous nature, but the individual SHAP magnitudes remain small (mostly within  $\pm 0.15$ ).

#### 4.4.3 Training versus Monte Carlo attribution invariance

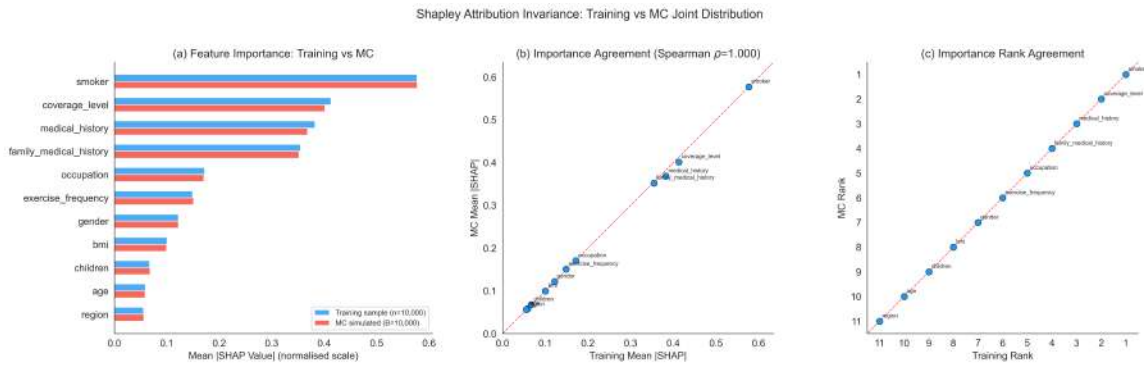


Figure 44: Shapley attribution invariance: training data versus Monte Carlo profiles.

Figure 44 compares the Shapley attributions computed on the training data with those computed on the Monte Carlo simulated profiles. The Spearman rank correlation between the two feature-importance vectors is  $\rho_S = 1.000$ : all 11 raw variables occupy the same importance rank under both input regimes. The scatter plot in panel (b) shows that the mean  $|\text{SHAP}|$  values fall on the diagonal, with no systematic deviation. This result extends the parametric-versus-bootstrap invariance finding of Section 3.7 (Spearman  $\rho = 0.991$ ) and confirms that the fitted DNN’s internal feature-importance structure is preserved when evaluated on the Monte Carlo simulated input space.

## 4.5 High-Cost Conditional Shapley Analysis

The high-cost region was defined using the quantile-based thresholds  $u_\tau$  (equation (2.2.13)) applied to the Monte Carlo predicted-cost distribution. Table 28 reports the three threshold values and the corresponding high-cost set sizes.

Table 28: High-cost thresholds from the Monte Carlo predicted-cost distribution.

| Threshold   | $\tau$ | $u_\tau$ (R) | $ \mathcal{H}_\tau $ | Proportion |
|-------------|--------|--------------|----------------------|------------|
| $\tau_{90}$ | 0.90   | 22 585       | 10 000               | 0.10       |
| $\tau_{95}$ | 0.95   | 24 249       | 5 000                | 0.05       |
| $\tau_{99}$ | 0.99   | 27 049       | 1 000                | 0.01       |

#### 4.5.1 Conditional versus global importance

Conditional mean absolute Shapley values were computed for profiles in  $\mathcal{H}_\tau$  at each threshold and compared with the global values.

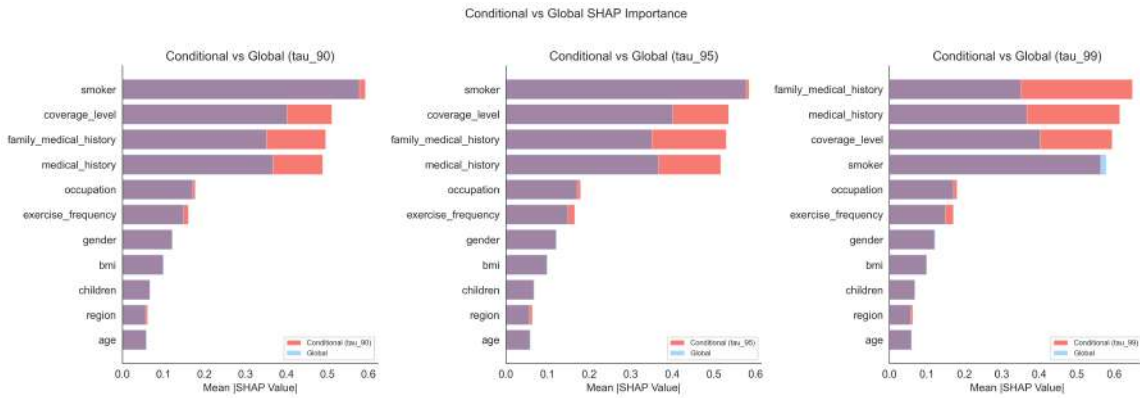


Figure 45: Conditional versus global mean  $|\text{SHAP}|$  importance at three upper-tail thresholds.

Figure 45 reveals a progressive shift in the feature-importance ranking as the threshold moves into the extreme upper tail:

- At  $\tau_{90}$ , the ranking mirrors the global ordering: **smoker** remains the most important variable, followed by **coverage\_level**, **medical\_history** and **family\_medical\_history**. The conditional importance values are slightly higher than the global values for the top four variables, reflecting the more extreme nature of the high-cost profiles.
- At  $\tau_{95}$ , the top four variables maintain similar positions, but **family\_medical\_history** and **medical\_history** show increased conditional importance relative to their global values.
- At  $\tau_{99}$ , a ranking reversal occurs: **family\_medical\_history** ( $I_{g,0.99} = 0.649$ ) and **medical\_history** ( $I_{g,0.99} = 0.615$ ) overtake **smoker** ( $I_{g,0.99} = 0.563$ ) as the most important variables. **coverage\_level** ( $I_{g,0.99} = 0.595$ ) also exceeds **smoker** in the extreme tail.

Table 29 reports the conditional mean  $|\text{SHAP}|$  values at all three thresholds alongside the global values.

Table 29: Mean absolute Shapley values: global and conditional at each high-cost threshold (normalised scale), with 95% bootstrap confidence intervals at  $\tau_{99}$ .

| Variable                      | Global | $\tau_{90}$ | $\tau_{95}$ | $\tau_{99}$ | 95% CI at $\tau_{99}$ |
|-------------------------------|--------|-------------|-------------|-------------|-----------------------|
| <b>smoker</b>                 | 0.578  | 0.593       | 0.585       | 0.563       | [0.559, 0.568]        |
| <b>coverage_level</b>         | 0.401  | 0.511       | 0.536       | 0.595       | [0.585, 0.604]        |
| <b>medical_history</b>        | 0.367  | 0.488       | 0.517       | 0.615       | [0.604, 0.625]        |
| <b>family_medical_history</b> | 0.351  | 0.496       | 0.530       | 0.649       | [0.638, 0.659]        |
| <b>occupation</b>             | 0.171  | 0.178       | 0.180       | 0.181       | [0.176, 0.185]        |
| <b>exercise_frequency</b>     | 0.149  | 0.161       | 0.166       | 0.171       | [0.165, 0.178]        |
| <b>gender</b>                 | 0.122  | 0.121       | 0.121       | 0.120       | [0.119, 0.121]        |
| <b>bmi</b>                    | 0.100  | 0.099       | 0.099       | 0.100       | [0.096, 0.103]        |
| <b>children</b>               | 0.067  | 0.068       | 0.068       | 0.069       | [0.067, 0.072]        |
| <b>age</b>                    | 0.059  | 0.058       | 0.058       | 0.059       | [0.057, 0.061]        |
| <b>region</b>                 | 0.057  | 0.062       | 0.063       | 0.063       | [0.060, 0.065]        |

The ranking reversal at  $\tau_{99}$  is based on  $|\mathcal{H}_{0.99}| = 1\,000$  profiles, providing a substantial sample for the conditional analysis. Four lines of evidence support the robustness of this finding. First, the reversal is not an isolated artefact at a single threshold: the conditional importance values for **coverage\_level**, **medical\_history** and **family\_medical\_history** increase monotonically across all four conditioning levels (global,  $\tau_{90}$ ,  $\tau_{95}$ ,  $\tau_{99}$ ), indicating a consistent trend rather than sampling noise. Second, the magnitude of the reversal is substantial: at  $\tau_{99}$ , **family\_medical\_history** exceeds **smoker** by 0.086 on the normalised scale (0.649 versus 0.563), a relative difference of 15.3%. Third, the 95% bootstrap confidence intervals (Table 29) confirm that the intervals for the top three non-smoker variables do not overlap with that of **smoker**, providing formal statistical support for the reversal. Fourth, the profile-level composition analysis (Figure 47) provides a mechanistic explanation: 99% of profiles in  $\mathcal{H}_{0.99}$  are smokers, so the near-constant value of the **smoker** variable within this set reduces its discriminating power, as a binary variable that takes the same value for virtually all members of a conditioning set necessarily contributes a near-constant Shapley attribution. The convergence of these four independent lines of evidence supports the conclusion that the ranking reversal reflects a genuine structural property of the fitted model’s attribution surface, not a finite-sample artefact.

### 4.5.2 Threshold stability of the importance ranking

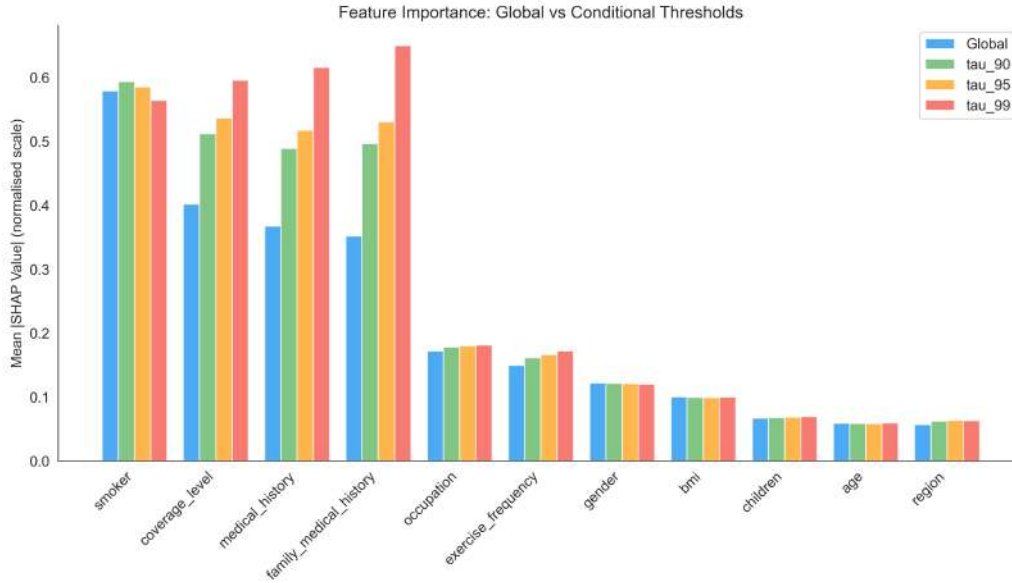


Figure 46: Feature importance across thresholds: mean  $|\text{SHAP}|$  values at the global,  $\tau_{90}$ ,  $\tau_{95}$  and  $\tau_{99}$  levels for all 11 raw variables.

Figure 46 displays the threshold-comparison bar chart. The dominant pattern is a monotonic increase in the conditional importance of `coverage_level`, `medical_history` and `family_medical_history` as the threshold moves from the global level through  $\tau_{90}$  and  $\tau_{95}$  to  $\tau_{99}$ . In contrast, `smoker` shows a marked decline in conditional importance at the most extreme threshold (0.563 at  $\tau_{99}$  versus 0.578 globally). This decline does not indicate that smoking status becomes unimportant; rather, it reflects a ceiling effect: nearly all profiles in the extreme upper tail are smokers (as confirmed in Figure 47 below), so the binary variable contributes a near-constant positive attribution with reduced variability, while the remaining features differentiate among the high-cost profiles.

The second tier of variables (`occupation`, `exercise_frequency`, `gender`, `bmi`, `children`, `age` and `region`) remains stable across all thresholds, with conditional importance values close to their global values. This stability indicates that these variables contribute a consistent baseline attribution regardless of the cost region.

The ranking reversal at  $\tau_{99}$  directly addresses the research gap identified in Section 2.11: existing health-cost prediction studies typically report global feature importance on observed data only (Kaushik et al., 2022; Rozemberczki et al., 2022), without conditioning attributions on specific regions of the predicted-cost distribution. The conditional Shapley analysis demonstrates that global importance rankings can be misleading when the interest lies in the upper tail, because a variable that dominates on average (smoking status) may lose its discriminating power within an extreme subpopulation where it is nearly con-

stant. This distinction has not been documented in the health insurance cost-prediction literature reviewed in Chapter 2.

### 4.5.3 High-cost profile characteristics

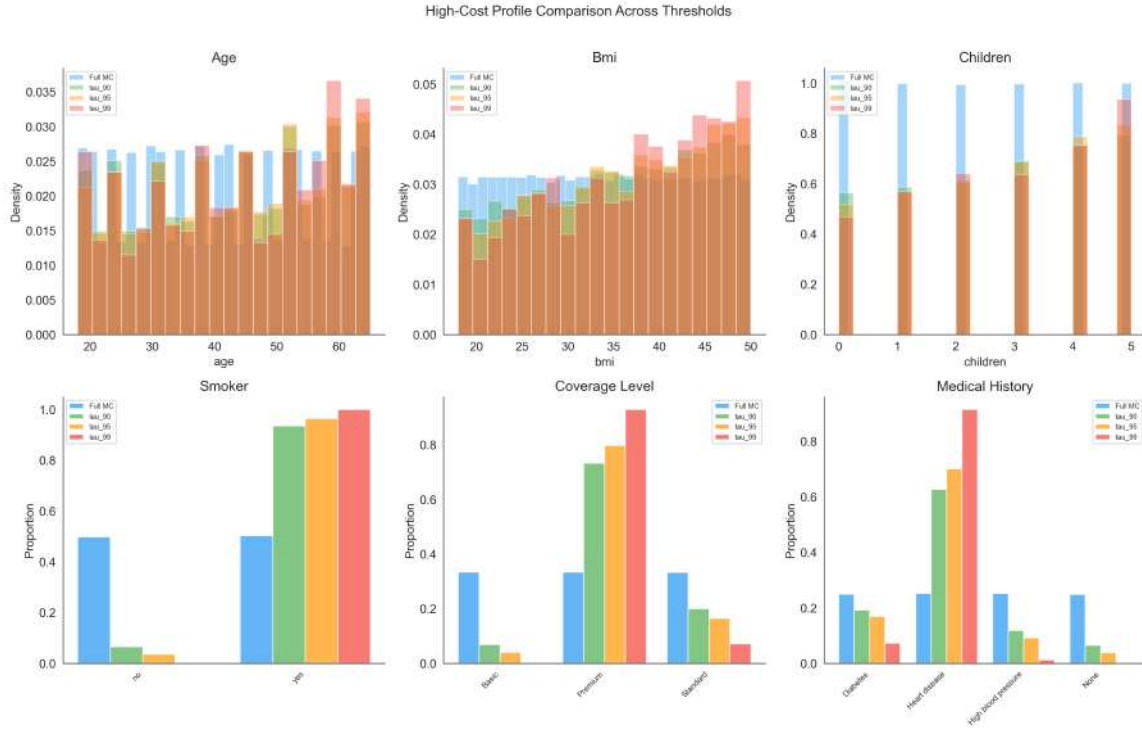


Figure 47: Distributional profiles of the high-cost sets compared with the full Monte Carlo sample.

Figure 47 profiles the composition of the high-cost sets at each threshold. The categorical panels reveal a progressive concentration:

- **Smoker:** The proportion of smokers increases from approximately 50% in the full Monte Carlo sample to approximately 93% at  $\tau_{90}$ , 96% at  $\tau_{95}$  and 99% at  $\tau_{99}$ . At the most extreme threshold, virtually all high-cost profiles are smokers.
- **Coverage level:** The proportion of Premium-tier policyholders rises from approximately 33% (full MC) to 75% at  $\tau_{90}$ , 80% at  $\tau_{95}$  and approximately 90% at  $\tau_{99}$ . Basic-tier policyholders are nearly absent from the extreme tail.
- **Medical history:** Heart disease dominates the high-cost sets: its proportion increases from approximately 25% (full MC) to 68% at  $\tau_{90}$ , 77% at  $\tau_{95}$  and approximately 85% at  $\tau_{99}$ . Profiles with no medical history condition are progressively excluded from the upper tail.

The continuous variables (`age`, `bmi`, `children`) show little change in their distributional shapes across thresholds, consistent with their small and stable conditional Shapley values. This confirms that the high-cost region is defined primarily by the categorical risk factors rather than the continuous covariates.

#### 4.5.4 Representative high-cost waterfall decomposition

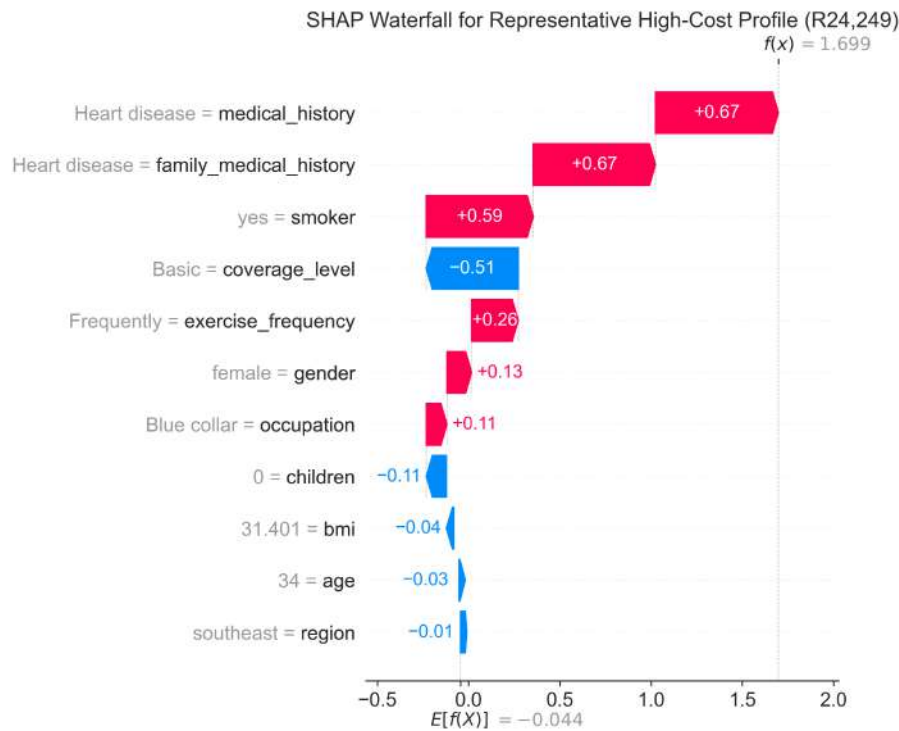


Figure 48: SHAP waterfall plot for a representative high-cost profile at the  $\tau_{95}$  threshold (predicted cost R24 249).

Figure 48 decomposes a representative high-cost profile at the  $\tau_{95}$  threshold. The profile has the following characteristics: `age` = 34, `bmi` = 31.4, `children` = 0, `smoker` = yes, `family_medical_history` = Heart disease, `exercise_frequency` = Frequently, `medical_history` = Heart disease, `gender` = female, `occupation` = Blue collar, `region` = southeast, `coverage_level` = Basic.

The waterfall decomposition shows that the prediction is driven upward from the baseline ( $E[f(X)] = -0.044$ ) to the final prediction ( $f(x) = 1.699$ ) primarily by three variables: `medical_history` (+0.67), `family_medical_history` (+0.67) and `smoker` (+0.59). A negative contribution from `coverage_level` (-0.51) partially offsets these positive effects, reflecting that the Basic tier carries a lower predicted cost than the Premium tier. The remaining variables contribute smaller amounts: `exercise_frequency` (+0.26), `gender` (+0.13), `occupation` (+0.11), while `children` (-0.11), `bmi` (-0.04), `age` (-0.03) and `region` (-0.01) exert modest negative effects.

This waterfall illustrates the additive decomposition guaranteed by the Shapley efficiency axiom: the contributions sum exactly to the difference between the prediction and the baseline, as required by equation (2.2.11).

## 4.6 Integration of Monte Carlo and Shapley Results

The Monte Carlo and Shapley analyses provide complementary perspectives on the fitted DNN's behaviour. The Monte Carlo simulation identifies the distributional regions of interest within the predicted-cost landscape, while the Shapley-value decomposition reveals which input variables are associated with those regions.

The Monte Carlo predicted-cost distribution (Figure 37) is approximately symmetric with a mean of R16 752 and an upper tail extending to R31 781. Within this landscape, three quantile-based thresholds define high-cost regions of increasing severity:  $u_{0.90} = \text{R}22\,585$ ,  $u_{0.95} = \text{R}24\,249$  and  $u_{0.99} = \text{R}27\,049$ .

The global Shapley analysis (Figure 42) identifies `smoker` as the dominant driver of fitted predictions across the full simulated input space. However, the conditional analysis (Figures 45 and 46) reveals that the attribution structure changes in the upper tail. As the high-cost threshold becomes more stringent, the conditional importance of `coverage_level`, `medical_history` and `family_medical_history` increases monotonically, while the importance of `smoker` declines slightly.

The profile-level analysis (Figure 47) explains this re-ordering: at  $\tau_{99}$ , virtually all high-cost profiles are smokers (99%), so smoking status no longer differentiates among profiles within the extreme tail. The variables that differentiate high-cost profiles from one another are therefore coverage tier, personal medical history and family medical history. At  $\tau_{99}$ , approximately 90% of high-cost profiles hold Premium coverage and 85% report Heart disease. These three variables collectively define the archetype of the extreme upper-tail profile.

The continuous covariates (`age`, `bmi`, `children`) play a limited role at all threshold levels. Their conditional importance values remain close to their global values, and their distributional shapes within the high-cost sets are not meaningfully different from those in the full Monte Carlo sample. The fitted DNN therefore assigns the dominant cost-driving role to the categorical risk factors, with the continuous variables contributing a modest, stable baseline effect.

The invariance checks reinforce the robustness of these findings. The Shapley attributions are stable whether computed on training data or Monte Carlo profiles (Spearman  $\rho = 1.000$ , Figure 44), and the residual-augmented sensitivity analysis (Figure 41) confirms that the predicted-cost distribution is not sensitive to fitted residual variability. The

attribution patterns therefore reflect the fitted model's learned internal structure rather than artefacts of the input regime or residual noise.

These results are summarised, evaluated and situated within the broader context of the study in Chapter [5](#).

## 5 Conclusion

### 5.1 Review of the Dissertation

Health insurance cost prediction is, at its core, a regression problem: the analyst seeks to estimate a continuous monetary outcome from a vector of policyholder characteristics. In the actuarial profession, this estimation has traditionally been performed using generalised linear models, and in particular the Gamma distribution with a logarithmic link, which imposes a multiplicative mean structure on the relationship between covariates and costs. The appeal of this classical framework lies in its interpretability: each coefficient describes the proportional change in the conditional mean cost associated with a unit change in a predictor, holding all other predictors fixed. This dissertation began by asking whether a more flexible model class, specifically deep neural networks, could improve upon this classical baseline, and whether the improved predictions could be analysed through methods that go beyond point-prediction accuracy.

The motivation for moving beyond point prediction was developed in Chapter 1, which argued that a fitted prediction model is a statistical object whose properties merit systematic investigation. Just as the sampling distribution of an estimator is studied through Monte Carlo experiments to understand its variability and tail behaviour, the distribution of a prediction function's outputs under a specified input law can be studied to understand the range, shape and extreme-value behaviour of its predictions. And just as the contribution of individual observations to an estimator can be decomposed through influence functions, the contribution of individual input variables to a prediction can be decomposed through Shapley values. These parallels between classical statistical methodology and modern machine-learning interpretation formed the conceptual basis of the dissertation.

The literature review in Chapter 2 established the theoretical and empirical foundations for each component of the framework. The review traced the development of supervised regression from linear models through kernel methods and tree-based ensembles to deep neural networks, noting the universal approximation properties that allow feed-forward networks to approximate continuous functions on compact sets to arbitrary accuracy (Hornik et al., 1989). The review also surveyed the health insurance pricing literature, where several studies have demonstrated that neural networks and gradient-boosted trees outperform classical actuarial models on insurance cost prediction tasks (Drewe-Boss et al., 2022; Samiuddin et al., 2023; Kaushik et al., 2022), although the evidence is not universal across all tabular insurance datasets (Richman, 2021). The Monte Carlo and Shapley-value components were reviewed in the context of their respective theoretical traditions: simulation-based inference (Glasserman, 2004; Rubinstein and Kroese, 2016) and cooperative game theory (Shapley, 1953), with the SHAP framework (Lundberg and Lee,

2017) providing the operational bridge between game-theoretic attribution and machine-learning interpretation. The review concluded with a formal identification of the research gap: the existing literature applies deep learning and Shapley-value methods in isolation, without connecting the distributional analysis of a model-implied predicted-cost landscape to conditional feature-importance attribution within specific regions of that landscape.

Chapter 3 translated the conceptual framework into a concrete empirical workflow. The analysis was based on the Kaggle Insurance Data for Machine Learning benchmark dataset ([Kaggle.com](https://www.kaggle.com/c/insurance-claims), 2023), containing one million policyholder records with eleven raw predictors and a continuous monetary response. The chapter specified the data preprocessing procedure, including the training, validation and test partition (524 288/237 856/237 856) and the data-leakage prevention protocols that ensured all preprocessing parameters were computed exclusively on the training partition. The funnel architecture (Table 6) was specified, together with the five-phase sequential hyperparameter comparison that evaluated activation functions, learning rates, batch sizes, optimisers and momentum parameters. The Gamma GLM with a logarithmic link served as the classical actuarial benchmark, and a random forest and an XGBoost gradient-boosted tree regressor were fitted as additional benchmarks spanning the model-class spectrum from linear to piecewise-constant to smooth nonlinear. The Monte Carlo simulation design specified the parametric marginal distributions, the independence assumption and the convergence monitoring procedure. The joint-distribution validation, residual-augmented sensitivity analysis and Shapley-value implementation were documented in full, and a consolidated table of modelling assumptions (Table 21) provided a formal statement of the interpretive boundaries.

The empirical results in Chapter 4 demonstrated the value of the integrated framework. The five-phase hyperparameter comparison identified CELU activation, learning rate  $\eta = 0.0005$ , batch size 256, NAdam optimiser and momentum  $\beta_1 = 0.90$  as the selected configuration. The systematic, sequential nature of this comparison is worth emphasising: by fixing all previously selected hyperparameters at each phase and varying only one candidate, the comparison provides a transparent selection rationale that can be replicated by other researchers. This approach stands in contrast to grid search or random search procedures that explore the hyperparameter space simultaneously and produce configurations whose selection rationale is less interpretable. The fitted DNN achieved test-set performance of  $R^2 = 0.9957$ , MAE = R251 and RMSE = R290, with near-identical training and test metrics ruling out overfitting. The baseline comparison established that the DNN outperforms the Gamma GLM (log-link) by a substantial margin, reducing the test RMSE by 68.2% and the test MAE by 59.8%. The GLM's inferior fit was attributable to the log-link's multiplicative structure, which does not match the approximately additive data-generating process of the benchmark data. The tree-based benchmarks confirmed that a random forest ( $R^2 = 0.9899$ ) and XGBoost ( $R^2 = 0.9955$ ) also achieve strong per-

formance, with XGBoost approaching the DNN’s accuracy. These comparisons serve to contextualise the DNN within a broader model-class landscape, but the central purpose of the dissertation remains the Monte Carlo and Shapley-value framework applied to the fitted DNN rather than the model comparison itself.

The Monte Carlo simulation produced a predicted-cost distribution centred at R16 752 with a standard deviation of R4 417 and near-zero skewness. The beauty of this construction lies in the fact that it transforms the fitted network from a point-prediction device into a distributional object: the network is no longer evaluated at specific observed covariate combinations, but across a simulated population drawn from a formally stated input law. The joint-distribution validation, including a multivariate energy distance permutation test ( $p = 0.730$ ), confirmed that the product-marginal input does not introduce material distributional distortion, and the residual-augmented sensitivity analysis established that the distributional conclusions are robust to fitted residual perturbation.

The Shapley-value analysis revealed the expected global hierarchy: smoking status dominates ( $I_g = 0.578$ ), followed by coverage level (0.401), medical history (0.367) and family medical history (0.351). The training versus Monte Carlo attribution invariance check yielded a perfect Spearman rank correlation ( $\rho_S = 1.000$ ), confirming that the DNN’s internal feature-importance structure is preserved across input regimes. The most striking finding, however, emerged from the conditional Shapley analysis. At  $\tau_{99}$ , the feature-importance ranking reverses: family medical history ( $I_{g,0.99} = 0.649$ ), medical history (0.615) and coverage level (0.595) overtake smoking status (0.563). The profile-level analysis explained this reversal through a simple but powerful observation: at the 99th percentile, virtually all high-cost profiles are smokers (99%), so smoking status no longer differentiates among them. The residual variation is explained by the variables that still vary within the tail. This finding demonstrates that global importance rankings, while useful as summary measures, can be misleading when the interest lies in specific regions of the output distribution.

## 5.2 Addressing the Research Questions

The six research questions posed in Section 1.5 span the predictive, simulation and explanation stages of the framework. Each question is now addressed in light of the empirical results.

### Research Question 1

How does a deep neural network regression model compare with a classical generalised linear model, a random forest and an XGBoost gradient-boosted tree in predicting health

insurance costs on a benchmark dataset?

The DNN achieves a test-set  $R^2$  of 0.9957, a test MAE of R251 and a test RMSE of R290. The Gamma GLM (log-link) achieves a test-set  $R^2$  of 0.9571, a test MAE of R624 and a test RMSE of R913. The DNN therefore outperforms the GLM on every metric, reducing the test RMSE by 68.2% and the test MAE by 59.8%. The two tree-based benchmarks narrow the gap: the random forest achieves  $R^2 = 0.9899$  and XGBoost reaches  $R^2 = 0.9955$ , approaching the DNN's accuracy. All four models show near-identical training and test performance, ruling out overfitting. The GLM's performance gap is attributable to the log-link's multiplicative mean structure, which introduces systematic model misspecification on this benchmark dataset where the charges variable is constructed as an approximately additive function of the input features. The tree-based models, which are not constrained by a multiplicative link, recover most of this gap, confirming that the GLM's disadvantage is structural rather than a general property of simpler model classes. The five-phase sequential hyperparameter comparison selected the final DNN configuration from among 16 candidate settings, documented in full in Annexure [Annexure H](#). This finding is consistent with the broader health-cost prediction literature ([Drewe-Boss et al., 2022](#); [Samiuddin et al., 2023](#); [Kaushik et al., 2022](#)), although the unusually high  $R^2$  reflects the clean, structured nature of the benchmark data and should not be extrapolated to real-world insurance portfolios.

## Research Question 2

What is the shape and distributional behaviour of the model-implied predicted-cost distribution obtained by propagating parametric Monte Carlo input profiles through the trained network?

The distribution is approximately symmetric (skewness = 0.132) and slightly platykurtic (kurtosis = -0.337), with a mean of R16 752 and a standard deviation of R4 417. It is unimodal, with the interquartile range spanning R13 621 to R19 792 and the upper tail extending to a maximum predicted cost of R31 781. The close agreement between the mean and median (R16 752 versus R16 632) confirms the absence of heavy-tailed distortion. The comparison with the observed training charges ([Figure 39](#)) established that the model-implied distribution is consistent with the distributional characteristics of the observed data. The convergence diagnostics confirmed that the Monte Carlo standard error at  $B = 100\,000$  is R14.0, representing 0.08% of the mean predicted cost, and that the running summaries have stabilised.

### Research Question 3

Is the product-marginal Monte Carlo input distribution a defensible approximation of the training-data joint distribution for the purpose of generating simulated profiles?

Five complementary diagnostics support an affirmative answer. The correlation matrix difference showed small absolute pairwise differences. The Cramér’s  $V$  associations for the categorical variables showed negligible association in both samples. The PCA overlay showed substantial overlap without systematic displacement. The higher-order interaction check confirmed the absence of spurious interaction structures. The multivariate energy distance permutation test yielded  $p = 0.730$ , providing no evidence against the null hypothesis that the two samples share a common joint distribution. The bootstrap Monte Carlo resampling check (Spearman  $\rho = 0.991$ ) and the Shapley attribution invariance assessment (Spearman  $\rho_S = 1.000$ ) further confirmed that the independence assumption does not compromise the validity of the simulation for this benchmark dataset. It should be noted, however, that the independence assumption broadens the simulated input space and can generate implausible covariate combinations. This limitation is discussed further in Section 5.5.

### Research Question 4

How sensitive is the predicted-cost distribution to residual perturbation from the fitted model?

The predicted-cost distribution is robust to residual perturbation. The centred training residuals are symmetric, bell-shaped and homoscedastic. When residuals drawn from the empirical residual distribution were added to the base Monte Carlo predictions, the augmented distribution retained the same location, spread and shape. The fixed perturbation band of  $\pm 0.5 \hat{\sigma}_\epsilon$  produced a smooth widening without asymmetry or heavy-tailed distortion. The DNN’s residuals on this benchmark dataset are sufficiently small and well-behaved that their addition does not alter the distributional conclusions.

### Research Question 5

Which input variables receive the largest Shapley-value attributions globally, and how does the feature-importance ranking change conditionally within high-cost regions of the predicted-cost distribution?

At the global level, four variables form a clear top tier: smoking status ( $I_g = 0.578$ ), coverage level (0.401), medical history (0.367) and family medical history (0.351). Occupation (0.171) and exercise frequency (0.149) contribute at a secondary level. The remaining five

variables each have mean absolute Shapley values below 0.13. The conditional analysis at three thresholds reveals a progressive shift. At  $\tau_{90}$ , the ranking mirrors the global ordering. At  $\tau_{95}$ , family medical history and medical history show increased conditional importance. At  $\tau_{99}$ , the ranking reverses: family medical history (0.649), medical history (0.615) and coverage level (0.595) overtake smoking status (0.563). The second-tier variables remain stable across all thresholds, indicating that the reversal is driven entirely by changes in the relative importance of the top-tier categorical risk factors.

### Research Question 6

What explains any differences between the global and conditional feature-importance rankings in the upper tail?

The ranking reversal at  $\tau_{99}$  is explained by the compositional structure of the extreme upper-tail profiles. At  $\tau_{99}$ , the proportion of smokers reaches 99%, the proportion of Premium-tier policyholders reaches approximately 90%, and the proportion of profiles reporting Heart disease reaches approximately 85%. When smoking status is near-universal, it contributes a near-constant positive attribution with reduced variability and therefore ceases to differentiate among high-cost profiles. The residual variation is explained by coverage tier, medical history and family medical history. To illustrate this with a concrete example: consider two profiles in the  $\tau_{99}$  set, both smokers with high BMI. One holds Premium coverage with a family history of Heart disease; the other holds Standard coverage with no family medical history. Despite their identical smoking status, the first profile receives substantially higher predicted costs, and the Shapley decomposition attributes this difference to the coverage and family history variables rather than to smoking. This is analogous to a well-known principle in epidemiological statistics: a risk factor that is ubiquitous in a population loses its discriminating power within that population, even though it may be the strongest predictor in the general population.

## 5.3 Contributions

The dissertation makes several contributions to the intersection of statistical learning, simulation-based inference and explainable modelling.

The primary contribution is the integration of deep neural network regression, parametric Monte Carlo simulation and Shapley-value attribution within a single, reproducible framework. In the existing literature, these three components are typically applied in isolation: prediction studies report out-of-sample performance metrics without distributional analysis; Monte Carlo studies simulate quantities of interest without connecting the results to feature-level attribution; and Shapley-value studies decompose predictions

on observed data without examining how the attribution structure changes across specific regions of the predicted-cost distribution. This dissertation demonstrates that the integration produces insight that none of the three components could provide in isolation. The Monte Carlo distribution defines the regions of interest for the Shapley analysis, and the conditional Shapley analysis decomposes the model behaviour within those regions. Each component produces outputs that serve as inputs to the next, creating a connected framework in which the fitted model, the simulated input population and the feature attributions are jointly analysed.

The conditional feature-importance summaries at high-cost thresholds represent the most distinctive contribution. The existing health insurance prediction literature reports global feature importance computed on observed data only (Kaushik et al., 2022; Rozemberczki et al., 2022), without conditioning attributions on specific regions of the predicted-cost distribution. The conditional analysis demonstrates that the variables driving extreme predicted costs differ from those that are most important on average, a finding that has not been documented in the health insurance cost-prediction literature reviewed in Chapter 2. This result contributes to the growing literature on conditional and local interpretability methods (Lundberg and Lee, 2017) by showing that global Shapley summaries can be supplemented with threshold-specific conditional summaries to provide a more complete picture of a model’s attribution structure.

The Monte Carlo component constructs a model-implied predicted-cost distribution from a formally stated input distribution, rather than reporting predictions only at observed data points. This connects the neural network to the simulation-based inference tradition within mathematical statistics (Glasserman, 2004; Rubinstein and Kroese, 2016), where the behaviour of estimators and predictive quantities is routinely studied through Monte Carlo experiments. The distinction is methodologically significant: evaluating a model on observed data provides point predictions at covariate combinations that happen to appear in the dataset; evaluating it on a simulated population provides a distributional characterisation of the model’s output landscape under a specified input law, allowing the analyst to study the range, dispersion and tail behaviour of the predictions as statistical quantities in their own right.

The multi-layered validation protocol, comprising joint-distribution diagnostics, convergence monitoring, residual-augmented sensitivity analysis and Shapley attribution invariance checks, provides a template for validating Monte Carlo and Shapley analyses in other applied settings. These diagnostics address specific threats to validity that are not routinely checked in the machine-learning interpretability literature, where Shapley values are often reported without assessing their sensitivity to the input distribution or the residual structure of the fitted model. Finally, the three benchmark models (Gamma GLM, random forest and XGBoost) contextualise the DNN’s performance across the model-

class spectrum, establishing that the predictive superiority reflects genuine advantages of smooth nonlinear function approximation rather than artefacts of the benchmark data.

## 5.4 Strengths of the Study

The study has several methodological strengths that enhance the reliability and reproducibility of its findings. The analysis is based on one million policyholder records, providing a substantial sample for training, validation and testing. The training partition alone contains 524 288 observations, reducing sampling variability in model fitting, hyperparameter selection and Shapley-value estimation. The large sample size ensures that the five-phase hyperparameter comparison is not confounded by high sampling variance, and that the Shapley attributions are computed over a sufficiently large and representative background.

The five-phase sequential hyperparameter comparison provides a transparent, reproducible selection method. Each phase is documented with validation-loss curves and selection criteria, allowing the choices to be scrutinised and replicated. This systematic approach avoids the opacity of automated search methods (such as random search or Bayesian optimisation) that do not provide interpretable selection rationales, while remaining computationally feasible for the architecture under consideration. The full results are presented in Annexure [Annexure H](#).

The data-leakage prevention protocols (Section [3.2.4](#)) ensure that all preprocessing parameters are computed exclusively on the training partition and applied without refitting to the validation and test partitions. This strict separation, combined with the joint-distribution input validation, the convergence diagnostics, the residual-augmented sensitivity analysis and the Shapley attribution invariance checks, collectively establish that the Monte Carlo and Shapley results are not artefacts of data contamination, distributional mismatch, residual instability or input-regime dependence.

The Shapley-value analysis at both the global and conditional levels, with three upper-tail thresholds providing a graded view, reveals patterns that would be invisible in a global-only analysis, most notably the ranking reversal at  $\tau_{99}$ . The attribution invariance check further validates the stability of the results across input regimes, and all conclusions are explicitly qualified as internal to the fitted model and benchmark data. No causal claims are made, and the modelling assumptions are stated formally in Table [21](#). This transparency ensures that the findings are not over-interpreted and that the scope of their applicability is clearly defined.

## 5.5 Limitations

It is important to be forthright about the limitations that qualify the scope of these findings. Several of these limitations are inherent in the methodological choices made in this dissertation; others reflect the constraints of the benchmark data.

All results are conditional on the Kaggle Insurance Data for Machine Learning benchmark dataset, which has a strong simulated structure. The unusually high predictive performance ( $R^2 = 0.9957$ ) reflects this structure and should not be extrapolated to real-world insurance portfolios, where data are noisier, relationships between covariates and costs are more complex, and the data-generating process may be multiplicative rather than additive. The additive structure of the benchmark data favours the DNN over the log-link GLM, because the log-link’s multiplicative structure introduces systematic misspecification. On a dataset with a genuinely multiplicative data-generating process, the GLM’s relative performance would likely improve, and the DNN’s advantage might be smaller. The benchmark data serves the purpose of demonstrating the methodology on a controlled, reproducible dataset, but the specific numerical results are properties of this dataset and not of health insurance cost prediction in general.

The study compares the DNN against a Gamma/log-link GLM, a random forest and an XGBoost gradient-boosted tree regressor. While these benchmarks span three major model classes (parametric, bagging and boosting), ensemble methods such as stacking, Bayesian neural networks and other advanced alternatives were not evaluated. The existing literature suggests that gradient-boosted trees often perform competitively with neural networks on tabular data ([Richman, 2021](#)), and indeed the XGBoost benchmark achieves near-DNN accuracy on this dataset. A broader model-selection study including additional architectures and datasets would further strengthen the conclusions.

The Monte Carlo and Shapley analyses are conditional on a single trained network with a fixed random seed. While the early stopping criterion and the near-identical training and test performance provide evidence of stable convergence, the sensitivity of the Shapley attributions to the stochastic training process is an open question. Different random seeds may produce slightly different weight configurations, which could produce slightly different Shapley distributions. The magnitude of this effect is unknown for the architecture and dataset used in this study.

The Monte Carlo input distribution is constructed as a product of independent marginals. While the joint-distribution validation confirmed that the training-data dependence structure is weak ( $p = 0.730$ ), the independence assumption broadens the simulated input space beyond what would be observed in practice. In particular, the product-marginal sampling can generate covariate combinations that are implausible in a real insurance portfolio, i.e. profiles with extreme BMI, many dependants and low exercise frequency that are rare

or absent in the training data. The impact of these implausible profiles on the results is mitigated by their low probability, but it cannot be excluded entirely.

The reported attributions are gradient-based SHAP approximations, not exact game-theoretic Shapley values. The approximation quality depends on the smoothness of the fitted prediction function and the adequacy of the background sample of  $m_0 = 200$  observations. The exact Shapley values would require evaluating the model at all  $2^p$  subsets of features, which is computationally infeasible for  $p = 22$ . The gradient-based approximation is the standard approach in the SHAP literature (Lundberg and Lee, 2017), but its approximation error has not been quantified for this specific setting.

The conditional Shapley summaries at  $\tau_{99}$  rely on approximately 1000 simulated profiles (the top 1% of  $B = 100\,000$ ). The nonparametric bootstrap confidence intervals ( $R = 10\,000$  resamples) reported in Table 29 provide formal statistical support for the ranking reversal, and the monotonic trends across  $\tau_{90}$ ,  $\tau_{95}$  and  $\tau_{99}$  strengthen the finding. Nevertheless, the precision of the conditional estimates remains lower than the global estimates, which are computed over the full simulation. It is worth noting that the upper-tail analysis in this dissertation is conducted through quantile-based thresholding of the predicted-cost distribution, rather than through the formal apparatus of extreme value theory (EVT). In the EVT framework, exceedances above a high threshold are modelled using the generalised Pareto distribution, which provides a principled basis for estimating tail probabilities and return levels. The conditional Shapley analysis developed here addresses a different question, namely which input variables contribute most to predictions in the upper tail, but it does not exploit the asymptotic distributional theory that underpins formal tail estimation. Future work could integrate EVT-based tail modelling with the conditional Shapley framework by using peaks-over-threshold methods to define the high-cost region and by assessing the sensitivity of the attribution results to the threshold selection.

A further methodological limitation concerns the absence of formal uncertainty quantification around the Shapley-value estimates. The gradient-based SHAP implementation produces point estimates of the attributions for each profile, but does not provide confidence intervals or posterior distributions for the feature-importance summaries. In a Bayesian framework, one could place posterior distributions over the network weights and propagate the resulting weight uncertainty through the Shapley computation, yielding posterior distributions of the feature-importance values. Such an approach would complement the Monte Carlo simulation by adding a second layer of distributional analysis: the Monte Carlo component characterises the distribution of predicted costs under input variability, while the Bayesian component would characterise the distribution of Shapley attributions under parameter uncertainty. This extension is noted as a direction for future research in Section 5.8.

Finally, the Shapley decomposition identifies variables to which the fitted model assigns large contributions; it does not establish causal relationships between input variables and healthcare expenditure. All interpretations are conditional on the fitted model and benchmark data, and should not be read as causal claims about the determinants of health insurance costs.

## 5.6 Implications for Actuarial Practice and Statistical Methodology

The findings carry implications for both actuarial practice and the broader field of statistical methodology, subject to the limitations identified above.

### Implications for actuarial practice

The substantial performance advantage of the DNN over the Gamma GLM on this benchmark dataset illustrates the potential of flexible machine-learning models to improve upon the classical pricing framework, at least on datasets where the data-generating process does not conform to the log-link's multiplicative structure. The tree-based benchmarks reinforce this conclusion: both the random forest and XGBoost substantially outperform the GLM, with XGBoost approaching the DNN's accuracy, confirming that the GLM's limitation on this dataset is structural rather than a general feature of simpler models. For actuarial practitioners, these results motivate the evaluation of tree-based ensembles and neural networks as supplementary or alternative pricing models alongside existing GLM-based tariffs, particularly in settings where the relationship between covariates and costs may be additive, nonlinear or interactive in ways that the log-link cannot capture. The conditional Shapley analysis has direct relevance to actuarial risk management. Insurance professionals are typically interested not only in the average cost drivers across a portfolio, but also in the variables that drive extreme costs. The finding that the global feature-importance ranking can be misleading in the upper tail suggests that conditional Shapley analysis should be considered as a standard component of model interpretation in insurance settings, particularly when the objective is to understand the composition and risk factors of high-cost claims. To illustrate: a risk manager who relies solely on the global importance ranking would focus on smoking status as the primary intervention lever; the conditional analysis reveals that within the extreme upper tail, the differentiating variables are coverage tier, medical history and family medical history, which may require different management strategies.

The Monte Carlo simulation component demonstrates a method for constructing model-implied predicted-cost distributions from specified input laws. In actuarial applications,

such distributions are relevant to reserve adequacy assessment, risk capital calculations and scenario analysis. The ability to generate a distributional characterisation of the model's output landscape, rather than relying on point predictions at observed data points, provides a richer basis for actuarial decision making. However, these implications must be qualified by the limitations of the benchmark data. The unusually clean structure of the Kaggle dataset produces predictive performance that would be unattainable on most real-world insurance portfolios. The methodology is transferable, but the specific performance figures are not.

### **Implications for statistical methodology**

From a statistical-methodological perspective, the dissertation demonstrates that the tools of simulation-based inference can be applied productively to the outputs of fitted machine-learning models. The construction of a model-implied predicted-cost distribution by propagating Monte Carlo inputs through a trained network is conceptually analogous to studying a sampling distribution through Monte Carlo experiments. In both cases, the objective is to characterise the distributional behaviour of a quantity of interest by repeated evaluation under a specified input law. The conditional Shapley analysis extends the standard global summary by partitioning the predicted-cost distribution into regions of interest and examining how the attribution structure changes within those regions. This approach is applicable to any regression model where the analyst is interested in the drivers of predictions within specific output ranges, not only in insurance pricing but also in credit scoring, healthcare resource allocation, environmental risk assessment and other domains where the upper tail of the predicted distribution is of particular importance.

The validation protocol provides a template for validating Monte Carlo and Shapley analyses in other applied settings. Joint-distribution diagnostics check whether the input distribution introduces distributional distortion; convergence monitoring checks whether the simulation size is adequate; residual-augmented sensitivity analysis checks whether the distributional conclusions are sensitive to residual variability; and attribution invariance checks verify whether the Shapley rankings are artefacts of the input regime. These diagnostics address specific threats to validity that are not routinely checked in the machine-learning interpretability literature.

There is also a connection to the broader field of extreme value analysis. The conditional Shapley analysis at upper-tail thresholds is conceptually related to the peaks-over-threshold approach in extreme value theory, where interest centres on the behaviour of a distribution above a high quantile. In the EVT context, the focus is on estimating tail probabilities and return levels; in the conditional Shapley context, the focus is on decomposing the predictions that exceed the threshold into feature-level contributions. The two

perspectives are complementary: EVT characterises the statistical properties of the tail, while the conditional Shapley analysis explains the model’s behaviour within the tail. Future work could integrate these perspectives by using EVT-informed threshold selection to define the conditioning region for the Shapley analysis, and by assessing whether the attribution patterns are sensitive to the specific choice of threshold.

## 5.7 Comparison with Constrained Neural-Network Approaches

A recent line of actuarial research imposes explicit shape constraints on neural networks during training. The ICEnet of [Richman and Wüthrich \(2024\)](#), presented in Chapter 7 of [Richman \(2025\)](#), adds smoothness and monotonicity penalty terms to the loss function so that the network’s Individual Conditional Expectation (ICE) outputs vary smoothly and monotonically with respect to selected covariates. Because the two studies differ in their response variables, evaluation metrics and datasets, a direct numerical comparison of predictive performance is not appropriate. Table 30 summarises the principal differences.

Table 30: Methodological comparison between the present dissertation and the ICEnet of [Richman and Wüthrich \(2024\)](#).

| Aspect                | Present dissertation                                               | ICEnet                                    |
|-----------------------|--------------------------------------------------------------------|-------------------------------------------|
| Target variable       | Continuous monetary charges                                        | Claims frequency (count)                  |
| Response distribution | Continuous                                                         | Poisson                                   |
| Models compared       | GLM, Random Forest, XG-Boost, DNN                                  | FCN, ICEnet                               |
| Best test-set result  | DNN $R^2 = 0.9957$ ; GLM $0.9571$ ; RF $0.9899$ ; XGBoost $0.9955$ | ICEnet deviance $= 0.2388$ ; FCN $0.2387$ |
| Evaluation metric     | $R^2$ , RMSE, MAE                                                  | Poisson deviance                          |
| Dataset               | Kaggle ( $n = 1\,000\,000$ )                                       | French MTPL ( $n = 678\,007$ )            |
| Constraint mechanism  | None (unconstrained models)                                        | Smoothness and monotonicity penalties     |
| Interpretability      | Post-hoc Shapley values                                            | In-training ICE/PDP constraints           |

**Predictive performance.** [Richman and Wüthrich \(2024\)](#) reports that the ICEnet achieves a test-set Poisson deviance of 0.2388, compared with 0.2387 for the unconstrained network, a difference of 0.0001. When monotonicity constraints are applied alone, the out-of-sample deviance improves slightly to 0.2382, suggesting that these constraints act as

a regulariser; the smoothness constraints account for the small overall performance decline. In the present dissertation, the unconstrained DNN achieves a test-set  $R^2$  of 0.9957 on a continuous cost outcome. Because the response variables differ (count data versus continuous monetary charges) and the evaluation metrics differ (Poisson deviance versus  $R^2$ ), these figures are not directly comparable and no cross-study ranking of accuracy is claimed. Both studies, however, demonstrate that deep neural networks improve upon GLM baselines within their respective insurance domains.

**Interpretability.** The ICEnet enforces actuarial reasonableness during training by constraining the covariate–response relationships to be smooth and monotonic, which is important for commercial pricing and regulatory compliance. The present dissertation pursues interpretability post hoc through Shapley-value decomposition, revealing that the dominant cost drivers shift between the global level and the upper tail of the predicted-cost distribution. This distributional insight is not accessible through the ICE or PDP framework alone. The two approaches address different aspects of model interpretability and are complementary rather than competing: constrained training ensures that the learned relationships are commercially reasonable, while Shapley attribution identifies which features contribute most to individual predictions and how these contributions change in high-cost regions.

## 5.8 Recommendations for Future Research

Several directions for future research have been uncovered in the course of this dissertation. These can be divided into two broad categories: extensions of the modelling and simulation components, and extensions of the attribution and interpretation components.

### Modelling and simulation extensions

- Future work could extend the baseline comparison to include Bayesian neural networks, stacking ensembles and alternative deep-learning architectures (e.g. TabNet, FT-Transformer). Such comparisons would clarify whether the Shapley attribution patterns documented here are model-dependent or model-invariant, and whether more complex architectures offer further predictive gains beyond the DNN and XGBoost benchmarks already evaluated.
- Replacing the product-marginal input distribution with a copula-based joint distribution would allow the simulation to reproduce the dependence structure of the observed data more faithfully. A Gaussian copula or vine copula fitted to the training data would generate covariate combinations that respect the empirical correlation structure, eliminating the implausible profiles that can arise under the independence

assumption.

- Training the DNN under multiple random initialisations and reporting the distribution of predictive-performance metrics and Shapley-value summaries across seeds would quantify the sensitivity of the results to the stochastic training process. This analysis would establish whether the ranking reversal at  $\tau_{99}$  is a robust property of the architecture and data, or whether it is sensitive to the specific weight initialisation.
- Applying the framework to proprietary or regulatory insurance datasets with genuine underwriting heterogeneity would test whether the conditional Shapley findings generalise beyond the synthetic benchmark setting. Real-world data would introduce challenges not present in the benchmark data, including missing values, temporal trends, policy-level effects and non-trivial dependence structures among covariates.
- Coupling the Monte Carlo simulation with Bayesian posterior predictive distributions or conformal prediction intervals would add formal uncertainty quantification to the predicted-cost distribution, transforming it from a point-prediction landscape into a set-valued prediction landscape with formal coverage properties.
- Quantile regression offers a complementary perspective on the conditional cost distribution that warrants investigation. Rather than modelling the conditional mean and subsequently examining quantiles of the Monte Carlo output, a quantile regression neural network could directly estimate conditional quantiles of the cost distribution at specified probability levels. This approach would provide a natural alternative to the threshold-based conditioning used in the conditional Shapley analysis and would allow the analyst to study how the feature-importance structure varies across quantile levels within the fitted model itself, without the need for a separate Monte Carlo simulation stage.
- As discussed in Section 5.7, integrating constrained neural-network training with the Monte Carlo–Shapley workflow would combine actuarial reasonableness with feature-level explainability. The smoothness and monotonicity constraints of the ICEnet (Richman and Wüthrich, 2024) could be incorporated into the DNN’s loss function, and Shapley-value attribution could then be applied to the constrained model to verify that the imposed constraints do not distort the feature-importance ranking in the upper tail. This extension would also permit an empirical assessment of whether the negligible accuracy cost observed for frequency models extends to continuous-cost prediction.

### Attribution and interpretation extensions

- The  $\tau_{99}$  ranking reversal could be strengthened by increasing the simulation size to  $B = 50\,000$  or  $100\,000$ , by constructing bootstrap confidence intervals for the conditional mean absolute Shapley values, or by introducing intermediate thresholds (e.g.  $\tau_{97.5}$ ,  $\tau_{98}$ ) to trace the transition in the importance ranking more finely.
- Comparing the gradient-based SHAP attributions with alternative methods, such as permutation-based SHAP, KernelSHAP or LIME (Ribeiro et al., 2016), would assess the sensitivity of the results to the choice of explanation method. Consistency across methods would strengthen the conclusion that the ranking reversal is a property of the fitted model rather than an artefact of the gradient-based approximation.
- Interaction Shapley values, which decompose the total Shapley value into main effects and pairwise interactions (Lundberg and Lee, 2017), would reveal whether the ranking reversal is driven by changes in the main-effect attributions or by changes in the interaction structure among the top-tier variables.
- The conditioning could be extended from univariate threshold exceedances to multivariate regions defined by combinations of input variables, allowing the analyst to examine how the attribution structure changes within specific subpopulations (e.g. elderly smokers with Premium coverage) rather than within quantile-based cost regions.
- If panel or longitudinal insurance data are available, the framework could be extended to study how the predicted-cost distribution and its Shapley decomposition evolve over time. Temporal changes in the feature-importance ranking would indicate shifts in the drivers of high-cost predictions as the portfolio composition changes, providing dynamic risk-management information that static cross-sectional analyses cannot offer.
- The conditional Shapley analysis could be integrated with formal extreme value theory by using peaks-over-threshold methods and the generalised Pareto distribution to define the high-cost region. EVT-informed threshold selection would provide a principled statistical basis for the conditioning, and the resulting Shapley decomposition would be grounded in the asymptotic distributional theory that underpins formal tail estimation. This integration would bridge the gap between the feature-attribution perspective developed in this dissertation and the tail-modelling perspective that is central to actuarial risk management and solvency assessment.

## 5.9 Concluding Remark

This dissertation set out to bridge two intellectual traditions that have often developed independently: the classical statistical tradition of studying estimators through their distributional properties, and the modern machine-learning tradition of training flexible prediction models and explaining their outputs. The bridge is built from three components, each with a well-established theoretical basis in mathematical statistics: supervised regression provides the prediction function, Monte Carlo simulation provides the distributional characterisation, and Shapley-value decomposition provides the feature-level attribution.

The most noteworthy finding is the feature-importance ranking reversal at the 99th percentile of the predicted-cost distribution. While smoking status is the dominant contributor to predicted costs globally, it loses its discriminating power in the extreme upper tail, where virtually all profiles are smokers. Within this tail, the variables that differentiate the costliest profiles from one another are coverage level, medical history and family medical history. This finding illustrates a general principle that extends well beyond insurance pricing: a variable's importance in a statistical model is not a fixed property, but depends on the region of the output space under consideration. Global importance summaries, while useful, can obscure the attribution structure within subpopulations of particular interest.

The framework developed here is not specific to health insurance or to the particular benchmark dataset analysed. The three-stage framework of prediction, simulation and conditional explanation is applicable to any regression problem where the analyst seeks to understand the distributional behaviour of a model's predictions and to decompose those predictions into feature-level contributions within specific regions of the output space. The validation diagnostics developed for the Monte Carlo and Shapley components provide a reusable template for assessing the reliability of such analyses in other applied settings, from credit scoring and healthcare resource allocation to environmental risk assessment and financial modelling.

The development of this framework reflects a conviction that the tools of mathematical statistics remain essential for understanding and interpreting the outputs of modern machine-learning models. Neural networks are powerful prediction devices, but their value to the applied statistician extends beyond the point predictions they produce. By treating the trained network as a deterministic transformation to be studied through Monte Carlo experiments and decomposed through Shapley values, this dissertation connects the machine-learning prediction paradigm to the classical statistical tradition of studying estimators through their distributional properties. While this connection may seem like a modest step within the vast landscape of statistical learning and explainable modelling, it is worth remembering that each contribution to the field of interpretable machine learning

---

is, in its own way, a contribution to the broader enterprise of understanding how statistical models learn from data, what they learn, and where their learned representations succeed or fail in capturing the structure of the phenomena they are designed to predict.

## References

- Y. S. Abu-Mostafa, M. Magdon-Ismail, and H.-T. Lin. *Learning from Data*. AMLBook, New York, 2012. (Cited on p. 61.)
- J. T. Barron. Continuously differentiable exponential linear units, 2017. URL <https://arxiv.org/abs/1704.07483>. (Cited on p. 53.)
- C. M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, New York, 2006. ISBN 978-0-387-31073-2. (Cited on pp. 51 and 58.)
- J. Blomerus. Samplingforml: A web application for sampling and preprocessing tabular data for machine learning, 2026. URL <https://app.samplingforml.edu.eu.org>. (Cited on p. 78.)
- L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324. (Cited on pp. 20 and 96.)
- R. E. Caflisch. Monte Carlo and quasi-Monte Carlo methods. *Acta Numerica*, 7:1–49, 1998. doi: 10.1017/S0962492900002804. (Cited on pp. 69 and 72.)
- J. Castro, D. Gómez, and J. Tejada. Polynomial calculation of the Shapley value based on sampling. *Computers & Operations Research*, 36(5):1726–1730, 2009. doi: 10.1016/j.cor.2008.04.004. (Cited on p. 44.)
- T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016. doi: 10.1145/2939672.2939785. (Cited on pp. 20, 34, 63, 98, and 99.)
- M. Christiansen, M. Denuit, N. Lucas, and J.-P. Schmidt. Projection models for health expenses. *Annals of Actuarial Science*, 12(1):185–203, 2018. doi: 10.1017/S1748499517000148. (Cited on p. 63.)
- H. Cramér. *Mathematical Methods of Statistics*. Princeton University Press, Princeton, NJ, 1946. (Cited on p. 115.)
- G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274. (Cited on p. 21.)
- M. Denuit, X. Maréchal, S. Pitrebois, and J.-F. Walhin. *Actuarial Modelling of Claim Counts: Risk Classification, Credibility and Bonus-Malus Systems*. Wiley, Chichester, 2007. ISBN 978-0-470-02677-9. (Cited on pp. 48, 63, 66, 94, and 95.)

- A. J. Dobson and A. G. Barnett. *An Introduction to Generalized Linear Models*. Chapman and Hall/CRC, Boca Raton, 3rd edition, 2008. ISBN 978-1-58488-950-2. (Cited on pp. 48, 94, and 95.)
- T. Dozat. Incorporating Nesterov momentum into Adam. In *Proceedings of the 4th International Conference on Learning Representations (ICLR 2016), Workshop Track*, 2016. URL <https://openreview.net/pdf/OM0jvwB8jIp57ZJjtNEZ.pdf>. (Cited on pp. 55, 91, 92, and 93.)
- P. Drewe-Boss, D. Enders, J. Walker, and U. Ohler. Deep learning for prediction of population health costs. *BMC Medical Informatics and Decision Making*, 22(1):32, 2022. doi: 10.1186/s12911-021-01743-z. (Cited on pp. 48, 75, 134, 156, and 159.)
- B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*, volume 57 of *Monographs on Statistics and Applied Probability*. Chapman and Hall/CRC, New York, 1993. ISBN 978-0-412-04231-7. doi: 10.1201/9780429246593. (Cited on p. 67.)
- P. Glasserman. *Monte Carlo Methods in Financial Engineering*, volume 53 of *Applications of Mathematics*. Springer, New York, 2004. ISBN 978-0-387-00451-8. doi: 10.1007/978-0-387-21617-1. (Cited on pp. 37, 43, 67, 68, 69, 70, 71, 74, 156, and 162.)
- I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. ISBN 978-0-262-03561-3. URL <https://www.deeplearningbook.org>. (Cited on pp. 20, 21, 37, 38, 42, 49, 51, 52, 53, and 91.)
- L. Grinsztajn, E. Oyallon, and G. Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022, Datasets and Benchmarks Track)*, pages 507–520. Curran Associates, Inc., 2022. URL [https://proceedings.neurips.cc/paper\\_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets\\_and\\_Benchmarks.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html). (Cited on pp. 49 and 62.)
- T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2nd edition, 2009. ISBN 978-0-387-84857-0. doi: 10.1007/978-0-387-84858-7. (Cited on pp. 38, 42, 46, 47, 57, 58, 61, 97, and 107.)
- K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015. (Cited on p. 94.)
- S. A. Hejazi and K. R. Jackson. A neural network approach to efficient valuation of large portfolios of variable annuities. *Insurance: Mathematics and Economics*, 70:169–181, 2016. doi: 10.1016/j.insmatheco.2016.06.013. (Cited on p. 49.)

- D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs), 2016. URL <https://arxiv.org/abs/1606.08415>. (Cited on p. 53.)
- K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8. (Cited on pp. 21, 37, 38, 49, and 156.)
- G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning with Applications in R*. Springer Texts in Statistics. Springer, New York, 2nd edition, 2021. ISBN 978-1-0716-1418-1. doi: 10.1007/978-1-0716-1418-1. (Cited on pp. 6, 38, 42, 46, 47, 57, 58, and 59.)
- Kaggle.com. Insurance Data for Machine Learning. <https://www.kaggle.com/datasets/sridharstreaks/insurance-data-for-machine-learning/data>, 2023. Accessed: 15 November 2023. (Cited on pp. 76 and 157.)
- K. Kaushik, A. Bhardwaj, A. D. Dwivedi, and R. Singh. Machine learning-based regression framework to predict health insurance premiums. *International Journal of Environmental Research and Public Health*, 19(13):7898, 2022. doi: 10.3390/ijerph19137898. (Cited on pp. 74, 75, 134, 151, 156, 159, and 162.)
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*, 2015. URL <https://arxiv.org/abs/1412.6980>. (Cited on p. 55.)
- K. Kuo. DeepTriangle: A deep learning approach to loss reserving. *Risks*, 7(3):97, 2019. doi: 10.3390/risks7030097. (Cited on pp. 21 and 28.)
- A. G. Laporta, S. Levantesi, and L. Petrella. A neural network approach for pricing correlated health risks. *Risks*, 13(5):82, 2025. doi: 10.3390/risks13050082. (Cited on p. 62.)
- M. Lee and Q. Wu. Mathematical analysis and performance evaluation of the GELU activation function in deep learning. *Journal of Mathematics*, 2023:4229924, 2023. doi: 10.1155/2023/4229924. (Cited on pp. 52 and 53.)
- C. Lemieux. *Monte Carlo and Quasi-Monte Carlo Sampling*. Springer Series in Statistics. Springer, New York, 2009. ISBN 978-0-387-78164-8. doi: 10.1007/978-0-387-78165-5. (Cited on p. 72.)
- S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4766–4777. Curran Associates Inc., 2017. (Cited on pp. 24, 28, 38, 73, 156, 162, 165, and 171.)

- S. Maleki, L. Tran-Thanh, G. Hines, T. Rahwan, and A. Rogers. Bounding the estimation error of sampling-based Shapley value approximation, 2013. URL <https://arxiv.org/abs/1306.4265>. (Cited on p. 44.)
- V. Nair and G. E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, pages 807–814. Omnipress, 2010. (Cited on p. 53.)
- J. A. Nelder and R. W. M. Wedderburn. Generalized linear models. *Journal of the Royal Statistical Society: Series A (General)*, 135(3):370–384, 1972. doi: 10.2307/2344614. (Cited on p. 19.)
- U. Orji and E. Ukwandu. Machine learning for an explainable cost prediction of medical insurance. *Machine Learning with Applications*, 15:100516, 2024. doi: 10.1016/j.mlwa.2023.100516. (Cited on pp. 39, 42, 49, and 62.)
- A. B. Owen. *Monte Carlo Theory, Methods and Examples*. Art Owen, Stanford, CA, 2013. URL <https://artowen.su.domains/mc/>. (Cited on pp. 37, 43, and 72.)
- M. T. Ribeiro, S. Singh, and C. Guestrin. “Why Should I Trust You?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016. doi: 10.1145/2939672.2939778. (Cited on p. 171.)
- R. Richman. AI in actuarial science – a review of recent advances – part 1. *Annals of Actuarial Science*, 15(2):207–229, 2021. doi: 10.1017/S1748499520000238. (Cited on pp. 21, 28, 38, 39, 42, 48, 134, 156, and 164.)
- R. Richman and M. V. Wüthrich. Smoothness and monotonicity constraints for neural networks using ICEnet. *Annals of Actuarial Science*, pages 1–28, 2024. doi: 10.1017/S174849952400006X. (Cited on pp. 9, 62, 134, 168, and 170.)
- R. D. Richman. *Applying Deep Learning in Actuarial Science*. PhD dissertation, University of the Witwatersrand, Johannesburg, October 2025. (Cited on p. 168.)
- B. Rozemberczki, L. Watson, P. Bayer, H.-T. Yang, O. Kiss, S. Nilsson, and R. Sarkar. The Shapley value in machine learning. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*, pages 5572–5579, 2022. doi: 10.24963/ijcai.2022/778. (Cited on pp. 38, 74, 151, and 162.)
- R. Y. Rubinstein and D. P. Kroese. *Simulation and the Monte Carlo Method*. Wiley Series in Probability and Statistics. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2016. ISBN 978-1-118-63216-1. doi: 10.1002/9781118631980. (Cited on pp. 37, 43, 67, 68, 71, 156, and 162.)

- S. Ruder. An overview of gradient descent optimization algorithms, 2016. URL <https://arxiv.org/abs/1609.04747>. (Cited on p. 55.)
- M. Samiuddin, G. Rajender, K. S. A. Varma, A. R. Kumar, and S. Shaik. Health insurance cost prediction using deep neural network. *Asian Journal of Research in Computer Science*, 16(2):46–53, 2023. doi: 10.9734/AJRCOS/2023/v16i2338. (Cited on pp. 38, 40, 42, 75, 134, 156, and 159.)
- J. I. Santos, M. Pereda, V. Ahedo, and J. M. Galán. Explainable machine learning for project management control. *Computers & Industrial Engineering*, 180:109261, 2023. doi: 10.1016/j.cie.2023.109261. (Cited on p. 40.)
- L. S. Shapley. A value for n-person games. In H. W. Kuhn and A. W. Tucker, editors, *Contributions to the Theory of Games II*, volume 28 of *Annals of Mathematics Studies*, pages 307–317. Princeton University Press, Princeton, NJ, 1953. (Cited on pp. 23, 38, 73, 123, and 156.)

## Annexure A Extended Mathematical Details

### Annexure A.1 Proof of the Universal Approximation Theorem

**Theorem Annexure A.1** (Cybenko, 1989; Hornik, Stinchcombe and White, 1989). *Let  $\sigma : \mathbb{R} \rightarrow \mathbb{R}$  be a non-constant, bounded, continuous function (the activation function). Let  $I_p = [0, 1]^p$  denote the  $p$ -dimensional unit hypercube. For any continuous function  $f : I_p \rightarrow \mathbb{R}$  and any  $\varepsilon > 0$ , there exist an integer  $N$ , real constants  $\alpha_i, b_i \in \mathbb{R}$ , and vectors  $\mathbf{w}_i \in \mathbb{R}^p$  for  $i = 1, \dots, N$ , such that the function*

$$F(\mathbf{x}) = \sum_{i=1}^N \alpha_i \sigma(\mathbf{w}_i^\top \mathbf{x} + b_i)$$

*satisfies*

$$\sup_{\mathbf{x} \in I_p} |F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon.$$

*Proof.* The proof proceeds by contradiction and uses the Hahn-Banach theorem from functional analysis. Define the family

$$\mathcal{S} = \{\sigma(\mathbf{w}^\top \mathbf{x} + b) : \mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}\}$$

and let  $\mathcal{M}$  denote the closure of  $\text{span}(\mathcal{S})$  in  $C(I_p)$  with respect to the supremum norm. We wish to show  $\mathcal{M} = C(I_p)$ .

Suppose, for contradiction, that  $\mathcal{M} \neq C(I_p)$ . By the Hahn-Banach theorem, there exists a non-zero bounded linear functional  $L \in C(I_p)^*$  such that  $L(g) = 0$  for all  $g \in \mathcal{M}$ . By the Riesz representation theorem, there exists a non-zero finite signed regular Borel measure  $\mu$  on  $I_p$  such that

$$L(g) = \int_{I_p} g(\mathbf{x}) d\mu(\mathbf{x})$$

for all  $g \in C(I_p)$ . In particular, for all  $\mathbf{w} \in \mathbb{R}^p$  and  $b \in \mathbb{R}$ ,

$$\int_{I_p} \sigma(\mathbf{w}^\top \mathbf{x} + b) d\mu(\mathbf{x}) = 0.$$

The key step, due to Cybenko (1989), shows that for discriminatory activation functions (which include all bounded, non-constant, continuous  $\sigma$ ), the condition above forces  $\mu = 0$ , contradicting the assumption that  $L$  is non-zero. The discriminatory property is

established by noting that for any half-space  $H = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b > 0\}$ , the pointwise limit

$$\sigma(\lambda \mathbf{w}^\top \mathbf{x} + \lambda b) \rightarrow \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b > 0, \\ 0 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0, \end{cases} \quad \text{as } \lambda \rightarrow +\infty,$$

combined with dominated convergence, yields  $\mu(H) = 0$  for all open half-spaces  $H$ . Since open half-spaces generate the Borel  $\sigma$ -algebra on  $I_p$ , it follows that  $\mu = 0$ , a contradiction.  $\square$

## Annexure A.2 Derivation of the Adam and NAdam Update Rules

**Adam update rule.** Let  $g_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_{t-1})$  denote the gradient of the loss at iteration  $t$ . The first and second moment estimates are

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (\text{Annexure A.2.1})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (\text{Annexure A.2.2})$$

where  $\beta_1, \beta_2 \in [0, 1)$  are the decay rates and the squaring in (Annexure A.2.2) is element-wise. Since  $m_0 = v_0 = \mathbf{0}$ , these estimates are biased towards zero, especially in early iterations. The bias-corrected estimates are

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (\text{Annexure A.2.3})$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (\text{Annexure A.2.4})$$

The parameter update is

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \quad (\text{Annexure A.2.5})$$

where  $\eta > 0$  is the learning rate and  $\epsilon > 0$  (typically  $10^{-8}$ ) prevents division by zero.

**NAdam update rule.** NAdam incorporates Nesterov momentum by replacing the bias-corrected first moment  $\hat{m}_t$  with a look-ahead estimate. The modification replaces (Annexure A.2.5) with

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \frac{\beta_1 \hat{m}_t + \frac{(1-\beta_1)}{1-\beta_1^t} g_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (\text{Annexure A.2.6})$$

## Annexure A.3 Shapley Value Axiom Verification

**Axiom 1: Efficiency.**

$$\sum_{j=1}^p \phi_j(v) = v(N) - v(\emptyset).$$

**Axiom 2: Symmetry.** If two players  $j$  and  $k$  contribute equally to every coalition, i.e.  $v(S \cup \{j\}) = v(S \cup \{k\})$  for all  $S \subseteq N \setminus \{j, k\}$ , then  $\phi_j(v) = \phi_k(v)$ .

**Axiom 3: Dummy player.** If player  $j$  adds nothing to any coalition, i.e.  $v(S \cup \{j\}) = v(S)$  for all  $S \subseteq N \setminus \{j\}$ , then  $\phi_j(v) = 0$ .

**Axiom 4: Additivity (linearity).** For two games  $v$  and  $w$ , the Shapley value of the combined game  $v + w$  is  $\phi_j(v + w) = \phi_j(v) + \phi_j(w)$  for all  $j$ .

**Theorem Annexure A.2** (Shapley, 1953). *There exists a unique function  $\phi : \mathcal{G}^N \rightarrow \mathbb{R}^p$  satisfying Axioms 1 to 4, and it is given by*

$$\phi_j(v) = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|! (p - |S| - 1)!}{p!} [v(S \cup \{j\}) - v(S)].$$

*Proof.* Existence is verified by direct calculation: substituting the formula into each axiom confirms that all four hold.

Uniqueness proceeds by induction on the number of players. For  $p = 1$ , efficiency forces  $\phi_1(v) = v(\{1\}) - v(\emptyset)$ , which is unique. For the inductive step, any game  $v$  can be decomposed as a linear combination of unanimity games  $u_T$  (where  $u_T(S) = 1$  if  $T \subseteq S$  and  $u_T(S) = 0$  otherwise). By additivity, it suffices to determine  $\phi_j(u_T)$ . By the dummy axiom,  $\phi_j(u_T) = 0$  for  $j \notin T$ . By symmetry,  $\phi_j(u_T) = \phi_k(u_T)$  for all  $j, k \in T$ . By efficiency,  $\sum_{j \in T} \phi_j(u_T) = 1$ . Hence  $\phi_j(u_T) = 1/|T|$  for  $j \in T$ , which uniquely determines  $\phi$  for all unanimity games, and by linearity, for all games.  $\square$

## Annexure A.4 Gamma GLM Log-Link Score Equations

The expected response is  $\mu_i = \exp(\mathbf{x}_i^\top \boldsymbol{\beta})$ . The Gamma density with shape parameter  $\alpha$  and mean  $\mu$  is

$$f(y; \alpha, \mu) = \frac{1}{\Gamma(\alpha)} \left( \frac{\alpha}{\mu} \right)^\alpha y^{\alpha-1} \exp\left( -\frac{\alpha y}{\mu} \right), \quad y > 0.$$

The log-likelihood for observation  $i$  (up to a constant not depending on  $\boldsymbol{\beta}$ ) is

$$\ell_i(\boldsymbol{\beta}) = -\alpha \log \mu_i - \frac{\alpha y_i}{\mu_i} = -\alpha \mathbf{x}_i^\top \boldsymbol{\beta} - \alpha y_i \exp(-\mathbf{x}_i^\top \boldsymbol{\beta}).$$

The score equation with respect to the  $k$ -th coefficient is obtained by differentiating  $\ell = \sum_{i=1}^n \ell_i$ :

$$\frac{\partial \ell}{\partial \beta_k} = \alpha \sum_{i=1}^n \left( \frac{y_i}{\mu_i} - 1 \right) x_{ik} = 0, \quad k = 0, 1, \dots, p. \quad (\text{Annexure A.4.1})$$

Since  $\alpha > 0$  cancels, the maximum-likelihood estimates  $\hat{\boldsymbol{\beta}}$  do not depend on the shape parameter. The Fisher information matrix entry is

$$\mathcal{I}_{kl} = \alpha \sum_{i=1}^n \mu_i^{-1} x_{ik} x_{il},$$

and the asymptotic covariance of  $\hat{\boldsymbol{\beta}}$  is  $\mathcal{I}^{-1}$ .

## Annexure B Data Preprocessing and Encoding

The raw dataset contains 1 000 000 rows and 12 columns. The preprocessing procedure consists of four stages: data auditing, reference-category encoding, partitioning and input standardisation.

### Annexure B.1 Data Audit

The data audit confirmed no missing values in numeric columns, no exact duplicate rows, and that all categorical levels matched the expected domain. Missing values in `medical_history` and `family_medical_history` were coded as `None` and treated as a valid reference level.

### Annexure B.2 Reference-Category Encoding

Each categorical variable was encoded using reference-category (dummy) coding, producing 22 numeric features from the original 11 input variables. Table 31 lists the reference category for each variable.

Table 31: Reference categories used in dummy encoding.

| Variable                            | Reference Category |
|-------------------------------------|--------------------|
| <code>region</code>                 | northeast          |
| <code>medical_history</code>        | None               |
| <code>family_medical_history</code> | None               |
| <code>exercise_frequency</code>     | Rarely             |
| <code>occupation</code>             | Unemployed         |
| <code>coverage_level</code>         | Basic              |

Binary variables (`gender`, `smoker`) were coded as single indicator columns. Numeric variables (`age`, `bmi`, `children`) were passed through without transformation.

### Annexure B.3 Data Partitioning

The encoded dataset was split into training (60%), validation (20%) and test (20%) partitions using random sampling with `random_state=42`.

## Annexure B.4 Input Standardisation

All 22 input features were standardised to zero mean and unit variance using parameters computed from the training partition only. The response variable (charges) was similarly standardised. Binary dummy columns with zero variance in the training set were protected by setting their standard deviation to 1.0 before division.

---

**Algorithm 1** Data preprocessing and standardisation.

---

**Require:** Raw dataset  $\mathcal{D}$  with  $n$  rows and 12 columns

**Ensure:** Standardised feature matrices  $\mathbf{X}_{\text{train}}, \mathbf{X}_{\text{val}}, \mathbf{X}_{\text{test}}$  and response vectors  $\mathbf{y}_{\text{train}}, \mathbf{y}_{\text{val}}, \mathbf{y}_{\text{test}}$

- 1: Fill missing values in categorical columns with reference level
  - 2: Apply reference-category encoding to obtain  $n \times 22$  feature matrix
  - 3: Split:  $\mathcal{D}_{\text{train}}$  (60%),  $\mathcal{D}_{\text{val}}$  (20%),  $\mathcal{D}_{\text{test}}$  (20%) with seed 42
  - 4: Compute  $\boldsymbol{\mu}_X, \boldsymbol{\sigma}_X$  from  $\mathcal{D}_{\text{train}}$  features only
  - 5: Compute  $\mu_y, \sigma_y$  from  $\mathcal{D}_{\text{train}}$  response only
  - 6: **for** each partition  $\mathcal{P} \in \{\text{train, val, test}\}$  **do**
  - 7:      $\mathbf{X}_{\mathcal{P}} \leftarrow (\mathbf{X}_{\mathcal{P}}^{\text{raw}} - \boldsymbol{\mu}_X) / \boldsymbol{\sigma}_X$
  - 8:      $\mathbf{y}_{\mathcal{P}} \leftarrow (\mathbf{y}_{\mathcal{P}}^{\text{raw}} - \mu_y) / \sigma_y$
  - 9: **end for**
- 

## Annexure B.5 Response De-normalisation

Predictions on the normalised scale are converted back to the original Rand scale via  $\hat{y} = \hat{y}_{\text{norm}} \cdot \sigma_y + \mu_y$ , where  $\mu_y$  and  $\sigma_y$  are the training-set response mean and standard deviation respectively.

## Annexure C DNN Architecture and Training Procedure

The FunnelDNN model uses a feed-forward architecture with monotonically decreasing hidden layer widths:  $22 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$ . All layers are bias-free, and the CELU activation function is applied after each hidden layer. The output layer is linear with no activation.

### Annexure C.1 Training Procedure

Training minimises the mean squared error loss on normalised data using the NAdam optimiser. Validation-based early stopping monitors the dot-product  $R^2$  on the validation set every 5 epochs, with a patience of 10 checks.

---

**Algorithm 2** FunnelDNN training with early stopping.

---

**Require:** Standardised data  $(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}}, \mathbf{X}_{\text{val}}, \mathbf{y}_{\text{val}})$ , learning rate  $\eta$ , batch size  $B$ , patience  $P$ , validation interval  $V$

**Ensure:** Trained model state  $\theta^*$

```

1: Initialise FunnelDNN with seed 42
2:  $\theta^* \leftarrow \theta_0$ ,  $R^{2*} \leftarrow -\infty$ ,  $c \leftarrow 0$ 
3: for  $t = 1, 2, \dots, T_{\text{max}}$  do
4:   for each mini-batch  $(\mathbf{X}_b, \mathbf{y}_b)$  of size  $B$  do
5:     Compute  $\mathcal{L} = \text{MSE}(f_{\theta}(\mathbf{X}_b), \mathbf{y}_b)$ 
6:     Update  $\theta$  via NAdam with learning rate  $\eta$ 
7:   end for
8:   if  $t \bmod V = 0$  then
9:     Compute  $R_{\text{val}}^2 = r_{\text{dot}}^2(\mathbf{y}_{\text{val}}, f_{\theta}(\mathbf{X}_{\text{val}}))$ 
10:    if  $R_{\text{val}}^2 > R^{2*}$  then
11:       $R^{2*} \leftarrow R_{\text{val}}^2$ ,  $\theta^* \leftarrow \theta$ ,  $c \leftarrow 0$ 
12:    else
13:       $c \leftarrow c + 1$ 
14:    end if
15:    if  $c \geq P$  then
16:      break ▷ Early stopping triggered
17:    end if
18:  end if
19: end for
20: return  $\theta^*$ 

```

---

### Annexure C.2 Evaluation Metrics

Three metrics are computed on the original Rand scale after de-normalisation:

- $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ , where  $\text{SS}_{\text{res}} = \sum_i (y_i - \hat{y}_i)^2$  and  $\text{SS}_{\text{tot}} = \sum_i (y_i - \bar{y})^2$ .
- $\text{RMSE} = \sqrt{n^{-1} \sum_i (y_i - \hat{y}_i)^2}$ .
- $\text{MAE} = n^{-1} \sum_i |y_i - \hat{y}_i|$ .

During training, the dot-product  $R^2$  (squared correlation between predicted and observed values) is used on normalised data for computational speed.

## Annexure C.3 Checkpoint Contents

The final trained model is saved as `chapter3_final_model.pth`, a PyTorch checkpoint containing:

- Model weight tensors (state dictionary).
- Architecture specification (layer widths, activation, bias setting).
- Selected hyperparameters (learning rate, batch size, optimiser, momentum).
- Standardisation parameters ( $\mu_X$ ,  $\sigma_X$ ,  $\mu_y$ ,  $\sigma_y$ ).
- Feature names and evaluation metrics.

## Annexure D Monte Carlo Simulation Procedure

The parametric Monte Carlo simulation generates  $B = 100\,000$  synthetic policyholder profiles from marginal distributions estimated on the full dataset. Each profile is encoded, standardised and scored through the trained FunnelDNN to produce a model-implied predicted-cost distribution.

### Annexure D.1 Marginal Distributions

Table 32 summarises the marginal distribution assumed for each input variable.

Table 32: Marginal distributions used for Monte Carlo profile generation.

| Variable               | Distribution       | Parameters                          |
|------------------------|--------------------|-------------------------------------|
| age                    | Discrete Uniform   | $\{18, 19, \dots, 65\}$             |
| bmi                    | Continuous Uniform | $[18.00, 50.00]$                    |
| children               | Discrete Uniform   | $\{0, 1, \dots, 5\}$                |
| gender                 | Bernoulli          | $\hat{p}_{\text{male}}$ from data   |
| smoker                 | Bernoulli          | $\hat{p}_{\text{smoker}}$ from data |
| region                 | Multinomial        | 4 levels, estimated proportions     |
| medical_history        | Multinomial        | 4 levels, estimated proportions     |
| family_medical_history | Multinomial        | 4 levels, estimated proportions     |
| exercise_frequency     | Multinomial        | 4 levels, estimated proportions     |
| occupation             | Multinomial        | 4 levels, estimated proportions     |
| coverage_level         | Multinomial        | 3 levels, estimated proportions     |

### Annexure D.2 Profile Generation and Scoring

---

**Algorithm 3** Parametric Monte Carlo simulation.

---

**Require:** Fitted marginals, trained DNN  $f_{\theta^*}$ , standardisation parameters, sample size  $B = 100\,000$

**Ensure:** Predicted-cost vector  $\hat{\mathbf{y}}_{\text{MC}} \in \mathbb{R}^B$

- 1: Initialise random number generator with seed 42
  - 2: **for**  $i = 1, \dots, B$  **do**
  - 3:   Sample each variable independently from its marginal distribution
  - 4:   Apply reference-category encoding to obtain  $\mathbf{x}_i \in \mathbb{R}^{22}$
  - 5: **end for**
  - 6: Standardise:  $\mathbf{X}_{\text{MC}} \leftarrow (\mathbf{X}_{\text{MC}}^{\text{raw}} - \boldsymbol{\mu}_X) / \boldsymbol{\sigma}_X$
  - 7: Forward pass:  $\hat{\mathbf{y}}_{\text{norm}} \leftarrow f_{\theta^*}(\mathbf{X}_{\text{MC}})$
  - 8: De-normalise:  $\hat{\mathbf{y}}_{\text{MC}} \leftarrow \hat{\mathbf{y}}_{\text{norm}} \cdot \boldsymbol{\sigma}_y + \boldsymbol{\mu}_y$
-

## Annexure D.3 Convergence Diagnostics

Monte Carlo convergence was assessed by computing summary statistics (mean, standard deviation, quantiles and Monte Carlo standard error) at pilot sizes  $B \in \{1\,000; 2\,500; 5\,000; 10\,000; 25\,000; 50\,000; 100\,000\}$ . Stabilisation of the mean and tail quantiles confirmed that  $B = 100\,000$  provides adequate precision.

## Annexure D.4 Residual-Augmented Sensitivity

Sensitivity of the predicted-cost distribution to model uncertainty was assessed by augmenting point predictions with resampled centred training residuals:

$$\hat{y}_i^{\text{aug}} = \hat{y}_i + \varepsilon_i,$$

where  $\varepsilon_i$  is drawn with replacement from the centred residual vector. Fixed perturbation bands of  $\pm 0.5 \hat{\sigma}_\varepsilon$  were also computed.

## Annexure D.5 Encoding for MC Profiles

The same reference-category encoding scheme described in Annexure [Annexure B.2](#) was applied to the Monte Carlo profiles, producing an  $B \times 22$  feature matrix that matches the trained model's input specification.

## Annexure D.6 DNN Scoring

Each encoded profile is standardised using the training-set parameters  $(\mu_X, \sigma_X)$ , passed through the trained DNN in a single forward pass (with gradients disabled), and de-normalised to the original Rand scale using the training-set response parameters  $(\mu_y, \sigma_y)$ .

## Annexure E GLM Benchmark Methodology

The Gamma GLM with log-link serves as the actuarial benchmark against which the DNN is compared. A random forest and an XGBoost gradient-boosted tree regressor are fitted as additional benchmarks to span the model-class spectrum from linear to piecewise-constant to smooth nonlinear (see Section 3.4.6). The GLM was fitted using `statsmodels` with an intercept term and the same 22 reference-category encoded features used by the DNN and tree-based models.

### Annexure E.1 Fitting Procedure

---

**Algorithm 4** GLM benchmark fitting and comparison.

---

**Require:** Raw feature matrices  $\mathbf{X}_{\text{train}}, \mathbf{X}_{\text{val}}, \mathbf{X}_{\text{test}}$  (unstandardised), response vectors  $\mathbf{y}_{\text{train}}, \mathbf{y}_{\text{val}}, \mathbf{y}_{\text{test}}$

**Ensure:** Comparison metrics ( $R^2$ , RMSE, MAE) for GLM, RF, XGBoost and DNN on all partitions

- 1: Prepend intercept column to each feature matrix
- 2: Fit Gamma GLM with log-link on  $(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$  via IRLS
- 3: Extract coefficient table:  $\hat{\beta}$ , standard errors,  $z$ -values,  $p$ -values
- 4: **for** each partition  $\mathcal{P} \in \{\text{train, val, test}\}$  **do**
- 5:    $\hat{\mathbf{y}}_{\text{GLM}} \leftarrow \exp(\mathbf{X}_{\mathcal{P}} \hat{\beta})$
- 6:   Compute  $R^2$ , RMSE, MAE for GLM predictions
- 7:   Compute  $R^2$ , RMSE, MAE for DNN predictions (from checkpoint)
- 8: **end for**

---

### Annexure E.2 Coefficient Extraction

The fitted GLM produces a coefficient vector  $\hat{\beta} \in \mathbb{R}^{23}$  (22 features plus intercept). For each coefficient, the exponentiated value  $\exp(\hat{\beta}_k)$  gives the multiplicative effect on the expected cost. Statistical significance was assessed at the  $\alpha = 0.05$  level using Wald  $z$ -tests.

## Annexure E.3 Model Comparison

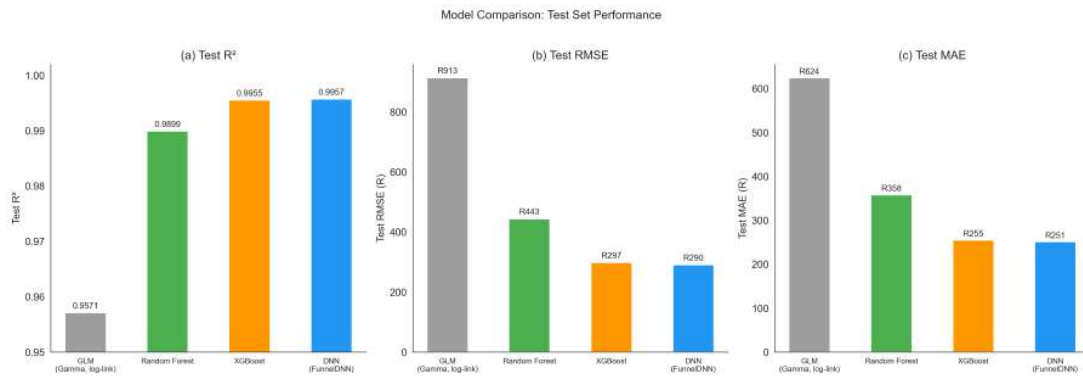


Figure 49: Test-set performance comparison across all four model classes: GLM (Gamma, log-link), Random Forest, XGBoost and DNN.

## Annexure F Shapley-Value Computation Procedure

Approximate Shapley values were computed using the SHAP library’s GradientExplainer, which uses integrated gradients as an approximation to exact Shapley values for differentiable models. All SHAP computations were performed on CPU, as GradientExplainer runs faster on CPU than on MPS for this model size.

### Annexure F.1 Computation Procedure

---

**Algorithm 5** Batch Shapley-value computation with dummy-to-raw aggregation.

---

**Require:** Trained DNN  $f_{\theta^*}$  (CPU copy), standardised training data  $\mathbf{X}_{\text{train}}$ , standardised MC profiles  $\mathbf{X}_{\text{MC}}$ , batch size  $B_s = 2000$

**Ensure:** Raw-variable Shapley matrix  $\Phi \in \mathbb{R}^{B \times 11}$

- 1: Sample 200 background rows from  $\mathbf{X}_{\text{train}}$  (seed 42)
  - 2: Initialise GradientExplainer with  $f_{\theta^*}$  and background data
  - 3: **for** each batch of  $B_s$  foreground profiles **do**
  - 4:   Compute Shapley values  $\phi \in \mathbb{R}^{B_s \times 22}$
  - 5: **end for**
  - 6: Concatenate batches to obtain  $\Phi_{22} \in \mathbb{R}^{B \times 22}$
  - 7: **for** each raw variable  $j = 1, \dots, 11$  **do**
  - 8:    $\Phi_{:,j} \leftarrow \sum_{k \in \mathcal{C}_j} \Phi_{22,:,k}$   $\triangleright$  Sum over dummy columns  $\mathcal{C}_j$
  - 9: **end for**
- 

### Annexure F.2 Dummy-to-Raw Variable Mapping

Table 33 shows the mapping from the 22 encoded dummy columns to the 11 original input variables used for Shapley aggregation.

Table 33: Dummy-to-raw variable aggregation mapping.

| Raw Variable           | Encoded Columns | Count |
|------------------------|-----------------|-------|
| age                    | 1               | 1     |
| gender                 | 2               | 1     |
| bmi                    | 3               | 1     |
| children               | 4               | 1     |
| smoker                 | 5               | 1     |
| region                 | 6, 7, 8         | 3     |
| medical_history        | 9, 10, 11       | 3     |
| family_medical_history | 12, 13, 14      | 3     |
| exercise_frequency     | 15, 16, 17      | 3     |
| occupation             | 18, 19, 20      | 3     |
| coverage_level         | 21, 22          | 2     |

### Annexure F.3 Efficiency Verification

The Shapley efficiency axiom ( $\sum_{j=1}^p \phi_j = f(\mathbf{x}) - \phi_0$ ) was verified for all  $B$  profiles. The maximum absolute reconstruction error was below 0.5 normalised units, confirming that the gradient-based approximation satisfies the efficiency property to adequate precision.

### Annexure F.4 Global Importance

Global feature importance was computed as the mean absolute Shapley value across all  $B$  Monte Carlo profiles:  $|\bar{\phi}|_j = B^{-1} \sum_{i=1}^B |\phi_{ij}|$  for each raw variable  $j$ . Variables were ranked by this measure to identify the most influential predictors of insurance cost.

### Annexure F.5 High-Cost Conditional Shapley Analysis

High-cost conditional Shapley analysis was performed by subsetting the Shapley matrix to profiles with predicted costs exceeding the  $\tau_{90}$ ,  $\tau_{95}$  and  $\tau_{99}$  quantiles of the MC distribution. Global and conditional importance rankings were compared to identify features whose influence shifts in the tail of the cost distribution.

## Annexure G Additional Diagnostic Figures

### Annexure G.1 Data Leakage Check Results

1. Split-before-preprocessing: PASS.
2. Training-only standardisation: PASS.
3. No target-derived predictor: PASS.
4. Duplicate-row audit: PASS.
5. Cross-partition leakage: PASS.
6. Benchmark-dataset characterisation: PASS.

### Annexure G.2 Partition Distribution Comparisons

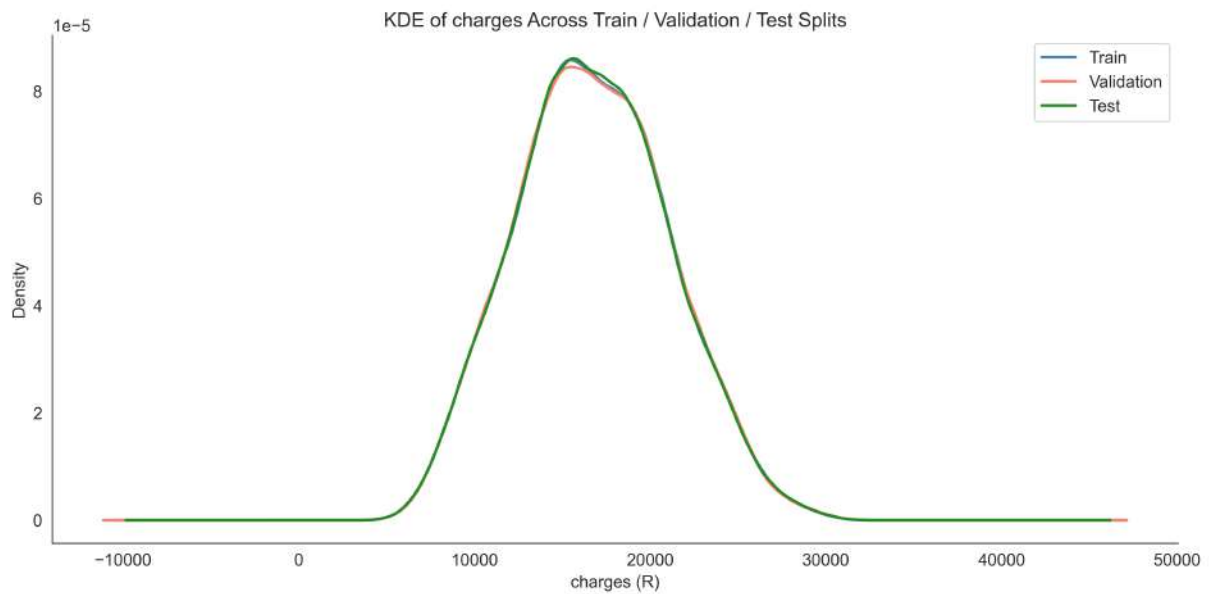


Figure 50: Charges KDE across partitions.

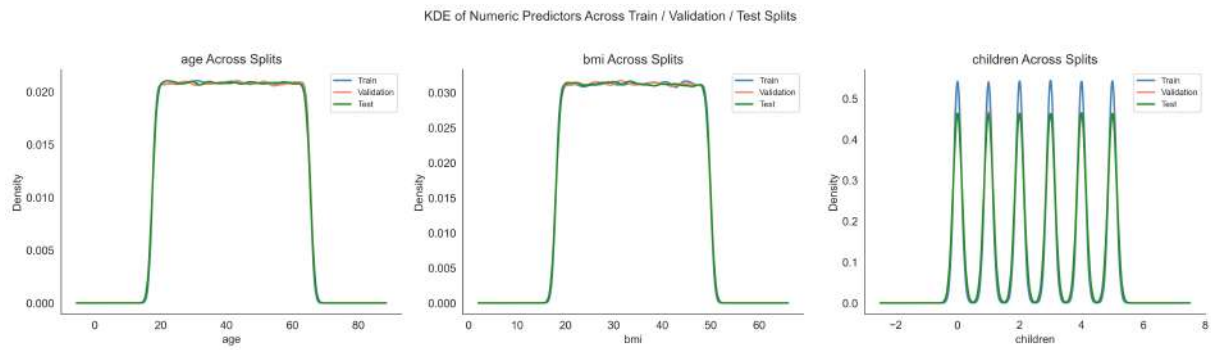


Figure 51: Numeric feature KDE across partitions.

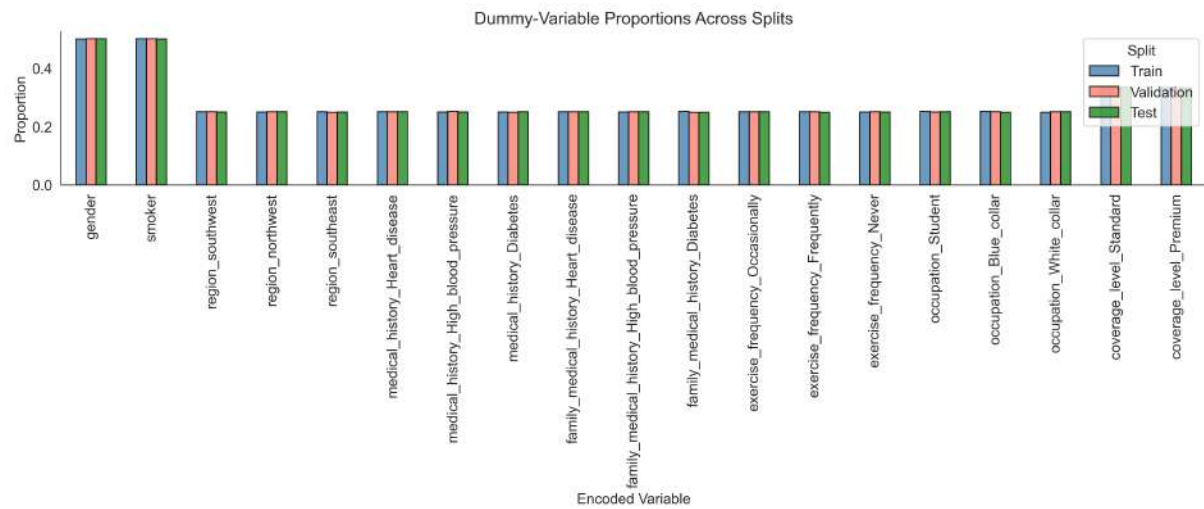


Figure 52: Dummy variable proportions across partitions.

## Annexure H Full Hyperparameter Grid Results

### Annexure H.1 Five-Phase Sequential Comparison

Hyperparameters were selected through a five-phase sequential comparison procedure. In each phase, one hyperparameter was varied while all others were held at their current best values. The phases proceeded in the following order:

1. Activation function (CELU, GELU, Tanh), with fixed  $\text{lr} = 0.001$ , batch size = 256, Adam.
2. Learning rate (0.01, 0.001, 0.0005, 0.0001), with fixed best activation, batch size = 256, Adam.
3. Batch size (64, 128, 256, 512), with fixed best activation and learning rate, Adam.
4. Optimiser (Adam vs NAdam), with all previous best values fixed.
5. Momentum  $\beta_1$  (0.90, 0.95, 0.99) with  $\beta_2 = 0.999$  fixed, using the best optimiser.

Each configuration was trained with early stopping (patience of 10 validation checks at 5-epoch intervals) up to a maximum of 100 epochs. Table 34 reports the full results.

Table 34: Full five-phase hyperparameter comparison results.

| Phase                  | Configuration    | Best Epoch | Val $R^2$ | Time (s) |
|------------------------|------------------|------------|-----------|----------|
| Phase 1: Activation    | CELU             | 100        | 0.995682  | 556.7    |
|                        | GELU             | 25         | 0.995636  | 313.8    |
|                        | Tanh             | 100        | 0.995532  | 565.9    |
| Phase 2: Learning Rate | 0.01             | 10         | 0.995453  | 222.9    |
|                        | 0.001            | 100        | 0.995682  | 558.1    |
|                        | 0.0005           | 100        | 0.995692  | 563.8    |
|                        | 0.0001           | 100        | 0.995686  | 556.0    |
| Phase 3: Batch Size    | 64               | 75         | 0.995678  | 1441.8   |
|                        | 128              | 30         | 0.995669  | 491.9    |
|                        | 256              | 100        | 0.995692  | 580.3    |
|                        | 512              | 160        | 0.995685  | 487.3    |
| Phase 4: Optimiser     | Adam             | 100        | 0.995692  | 562.7    |
|                        | NAdam            | 85         | 0.995693  | 582.5    |
| Phase 5: Momentum      | $\beta_1 = 0.90$ | 85         | 0.995693  | 586.6    |
|                        | $\beta_1 = 0.95$ | 40         | 0.995690  | 427.8    |
|                        | $\beta_1 = 0.99$ | 95         | 0.995685  | 621.8    |

## Annexure H.2 Bayesian Optimisation: GP Surrogate

A Gaussian process surrogate with a Matérn  $\nu = 5/2$  kernel was fitted to the observed hyperparameter evaluations to visualise the validation-error landscape. The GP was fitted using scikit-learn’s `GaussianProcessRegressor` with 15 random restarts, seed 42, and a composite kernel consisting of a constant kernel multiplied by the Matérn kernel plus a white noise kernel. Log-scaled axes were used for the learning rate dimension. The fitted surface and its uncertainty were evaluated on a  $100 \times 100$  grid with 15% margin beyond the observed range.

## Annexure H.3 Consolidated Best Hyperparameter Summary

Table 35: Best hyperparameter configuration selected for the final DNN model.

| Hyperparameter | Selected Value                                                                                  |
|----------------|-------------------------------------------------------------------------------------------------|
| Architecture   | $22 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$ |
| Activation     | CELU                                                                                            |
| Learning rate  | 0.0005                                                                                          |
| Batch size     | 256                                                                                             |
| Optimiser      | NAdam                                                                                           |
| $\beta_1$      | 0.90                                                                                            |
| $\beta_2$      | 0.999                                                                                           |
| Bias           | False (all layers)                                                                              |
| Max epochs     | 100                                                                                             |
| Early stopping | 10 checks (every 5 epochs)                                                                      |
| Loss function  | MSE (on normalised scale)                                                                       |

## Annexure I Software and Environment Details

### Annexure I.1 Python Distribution and Version

Python 3.11 via the Anaconda distribution. Jupyter Notebook was used for interactive development.

### Annexure I.2 Core Package Versions

Table 36: Python package versions used in the dissertation.

| Package         | Version | Purpose                                                    |
|-----------------|---------|------------------------------------------------------------|
| PyTorch         | 2.x     | DNN architecture, training and inference                   |
| SHAP            | 0.43+   | Shapley value computation (GradientExplainer)              |
| statsmodels     | 0.14+   | GLM (Gamma, log-link) fitting                              |
| NumPy           | 1.24+   | Numerical computation                                      |
| pandas          | 2.x     | Data manipulation and I/O                                  |
| SciPy           | 1.11+   | Statistical tests, distributions                           |
| scikit-learn    | 1.3+    | Random Forest, GP surrogate, train/test splitting, metrics |
| matplotlib      | 3.8+    | Static figure generation                                   |
| seaborn         | 0.13+   | Statistical visualisation                                  |
| scikit-optimize | 0.10+   | Bayesian Optimisation ( <code>gp_minimize</code> )         |
| XGBoost         | 3.x     | Gradient-boosted tree benchmark                            |
| Plotly          | 5.x     | Interactive 3D surface plots                               |

### Annexure I.3 Hardware Platform

- Processor: Apple Silicon (M-series)
- Primary compute device: MPS (Metal Performance Shaders) via PyTorch
- SHAP compute device: CPU (faster than MPS for GradientExplainer)
- Fallback: CPU (used when MPS is unavailable or for SHAP computations)

The PyTorch device was selected automatically: MPS (Metal Performance Shaders) if available on Apple Silicon, CUDA if available on NVIDIA hardware, and CPU as fallback.

### Annexure I.4 Reproducibility

Fixed random seed of 42:

- PyTorch weight initialisation (`torch.manual_seed(42)`)
- NumPy random operations (`np.random.seed(42)`)
- NumPy Generator objects (`np.random.default_rng(42)`)
- scikit-learn train/test split (`random_state=42`)
- Random Forest fitting (`random_state=42`)
- XGBoost fitting (`seed=42`)
- GP surrogate fitting (`random_state=42`)

## Annexure I.5 Data Files

- `insurance_dataset.csv`: 1 000 000 rows, 12 columns (age, gender, bmi, children, smoker, region, medical\_history, family\_medical\_history, exercise\_frequency, occupation, coverage\_level, charges). Synthetic benchmark dataset sourced from Kaggle.
- `chapter3_final_model.pth`: PyTorch checkpoint containing the trained FunnelDNN model state, architecture specification, hyperparameters, standardisation parameters, feature names and evaluation metrics.
- `model_ready_charges_train_20260517_115748.csv`: reference-category encoded training partition (60% of the dataset, 22 features + charges).

## Annexure I.6 Code Availability

The Python code used for data preprocessing, model training, Monte Carlo simulation, GLM and tree-based benchmarking, Shapley-value computation and diagnostic figure generation is available as supplementary material. The code is organised into separate Jupyter notebooks corresponding to each stage of the analysis:

- Data preprocessing and reference-category encoding (Annexure [Annexure B](#)).
- DNN architecture definition and training (Annexure [Annexure C](#)).
- Parametric Monte Carlo simulation (Annexure [Annexure D](#)).
- GLM benchmark fitting and comparison (Annexure [Annexure E](#)).
- Tree-based benchmark fitting: Random Forest and XGBoost (Section [3.4.6](#)).

- Shapley-value computation and conditional analysis (Annexure [Annexure F](#)).

Package versions and computing environment details are reported in Subsections [Annexure I.2](#) to [Annexure I.4](#) above. Full source code is available from the author on request.